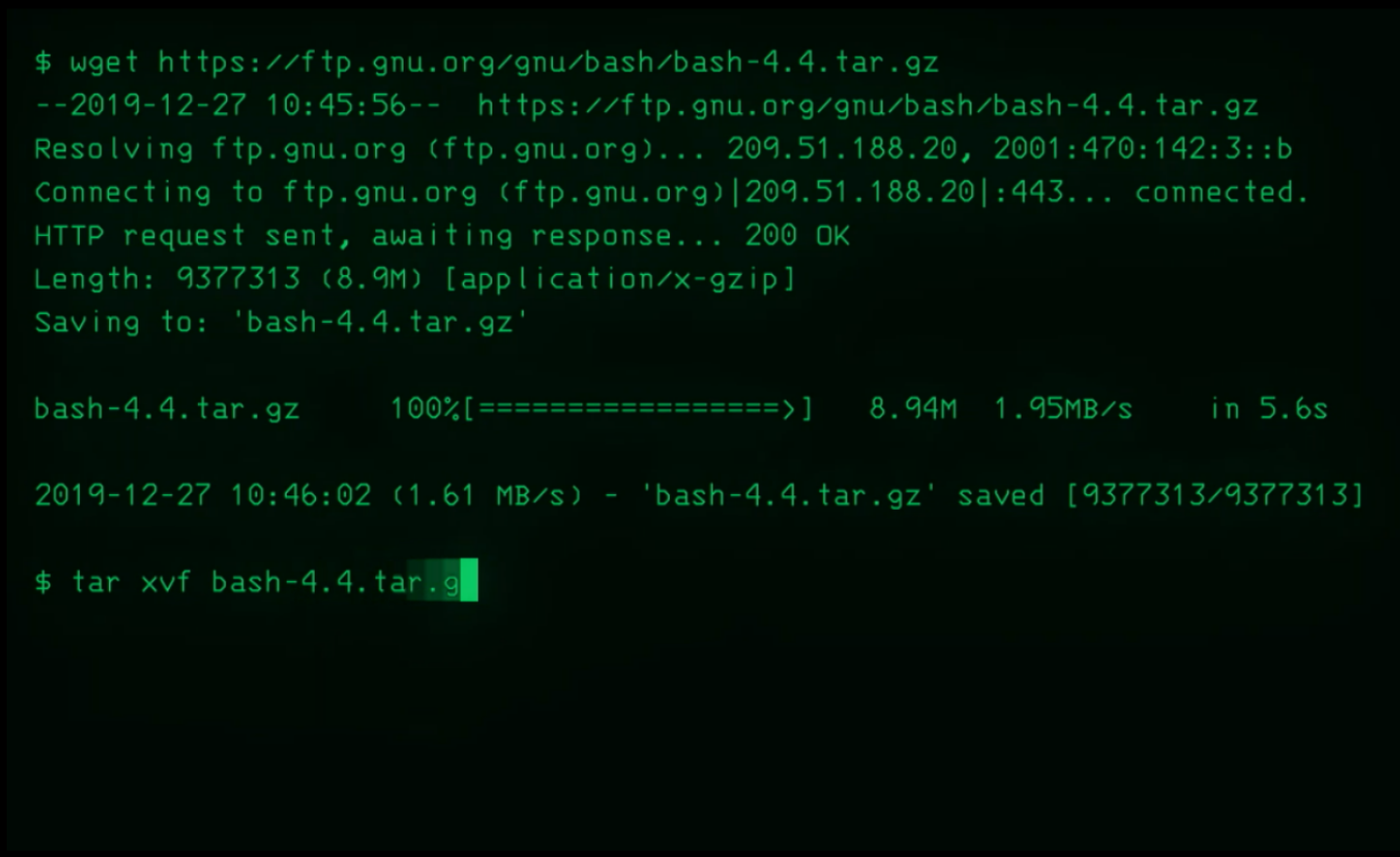
# Bash programming from scratch




Ilya Shpigor

# Bash programming from scratch

Ilya Shpigor

This book is for sale at http://leanpub.com/bash-programming-from-scratch

This version was published on 2021-05-08




This is a Leanpub book. Leanpub empowers authors and publishers with the Lean Publishing process. Lean Publishing is the act of publishing an in-progress ebook using lightweight tools and many iterations to get reader feedback, pivot until you have the right book and build traction once you do.

© 2020 - 2021 Ilya Shpigor

# Contents

**Introduction** . . . . . **1**

**General Information** . . . . . **3**
Operating Systems . . . . . 3
Computer Program . . . . . 28

**Bash Shell** . . . . . **43**
Development Tools . . . . . 43
Command Interpreter . . . . . 47
Navigating the File System . . . . . 50
Command Information . . . . . 79
Actions on Files and Directories . . . . . 85
Extra Bash Features . . . . . 98

**Bash Scripts** . . . . . **121**
Development Tools . . . . . 121
Why Do We Need Scripts? . . . . . 126
Variables and Parameters . . . . . 143
Conditional Statements . . . . . 172
Arithmetic Expressions . . . . . 195
Loop Constructs . . . . . 228
Functions . . . . . 253

**Package Manager** . . . . . **272**
Repository . . . . . 272
Package Operating . . . . . 272

**Finalwords** . . . . . **276**

**Acknowledgements** . . . . . **279**

**Glossary** . . . . . **280**
A . . . . . 280
B . . . . . 281
C . . . . . 282
D . . . . . 283

E . . . . . 283
F . . . . . 284
G . . . . . 284
H . . . . . 285
I . . . . . 285
K . . . . . 285
L . . . . . 285
M . . . . . 286
N . . . . . 286
O . . . . . 286
P . . . . . 287
Q . . . . . 288
R . . . . . 288
S . . . . . 289
T . . . . . 290
U . . . . . 290
V . . . . . 290
W . . . . . 291

**Solutions for Exercises . . . . . 292**
General Information . . . . . 292
Bash Shell . . . . . 292

**Helpful Links . . . . . 315**
General Information . . . . . 315
Bash . . . . . 315
Unix Environment . . . . . 316

# Introduction

Learning to program is not easy. It is even more challenging if you start from scratch and learn it on your own without mentors. This task is feasible, but its result depends on you.

You are going to study a new and complex subject. It will require strong motivation. Therefore, consider your goals before you continue reading this book. What do you expect from your new skills? What tasks will your programs solve? Answers to these questions help you to choose an effective way to study.

If you firmly intend to become a professional programmer in the shortest possible time, you need a mentor. Enroll yourself in a full-time or online course. It accelerates your progress significantly. You will get a chance to communicate directly with the mentor, ask him questions, and clarify unclear topics.

You can learn programming without a mentor if you do it for curiosity. Do you want to get a new hobby or learn a new fancy thing? In this case, self-study with the book is the right way. It will bring you practical benefits for relatively small efforts. After all, basic programming skills are helpful for anyone who works with the computer. Perhaps this book would be a starting point for you. It will help you to choose a direction for your further development.

Today there are plenty of books available in online shops and libraries. A reader with any technical level can find appropriate material for him. You can ask about the reason for writing this particular book.

Programming is a practical domain. Yes, it has a lot of theory. But mathematics has a lot of theory too. However, just knowing formulas does not make you a mathematician. In the same way, knowing the fundamental principles of software development does not make you a programmer.

You have to write a lot of code on your own to become a programmer. At first, this code won't work. Then, it will contain errors. Gradually, you will learn how to anticipate and fix the mistakes in advance. Knowledge of a particular language does not indicate your progress. But an evaluation of your old code does it. When you read your code several months later and notice its disadvantages, it confirms your progress.

So what is wrong with existing books? Many of them focus on a specific language or technology. They consider the chosen theme in detail. In this case, the author does not pay enough attention to practical exercises. A beginner programmer does not need such a volume of specialized knowledge. It brings him the wrong idea of how to learn to program. It is rarely that somebody reads such books for specific technologies from cover to cover. More often, they are used as a reference manual when the specific practical question arises.

Books of another type teach you some programming language by examples. When you study such a book, your progress goes faster. But there is one problem here. Many readers do not find the

motivation to work with examples. The author offers you to read and understand a lot of code. Often, this code demonstrates a specific principle and does not have a practical use case. Therefore, such examples are not attractive to readers.

Modern **general-purpose languages**[1] are complex. It means that you should know a lot about the language before dealing with real-life tasks. Here we meet a vicious circle. Examples are not interesting because they are useless. But useful real-life programs are too hard to understand for a beginner.

This book follows another approach. It starts with a general theory about a computer. Here we pay attention to the reasons for the technical solutions of the past. These solutions have defined the basic features of a modern computer. Following the reasons helps you to learn the material quickly.

The general theory about a computer helps you when you start programming in a particular language. You will certainly meet problems. For example, your program runs too slow or ends up with an error all the time. Knowing the internals of a computer helps you to understand the reasons for such behavior.

The next part of this book introduces you to the Bash programming language. Contrary to popular belief, it is a complex domain-specific language. However, there are some practical tasks that it solves easily and laconically. We will use them as examples. This way, we will learn the basic concepts of programming.

Our first step is replacing the graphical user interface with the shell. It will teach you basic operations on files and directories using Bash commands. When we get this basic syntax, we apply it and write our first Bash programs.

Please do not consider learning Bash as a necessary but useless exercise. Every professional programmer faces tasks of automating some processes and executing commands on a Unix-like system. Bash knowledge is irreplaceable in both cases.

If you are not able to complete some example or exercise, do not be upset. It means that the book does not disclose material sufficiently. Please write[2] to me about this. We will consider it out together.

There is a glossary at the end of the book. There you can clarify an unknown term that you met while reading the book.

[1] https://en.wikipedia.org/wiki/General-purpose_language
[2] mailto:petrsum@gmail.com

# General Information

This chapter explains the basics of how a computer works. It starts with describing operating systems and their history. Then the chapter considers families of modern operating systems and their features. The last section explains a computer program and its execution.

## Operating Systems

### History of OS Origin

Most of the computer users understand why **operating system**[3] (OS) is needed. Before launching a new **application**[4], they usually check its system requirements. These requirements specify the **hardware**[5] and operating system that a user needs to launch the application.

The system requirements bring us the idea that the OS is a software platform. The application requires this platform for correct working. But where did this requirement come from? Why can't you just buy a computer and launch the application without any OS?

Our questions seem meaningless at first blush. But let's consider them from the following point of view. Modern operating systems are multipurpose, and they offer users many features. Each specific user would not use most of these features. However, you cannot disable them. The OS utilizes the computer's resources intensively for maintaining its features. As a result, the user has much fewer computation resources for his applications. This overhead leads to the computer's slow work, hanging of the programs, and even rebooting the whole system.

Let's turn to history to find out the origins of operating systems. The first commercial OS was called GM-NAA I/O[6]. It appeared for the computer IBM 704[7] in 1956. All earlier computers have worked without any OS. Why didn't they need it?

The main reason for having an OS is high computational speed. For example, let's consider the first **electromechanical computer**[8]. Herman Hollerith[9] constructed it in 1890. This computer was called a tabulator. It does not require an OS and a program[10] in the modern sense. The tabulator performs a limited set of arithmetic operations only. Its hardware design defines this set of operations.

[3] https://en.wikipedia.org/wiki/Operating_system
[4] https://en.wikipedia.org/wiki/Application_software
[5] https://en.wikipedia.org/wiki/Computer_hardware
[6] https://en.wikipedia.org/wiki/GM-NAA_I/O
[7] https://en.wikipedia.org/wiki/IBM_704
[8] https://en.wikipedia.org/wiki/Tabulating_machine#1890_census
[9] https://en.wikipedia.org/wiki/Herman_Hollerith
[10] https://en.wikipedia.org/wiki/Computer_program

The tabulator loads input data for computation from **punched cards**[11]. These cards look like sheets of thick paper with punched holes. A human operator prepares these sheets manually and stacks them in special receivers. The sheets are threaded on the needles in the receivers. Then a short circuit happens in each punched hole. Each short circuit increases the mechanical counter, which is a rotating cylinder. The tabulator displays calculation results on dials that resemble watches.

Figure 1-1 shows the tabulator that Hermann Hollerith has constructed.



**Figure 1-1. Hollerith tabulating machine**

By modern standards, the tabulator works very slowly. There are several reasons for this. The first one is you need a lot of manual work. At the beginning, you should prepare input data. There was no way to punch the cards automatically. Then you manually load punched cards into the receivers. Both operations require significant efforts and time.

The second reason for low computation speed is mechanical parts. The tabulator contains many of them: needles to read data, rotating cylinders as counters, dials to output the result. All these mechanics work slowly. It takes about one second to perform one elementary operation. No automation can accelerate these processes.

If you look at how the tabulator works, you find no place for OS there. OS just does not have any tasks to do on such a kind of computer.

Tabulators used rotating cylinders for performing calculations. The next generation of computers replaced the cylinders with **relays**[12]. A relay is a mechanical element that changes its state due to an electric current.

The German engineer Konrad Zuse[13] designed one of the first relay computers[14] called Z2 in 1939. Then he improved this computer in 1941 and called it Z3. These machines perform a single

[11] https://en.wikipedia.org/wiki/Punched_card
[12] https://en.wikipedia.org/wiki/Relay
[13] https://en.wikipedia.org/wiki/Konrad_Zuse
[14] https://en.wikipedia.org/wiki/Z2_(computer)

elementary operation in milliseconds. They are much faster than tabulators. This performance gain happens thanks to applying relays instead of rotating cylinders.

The increased computation speed is the first feature of Zuse's computers. The second feature is the concept of a computer program. The idea of the universal machine, which follows the **algorithm**[15] you choose, was fundamentally new at that time.



An algorithm is a finite sequence of instructions to perform a particular calculation or task.

Z3 computer uses two input devices in parallel for supporting programs. The first one is a receiver for punched cards that resembles the tabulator's receiver. It reads the program to execute from the card. The second input device is the keyboard. It allows the user to type input data for the program.

The computers with the feature of changing their algorithms became known as **programmable**[16] or **general-purpose**.

The invention of programmable computers was a milestone in the development of computer science. Until this moment, machines were able to perform highly specialized tasks only. The construction of these machines was too expensive and unprofitable. This was a reason why commercial investors did not join the projects to design new computers. Only governments invested money there. However, this situation has changed since programmable computers came.

The next step in computer design is the construction of the **ENIAC**[17] (see Figure 1-2) computer in 1946 by John Eckert[18] and John Mauchly[19]. ENIAC uses a new type of element for performing computations. **Vacuum tubes**[20] replaced relays there. The tube is a purely electronic device. It does not have any mechanical component as the relay has. Therefore, when the electrical signal comes, the tube's reaction time is much faster than the relay one. This change increased ENIAC performance by order of magnitude comparing to relay-based machines. The new computer performs single elementary operation in 200 microseconds instead of milliseconds.

[15] https://en.wikipedia.org/wiki/Algorithm
[16] https://en.wikipedia.org/wiki/General_purpose_computer
[17] https://en.wikipedia.org/wiki/ENIAC
[18] https://en.wikipedia.org/wiki/J._Presper_Eckert
[19] https://en.wikipedia.org/wiki/John_Mauchly
[20] https://en.wikipedia.org/wiki/Vacuum_tube

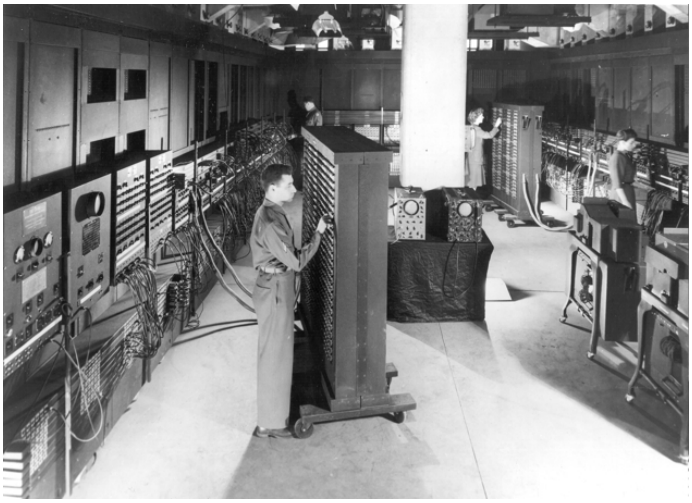


**Figure 1-2. ENIAC**

Most computer engineers were skeptical about vacuum tubes in the 1940s. The tubes were known for their low reliability and high power consumption. Nobody believed that a tube-based machine could work at all. ENIAC contains around 18,000 vacuum tubes. They burn out often. However, the computer performed the calculations efficiently between the failures. ENIAC was the first successful case of applying vacuum tubes. This case convinced many engineers that such an approach works well.

ENIAC is a programmable computer. You can set its algorithm using a combination of switches and jumpers on the control panels. This task requires considerable time and simultaneous work of several people. Figure 1-3 shows one of the panels for programming ENIAC.

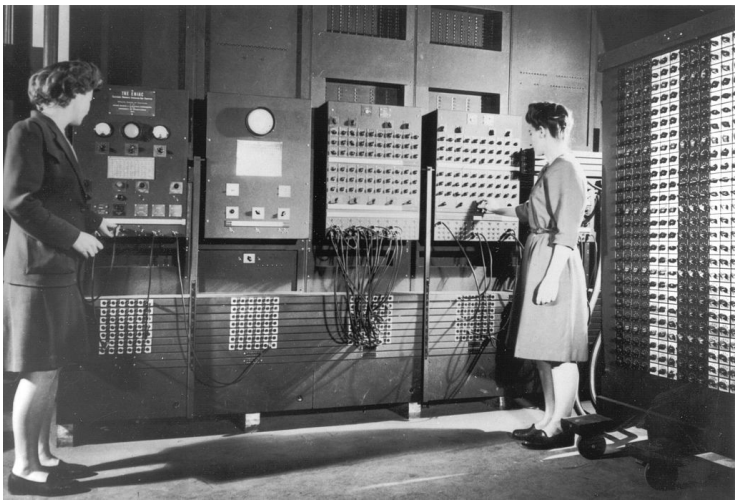


**Figure 1-3. ENIAC control panel**

ENIAC uses punched cards for reading input data and to output results. This is the same approach as previous models of computers have used. The new feature of ENIAC was storing intermediate results on the cards.

When the ENIAC should calculate a complex task, it can come to a leak of computation resources. However, you can split the task into several subtasks. Then the computer can solve each subtask separately. Storing the intermediate results on the cards helps you to move data from one subtask to another while you reprogram the computer.

Using ENIAC gave a new experience to engineers. It shows that all mechanical operations limit computer performance. These are the mechanical operations in ENIAC: manual reprogramming with switches and jumpers, reading and punching the cards. Each of these operations takes a significant time. Therefore, solving practical tasks with ENIAC is ineffective. The computer itself had an unprecedented performance at that time. However, it was idling most of the time and waiting for reprogramming or receiving input data. These findings initiated the development of new devices for both data input and output.

The next step in computer design is replacing vacuum tubes with **transistors**[21]. It again increased the computation speed by an order of magnitude. Transistors came together with new **input/output**[22] devices. It allowed to increase the loading of the computers and reprogram them more often.

When the epoch of transistors started, computers spread beyond government and military projects. Banks and corporations began to exploit machines too. It increases the number and variety of programs running on computers significantly.

Commercial usage of computers brings new requirements. When the computer works in a company, it should execute programs one after another without any delays. Otherwise, the machine does not justify the money spent on it.

New solutions were needed to meet new requirements. The most time-consuming task was switching between programs. Therefore, engineers of General Motors and North American Aviation came to an idea to automate it. They created the first commercial operating system **GM-NAA I/O**[23]. The primary goal of the system was managing the execution of programs.

The heavy load of computers and the variety of their programs brought another new task. When the computer of that time loads a program, the program defines the hardware's available features. For example, if the program includes a code to control an output device, you can use it. Otherwise, this device is unavailable.

Suppose that you are a company. You use a particular computer model all the time. The hardware always stays the same. Therefore, you do not change the code that controls your hardware. You just copy this code from one program to another. It takes extra efforts.

Copying hardware-specific code brought engineers to an idea of a special service program. The service program loads together with the user's application and provides the hardware support. These service programs became part of the first-generation operating systems after a while.

Now it is time to come back to the question, why do you need an OS. We found out that applications could work without them in general. Such applications are still in use today. For example, they are

[21] https://en.wikipedia.org/wiki/Transistor
[22] https://en.wikipedia.org/wiki/Input/output
[23] https://en.wikipedia.org/wiki/GM-NAA_I/O

**utilities**[24] for memory checks and disk partitioning, antiviruses, recovery tools. However, developing such applications takes more time and efforts. They should include the code for supporting all required hardware.

OS usually provides the code for supporting hardware. Why would you not use it? Most modern developers choose this way. It reduces the amount of work and speeds up the release of the applications.

However, a modern OS is a vast complex system. It provides much more features than just hardware support. Let's consider them too.

## OS Features

Why did we start studying programming by considering the OS? OS features are the basis for the application. Let's consider how it works.

Figure 1-4 demonstrates the interaction between the OS, an application and hardware. Applications are programs that solve practical user tasks. Examples are text editor, calculator, browser. Hardware is all electronic and mechanical components of a computer. For example, these are keyboard, monitor, central processor, video card.

[24] https://en.wikipedia.org/wiki/Utility_software

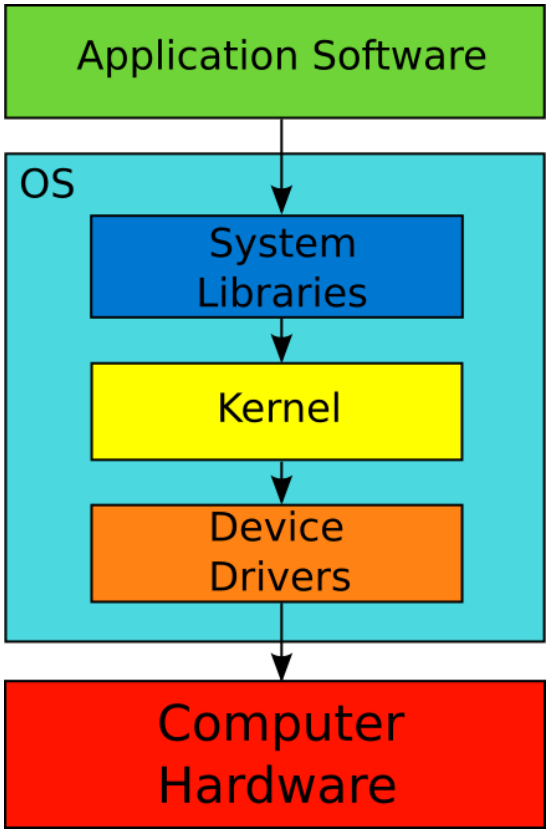



**Figure 1-4. The interaction between the OS, an application and hardware**

According to Figure 1-4, the application does not interact with hardware directly. The program does it through **system libraries**[25]. The system libraries are part of the OS. There are rules to access the system libraries. Each application should follow them.

**Application Programming Interface**[26] or API is the interface that the OS provides to an application to interact with system libraries. In general, the API term means a set of agreements for interacting components of the information system. These agreements become a well-known standard often. For example, the POSIX standard describes the API for portable OSes. The standard guarantees the compatibility of the OS and applications.

OS **kernel**[27] and **device drivers**[28] are part of OS. They dictate which hardware features the application can access. When the application interacts with system libraries, the libraries request capabilities of kernel and drivers. If you need the hardware feature and OS does not support it, you cannot use it.

When the application accesses the system library, it calls a library's **function**[29]. A function is a program fragment or an independent block of code that performs a certain task. You can imagine

[25] https://en.wikipedia.org/wiki/Library_(computing)
[26] https://en.wikipedia.org/wiki/API
[27] https://en.wikipedia.org/wiki/Kernel_(operating_system)
[28] https://en.wikipedia.org/wiki/Device_driver
[29] https://en.wikipedia.org/wiki/Subroutine

the API as a list of all available functions that the application can call. Besides that, API describes the following aspects of the interaction between the OS and applications:

1. What action does the OS perform when the application calls the specific system function?
2. What data does the function receive as input?
3. What data does the function return as a result?

Both the OS and application should follow the API agreements. It guarantees the compatibility of their current versions and future modifications. Such compatibility is impossible without a well-documented and standardized interface.

We have discovered that some applications work without an OS. They are called bare-metal software[30]. This approach works well in some cases. However, the OS provides ready-made solutions for interaction with the computer's hardware. Without these solutions, developers should take responsibility for managing hardware. It requires significant efforts. Imagine the variety of devices of a modern computer. The application should support all popular models of each device (for example, video cards). Without such support, the application would not work for all users.

Let's consider the features that the OS provides via the API. We can treat the computer's hardware as resources. The application uses these resources for performing calculations. The API reflects the list of hardware features that the program can use. Also, the API dictates the order of interaction between several applications and the hardware.

There is an example. Two programs cannot write data to the same area of the hard disk[31] simultaneously. There are two reasons for that:

1. A single magnetic head records data on the hard disk. The head can do one task at a time.
2. One program can overwrite data of another program in the same memory area. It leads to losing data.

You should place all requests to write data on the disk in a queue because of these two problems. Then each request should be performed separately. The OS takes care of this task.

The kernel (see Figure 1-4) of the OS provides a mechanism for managing access to the hard drive. This mechanism is called **file system**[32]. Similarly, the OS manages access to all **peripheral**[33] and internal devices of the computer. Besides kernel, there are special programs called device drivers (see Figure 1-4). They help the OS to control devices.

We have mentioned peripheral and internal devices. What is the difference between them? Peripherals are all devices that are responsible for inputting, outputting, and storing data permanently. Here are the examples:

[30] https://www.quora.com/What-is-bare-metal-programming-in-Embedded-systems
[31] https://en.wikipedia.org/wiki/Hard_disk_drive#Magnetic_recording
[32] https://en.wikipedia.org/wiki/File_system
[33] https://en.wikipedia.org/wiki/Peripheral

- Keyboard
- Mouse
- Microphone
- Monitor
- Speakers
- Hard drive

Internal devices are responsible for processing data, i.e. for executing programs. These are internal devices:

- **Central Processing Unit**[34] (CPU)
- **Random-Access Memory**[35] (RAM)
- **Video Card**[36] (graphics processing unit or GPU).

The OS provides access to the computer's hardware. At the same time, the OS has something besides the hardware management to share with user's applications. The system libraries have grown from the program modules to serve the devices. However, some libraries of modern OSes provide complex algorithms for processing data. Let's consider an example.

There is the Windows OS component called **Graphics Device Interface**[37] (GDI). It provides algorithms for manipulating graphic objects. GDI allows you to create a user interface for your application with minimal efforts. Then you can use the monitor to display this interface.

The system libraries with useful algorithms (like GDI) are software resources of the OS. These resources are already installed on your computer. You just need to know how to use them. Besides that, the OS also provides access to third-party libraries and their algorithms. You can install these libraries separately and use them in your applications.

The OS manages hardware and software resources. Also, it organizes the joint work of running programs. The OS performs several non-trivial tasks to launch an application. Then the OS tracks its work. If the application violates some agreements (like memory usage), the OS terminates it. We will consider launching and executing the program in detail in the next section.

If the OS is multi-user, it controls access to the data. It is an important security feature. Thanks to such control, each user can access his own files only. Therefore, they can work with the same computer safely.

Here is the summary. The modern OS has all following features:

1. It provides and manages access to hardware resources of the computer.
2. It provides its own software resources.
3. It launches applications.

[34] https://en.wikipedia.org/wiki/Central_processing_unit
[35] https://en.wikipedia.org/wiki/Random-access_memory
[36] https://en.wikipedia.org/wiki/Video_card
[37] https://en.wikipedia.org/wiki/Graphics_Device_Interface

4. It organizes the interaction of applications with each other.
5. It controls access to users' data.

You can guess that it is impossible to launch several applications in parallel without the OS. You are right. When you develop an application, you have no idea how a user will launch it. The user can launch your application together with another one. You cannot foresee this use case. However, the OS responds for launching all applications. It means that the OS has enough information to allocate computer resources properly.

## Modern OSes

We have reviewed the OS features in general. Now we will consider modern operating systems. Today you can pick any OS and get very similar features. However, their developers follow different approaches. This leads to implementation difference that can be important for some users.

There is the **software architecture**[38] term. It means the implementation aspects of the specific software and the solutions that led to them.

All modern OSes have two features that determine the way how users interact with them. These features are multitasking and the graphical user interface. Let's take a closer look at them.

### Multitasking

All modern OSes support **multitasking**[39]. It means that they can execute multiple programs in parallel. The systems with this feature displaced OSes without it. Why does multitasking so important?

The challenge of efficient usage of computers came in the 1960s. Computers were expensive at that time. Only large companies and universities were able to buy them. These organizations counted every minute of working with their machines. They did not accept any idle time of the hardware because of its huge cost.

Early operating systems executed programs one after another without delays. This approach saves time for switching computer tasks. If you use such an OS, you should prepare several programs and their input data in advance. Then you should write them on the storage device (e.g., magnetic tape). You load the tape to the computer's reading device. Afterward, the computer executes the programs sequentially and prints their results to an output device (e.g., a printer). This mode of operation is called **batch processing**[40].

Batch processing increased the efficiency of using computers in the 1960s. This approach has automated program loading. The human operators became unnecessary for this task. However, the computers still had the **bottleneck**[41]. The computational power of processors was increasing

[38] https://en.wikipedia.org/wiki/Software_architecture
[39] https://en.wikipedia.org/wiki/Computer_multitasking
[40] https://en.wikipedia.org/wiki/Batch_processing
[41] https://en.wikipedia.org/wiki/Bottleneck_(production)

significantly every year. The speed of peripherals has remained almost the same. It led to CPU idles all the time while waiting for input/output.



Bottleneck is a component or resource of an information system that limits its overall performance or bandwidth.

Why does CPU idle and wait for peripheral devices? Here is an example. Suppose that the computer from the 1960s runs programs one by one. It reads data from a magnetic tape and prints the results on the printer. The OS loads each program and executes its instructions. Then it loads the next one, and so on.

The problem happens when reading data and printing the results. The time for reading data on the magnetic tape is huge on the CPU scale. This time is enough for the processor to perform many calculations. However, it does not do that. The reason for that is the currently loaded program occupies all computer resources. The same CPU idle happens when printing the results. The printer is a slow mechanical device.

The CPU idle led OS developers to the concept of **multiprogramming**[42]. This concept implies that OS loads several programs into the computer memory at the same time. The first program works as long as all resources it needs are available. It stops executing once a required resource is busy. Then OS switches to another program.

Here is an example. Suppose that your application wants to read data from your hard disk. While the disk controller reads the data, it is busy. It cannot process the following requests from the program. Thus, the application waits for the controller. In this case, OS stops executing it and switches to the second program. The computer executes it to the end or until it has all the required resources. When the second program finishes or stops, OS switches tasks again.

Multiprogramming differs from multitasking that modern OSes have. Multiprogramming fits the batch processing mode very well. However, this load balancing concept is not suitable for **interactive systems**[43]. An interactive system considers each user action as an event (for example, a keystroke). The system should process events immediately when they happen. Otherwise, the user cannot work with the system.

Here is an example of workflow with an interactive system. Suppose that you are typing text in the MS Office document. You press a key and expect to see this symbol on the screen. If the computer requires several seconds to process your keystroke and display the symbol, you cannot work like that. Most of the time, you will wait to check if the computer has processed your keystroke or not. This is inefficient.

Multiprogramming cannot handle events in the interactive system. It happens because you cannot predict when task switching happens next time. OS switches tasks when a program finishes or when it requires a busy resource. Suppose that your text editor is not an active program now. Then you

[42] https://en.wikipedia.org/wiki/Computer_multitasking#Multiprogramming
[43] https://en.wikipedia.org/wiki/Interactivity#Computing_science

do not know when it can process your keystroke. It can happen in a second or in several minutes. This is unacceptable for a good user interface.

The multitasking concept solves the task of processing events in interactive systems. There were several versions of multitasking before it comes to the current state. Modern OSes use **displacing multitasking**[44] with pseudo-parallel tasks processing. The idea behind it is to allow the OS to choose an appropriate task for executing at the moment. The OS takes into account the priorities of the running programs. Therefore, a higher priority program receives hardware resources more often than a lower priority one. OS kernel provides this task-switching mechanism. It is called **task scheduler**[45].

Pseudo-parallel processing means that the computer executes one task only at any given time. However, the OS switches tasks so quickly that you can suppose the processing of several programs at once. This concept allows the OS to react immediately to any event. Even though every program and OS component hardware resources at strictly defined moments.

There are computers with multiple processors or with multi-core processors[46]. Only these computers can execute several programs at once. The number of the running programs equals the number of cores of all processors approximately. The preemptive multitasking mechanism with constant task switching works on such systems well. It is universal and balances the load regardless of the number of cores. This way, the computer responds to the user's actions quickly. The number of processor cores does not matter.

## User Interface

Modern OSes are able to solve a wide range of tasks. These tasks depend on the computer type where you run the OS. Here are the main types of computers:

- Personal computers[47] (PC) and notebooks.
- Smartphones and tablets.
- Servers.
- Embedded systems[48].

We will consider OSes for PCs and notebooks only. Apart from multitasking, they provide **graphic user interface**[49] (GUI). This interface means the way to interact with the system. You launch applications, configure computer devices and OS components via the user interface. Let's take a look at its history and find how it has reached the current state.

Nobody works with commercial computers interactively before 1960. Digital Equipment Corporation[50] implemented the interactive mode for their minicomputer[51] PDP-1[52] in 1959. It was a

[44] https://en.wikipedia.org/wiki/Preemption_(computing)
[45] https://en.wikipedia.org/wiki/Scheduling_(computing)
[46] https://en.wikipedia.org/wiki/Multi-core_processor
[47] https://en.wikipedia.org/wiki/Personal_computer
[48] https://en.wikipedia.org/wiki/Embedded_system
[49] https://en.wikipedia.org/wiki/Graphical_user_interface
[50] https://en.wikipedia.org/wiki/Digital_Equipment_Corporation
[51] https://en.wikipedia.org/wiki/Minicomputer
[52] https://en.wikipedia.org/wiki/PDP-1

fundamentally new approach. Before that, IBM computers dominated the market in the 1950s. They worked in batch processing mode only. IBM OSes with multiprogramming automated program loading and provided high performance for calculation tasks.

The idea of interactive work with the computer appeared first in the SAGE military project. The US Air Force was its customer. The goal of the project was to develop an automated air defense system to detect Soviet bombers.

When working on the SAGE project, engineers faced the problem. The user of the system should analyze data from radars in real-time. If he detects a threat, he should react as quickly as possible and command to intercept the bombers. However, the existed methods of interaction with the computer did not fit this task. They did not allow showing information to the user in real-time and receive his input at any moment.

Engineers came to the idea of the interactive mode. They implemented it in the first interactive computer AN/FSQ-7[53] in 1955 (see Figure 1-5). The computer used the monitor with a **cathode-ray tube**[54] to display information from radars. The **light pen**[55] allowed the user to command the system.



**Figure 1-5. Computer AN/FSQ-7**

The new way of interaction with computers became known in scientific circles. It gained popularity

[53] https://en.wikipedia.org/wiki/AN/FSQ-7_Combat_Direction_Central
[54] https://en.wikipedia.org/wiki/Cathode-ray_tube
[55] https://en.wikipedia.org/wiki/Light_pen

quickly. The existing batch processing mode coped with program execution well. However, this mode was inconvenient for development and **debugging**[56] applications.



Debugging a program is searching and eliminating its errors.

Suppose that you are writing a program for the computer with batch processing. You should prepare your code and write it to the storage device. When the device is ready, you put it in a queue. The computer's operator takes devices from the queue and loads them to the computer one by one. Therefore, your task can wait in the queue for several hours. Now assume that an error happened in your program. In this case, you should fix it, prepare the new version of the program and write it to the device. You put it in the queue again and wait for hours. Because of this workflow, you need several days to make even a small program working.

The software development workflow differs when you use an interactive mode. You prepare your program and load it to the computer. Then it launches the program and shows you results. Immediately, you can analyze a possible error, fix it and relaunch the program. This workflow accelerates software development and debugging tasks significantly. Now you spend few hours on the task that requires days with batch processing mode.

The interactive mode brought new challenges for computer engineers. This mode requires the system that reacts to user actions immediately. And providing a short reaction time required a new load-balancing mechanism. The multitasking concept became a solution for this task.

There are alternative solutions for providing interactive mode. For example, there are interactive single-tasking OSes like MS-DOS[57]. MS-DOS was the system for cheap PCs of the 1980s.

However, it was inadvisable to apply single-tasking in the 1960s when computers were too expensive. These computers executed many programs in parallel. Such a mode was called **time-sharing**[58]. It allows sharing expensive hardware resources among several users. The single-tasking approach does not fit such a use case because it is not compatible with time-sharing.

When the first relatively cheap personal computers appeared in the 1980s, they used single-tasking OSes. Such systems require fewer hardware resources than their analogs with multitasking. Despite its simplicity, single-tasking OSes support interactive mode for the running program. This mode became especially attractive for PC users.

When interactive mode became more and more popular, computer engineers meet a new challenge. The existing devices for interacting with computers turned out to be inconvenient. Magnetic tapes and printers were widespread through the 1950s and early 1960s. They did not fit interactive mode absolutely.

**Teletype**[59] (TTY) became a prototype of a device for interactive work with a computer. Figure 1-6 shows the Model 33 teletype. It is an electromechanical typewriter. It is connected to the same

[56] https://en.wikipedia.org/wiki/Debugging
[57] https://en.wikipedia.org/wiki/MS-DOS
[58] https://en.wikipedia.org/wiki/Time-sharing
[59] https://en.wikipedia.org/wiki/Teleprinter

typewriter through wires. Once two teletypes are connected, operators can send text messages to each other. The sender types text on his device. The keystrokes are transmitted to the receiver's device. It prints out each received letter on paper.




**Figure 1-6. A Teletype Model 33**

**Teletype**[60] (TTY) became a prototype of a device for interactive work with a computer. The original idea behind this device was to connect two of them via wires. It allows users on both sides to send each other text messages. One user types the message on the keyboard. Then his device transmits the keystrokes to the receiver. When the teletype on the other side receives data, it prints the text on paper.

Computer engineers connected the teletype to the computer. This solution allowed the user to send text commands to the machine and receive results. Such a workflow became known as a **command-line interface**[61] (CLI). Figure 1-6 shows the Model 33 teletype. It was one of the most popular devices in the 1960s.

Teletype uses the printer as an output device. It works very slow and requires around 10 seconds to print a single line. The next step of developing the user interface was replacing the printer with the monitor. This increased the data output speed several times. The new device with a keyboard and

[60] https://en.wikipedia.org/wiki/Teleprinter
[61] https://en.wikipedia.org/wiki/Command-line_interface

monitor was called the **terminal**[62]. It replaced teletypes in the 1970s.

Figure 1-7 shows a modern command-line interface. You can see the **terminal emulator**[63] application there. This application emulates the terminal device for the sake of compatibility. It allows you to run programs that work with the real terminal only. The emulator application in Figure 1-7 is called Terminator[64]. The Bash command-line interpreter is running in the Terminator window. The window displays the results of the ping and ls programs.

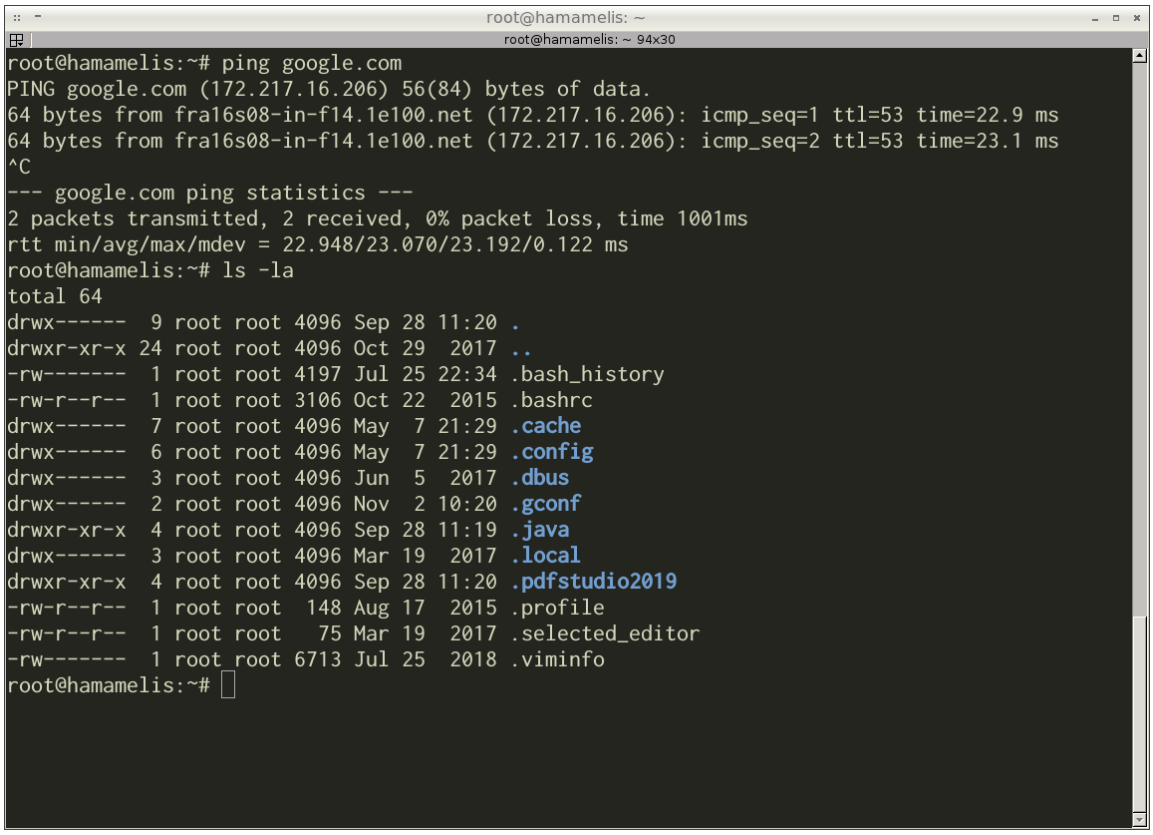



**Figure 1-7. Command-line interface**

The command-line interface is still in demand today. It has several advantages over the graphical interface. For example, CLI does not require as many resources as GUI. CLI runs reliably on low-performance embedded computers as well as on powerful servers. If you use CLI to access the computer remotely, you can use a low bandwidth communication channel. The server will receive your commands in this case.

The command-line interface has some disadvantages. Learning to use CLI effectively is a serious challenge. You have to remember around a hundred commands. Each command has several modes that change its behavior. Therefore, you should keep in mind these modes too. It takes some time to remember at least a minimum set of commands for daily work.

[62] https://en.wikipedia.org/wiki/Computer_terminal
[63] https://en.wikipedia.org/wiki/Terminal_emulator
[64] https://en.wikipedia.org/wiki/Terminator_(terminal_emulator)

There is an option to make the command-line interface more user-friendly. You can give a hint to the user about available commands. It was done in the **text-based interface**[65] (TUI). The interface uses **box-drawing characters**[66] along with alphanumeric symbols. The box-drawing characters display the graphic primitives on the screen. Primitives are lines, rectangles, triangles, etc. They guide the user about the available actions he can do with the application.

Figure 1-8 shows a typical text-based interface. There is an output of system resource usage by the htop program.

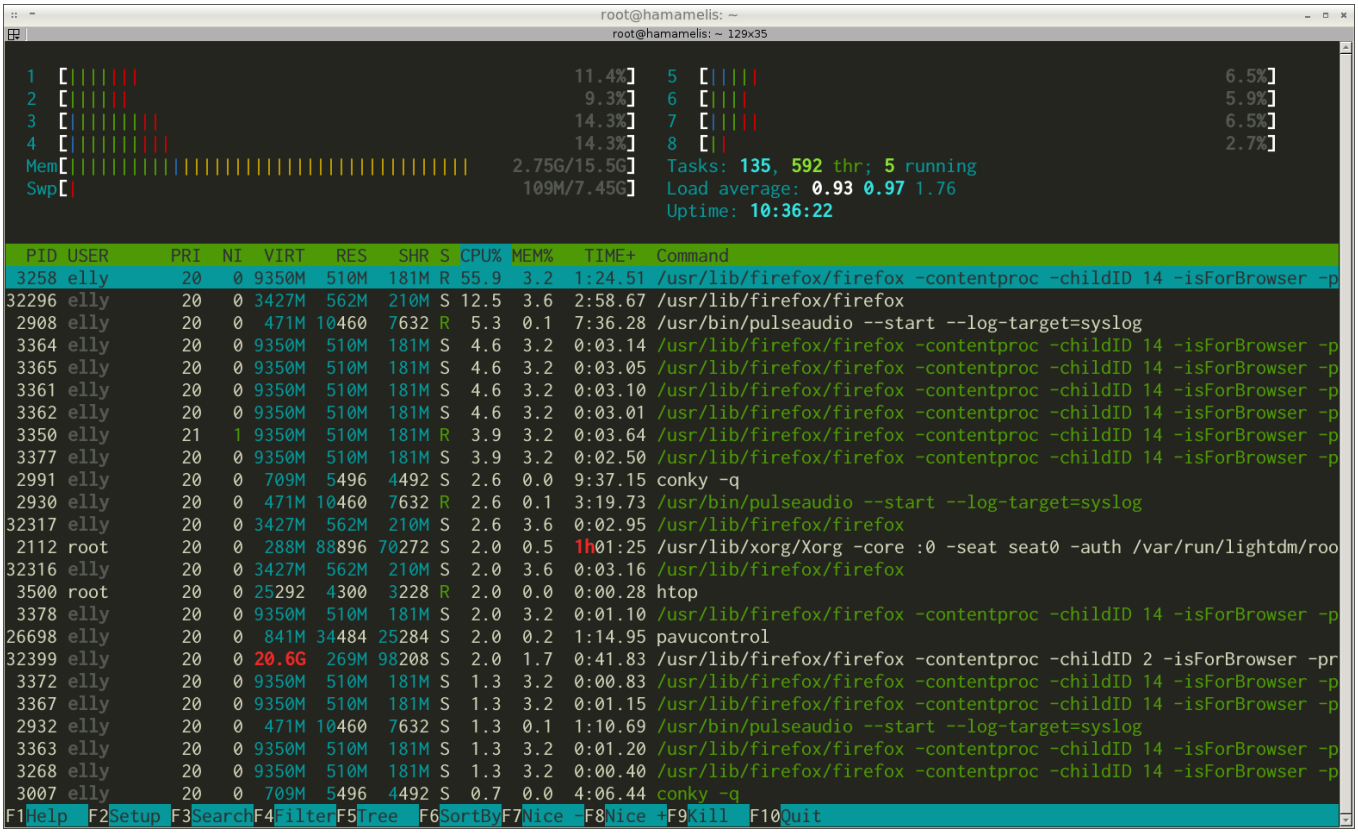



**Figure 1-8. Text-based user interface**

The further performance gain of computers allowed OS developers to replace box-drawing characters with real graphic elements. There are examples of such elements: windows, icons, buttons, etc. It was a moment when the full-fledged graphical interface came. Modern OSes provide this kind of interface.

Figure 1-9 demonstrates the Windows GUI. You can see the desktop. There are windows of three applications there. The applications are Explorer, Notepad and Calculator. They work in parallel.

[65] https://en.wikipedia.org/wiki/Text-based_user_interface
[66] https://en.wikipedia.org/wiki/Box-drawing_character

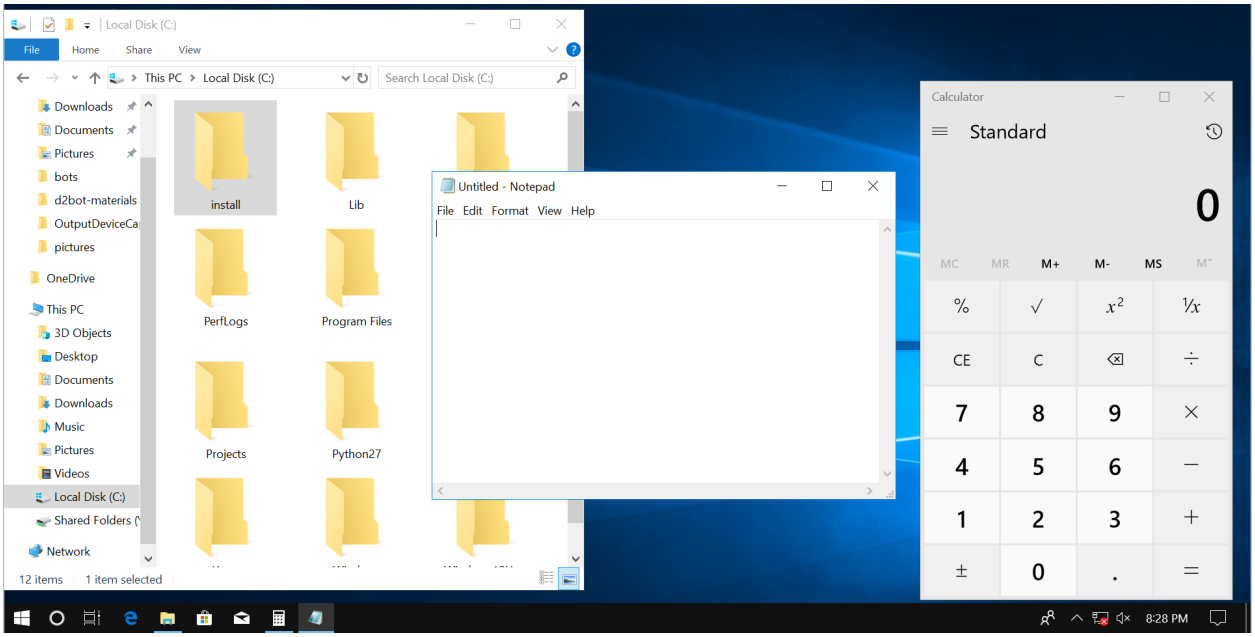



**Figure 1-9. Windows GUI**

The first GUI appeared in the Xerox Alto[67] minicomputer (see Figure 1-10). It was developed in the research center Xerox PARC[68] in 1973. However, the graphical interface did not spread widely until the 1980s. It happens because GUI requires a lot of memory and high computer performance. PCs with such features were too expensive for ordinary users at that time.

Apple produced the first relatively cheap PC with GUI in 1983. It was called Lisa.

[67] https://en.wikipedia.org/wiki/Xerox_Alto
[68] https://en.wikipedia.org/wiki/PARC_(company)



**Figure 1-10. Minicomputer Xerox Alto**

## Families of OSes

There are three families of OSes[69] that dominate the market today. Here are these families:

- Windows[70]
- Linux[71]
- macOS[72]

The term "family" means several OS versions that follow the same architectural solutions. Therefore, most functions in these versions are implemented in the same way.

The developers of the OS family follow the same architecture. They do not offer something fundamentally new in the upcoming versions of their product. Why?

Actually, changes in modern OSes happen gradually and slowly. The reason for this is a **backward compatibility**[73] problem. This compatibility means that newer OS versions provide the features of

[69] https://en.wikipedia.org/wiki/Category:Operating_system_families
[70] https://en.wikipedia.org/wiki/Microsoft_Windows
[71] https://en.wikipedia.org/wiki/Linux
[72] https://en.wikipedia.org/wiki/MacOS
[73] https://en.wikipedia.org/wiki/Backward_compatibility

older versions. Most existing programs require these features for their work. You can suppose that backward compatibility is an optional requirement. However, it is a severe limitation for software development. Let's find out why it is.

Imagine that you wrote a program for Windows and sell it. Sometimes users meet errors in the program. You receive bug reports and fix them. Also, you add new features from time to time.

Your business goes well until the new Windows version comes. Let's assume that Microsoft has changed its architecture completely. Therefore, your program does not work on the new OS version. This leads users of your program to the following choice:

- Update Windows and wait for the new version of your program that works there.
- Do not update Windows and continue to use your program.

If users need your program for daily work, they refuse the Windows update. Using the program is more important than getting new OS features.

We know that Microsoft has changed the Windows architecture completely. It means that you should rewrite your program from scratch. Now count all the time that you have spent fixing bugs and adding new features. You should repeat all this work as soon as possible. Most likely, you will give up this idea and suggest that users of your program stay on the old Windows version.

Windows is a very popular and widespread OS. It means that there are many programs like yours. Their developers will come to the same decision as you. As a result, nobody updates to the new Windows version. This situation demonstrates the backward compatibility problem. This problem forces OS developers to be careful with changing their products. The best solution for them is to make a family of similar OSes.

There is a significant influence of user applications on OS development. For example, Windows and IBM computers owe their success to a table processor Lotus 1-2-3[74]. You need both IBM PC and Windows to lunch Lotus 1-2-3. For the sake of Lotus 1-2-3, users bought both the computer and OS. The specific combination of the hardware and software is called the **computing platform**[75]. The popular application, which brings the platform to the broad market, is called **killer application**[76].

The tabular processor VisiCalc[77] was another killer application. It promoted the distribution of the Apple II[78] computers. In the same way, free compilers for C, Fortran and Pascal languages help Unix OS to become popular in university circles.

There was the killer application behind each of the modern OS families. This application gives the OS the initial success. Further distribution of the OS happens thanks to the network effect[79]. This effect means that developers tend to choose the most widespread computing platforms for their new applications.

[74] https://en.wikipedia.org/wiki/Lotus_1-2-3
[75] https://en.wikipedia.org/wiki/Computing_platform
[76] https://en.wikipedia.org/wiki/Killer_application
[77] https://en.wikipedia.org/wiki/VisiCalc
[78] https://en.wikipedia.org/wiki/Apple_II_series
[79] https://en.wikipedia.org/wiki/Network_effect

What are the differences between the OS families? Windows and Linux are remarkable because they do not depend on the hardware. It means that you can install them on any modern PC or laptop. macOS runs on Apple computers only. If you want to use macOS on different hardware, you would need the unofficial modified version[80] of OS.

Hardware compatibility is an example of the design decision of the OS development. There are many such decisions. Together they define the features and design of each OS family.

There is one more important point for software development besides the OS design. OS dictates the infrastructure for the programmer. The infrastructure means development tools. Examples of these tools are IDE, compiler, build system. Tools together with OS API impose some design decisions for the applications. It leads to a specific culture for program development. Please keep in mind that you should develop applications differently for each OS. Take it into account when you design your programs.

Let's consider the origins of software development cultures for Windows and Linux.

## Windows

Windows is proprietary software[81]. The source code of such software is unavailable for users. You cannot read or modify it as you want. In other words, there is no legal way to know about proprietary software more than its documentation tells you.

If you want to install Windows on your computer, you should buy it from Microsoft. However, manufacturers of computers pre-install Windows on their devices often. In this case, the final cost of the computer includes the price of the OS.

The target devices for Windows are relatively cheap PCs and laptops. Many people can buy such a device. Therefore, there is a huge market of potential users. Microsoft tends to keep its competitive edge in this market. The best way to reach it is to prevent appearing of Windows analogs with the same features from other companies. For reaching this goal, Microsoft takes care of protecting its intellectual property. The company does it in both technical and legal ways. An example of legal protection is the user agreement. It prohibits you to explore the internals of the OS.

The Windows OS family has a long history. Also, it is popular and widespread. It leads many developers to chose this OS as the target for their applications. However, the Microsoft company has developed the first Windows applications on its own. An example is the package of office programs Microsoft Office[82]. Such applications became a standard to follow for other developers.

Microsoft followed the same principle when developing both Windows and applications for it. It is a secrecy principle:

- Source codes are not available to users.
- Data formats are undocumented.

[80] https://en.wikipedia.org/wiki/Hackintosh
[81] https://en.wikipedia.org/wiki/Proprietary_software
[82] https://en.wikipedia.org/wiki/Microsoft_Office

- Third-party utilities do not have access to software features.

The goal of these decisions is to protect intellectual property.

Other software developers have followed the example of Microsoft. They stuck with the same philosophy of secrecy. As a result, their applications are self-contained and independent of each other. The formats of their data are encoded and undocumented.

If you are an experienced computer user, you immediately recognize a typical Windows application. It has a window with control elements[83] like buttons, input fields, tabs, etc. You manipulate some data using these control elements. Examples of data are text, image, sound record, video. When you are done, you save your results on the hard disk. You can open it again in the same application. If you write your own Windows program, it will look and work similarly. This succession of solutions is called the development culture.

## Linux

Linux has borrowed most of the ideas and solutions from the Unix[84]. Both OSes follow the set of standards that is called POSIX[85] (Portable Operating System Interface). POSIX defines interfaces between applications and the OS. Linux and Unix got the same design because they follow the same standard. We should have a look at the Unix origins to get this design.

The Unix appeared in the late 1960s. Two engineers from the Bell Labs company have developed it. Unix was a hobby project of Ken Thompson[86] and Dennis Ritchie[87]. In their daily work, they developed the **Multics**[88] OS. It was a joint project of the Massachusetts Institute of Technology (MIT), General Electric (GE) and Bell Labs. General Electric planned to use Multics for its new computer GE-645. Figure 1-11 demonstrates this machine.

[83] https://en.wikipedia.org/wiki/Graphical_widget
[84] https://en.wikipedia.org/wiki/Unix
[85] https://en.wikipedia.org/wiki/POSIX
[86] https://en.wikipedia.org/wiki/Ken_Thompson
[87] https://en.wikipedia.org/wiki/Dennis_Ritchie
[88] https://en.wikipedia.org/wiki/Multics



**Figure 1-11. Mainframe GE-645**

The Multics developers have invented several innovative solutions. One of them was time-sharing. It allows several users to work with the computer at the same time. Multics uses the multitasking concept to share resources among all users.

Because of many innovations and high requirements, Multics turned out to be too complicated. The project consumed more time and money than it was planned. This was a reason why Bell Labs left the project.

The Multics project was interesting from a technical point of view. Therefore, many Bell Labs engineers wanted to continue working on it. Ken Thompson was one of them. He decided to create his own operating system for the computer GE-645. Thompson started to write the system kernel and duplicated some Multics mechanisms. However, General Electric demanded the return of its GE-645 soon. Bell Labs has received this computer on loan only. As a result, Thompson lost a hardware platform for his development.

When working on the Multics analog, Thompson had a pet project to create a computer game. It was called Space Travel[89]. He launched the game on the past generation computer GE-635 from General Electric. It had the GECOS[90] OS. GE-635 consisted of several modules. Each module was a cabinet with electronics. The overall computer cost was about $7500000. Bell Labs engineers actively used this machine. Therefore, Thompson was rarely able to work with it to develop his game.

The limited access to the GE-635 machine was a problem. Therefore, Thompson decided to port his game to a relatively inexpensive and rarely used computer PDP-7[91] (see Figure 1-12). Its cost was about $72000. When doing that, Thompson met one problem. Space Travel used the features of the GECOS OS. The software of PDP-7 did not provide such features. Thompson was joined by his colleague Dennis Ritchie. They implemented GECOS features for PDP-7 together. It was a set of libraries and subsystems. Over time, these modules were developed into a self-sufficient OS. It was called Unix.

[89] https://en.wikipedia.org/wiki/Space_Travel_(video_game)
[90] https://en.wikipedia.org/wiki/General_Comprehensive_Operating_System
[91] https://en.wikipedia.org/wiki/PDP-7



**Figure 1-12. Minicomputer PDP-7**

Thompson and Ritchie were not going to sell Unix. Therefore, they never had a goal to protect their intellectual property. They developed Unix for their own needs. Afterward, they distributed it for free with the source code. Everyone could copy and use this OS. It was reasonable because the first Unix users were Bell Labs employees only.

Unix became popular among Bell Labs employees. Thompson and Ritchie presented the OS at the Symposium on Operating Systems Principles conference. Then they got a lot of requests for the system. However, Bell Labs belonged to AT&T company. Therefore, Bell Labs did not have the right to distribute any software on its own.

AT&T noticed the new perspective OS. The company started to sell its source code to universities for $20000. Thus, university circles got a chance to improve and develop Unix.

Linus Torvalds[92] met Unix when he had studied at the University of Helsinki. Unix encouraged him to create his own OS called Linux. It was not a pet project for fun. Torvalds met a practical problem. He needed a Unix-compatible OS for his PC to do university tasks at home. Such OS was not available at that moment.

At the University of Helsinki, students performed study assignments using the MicroVAX computer running Unix. Many of them had PCs at home. However, there was no Unix version for PC. The only Unix alternative for students was Minix[93] OS.

Andrew Tanenbaum developed this OS for IBM PCs with Intel 80268 processors in 1987. He created Minix for educational purposes only. This was a reason why he refused to apply changes to his OS for supporting modern PCs. Tanenbaum was afraid that his system becomes too complicated. Then he cannot use it for teaching students.

Torvalds had a goal to write a Unix-compatible OS for his new IBM computer with Intel 80386 processor. He took Minix as the prototype for his work. Like the Unix creators, Torvalds had no commercial interests and was not going to sell his software. He developed the OS for his own

[92] https://en.wikipedia.org/wiki/Linus_Torvalds
[93] https://en.wikipedia.org/wiki/MINIX

needs and wanted to share it with everyone. Linux OS became free in this way. Torvalds decided to distribute it with source code for free via the Internet. This decision made Linux known and popular.

Torvalds developed the kernel of OS only. The kernel provides memory management, file system, peripherals drivers and processor time scheduler. However, users needed an interface to access the kernel's features. It means that the Linux OS was not ready for use as it is.

The solution to the problem came from the GNU software project[94]. Richard Stallman[95] started this project at MIT in 1983. His idea was to develop the most necessary software for computers and make it free. The major products of the GNU project are the GCC compiler, glibc system library, system utilities and Bash shell. Torvalds included these products in his project and released the first Linux distribution[96] in 1991.

The first versions of Linux did not have a graphical interface. The only way to interact with the system was a command-line shell. Some complex applications had a text interface, but they were the minority. Linux got a GUI in the middle of the 1990s. This interface was based on X Window System[97] free software. X Window allowed developers to create graphical applications.

Unix and Linux evolved in very specific conditions. They differ from a typical cycle of commercial software development. These conditions made a unique development culture. Both systems developed in university circles. Computer science teachers and students used the OSes in daily work. They understood this software well. Therefore, they fixed software errors and added new features there willingly.

Let's have a look at what is the Unix development culture. Unix users prefer to use highly specialized command-line utilities. You can find a tool almost for each specific task. Such tools are well written, tested many times and worked as efficiently as possible. However, all features of one utility are focused on one specific task. The utility is not universal software to cover most of your needs.

When you meet a complex task, there is no single utility to solve it. However, you can easily combine several utilities and solve your task this way. Such an interaction becomes available thanks to a clear data format. Most Unix utilities use the text data[98] format. It is simple and self-explained. You can start working with it immediately.

The Linux development culture follows the Unix traditions. It differs from the standards that are adopted in Windows. Every application is monolithic and performs all its tasks by itself in the Windows world. The application does not rely on third-party utilities. The reason is the most software for Windows costs money and can be unavailable to the user. Therefore, each developer relies on himself. He cannot force the user to buy something extra to make the specific application working.

The software dependency looks different in Linux. Most of the utilities are free, interchangeable and accessible via the Internet. Therefore, it is natural that one program requires you to download and install a missing system component or another program.

[94] https://en.wikipedia.org/wiki/GNU_Project
[95] https://en.wikipedia.org/wiki/Richard_Stallman
[96] https://en.wikipedia.org/wiki/Linux_distribution
[97] https://en.wikipedia.org/wiki/X_Window_System
[98] https://en.wikipedia.org/wiki/Plain_text

The interaction of programs is crucial in Linux. Even monolithic graphical Linux applications usually provide a command-line interface. This way, they fit smoothly into the ecosystem. It leads that you can integrate them with other utilities and applications.

Suppose that you are solving a complex task in Linux. You should assemble a single computing process from a combination of highly specialized utilities. It means that you make a computation algorithm that can be complex by itself. Linux provides a tool for this specific task. The tool is called shell[99]. Using the shell, you type commands and the system performs them. The first Unix shell appeared in 1979. It was called Bourne shell[100]. Now it is deprecated. The Bash[101] shell has replaced it in most Linux distributions. We will consider Bash in this book.

We have considered Linux and Windows cultures. You cannot give a preference to one or another. Comparing them causes endless disputes on the Internet. Each culture has its advantages and disadvantages. For example, the Window-style monolithic applications cope well the tasks that require intensive calculations. When you combine specialized Linux utilities for the same task, you get an overhead. The overhead happens because of launching many utilities and transferring data between them. This requires extra time. As a result, you wait longer to complete your task.

Today, we observe a synthesis of Windows and Linux cultures. More and more commercial applications are being ported to Linux: browsers, development tools, games, messengers, etc. However, their developers are not ready for changes that the Linux culture dictates. Such changes require too much time and effort. They also make it more challenging to maintain the product. Instead of one application, there are two: each platform has a different version. Therefore, developers port their applications without significant changes. As a result, you find more and more Windows-style applications on Linux. One can argue about the pros and cons of this process. However, the more applications run on the specific OS, the more popular it becomes, thanks to the network effect.



Read more about the development culture in Unix and Linux in Eric Raymond's book "The Art of Programming in Unix"[102].

# Computer Program

We got acquainted with operating systems. They are responsible for starting and running computer programs. The program or application solves the user's specific task. For example, a text editor allows you to write and edit documents.

A program is a set of elementary steps. They are called instructions. The computer performs these steps sequentially. It follows the strict order of actions and copes with complex tasks. Let's consider how the computer launches and executes the program in detail.

[99] https://en.wikipedia.org/wiki/Unix_shell
[100] https://en.wikipedia.org/wiki/Bourne_shell
[101] https://en.wikipedia.org/wiki/Bash_(Unix_shell)
[102] https://en.wikipedia.org/wiki/Unix_philosophy#Eric_Raymond's_17_Unix_Rules

## Computer Memory

A hard disk stores all instructions of the program. If the program is relatively small and simple it fits a single file. Complex applications occupy several files.

Suppose that you have a single file program. When you launch it, the OS loads the file into the computer memory called RAM. Then the OS allocates a part of processor time for the new task. This way, the processor performs the program's instructions at specified intervals.

The first step of launching a program is to load it into RAM. We should consider the computer memory internals to understand this step better.

The single unit of the computer memory is **byte**[103]. The byte is the minimum amount of information that the processor can reference and load into its memory. However, the CPU can handle smaller amounts of data if you apply special techniques. You operate bits in this case. A **bit**[104] is the smallest amount of information you cannot divide. You can imagine the bit as a single logical state. It has one of two possible values. There are several ways to interpret them:

- 0 or 1
- True or False
- Yes or No
- + or —
- On or Off.

Another way to imagine one bit is to compare it with a lamp switch. It has two possible states:

- The switch closes the circuit. Then the lamp is on.
- The switch opens the circuit. Then the lamp is off.

A byte is a memory block of eight bits. Here you can ask why do we need this packaging? CPU can operate a single bit, right? Then it should be able to refer to a specific bit in memory.

CPU cannot refer to a single bit. There are technical reasons for that. The primary task of the first computers was arithmetic calculations. For example, these computers calculated the ballistic tables[105]. You should operate integers and fractional numbers to solve such a task. The computer does the same. However, a single bit is not enough to store a number in memory. Therefore, the computer needs memory blocks to store numbers. The bytes became such blocks.

Introducing bytes affected the architecture of processors. Engineers have expected that the CPU performs most operations over numbers. Therefore, they added a feature to load and process all bits of the number at once. This solution increased computers' performance by order of magnitude. At the same time, loading of the single bit in the CPU happens rarely. Supporting this feature in hardware brings significant overhead. Therefore, engineers have excluded it from modern processors.

[103] https://en.wikipedia.org/wiki/Byte
[104] https://en.wikipedia.org/wiki/Bit
[105] https://www.wikiwand.com/en/Ballistic_table

There is one more question. Why does a byte consist of eight bits? It was not always this way. The byte was equal to six bits[106] in the first computers. Such a memory block was enough to encode all the English alphabet characters in upper and lower case, numbers, punctuation marks and mathematical operations.

Six-bits encodings were insufficient for representing control and box-drawing characters. Therefore, these encodings were extended to seven bits in the early 1960s. The ASCII encoding[107] appeared at that moment. It became the standard for encoding characters. ASCII defines characters for codes from 0 to 127. The maximum seven-bit number 127 limits this range.

Then IBM released the computer IBM System/360[108] in 1964. The size of a byte was eight bits in this computer. IBM chose this size for supporting old character encodings from the past projects. The IBM System/360 computer was popular and widespread. It led that eight-bit packaging became the industry standard.

Table 1-1 shows frequently used units of information[109] besides bits and bytes.

**Table 1-1. Units of information**

| Title | Abbreviation[110] | Number of bytes | Number of bits |
|---|---|---|---|
| kilobyte | KB | 1000 | 8000 |
| megabyte | MB | 1000000 | 8000000 |
| gigabyte | GB | 1000000000 | 8000000000 |
| terabyte | TB | 10000000000 | 8000000000000 |

Table 1-2 shows standard storage devices and their capacity. You can compare them using Table 1-1.

**Table 1-2. Storage devices**

| Storage device | Capacity |
|---|---|
| Floppy disk 3.5"[111] | 1.44 MB |
| Compact disk[112] | 700 MB |
| DVD[113] | up to 17 GB |
| USB flash drive[114] | up to 2 TB |
| Hard disk drive[115] | up to 16 TB |
| Solid State Drive[116] | up to 100 TB |

We got acquainted with units of information. Now let's get back to the execution of the program. Why does the OS load it into RAM? In theory, the processor can read the program instructions

[106] https://en.wikipedia.org/wiki/Six-bit_character_code
[107] https://en.wikipedia.org/wiki/ASCII
[108] https://en.wikipedia.org/wiki/IBM_System/360
[109] https://en.wikipedia.org/wiki/Units_of_information
[111] https://en.wikipedia.org/wiki/Floppy_disk
[112] https://en.wikipedia.org/wiki/Compact_disc
[113] https://en.wikipedia.org/wiki/DVD
[114] https://en.wikipedia.org/wiki/USB_flash_drive
[115] https://en.wikipedia.org/wiki/Hard_disk_drive
[116] https://en.wikipedia.org/wiki/Solid-state_drive

directly from the hard disk drive, right?

A modern computer has four levels of the memory hierarchy[117]. Each level matches the red rectangle in Figure 1-13. Each rectangle match a separate device. The only exception is the CPU chip. It contains both registers and a memory cache. These are separate modules of the chip.

You see the arrows in Figure 1-13. They represent data flows. Data transfer occurs between adjacent memory levels.

Suppose that you want to process some data on the CPU. Then you should load these data to its registers. This is the only place where the processor can take data for calculations. If the CPU needs something from the disk drive, the following data transfers happen:

1. Disk drive -> RAM
2. RAM -> Processor cache
3. Processor cache -> Processor registers

When the CPU writes data back to the disk, it happens in the reverse order of steps.

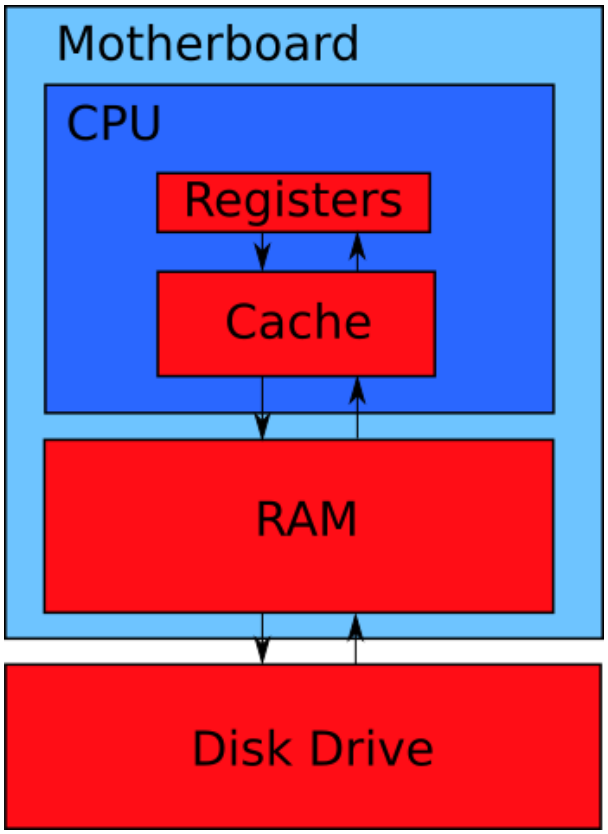



**Figure 1-13. Memory hierarchy**

Data storage devices have the following parameters:

[117] https://en.wikipedia.org/wiki/Memory_hierarchy

1. **Access speed** defines the amount of data that you can read or write to the device per unit of time. Units of measure are bytes per second (KBps).
2. **Capacity** is the maximum amount of data that a device can store. The units are bytes.
3. **Cost** is a price of a device concerning its capacity. The units are dollars or cents per byte or bit.
4. **Access time** is the time between the moment when the process needs some data from the device and when it receives them. Units are **clock signals**[118] of the CPU.

These parameters vary for devices on each level of the memory hierarchy. Table 1-3 shows the ratio of parameters for modern storage devices.

**Table 1-3. Memory levels**

| Level | Memory | Capacity | Access speed | Access time | Cost |
|---|---|---|---|---|---|
| 1 | **CPU registers**[119] | up to 1000 bytes | - | 1 tick | - |
| 2 | **CPU cache**[120] | from one KB to several MB | from 700 to 100 GBps | from 2 to 100 cycles | - |
| 3 | RAM | dozens of GB | 10 GBps | up to 1000 clock cycles | $10-$^{9}$/byte |
| 4 | Disk drive (**hard drive**[121] or **solid drive**[122]) | several TB | 2000 Mbps | up to 10000000 cycles | $10-$^{12}$/byte |

Table 1-3 raises questions. You can read the data from the disk drive at high speed. Why is there no way to read these data to the CPU registers directly? The high access speed is not so crucial for performance in practice. The critical parameter here is the access time. It measures the idle time of the CPU until it receives the required data. You can measure this idle time in clock signals or cycles. Such a signal synchronizes all operations of the processor. The CPU requires roughly from 1 to 10 clock cycles to execute one instruction of the program.

High access time can cause serious performance issues. For example, suppose that the CPU reads the program instructions directly from the hard disk. The problem happens because CPU registers have a small capacity. There is no chance to load the whole program from the hard disk to the registers. Therefore, when the CPU did one part of the program, it should load the next one. This loading operation takes up to 10000000 clock cycles. It means that loading data from the disk takes a much longer time than processing them. The CPU spends most of the time idling. The memory hierarchy solves exactly this problem.

Let's consider data flow between memory levels by example. Suppose that you launch a simple program. It reads a file from the hard disk and displays its contents on the screen. Reading data

[118] https://en.wikipedia.org/wiki/Clock_signal
[119] https://en.wikipedia.org/wiki/Processor_register
[120] https://en.wikipedia.org/wiki/CPU_cache
[121] https://en.wikipedia.org/wiki/Hard_disk_drive
[122] https://en.wikipedia.org/wiki/Solid-state_drive

from the disk happens in several steps. The hardware does them.

The first step is reading data from the hard disk into the RAM according to Figure 1-13. The next step is loading data from RAM to the CPU cache. There is a sophisticated **caching mechanism**. It guesses the data from RAM that the CPU requires next. This mechanism reduces the access time to the data and decreases the idle time of the CPU.

When data comes to the CPU chip, it manages them on its own. The processor reads the required data from the cache to registers and manipulates them. The program instructions reach the CPU the same way as the data.

The program displays data on the screen in our example. It should call the corresponding API function for that. Then the system library changes the screen picture. The CPU does the actual work here. It loads the instructions of the system library and the video card driver. Then the CPU applies these instructions to the data in its registers. This way, the video card driver displays the data on the screen.

The required data may absent in the specific memory level. Here are few examples. Suppose that the CPU needs data to process them in the video driver code. If these data are in the CPU cache but not in the registers, the processor waits for 2-100 clock cycles to get them. If data are in the RAM, the CPU's waiting time increases by order of magnitude up to 1000 cycles.

Our program can display both small and big files. Some big file does not fit the RAM. Then the RAM contains only part of it. The CPU can require the missing file part. In this case, the CPU idle time increases by four orders of magnitude up to 10000000 clock cycles. For comparison, the processor could execute about 1000000 program instructions instead of this idle time. This is really a lot.

Both CPU and disk drives use hardware caching mechanisms. The idea of caching for disk drives[123] is to store some data in the small and fast access memory. It speeds up reading and writing blocks of data. There are caching mechanisms on the software level too. They are parts of the OS in most cases.

All caching mechanisms increase a computer's performance significantly. When such a mechanism makes a mistake, it leads to the CPU idle. This mistake is called **cache miss**. Any cache miss is expensive from the performance point of view. Therefore, remember the memory hierarchy and caching mechanisms. Consider them when developing algorithms. Some algorithms and data structures cause more cache misses than others.

The storage devices with lower access times are placed closer to the CPU. Figure 1-14 demonstrates this principle. For example, registers and cache is the internal CPU memory. They are part of the processor's chip.

Both CPU and RAM are two chips that are plugged into the **motherboard**[124] near each other. The high-frequency **data bus**[125] connects them. This data bus provides low access time.

[123] https://en.wikipedia.org/wiki/Disk_buffer
[124] https://en.wikipedia.org/wiki/Motherboard
[125] https://en.wikipedia.org/wiki/Bus_(computing)

The motherboard is the printed circuit board[126] that connects computer components. You can plug in some chips right into the motherboard. However, there are devices that you should connect via cables there. The disk drive is an example of such a device. It connects to the motherboard via a relatively slow interface. There are several standards for such an interface: ATA, SATA, SCSI, PCI Express.

The old motherboard models have an embed chip that transfers data between CPU and RAM. This chip is called **northbridge**[127]. Thanks to improving technologies for chip manufacturing, the CPUs take northbridge's functions since 2011.

The **southbridge**[128] is another motherboard's chip. It presents there nowadays. The southbridge transfers data between RAM and devices that are connected via slow interfaces like PCI, USB, SATA, etc.

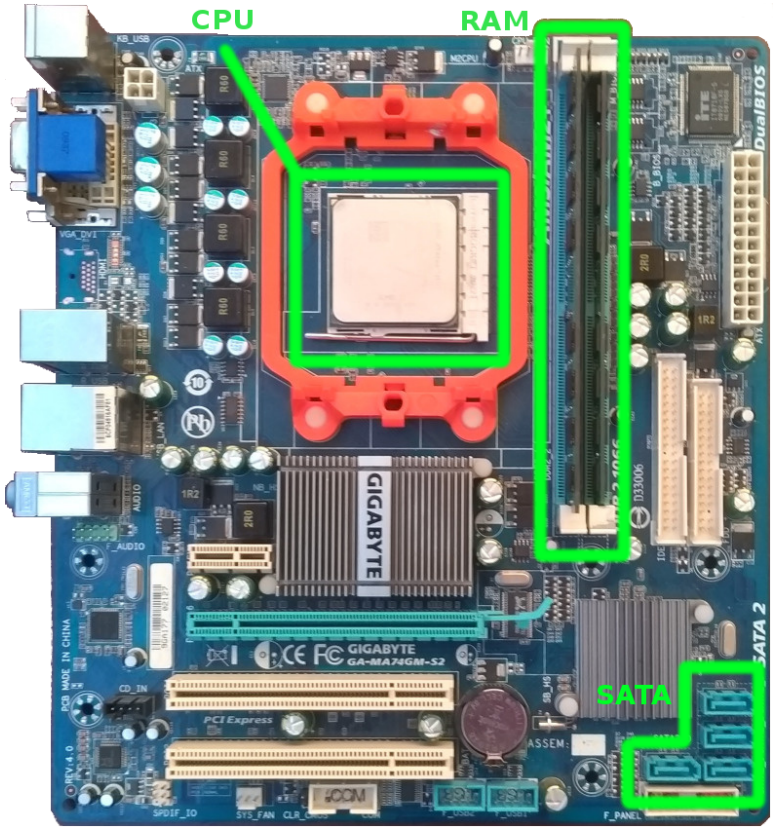



**Figure 1-14. PC motherboard**

## Machine code

Suppose that the OS has loaded the contents of an executable file into RAM. This file contains both instructions and data of the program. Examples of data are text strings, box-drawing characters,

[126] https://en.wikipedia.org/wiki/Printed_circuit_board
[127] https://en.wikipedia.org/wiki/Northbridge_(computing)
[128] https://en.wikipedia.org/wiki/Southbridge_(computing)

predefined constants, etc.

Program instructions are called **machine code**[129]. The processor executes them one by one. A single instruction is an elementary operation on the data from the CPU registers.

The CPU has logical blocks for executing each type of instruction. The available blocks determine the operations that the CPU can perform. If the processor does not have an appropriate logical block to accomplish a specific instruction, it combines several blocks to do this job. The execution takes more clock cycles in this case.

When the OS has loaded the program instructions and its data into RAM, it allocates the CPU time slots for that. The program becomes a **computing process**[130] or process since this moment. The process means the running program and the resources it uses. Examples of the resources are memory area and OS objects.

How do the program instructions look like? We can see them using the special program for reading and editing executable files. Such a program is called **hex editor**[131]. The editor represents the program's machine code in **hexadecimal numeral system**[132]. The actual contents of the executable file is **binary code**[133]. This code is a sequence of zeros and ones. The CPU receives program instructions and data in this format. The hex editor makes them easy to read for humans.

There are advanced programs to read machine code. They are called **disassemblers**[134]. These programs guess how the program instructions look like in terms of the CPU commands. You can get a better representation of the program using the disassembler than the hex editor.

If you take a specific number, it looks different in various numeral systems. The numeral system determines the symbols and their order to write a number. For example, binary allows 0 and 1 symbols only.

Table 1-4 shows matching of numbers in binary (BIN), decimal (DEC), and hexadecimal (HEX).

**Table 1-4. Numbers in decimal, hexadecimal and binary**

| Decimal | Hexadecimal | Binary |
|---|---|---|
| 0 | 0 | 0000 |
| 1 | 1 | 0001 |
| 2 | 2 | 0010 |
| 3 | 3 | 0011 |
| 4 | 4 | 0100 |
| 5 | 5 | 0101 |
| 6 | 6 | 0110 |
| 7 | 7 | 0111 |
| 8 | 8 | 1000 |

[129] https://en.wikipedia.org/wiki/Machine_code
[130] https://en.wikipedia.org/wiki/Process_(computing)
[131] https://en.wikipedia.org/wiki/Hex_editor
[132] https://en.wikipedia.org/wiki/Hexadecimal
[133] https://en.wikipedia.org/wiki/Binary_code
[134] https://en.wikipedia.org/wiki/Disassembler

**Table 1-4. Numbers in decimal, hexadecimal and binary**

| Decimal | Hexadecimal | Binary |
|---|---|---|
| 9 | 9 | 1001 |
| 10 | A | 1010 |
| 11 | B | 1011 |
| 12 | C | 1100 |
| 13 | D | 1101 |
| 14 | E | 1110 |
| 15 | F | 1111 |



Most calculator applications can convert numeral systems. If you use Windows, convert numbers with the standard Windows calculator in the **programmer mode**[135].

Why do programmers use both binary and hexadecimal numeral systems? It is more convenient to use only one of them, right? We should know more about computer hardware to answer this question.

The binary numeral system and **Boolean algebra**[136] are the basis of **digital electronics**[137]. An electrical **signal**[138] is the smallest portion of the information in digital electronics. When you work with such signals, you need a way to encode them. Encoding means associating specific numbers with the signal states. The signal has two states only. It can present or absent. Therefore, the simplest way to encode the signal is to take the first two integers: zero and one. Then you use zero when there is no signal. Otherwise, you use the number one. Such encoding is very compact. You can use one bit to encode one signal.

The basic element of digital electronics is a **logic gate**[139]. It converts electrical signals. You can implement the logic gate using various physical devices. Examples of such devices are an electromagnetic relay, vacuum tube and transistor. Each device has its own physics in the background. However, they work in the same way in terms of signal processing. This processing contains two steps:

1. Receive one or more signals on the input.
2. Transmit the resulting signal to the output.

The signal processing follows the rules of Boolean algebra. This algebra has an elementary operation for each type of logic gate. When you combine logic gates together, you can calculate the result using a Boolean expression. Combining logic gates provides a complex behavior. You need it if your digital device follows some non-trivial logic. For example, the CPU is a huge network of logic gates.

When you deal with digital electronics, you should apply the binary numeral system. This way, you convert signal states to logical values that Boolean algebra operates. Then you can calculate the

[135] https://en.wikipedia.org/wiki/Windows_Calculator#Features
[136] https://en.wikipedia.org/wiki/Boolean_algebra
[137] https://en.wikipedia.org/wiki/Digital_electronics
[138] https://en.wikipedia.org/wiki/Signal#Digital_signal
[139] https://en.wikipedia.org/wiki/Logic_gate

resulting signals. The level of signals and logic gates is close to the computer hardware. You have to work on this level sometimes when programming. We can say that the hardware design dictates you to use the binary numeral system.

The binary numeral system is a language of hardware. Why do you need the hexadecimal system then? When you develop a program, you need decimal and binary systems only. Decimal is convenient for writing a general logic of the program. For example, you count repeating the same action in decimal.

You need the binary system when your program deals with computer hardware. For example, you send data to some device in binary format. There is one problem with binary. It is hard to write, read, memorize and pronounce the numbers in this system for humans. Conversion from decimal to binary takes effort. The hexadecimal system solves both problems of representing and converting numbers. It is a compact and convenient as the decimal system. At the same time, you can convert a number from hexadecimal to binary form easily.

Follow these steps to convert a number from binary to hexadecimal form:

1. Split the number into groups of four digits. Start the splitting from the end of the number.
2. If the last group is less than four digits, add zeros to the left side of the group.
3. Use Table 1-4 to replace each four-digit group with one hexadecimal number.

Here is an example of converting the binary number 110010011010111:

```
1  110010011010111 = 110 0100 1101 0111 = 0110 0100 1101 0111 = 6 4 D 7 = 64D7
```

**Exercise 1-1. Numbers conversion from binary to hexadecimal**

```
Convert the following numbers from binary to hexadecimal:
* 10100110100110
* 1011000111010100010011
* 11111011100010010101001100000000110101101
```

**Exercise 1-2. Numbers conversion from hexadecimal to binary**

```
Convert the following numbers from hexadecimal to binary:
* FF00AB02
* 7854AC1
* 1E5340ACB38
```

There are the answers for all exercises in the last section of the book. Check yourself there if you are unsure about your results.

Let's come back to executing the program. The OS loads its executable file from the disk drive into RAM. Then the OS loads all libraries that the program requires. The special OS component does

both these tasks. It is called **loader**[140]. Thanks to preloading libraries, the CPU does not idle too much when the program accesses them. The instructions of the required library are already in RAM. Therefore, the CPU waits for a few hundred clock cycles to access them. When the loader finishes his job, the program becomes a process. The CPU executes it, starting from the first instruction.

Each machine code instruction is called **opcode**[141]. The opcode dictates the CPU which logic gates it should apply for data in the specific registers. When the operation is done, the opcode specifies the register for storing the result. Opcodes have a binary format that is a natural language of the CPU.

While the program is running, its instructions, resources and required libraries occupy the RAM area. The OS clears this memory area when the program finishes. Then other applications can use it.

## Source Code

Machine code is a low-level representation of a program. This format is convenient for the processor. However, it is hard for a human to write a program in machine code. Software engineers developed the first programs this way. It was possible for early computers because of their simplicity. Modern computers are much more powerful and complex devices. Their programs are huge and have a lot of functions.

Computer engineers invented two types of special applications. They solve the problem of the machine code complexity. These applications are **compilers**[142] and **interpreters**[143]. They translate the program from a human-readable language into machine code. Compilers and interpreters solve this task differently.

Software developers use **programming languages**[144] in their work nowadays. Compilers and interpreters take programs written in such languages and produce the corresponding machine code.

Humans use one of **natural language**[145] to communicate with each other. Programming languages are different from them. They are formal and very limited. Using a programming language, you can express only actions that a computer can perform. There are strict rules on how you should write these actions. For example, you can use a small set of words and combine them in specific orders. **Source code**[146] is a text of the program you write in a programming language.

The compiler and interpreter process source code differently. The compiler reads the entire program text, generates machine code instructions and saves them on a disk drive. The compiler does not execute the resulting program on its own. The interpreter reads the source code in parts, generates machine code instructions and executes them immediately. The interpreter stores its results in RAM temporarily. When the program finishes, you lose these results.

[140] https://en.wikipedia.org/wiki/Loader_(computing)
[141] https://en.wikipedia.org/wiki/Opcode
[142] https://en.wikipedia.org/wiki/Compiler
[143] https://en.wikipedia.org/wiki/Interpreter_(computing)
[144] https://en.wikipedia.org/wiki/Programming_language
[145] https://en.wikipedia.org/wiki/Natural_language
[146] https://en.wikipedia.org/wiki/Source_code

Let's consider how the compiler works step by step. Suppose that you have written the program. You saved its source code to a file on the hard disk. Then you run a compiler that fits the language you have used. Each programming language has the corresponding compiler or interpreter. The compiler reads your file, processes it and writes the resulting machine code in the executable file on a disk. Now you have two files: one with source code and one with machine code. Every time you change the source code file, you should generate the new executable file. You can run the executable file to launch your program.

Figure 1-15 shows how the compiler processes a program written in C and C++ languages.

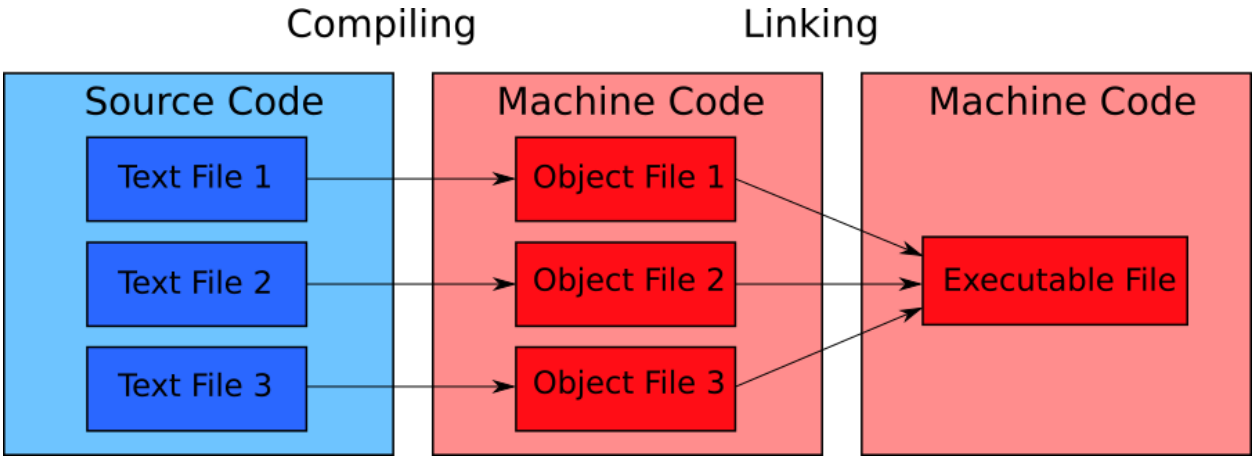



**Figure 1-15. The compilation process**

The compilation process consists of two steps. The compiler does the first step. The second step is called **linking**. The special program called **linker**[147] performs it.

The compiler produces intermediate **object files**. The linker takes them and converts them to one executable file.

Why do you need two steps for compiling the source code? In theory, you can combine the compiler and linker into a single application. However, such a solution has several problems.

The limited RAM size causes the first problem. There is a common practice to split source code into several files. Each file matches a separate part of the program that solves the specific task. This way, you simplify your work with the source code. The compiler processes these files separately. It produces an object file for each source code file. They store the intermediate results of compilation. If you combine the compiler and linker into one application, there is no chance to save the intermediate results to the disk. It means you should compile the whole program at once. If you deal with a big program, the compilation process consumes all your RAM and crashes.

The second problem of the compiler-linker application is resolving **dependencies**. There are blocks of commands that call each other in the source code. Such references are called dependencies. Tracking them is the linker task. However, if you combine compiler and linker, you need extra passes through the whole program source code for resolving dependencies. The compiler needs much more

[147] https://en.wikipedia.org/wiki/Linker_(computing)

time for a single pass over the source code than the linker needs it for object files. Therefore, when you have the compiler and linker separated, you speed up the overall compilation process.

The program can call blocks of commands from the library. The linker process the library file together with the object files of your program in this case. The compiler cannot process the library. Its file contains machine code but not the source code. Therefore, the compiler does not understand it. Splitting the compilation into two steps resolves the task of using libraries too.

We have considered the basics of how the compiler works. Now suppose that you choose an interpreter instead to execute your program. You have the file with its source code on the disk drive. The file is ready for execution. When you run it, the OS loads the interpreter first. Then the interpreter reads your source code file into RAM and executes it line by line. The translation of source code commands to machine code instructions happens in RAM. Some interpreters save files with an intermediate representation of the program to the disk drive. It speeds up the program execution if you restart it. However, you always need to run an interpreter first for executing your program.

Figure 1-16 shows the process of interpreting the program.

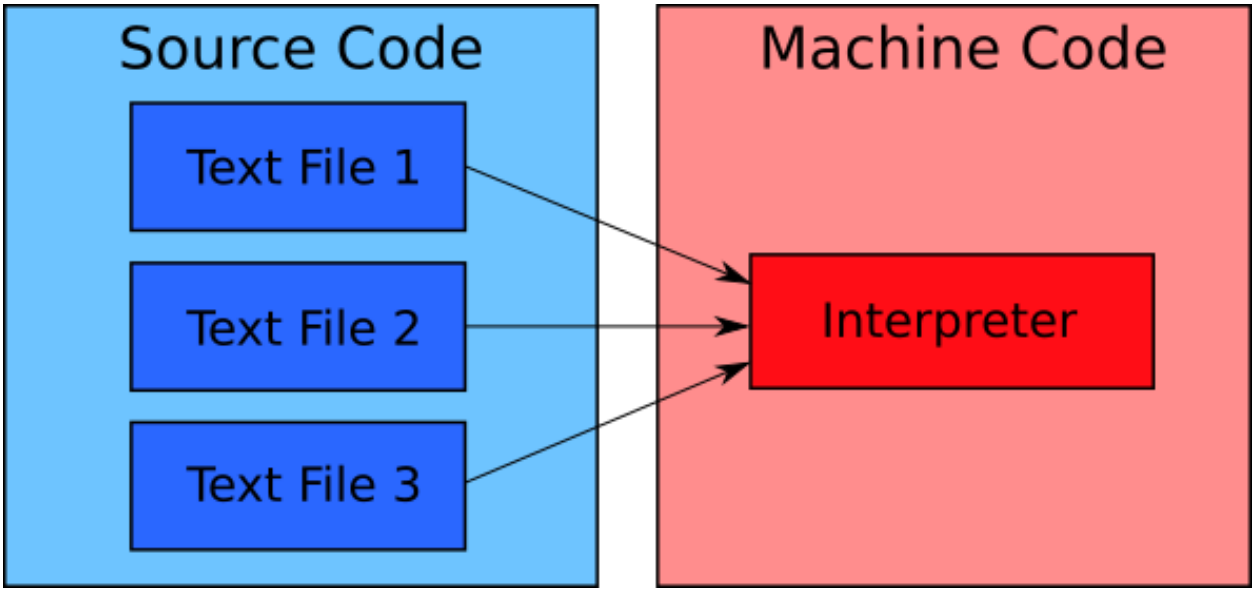



**Figure 1-16. Interpreting the program**

Figure 1-16 can give an idea that the interpreter works the same way as the compiler and linker combined into one application. The interpreter loads source code files into RAM and translates them into machine code. Why are there no problems with the RAM overflow and dependency resolution?

The interpreter avoids problems because it processes the source code differently than the compiler does. The interpreter processes and executes the program code line by line. Therefore, it does not store the machine code of the whole program in memory. The interpreter processes the parts of the source code file that it requires at the moment. When the interpreter executes them, it unloads these parts and frees the corresponding RAM area.

Interpreting the program looks more convenient for software development than compiling. However, it has some drawbacks.

First, all interpreters work slowly. It happens because every time you run the program, the interpreter should translate its source code to machine code. This process takes some time. You should wait for it. Another reason for the low performance of interpreters is disk operations. Loading the program's source code from the disk drive into RAM causes the CPU to idle. It takes up to 10000000 clock cycles, according to Table 1-3.

Second, the interpreter itself is a complex program. It requires some portion of the computer's hardware resources to launch and work. Therefore, the computer executes both interpreter and your program in parallel and shares resources among them. It is an extra overhead that reduces the performance of your program.

Interpreting the program is slow. Does it mean that compilation is better? The compiler generates an executable file with machine code. Therefore, you reach almost the program's performance when you compile it or write machine code on your own. However, you pay for using the programming language at the compilation stage. A couple of seconds and a few megabytes of RAM are enough to compile a small program. When you compile a large project (for example, the Linux kernel), it takes several hours. If you change the source code, you should recompile the project and wait hours again.

Keep in mind the overhead of interpreters and compilers when choosing a programming language for your project. The interpreter is a good choice in the following cases:

- You want to develop a program quickly.
- You do not care about the program's performance.
- You work on a small and relatively simple project.

The compiler would be better in the following cases:

- You work on a complex and large project.
- Your program should work as fast as possible.
- You want to speed up debugging of your program.

Both compilers and interpreters have an overhead. Does it make sense to discard a programming language and write a program in machine code? You do not waste your time waiting for compilation in this case. Your program works as fast as possible. These benefits sound reasonable. Please do not hurry with your conclusions.

One simple example helps you to realize all advantages of using a programming language. Listing 1-1 shows the source code written in C. This program prints the "Hello world!" text on the screen.

**Listing 1-1. Source code of the C program**

```c
#include <stdio.h>

int main(void)
{
    printf("Hello world!\n");
}
```

Listing 1-2 shows the machine code of this program in the hexadecimal format.

**Listing 1-2. Machine code of the program**

```
BB 11 01 B9 0D 00 B4 0E 8A 07 43 CD 10 E2 F9
CD 20 48 65 6C 6C 6F 2C 20 57 6F 72 6C 64 21
```

Even if you don't know the C language, you would prefer to deal with the code in Listing 1-1. You can read and edit it easily. At the same time, you need significant efforts to decode the numbers in Listing 1-2.

Perhaps a professional programmer with huge experience can write a small program in machine codes. However, another programmer will spend a lot of time and effort to figure it out. Developing a large project in machine codes is a challenging and time-consuming task for any developer.

Using programming language saves your effort and time significantly when developing programs. Also, it reduces the cost of maintaining the existing project. There is no way to develop modern complex software using the machine code only.

# Bash Shell

Programming is an applied skill. If you want to learn it, you should choose a programming language and solve tasks. This is the only way to get practical skills.

We use the Bash language in this book. This language is convenient for automating computer administration tasks. Here are few examples of what you can do with Bash:

- Create data backups.
- Manipulate directories[148] and files.
- Run programs and transfer data between them.

Bash was developed in the Unix environment. Therefore, it bears the imprint of the Unix philosophy[149]. Despite its roots, you can also use Bash on Windows and macOS.

## Development Tools

You need a Bash interpreter and a terminal emulator to run the examples of this chapter. You can install them on all modern operating systems. Let's take a look at how to do this.

### Bash Interpreter

Bash is a **script programming language**[150]. It has the following features:

1. It is interpreted language.
2. It operates existing programs or high-level commands.
3. You can use it as a shell to access the OS functions.

If you use Linux or macOS, you have the preinstalled Bash interpreter. If your OS is Windows, you need both Bash interpreter and POSIX-compatible environment. Bash needs this environment to work correctly. There are two ways to install it.



You can meet the "Unix environment" and "Linux environment" terms. They both mean a software environment that is compatible with POSIX standards.

[148] https://en.wikipedia.org/wiki/Directory_(computing)
[149] https://en.wikipedia.org/wiki/Unix_philosophy
[150] https://en.wikipedia.org/wiki/Scripting_language

The first option is to install the **MinGW**[151] toolkit. It contains the GNU compiler collection[152] in addition to Bash. If you do not need all MinGW features, you can install Minimal SYStem (MSYS) instead. MSYS is the MinGW component that includes Bash, a terminal emulator and GNU utilities[153]. These three things make up a minimal Unix environment.

It is always good to clarify the bitness of your Windows before installing any software. Here are steps to read it:

1. If you have a "Computer" icon on your desktop, right-click on it and select the "Properties" item.
2. If there is no "Computer" icon on your desktop, click the "Start" button. Find the "Computer" item in the menu. Right-click on it and select "Properties".
3. You have opened the "System" window. Locate the "System Type" item there as Figure 2-1 demonstrates. This item shows you the bitness of Windows.

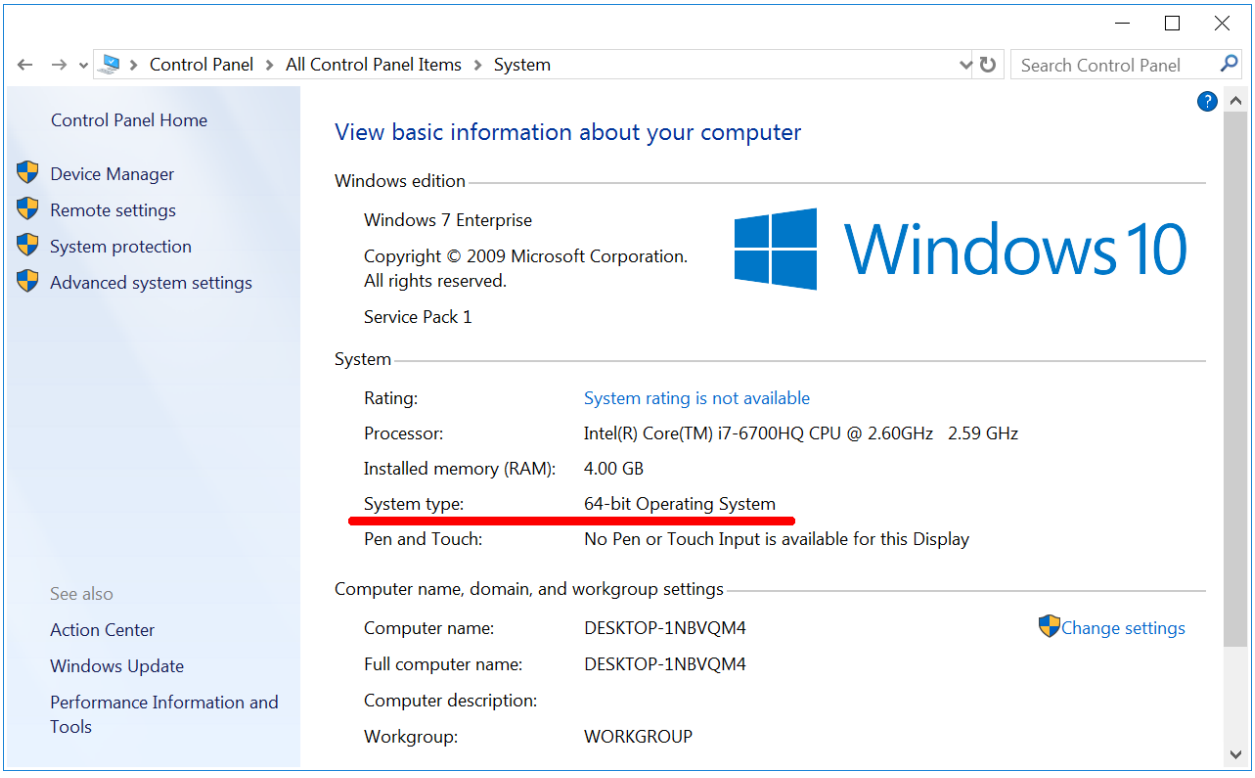



**Figure 2-1. System Type**

We are going to install the modern MSYS version called MSYS2. Download its installer from the official website[154]. You should choose the installer version that fits the bitness of your Windows.

Now we have everything to install MSYS2. Follow these steps for doing it:

[151] https://en.wikipedia.org/wiki/MinGW
[152] https://en.wikipedia.org/wiki/GNU_Compiler_Collection
[153] https://en.wikipedia.org/wiki/GNU_Core_Utilities
[154] https://www.msys2.org

1. Run the installer file. You will see the window as Figure 2-2 shows.

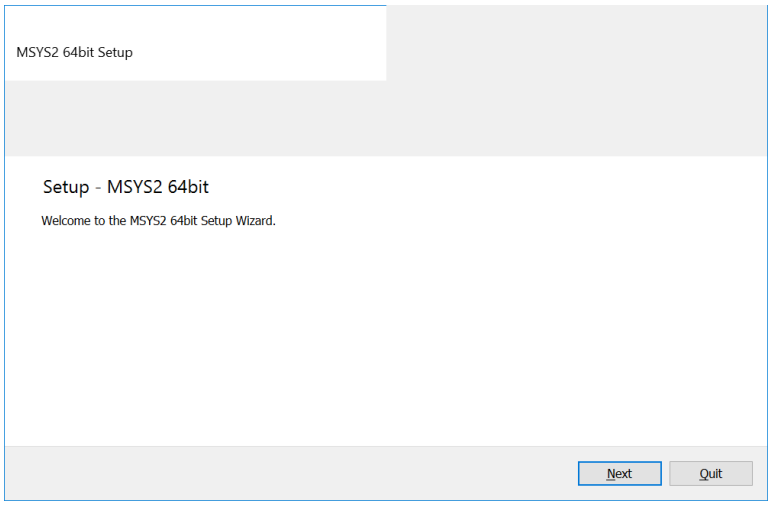



**Figure 2-2. MSYS2 installation dialog**

2. Click the "Next" button.
3. You see the new window as Figure 2-3 shows. Specify the installation **path**[155] there and press the "Next" button.

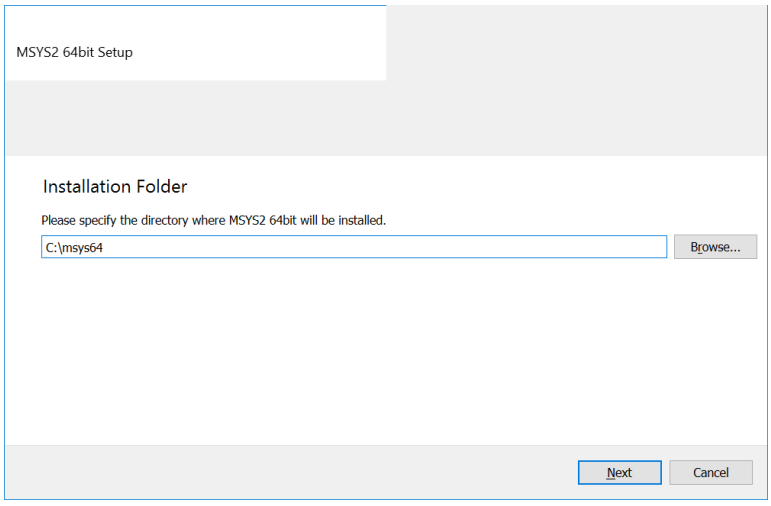



**Figure 2-3. Selecting the installation path**

4. The next window suggests you to choose the application name for the Windows "Start" menu. Leave it unchanged and click the "Next" button. Then the installation process starts.

4. When the installation finishes, click the "Finish" button. It closes the installer window.

You have installed the MSYS2 Unix environment on your hard drive. You can find its files in the `C:\msys64` directory if you did not change the default installation path. Go to this directory and run

[155] https://en.wikipedia.org/wiki/Path_(computing)

the `msys2.exe` file. It opens the window where you can work with the Bash shell. Figure 2-4 shows this window.

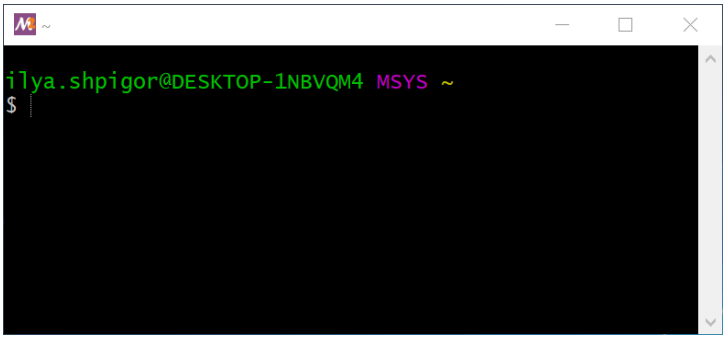



**Figure 2-4. The Bash shell**

The second option is to install a Unix environment from Microsoft. It is called Windows subsystem for Linux[156] (WSL). This environment is available for Windows 10 only. It does not work on Windows 8 and 7. You can find the manual to install WSL on the Microsoft website[157].

If you use Linux, you do not need to install Bash. You already have it. Just press the shortcut key Ctrl+Alt+T to open a window with the Bash shell.

If you use macOS, you have everything to launch Bash too. Here are the steps for doing that:

1. Click the magnifying glass icon in the upper right corner of the screen. It opens the Spotlight search program.
2. The dialog box appears. Enter the text "Terminal" there.
3. Spotlight shows you a list of applications. Click on the first line in the list with the "Terminal" text.

## Terminal emulator

Bash shell is not a regular GUI application. It even does not have its own window. When you run the `msys2.exe` file, it opens a window of the terminal emulator program.

An **emulator**[158] is a program that simulates the behavior of another program, OS or device. The emulator solves the compatibility task. For example, you want to run a Windows program on Linux. There are several ways to do that. One option is using the emulator of the Windows environment for Linux. It is called Wine[159]. Wine provides its own version of the Windows system libraries. When you run your program, it uses these libraries and supposes that it works on Windows.

The terminal emulator solves the compatibility task too. Command-line programs are designed to work through a terminal device. Nobody uses such devices today. Cheap personal computers and

[156] https://en.wikipedia.org/wiki/Windows_Subsystem_for_Linux
[157] https://docs.microsoft.com/en-us/windows/wsl/install-win10
[158] https://en.wikipedia.org/wiki/Emulator
[159] https://en.wikipedia.org/wiki/Wine_(software)

laptops have replaced them. However, there are still many programs that require a terminal for working. You can run them using the terminal emulator. It uses the shell to pass data to the program. When the program returns some result, the shell receives them and passes to the terminal emulator. Then the emulator displays the results on the screen.

Figure 2-5 explains the interaction between input/output devices, the terminal emulator, the shell and the command-line program.

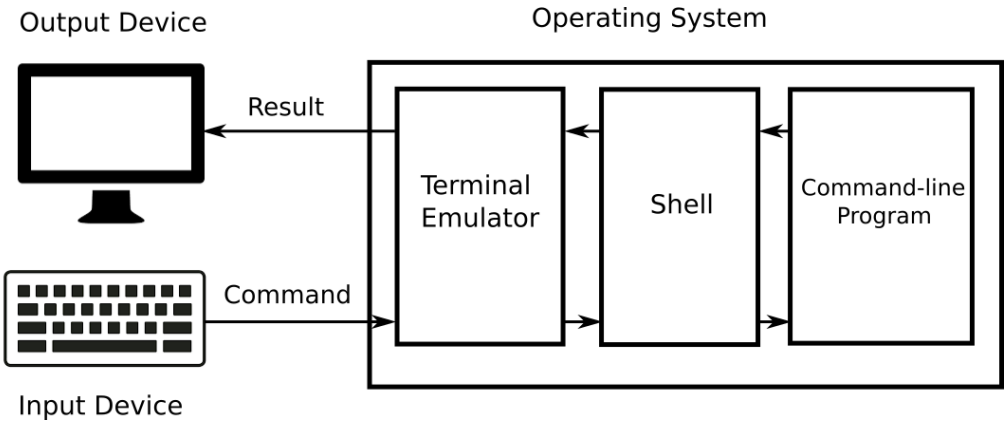



**Figure 2-5. The workflow of the terminal emulator**

The terminal emulator window in Figure 2-4 shows the following two lines:

```
ilya.shpigor@DESKTOP-1NBVQM4 MSYS ~
$
```

The first line starts with the username. The username is `ilya.shpigor` in my case. Then there is the @ symbol and the computer name `DESKTOP-1NBVQM4`. You can change the computer name via Windows settings. The word `MSYS` comes next. It means the platform where Bash is running. At the end of the line, there is the symbol ~. It is the path to the current directory.

# Command Interpreter

All interpreters have two working modes: non-interactive and interactive. When you choose the non-interactive mode, the interpreter loads the source code file from the disk and executes it. You do not need to type any commands or control the interpreter. It does everything on its own.

When you choose the interactive mode, you type each command to the interpreter manually. If the interpreter is integrated with the OS and works in the interactive mode, it is called a **command shell**[160] or shell.

A command shell provides access to the settings and functions of the OS. You can perform the following tasks using it:

[160] https://en.wikipedia.org/wiki/Shell_(computing)

- Run programs and system services.
- Manage the file system.
- Control peripherals and internal devices.
- Access the kernel features.

## Demand of the CLI

Why would somebody learn the command-line interface (CLI) today? It appeared 40 years ago for computers that are thousands of times slower than today. Then the graphical interface supplanted CLI on PCs and laptops. Everybody prefers to use GUI nowadays.

The CLI seems to be outdated technology. However, this statement is wrong. It should be a reason why developers include Bash in all modern macOS and Linux distributions[161]. Windows also has a command shell called Cmd.exe[162]. Microsoft has replaced it with PowerShell[163] in 2006. Just think about this fact. The developer of the most popular desktop OS has created a new command shell in the 2000s. All these points confirm that CLI is still in demand.

What tasks does the shell perform in modern OSes? First of all, it is a tool for system administration[164]. The OS consist of the kernel and software modules. These modules are libraries, services and system utilities. Most of the modules have settings and special modes of operation. You do not need them in your daily work. Therefore, you cannot access these settings via GUI in most cases.



The Bash shell has good integration with Linux and macOS. You can access most OS functions using Bash there. However, it does not work with Windows smoothly. Microsoft offers you their own PowerShell. We study Bash in this book for two reasons. First, it is compatible with the POSIX standard. Second, Bash is more common than PowerShell. You can use it on all modern OSes.

If the OS fails, you need system utilities to recover it. They have a command-line interface because a GUI often is not available after the failure.

Besides the administration tasks, you would need CLI when connecting computers over a network[165]. There are GUI programs for such connection. The examples are TeamViewer and Remote Desktop. They require a stable and fast network connection for working well. If the connection is not reliable, these programs are slow and fail often. The command interface does not have such a limitation. The remote server receives your command even if the link is poor.

You can say that a regular user does not deal with administration tasks and network connections. Even if you do not have such tasks, using command shell speeds up your daily work with the computer. Here are few things that you can do more effective with CLI than with GUI:

[161] https://en.wikipedia.org/wiki/Linux_distribution
[162] https://en.wikipedia.org/wiki/Cmd.exe
[163] https://en.wikipedia.org/wiki/PowerShell
[164] https://en.wikipedia.org/wiki/System_administrator
[165] https://en.wikipedia.org/wiki/Remote_desktop_software

- Operations on files and directories.
- Creating data backups.
- Downloading files from the Internet.
- Collecting statistics about your computer's resource usage.

An example will explain the effectiveness of CLI. Suppose you rename several files on the disk. You add the same suffix to their names. If you have a dozen of files, you can do this task with Windows Explorer[166] in a couple of minutes. Now imagine that you should rename thousands of files this way. You will spend the whole day doing that with the Explorer. If you use the shell, you need to launch a single command and wait for several seconds. It will rename all your files automatically.

The example with renaming files shows the strength of the CLI that is scalability. Scalability means that the same solution handles well both small and large amounts of input data. The solution implies a command when we are talking about the shell. The command renames ten and a thousand files with the same speed.

Experience with the command interface is a great benefit for any programmer. When you develop a complex project, you manage plenty of text files with the source code. You use the GUI editor to change the single file. It works well until you do not need to introduce the same change in many files. For example, it can be the new version of the license information in the file headers. You waste your time when solving such a task with the editor. Command-line utilities make this change much faster.

You need to understand the CLI to deal with compilers and interpreters. These programs usually do not have a graphical interface. You should run them via the command line and pass the names of the source code files. The reason for such workflow is the poor scalability of the GUI. If you have plenty of source code files, you cannot handle them effectively via the GUI.

There are special programs to edit and compile source code. Such programs are called **integrated development environments**[167] (IDE). You can compile a big project using IDE and its GUI. However, IDE calls the compiler via the command line internally. Therefore, you should deal with the compiler's CLI if you want to change its options or working mode.

If you are an experienced programmer, knowing the CLI encourages you to develop helper utilities. It happens because writing a program with a command interface is much faster than with a GUI. The speed of development is essential when solving one-off tasks.

Here is an example situation when you would need to write a helper utility. Suppose that you have to make a massive change in the source code of your project. You can do it with IDE by repeating the same action many times. Another option is to spend time writing a utility that will do this job. You should compare the required time for both ways of solving your task. If you are going to write a GUI helper utility, it takes more time than for a CLI utility. This can lead you to the wrong decision to solve the task manually using the IDE. Automating your job is the best option in most cases. It saves your time and helps to avoid mistakes.

[166] https://en.wikipedia.org/wiki/File_Explorer
[167] https://en.wikipedia.org/wiki/Integrated_development_environment

You decide if you need to learn the CLI. I have only given few examples when it is beneficial. It is hard to switch from using a GUI to a CLI. You have to re-learn many things that you do with Windows Explorer regularly. But once you get the hang of the command shell, your new productivity will surprise you.

# Navigating the File System

Let's start introducing the Unix environment and Bash with a **file system**[168]. A file system is a software that dictates how to store and read data from disks. It covers the following topics:

- API to access data on the disk that programs can use.
- Universal way for accessing different storage devices.
- Physical operations on the disk storage.

First, we will look at the differences between the directory structure in Unix and Windows. Then we will learn the Bash commands for navigating the file system.

## Directory Structure

There is an address bar at the top of the Windows Explorer window. It displays the **absolute path** to the current directory. An absolute path shows the place of the file system object regardless of the current directory.

Another way to specify the file system object place is using the **relative path**. It shows you how to reach the object from the current directory.

A directory[169] is a file system cataloging structure. It can contain references to files and other directories. Windows terminology calls it **folder**[170]. Both names mean the same kind of file system object.

Figure 2-6 shows an Explorer window. The address bar equals `This PC > Local Disk (C:) > msys64` there. It matches the `C:\msys64` absolute path. Thus, we see the contents of the `msys64` directory on the `C` drive in the Explorer window.

The letter `C` in the path denotes the local system disk drive. The local drive means the device that is connected to your computer physically. You can also have a network drive. You access such a device via the network. The system disk means one that has the Windows installation on it.

[168] https://en.wikipedia.org/wiki/File_system
[169] https://en.wikipedia.org/wiki/Directory_(computing)
[170] https://en.wikipedia.org/wiki/Directory_(computing)#Folder_metaphor

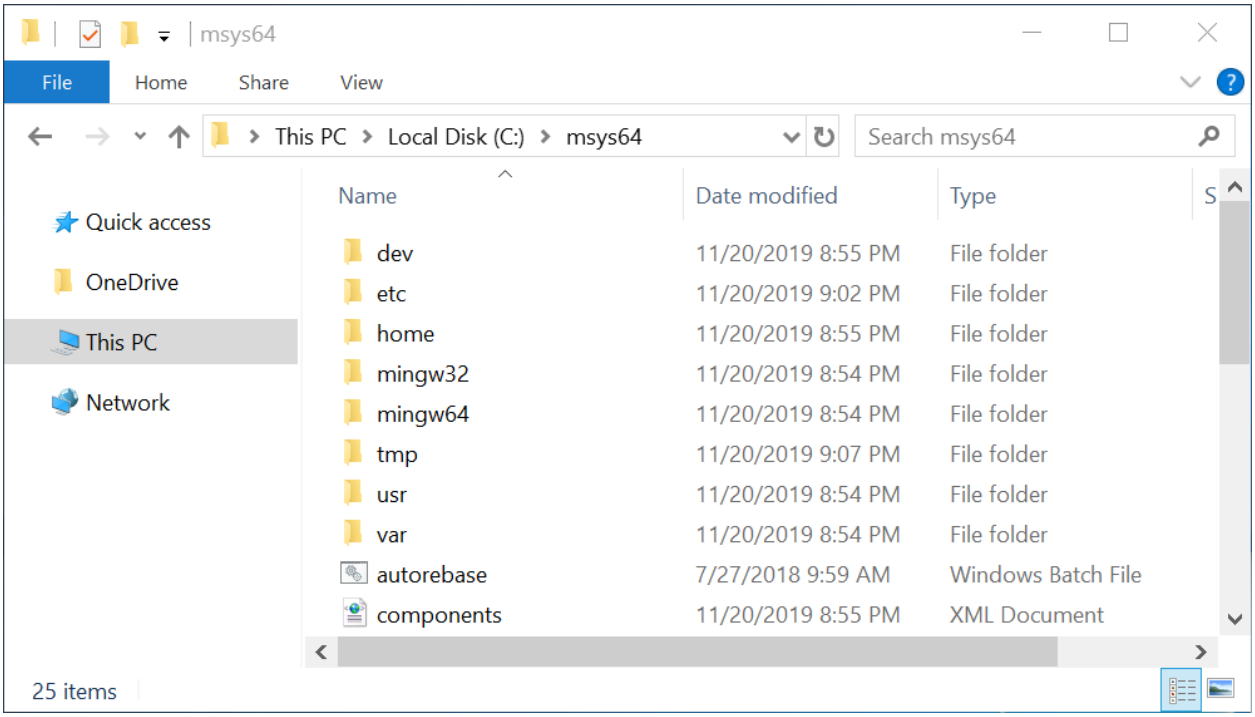



**Figure 2-6. Windows Explorer**

If you run the MSYS2 terminal emulator, it shows you the current absolute path at the end of the first line. This line behaves like the address bar of Windows Explorer. When you change the current directory, the current path changes too. However, you have to consider that the terminal and Explorer show you different paths for the same current directory. It happens because directory structures of the Unix environment and Windows do not match.

Windows marks each disk drive with a Latin letter. You can open the drive using Explorer as a regular folder. Then you access its content.

For example, let's open the `C` system drive. It has a standard set of directories[171]. Windows has created them during the installation process. If you open the `C` drive in Explorer, you see the following directories there:

- `Windows`
- `Program Files`
- `Program Files (x86)`
- `Users`
- `PerfLogs`

These directories store OS components, user applications and temporary files.

You can connect extra disk drives to your computer. Windows will assign them the following Latin letters: `D`, `E`, `F`, etc. You are allowed to create any directory structure on these disks. Windows does not restrict it in any way.

[171] https://en.wikipedia.org/wiki/Directory_structure#Windows_10

The File Allocation Table[172] (FAT) file system dictates how Windows manages disks and provides you access to them. Microsoft developed this file system for the MS-DOS[173] OS. The principles of FAT became the basis of the ECMA-107[174] standard. The next-generation file system from Microsoft is called NTFS[175]. It replaced the obsolete FAT in modern versions of Windows. However, the basic principles of disks and directory structure are the same in NAT and FAT. The reason for that is the backward compatibility requirement.

The Unix directory structure follows the POSIX standard[176]. This structure is more limited and strict than the Windows one. It has several predefined directories that you cannot move or rename. You are allowed to put your data in the specific paths only.

The POSIX standard says that the file system should have a top-level directory. It is called the **root directory**[177]. The slash sign / denotes it. All directories and files of all connected disk drives are inside the root directory.

If you want to access the contents of a disk drive, you should mount it. **Mounting** means embedding the contents of a disk into the root directory. When mounting is done, you can access the disk contents through some path. This path is called a **mount point**[178]. If you go to the mount point, you enter the file system of the disk.

Let's compare the Windows and Unix directory structures by example. Suppose that your Windows computer has two local disks `C` and `D`. Listing 2-1 shows their directory structure.

**Listing 2-1. The directory structure in Windows**

```
C:\
    PerfLogs\
    Windows\
    Program Files\
    Program Files (x86)\
    Users\

D:\
    Documents\
    Install\
```

Suppose that you have installed the Unix environment on your Windows. Then you run the terminal emulator and get the directory structure from Listing 2-2.

[172] https://en.wikipedia.org/wiki/File_Allocation_Table
[173] https://en.wikipedia.org/wiki/MS-DOS
[174] http://www.ecma-international.org/publications/standards/Ecma-107.htm
[175] https://en.wikipedia.org/wiki/NTFS
[176] https://pubs.opengroup.org/onlinepubs/9699919799/basedefs/V1_chap10.html#tag_10
[177] https://en.wikipedia.org/wiki/Root_directory
[178] https://en.wikipedia.org/wiki/Mount_(computing)

**Listing 2-2. The directory structure in Unix**

```
/
    c/
        PerfLogs/
        Windows/
        Program Files/
        Program Files (x86)/
        Users/

    d/
        Documents/
        Install/
```

Since you launch the MSYS2 terminal, you enter the Unix environment. Windows paths don't work there. You should use Unix paths instead. For example, you can access the `C:\Windows` directory via the `/c/Windows` path only.

There is another crucial difference between Unix and Windows file systems besides the directory structure. The character case[179] makes strings differ in the Unix environment. It means that two words with the same letters are not the same if their character case differs. For example, the `Documents` and `documents` words are not equal. Windows has no case sensitivity. If you type the `c:\windows` path in the Explorer address bar, it opens the `C:\Windows` directory. This approach does not work in the Unix environment. You should type all characters in the proper case.

Here is the last point to mention regarding Unix and Windows file systems. Use the slash sign / to separate directories and files in Unix paths. When you work with Windows paths, you use backslash \ for that.

## File System Navigation Commands

We are ready to learn our first Bash commands. Here are the steps to execute a shell command:

1. Make the terminal window active.
2. Type the command.
3. Press Enter.

The shell will process your input.

When the shell is busy, it cannot process your input. You can distinguish the shell's state by the **command prompt**[180]. It is a sequence of one or more characters. The default prompt is the dollar

[179] https://en.wikipedia.org/wiki/Case_sensitivity
[180] https://en.wikipedia.org/wiki/Command-line_interface#Command_prompt

sign $. You can see it in Figure 2-4. If the shell prints the prompt, it is ready for executing your command.

Windows Explorer allows you the following actions to navigate the file system:

- Display the current directory.
- Go to a specified disk drive or directory.
- Find a directory or file on the disk.

You can do the same actions with the shell. It provides you a corresponding command for each action. Table 2-1 shows these commands.

**Table 2-1. Commands and utilities for navigating the file system**

| Command | Description | Examples |
|---|---|---|
| `ls` | Display the contents of the directory.<br>If you call the command without parameters, it shows you the contents of the current directory. | `ls`<br>`ls /c/Windows` |
| `pwd` | Display the path to the current directory.<br>When you add the `-W` parameter, the command displays the path in the Windows directory structure. | `pwd` |
| `cd` | Go to the directory at the specified relative or absolute path. | `cd tmp`<br>`cd /c/Windows`<br>`cd ..` |
| `mount` | Mount the disk to the root file system. If you call the command without parameters, it shows a list of all mounted disks. | `mount` |
| `find` | Find a file or directory. The first parameter specifies the directory to start searching. | `find . -name vim`<br>`find /c/Windows -name *vim*` |
| `grep` | Find a file by its contents. | `grep "PATH" *`<br>`grep -Rn "PATH" .`<br>`grep "PATH" * .*` |

Bash can perform `pwd` and `cd` commands of Table 2-1 only. They are called **built-ins**[181]. Special utilities perform all other commands of the table. Bash calls an appropriate utility if it cannot execute your command on its own.

The MSYS2 environment provides a set of GNU utilities. These are auxiliary highly specialized programs. They give you access to the OS features in Linux and macOS. However, their capabilities are limited in Windows. Bash calls GNU utilities to execute the following commands of Table 2-1:

[181] https://www.gnu.org/software/bash/manual/html_node/Bash-Builtins.html

- `ls`
- `mount`
- `find`
- `grep`

When you read an article about Bash on the Internet, its author can confuse the "command" and "utility" terms. He names both things "commands". This is not a big issue. However, I recommend you to distinguish them. Calling a utility takes more time than calling Bash built-in. It causes performance overhead in some cases.

## pwd

Let's consider the commands in Table 2-1. We have just started the terminal. The first thing we do is to find out the current directory. You can get it from the command prompt, but it depends on your Bash configuration. You do not have this feature enabled by default in Linux and macOS.

When you start the terminal, it opens the home directory of the current user. Bash abbreviates this path by the tilde symbol ~. You see this symbol before the command prompt. Use tilde instead of the home directory absolute path. It makes your commands shorter.

Call the `pwd` command to get the current directory. Figure 2-7 shows this call and its output. The command prints the absolute path to the user's home directory. It equals `/home/ilya.shpigor` in my case.

If you add the `-W` **option** to the call, the command prints the path in the Windows directory structure. It is useful when you create a file in the MSYS2 environment and open it in a Windows application afterward. Figure 2-7 shows you the result of applying the `-W` option.

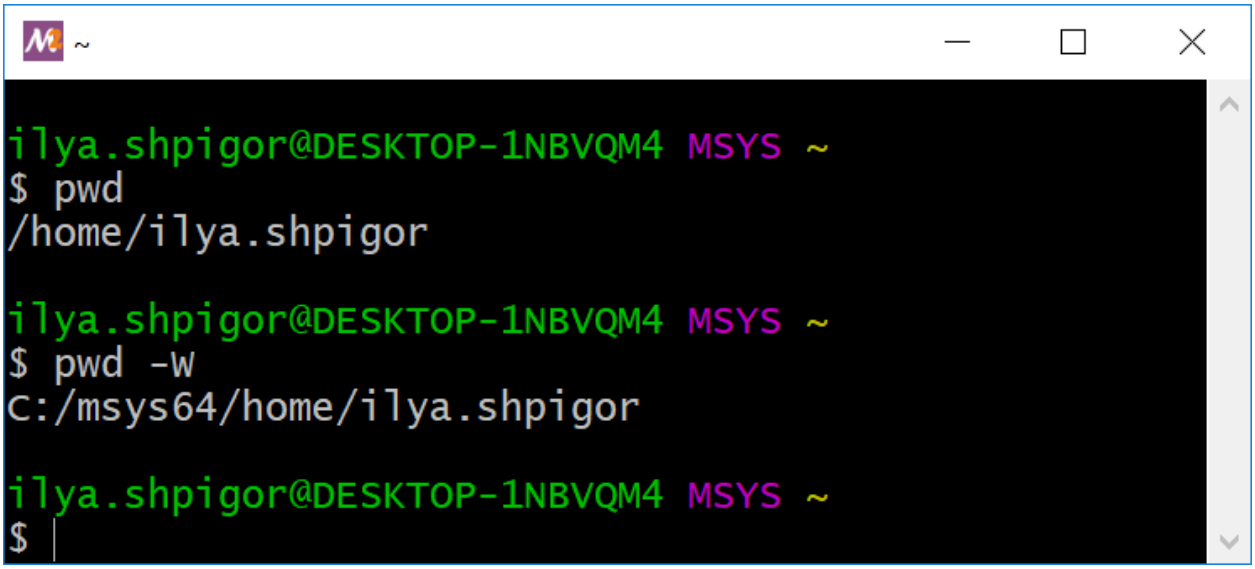



**Figure 2-7. The output of the `pwd` command**

What is a command option? When the program has a CLI only, you have very limited ways to interact with it. The program needs some data on input to do its job. The shell provides you a simple way to pass these data. Just type them after the command name. These data are called **arguments** of the program. Bash terminology distinguishes two kinds of arguments: parameter and option. A **parameter**[182] is the regular word or character you pass to the program. An **option** or **key** is an argument that switches the mode of a program. The standard dictates the option format. It is a word or character that starts with a dash `-` or a double dash `--`.

You pass data to the CLI programs and Bash built-ins in the same way. Use parameters and options for doing that.

Typing long commands is inconvenient. Bash provides the autocomplete feature to save your time. Here are the steps for using it:

1. Type the first few letters of the command.
2. Press the Tab key.
3. If Bash finds the command you mean, it completes it.
4. If several commands start with the typed letters, autocomplete does not happen. Press Tab again to see the list of these commands.

Figure 2-8 demonstrates how the autocomplete feature works. Suppose that you type the “pw” word. Then you press the Tab key twice. Bash shows you the commands that start with “pwd” as Figure 2-8 shows.

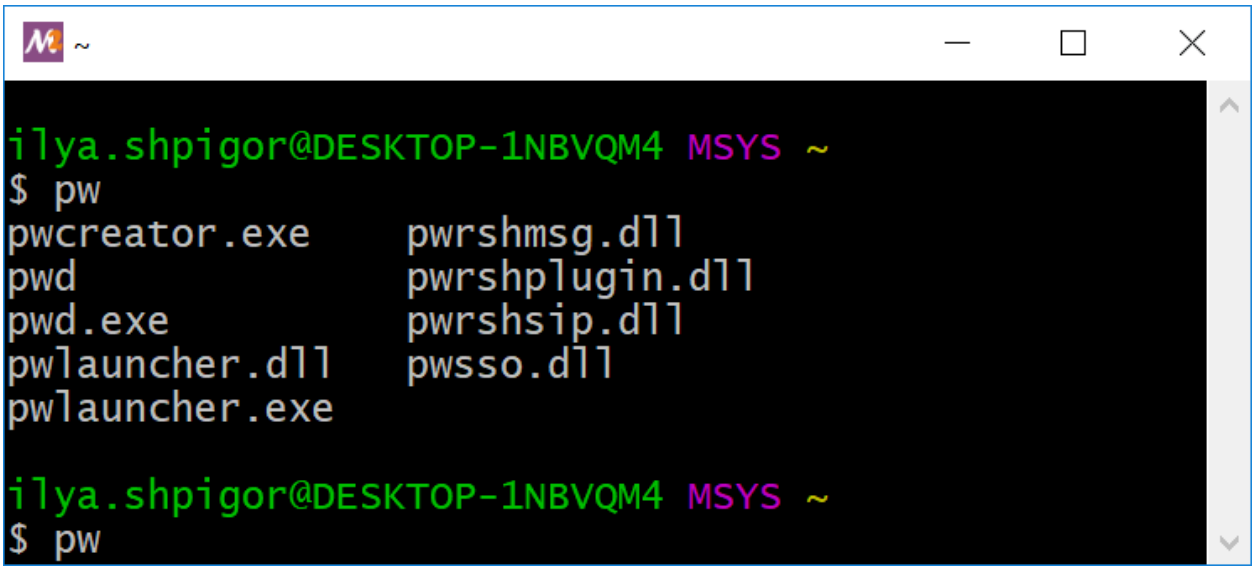



**Figure 2-8. Autocomplete for the pw command**

[182] https://en.wikipedia.org/wiki/Command-line_interface#Arguments

## ls

We got the current directory using the `pwd` command. The next step is checking the directory content. The `ls` utility does this task.

Suppose that you have just installed the MSYS2 environment. Then you launched the terminal first time. You are in the user's home directory. Call the "ls" command there. Figure 2-9 shows its result. The command output has nothing. It means that the directory is empty or has hidden files and directories only.

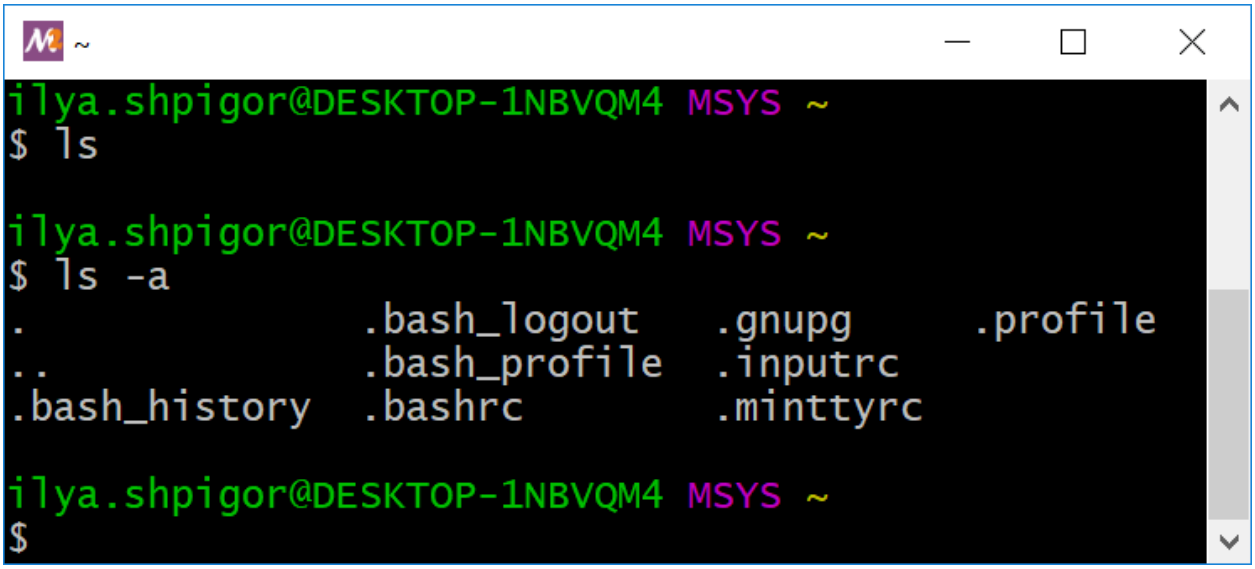



**Figure 2-9. The output of the `ls` utility**

Windows has a concept of hidden files and directories. The Unix environment also has it. Applications and OS create hidden files for their own needs. These files store configuration and temporary data.

Windows Explorer does not display hidden files and directories by default. Change the Explorer settings[183] to see them.

You can make the file hidden in Windows by changing its attribute. If you want to do the same in Unix, you should add a dot at the beginning of the filename.

When you launch the `ls` utility without parameters, it does not show you hidden objects. You can add the `-a` option to see them. Figure 2-9 shows a result of such a call.

The `ls` utility can show the contents of the specified directory. Pass a directory's absolute or relative path to the utility. For example, the following command shows the contents of the root directory:

```
ls /
```

Figure 2-10 shows the output of this command.

[183] https://support.microsoft.com/en-us/windows/show-hidden-files-0320fe58-0117-fd59-6851-9b7f9840fdb2

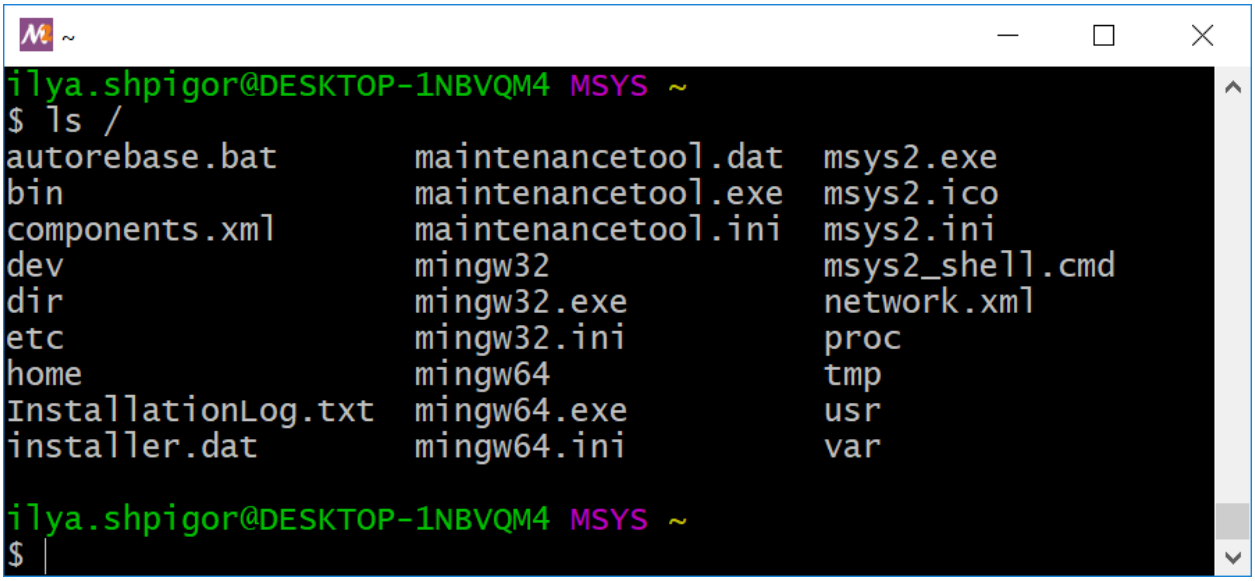



**Figure 2-10. The output of the `ls` utility**

There are no directories `/c` and `/d` in Figure 2-10. These are the mount points of C and D disk drives according to Listing 2-2. The mounting points are in the root directory. Why does not the `ls` utility print them? It happens because the Windows file system does not have a concept of mount points. Therefore, it does not have directories `/c` and `/d`. They are present in the Unix environment only. These are not real directories but paths where you can access the disk file systems. The `ls` utility reads the directory contents in the Windows file system. Thus, it does not show the mount points. The `ls` utility behaves differently in Linux and macOS. It shows mount points properly there.

## mount

If your computer has several disk drives, you can read their mount points. Call the `mount`[184] utility without parameters for doing that. Figure 2-11 shows its output.

[184] https://en.wikipedia.org/wiki/Mount_(Unix)

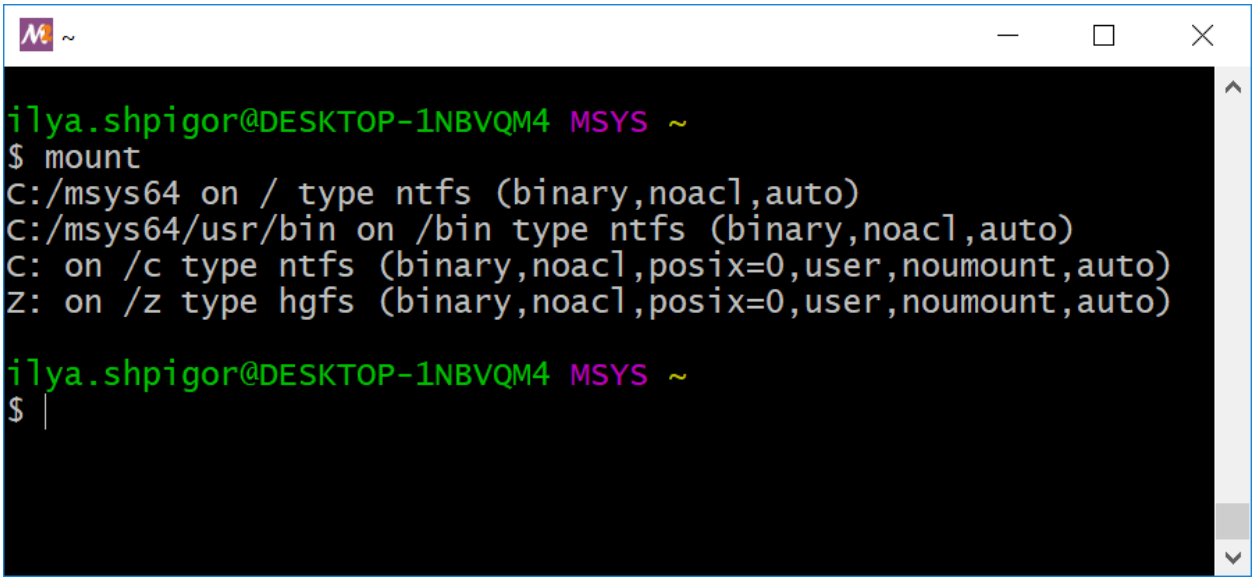



**Figure 2-11. The output of the `mount` utility**

Consider this output as a table with four columns. The columns display the following values:

1. The disk drive, its partition or directory. It is the object that the OS has mounted to the root directory.
2. Mount point. It is the path where you can access the mounted disk drive.
3. The file system type of the disk drive.
4. Mounting parameters. An example is access permissions to the disk contents.

If we split the `mount` utility output into these columns, we get Table 2-2.

**Table 2-2. The output of the `mount` utility**

| Mounted partition | Mount point | FS type | Mounting parameters |
|---|---|---|---|
| `C:/msys64` | `/` | `ntfs` | `binary,noacl,auto` |
| `C:/msys64/usr/bin` | `/bin` | `ntfs` | `binary,noacl,auto` |
| `C:` | `/c` | `ntfs` | `binary,noacl,posix=0,user,noumount,auto` |
| `Z:` | `/z` | `hgfs` | `binary,noacl,posix=0,user,noumount,auto` |

Table 2-2 confuses most Windows users. MSYS2 mounts `C:/msys64` as the root directory. Then it mounts the C and Z disks into the root. Their mount points are `/c` and `/z`. It means that you can access the C drive via the `C:/msys64/c` Windows path in the Unix environment. However, `C:/msys64` is the subdirectory of disk C in the Windows file system. We got a contradiction.

Actually, there is no contradiction. The `/c` path is the mount point that exists only in the Unix environment. It does not exist in the Windows file system. Therefore, Windows knows nothing about the `C:/msys64/c` path. It is just invalid if you try to open it via Explorer. You can imagine the

mount point `/c` as the shortcut[185] to drive C that exists in the MSYS2 environment only.

The output of the `mount` utility took up a lot of screen space. You can clear the terminal window by the Ctrl+L keystroke.

Another useful keystroke is Ctrl+C. It interrupts the currently running command. Use it if the command hangs or you want to stop it.

## cd

We have got everything about the current directory. Now we can change it. Suppose that you are looking for the Bash documentation. You can find it in the `/usr` system directory. Installed applications stores their non-executable files there. Call the `cd` command to go to the `/usr` path. Do it this way:

```
cd /usr
```

Do not forget about autocompletion. It works for both command and its parameters. Just type "cd /u" and press the Tab key. Bash adds the directory name `usr` automatically. Figure 2-12 shows the result of the command.

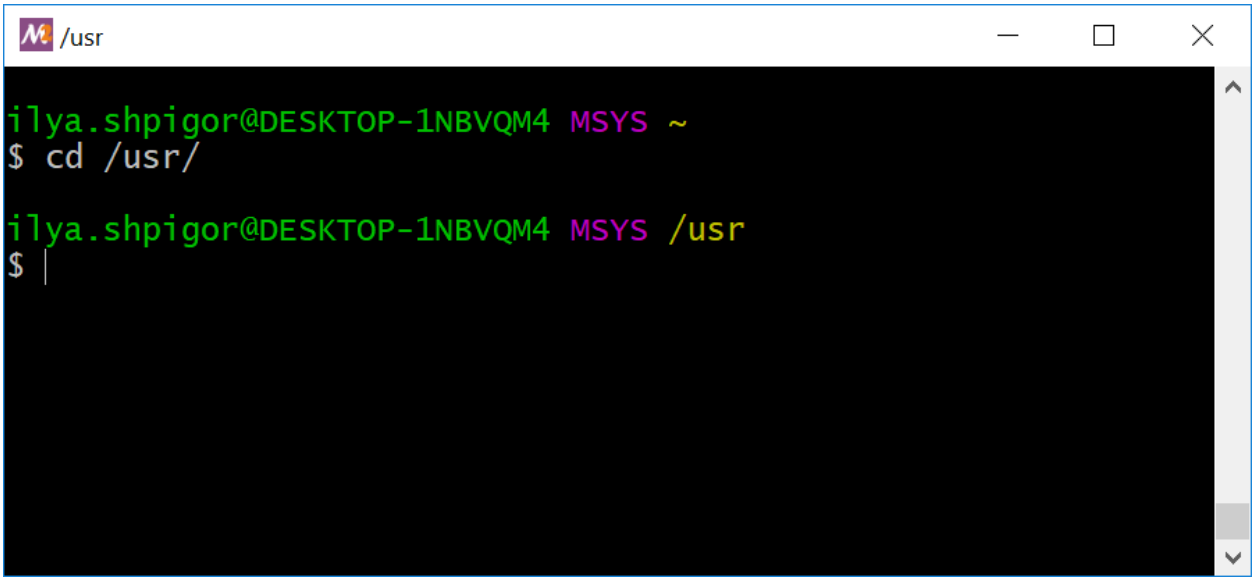



**Figure 2-12. The result of the `cd` command**

The `cd` command does not output anything if it succeeds. It changes the current directory and that is it. You can read the new path in the line above the command prompt. This line shows the `/usr` path after our `cd` call.

The `cd` command accepts both absolute and relative paths. Relative paths are shorter. Therefore, you type them faster. Prefer them when navigating the file system using a command shell.

[185] https://en.wikipedia.org/wiki/Shortcut_(computing)

There is a simple rule to distinguish the type of path. An absolute path starts with a slash /. An example is `/c/Windows/system32`. A relative path starts with a directory name. An example is `Windows/system32`.

Now you are in the `/usr` directory. You can get a list of its subdirectories and go to one of them. Suppose that you want to go one level higher and reach the root directory. There are two ways for doing that: go to the absolute path `/` or the special relative path `..`. The `..` path always points to the parent directory of the current one. Use it in the `cd` call this way:

```
cd ..
```



In addition to `..`, there is another special path `.`. It points to the current directory. If you execute the command "cd .", nothing happens. You stay in the same place. You need the `.` path to run programs from the current directory.

Come back to the `/usr` directory. Then run the `ls` utility there. It will show you the `share` subdirectory. Come to this directory and call `ls` again. You will find the `doc` directory there. It contains Bash documentation. Call the `cd` command this way to reach the documentation:

```
cd doc/bash
```

You are in the `/usr/share/doc/bash` directory now. Call the `ls` utility there. It will show you several files. One of them is `README`. It contains a brief description of the Bash interpreter.

You found the documentation file. The next step is to print its contents. The `cat` utility does that. Here is an example of how to run it:

```
cat README
```

Figure 2-13 shows the terminal window after the `cat` call.

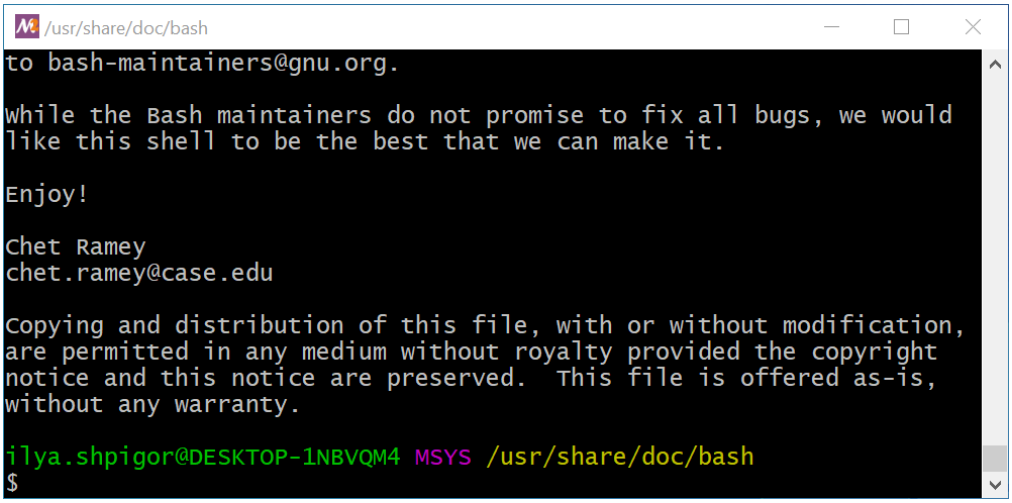



**Figure 2-13. The result of the `cat` utility**



Some Bash manuals do not recommend printing file contents using the `cat` utility. They said that the utility's purpose is to merge several files and print the result. The manuals suggest combining the `echo` command with a stream redirection. Here is an example of this approach:

```
echo "$(< README.txt)"
```

The `README` file contents do not fit the terminal window. Therefore, you see the tail of the file in Figure 2-13. Use the scroll bar on the window's right side to check the head of the file. Also, use the Shift+PageUp and Shift+PageDown hotkeys to scroll pages[186] up and down. The Shift+↑ and Shift+↓ keystrokes scroll the lines.

## Command History

Whenever you call a command, Bash saves it in the command history[187]. You can navigate the history by up and down arrow keys. Bash automatically types the corresponding command. You just need to press Enter for launching it. For example, you have called the "cat README" command. Press the up arrow and Enter to repeat it.

The Ctrl+R shortcut brings up a search over all command history. Press Ctrl+R and start typing. Bash will show you the last called command that begins with these characters. Press Enter to execute it.

The `history` command shows you the whole history. Run it without parameters this way:

[186] https://en.wikipedia.org/wiki/Page_Up_and_Page_Down_keys
[187] https://en.wikipedia.org/wiki/Command_history

```
history
```

The history stores the command that you have executed. It does not keep the command that you typed and then erased.

There is a trick to save the command to the history without executing it. Add the hash symbol # before the command and press Enter. Bash stores the typed line, but it does not execute it. This happens because the hash symbol means comment. When the interpreter meets a comment, it ignores this line. However, Bash adds the commented lines in the history because they are legal constructions of the language.

Here is an example of the trick with comment for our `cat` utility call:

```
#cat README
```

You have saved the commented command in the history. Now you can find it there by pressing the up arrow key. Remove the hash symbol at the beginning of the line and press Enter. Bash will execute your command.

You can do the comment trick by the Alt+Shift+3 shortcut. It works in most modern terminal emulators. Here are the steps for using the shortcut:

1. Type a command, but do not press Enter.
2. Press Alt+Shift+3.
3. Bash saves the command in the history without executing it.

Sometimes you need to copy text from the terminal window. It can be a command or its output. Here is an example. Suppose that some document needs a part of the Bash `README` file. Use the **clipboard**[188] to copy it. The clipboard is temporary storage for text data. When you select something in the terminal window with a mouse, the clipboard saves it automatically. Then you can paste this data to any other window.

These are the steps to copy text from the terminal window:

1. Select the text with the mouse. Hold down the left mouse button and drag the cursor over the required text.
2. Press the middle mouse button to paste the text from the clipboard into the same or another terminal window. You insert the text at the current cursor position.
3. Right-click and select the “Paste” item to paste the text to the application other than the terminal.

[188] https://en.wikipedia.org/wiki/Clipboard_(computing)

## find

It is inconvenient to search for a file or directory with `cd` and `ls` commands. The special `find` utility does it better.

If you run the `find` utility without parameters, it traverses the contents of the current directory and prints it. The output includes hidden objects. Figure 2-14 shows the result of running `find` in the home directory.

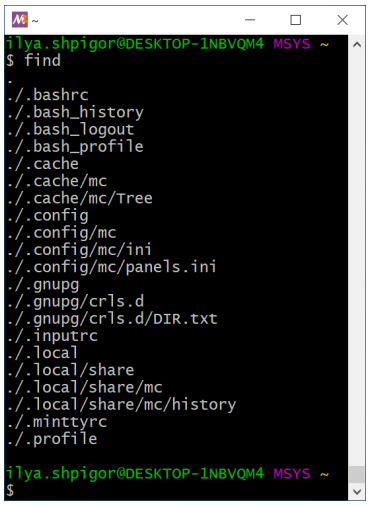



**Figure 2-14. The output of the `find` utility**

The first parameter of `find` is the directory to search in. The utility accepts relative and absolute paths. For example, the following command shows the contents of the root directory:

```
find /
```

You can specify search conditions starting from the second parameter. If the found object does not meet these conditions, `find` does not print it. The conditions form a single expression. The utility has an embedded interpreter that processes this expression.

An example of the `find` condition is the specific filename. When you call the utility with such a condition, it prints the found files with this name only.

Table 2-3 shows the format of commonly used conditions for the `find` utility.

**Table 2-3. Commonly used conditions for the `find` utility**

| Condition | Meaning | Example |
|---|---|---|
| `-type f` | Search files only. | `find -type f` |
| `-type d` | Search directories only. | `find -type d` |
| `-name <pattern>` | Search for a file or directory with the name that matches a **glob pattern**[189]. The pattern is case-sensitive. | `find -name README`<br>`find -name READ*`<br>`find -name READ??` |
| `-iname <pattern>` | Search for a file or directory with the name that matches a glob pattern. The pattern is case-insensitive. | `find -iname readme` |
| `-path <pattern>` | Search for a file or directory with the path that matches a glob pattern. The pattern is case-sensitive. | `find -path */doc/bash/*` |
| `-ipath <pattern>` | Search for a file or directory with the path that matches a glob pattern. The pattern is case-insensitive. | `find . -ipath */DOC/BASH/*` |
| `-a` or `-and` | Combine several conditions using the logical AND. If the found object fits all conditions, the utility prints it. | `find -name README -a -path */doc/bash/*` |
| `-o` or `-or` | Combine several conditions using the logical OR. If the found object fits at least one condition, the utility prints it. | `find -name README -o -path */doc/bash/*` |
| `!` or `-not` | The logical negation (NOT) of the condition. If the found object does not fit the condition, the utility prints it. | `find -not -name README`<br>`find ! -name README` |

A glob pattern is a search query that contains **wildcard characters**[190]. Bash allows three wildcard characters: *, ? and [. The asterisk stands for any number of any characters. A question mark means a single character of any kind.

Here is an example of glob patterns. The string `README` matches all following patterns:

- `*ME`
- `READM?`
- `*M?`
- `R*M?`

[189] https://en.wikipedia.org/wiki/Glob_(programming)
[190] https://en.wikipedia.org/wiki/Wildcard_character

Square brackets indicate a set of characters at a specific position. For example, the pattern "[cb]at.txt" matches the `cat.txt` and `bat.txt` files. You can apply this pattern to the `find` call this way:

```
find . -name "[cb]at.txt"
```

**Exercise 2-1. Glob patterns**

```
What of the following lines corresponds to the pattern "*ME.??" ?

* 00_README.txt
* README
* README.md
```

**Exercise 2-2. Glob patterns**

```
What of the following lines corresponds to the pattern "*/doc?openssl*" ?

* /usr/share/doc/openssl/IPAddressChoice_new.html
* /usr/share/doc_openssl/IPAddressChoice_new.html
* doc/openssl
* /doc/openssl
```

Let's apply glob patterns into practice. Suppose that you do not know the location of the Bash `README` file. You should use the `find` utility in this case. Start searching with the utility from the root directory. Now you need a search condition. It is a common practice to store documentation in directories called `doc` in Unix. Therefore, you can search files in these directories only. This way, you get the following `find` call:

```
find / -path */doc/*
```

The command shows you all documentation files on all mounted disks. This is a huge list. You can shorten it with an extra search condition. It should be a separate directory for the Bash documentation. The directory is called `bash`. Add this path as the second search condition. Then you get the following command:

```
find / -path */doc/* -path */bash/*
```

Figure 2-15 shows the result of this command.

The following `find` call provides the same result:

```
find / -path */doc/* -a -path */bash/*
```

Our `find` calls differ by the `-a` option between conditions. The option means logical AND. If you do not specify any logical operator between conditions, `find` inserts AND by default. This is a reason why both calls provide the same result.

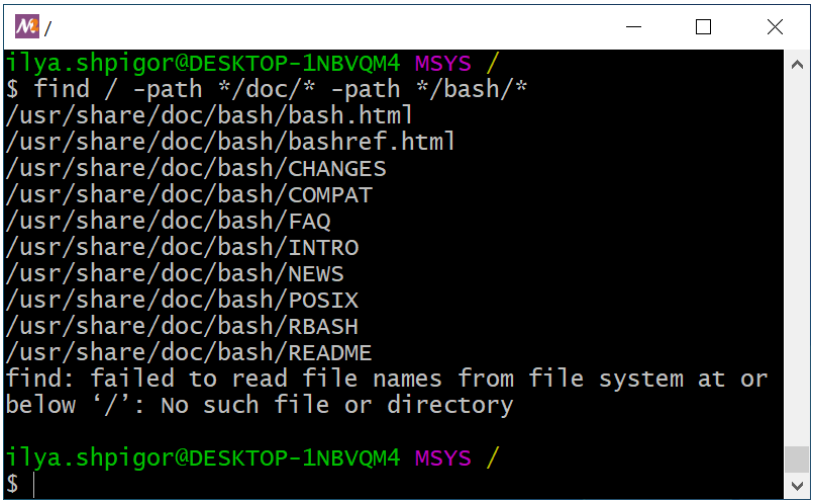



**Figure 2-15. The output of the `find` utility**

You can see that the `find` utility reports an error in Figure 2-15. The mount points of Windows disk drives cause it. The utility cannot access them when you start searching from the root directory. You can avoid the problem if you start searching from the `/c` mount point. Do it this way:

```
find /c -path */doc/* -a -path */bash/*
```

There is an alternative solution. You should exclude mount points from the search. The `-mount` option does this. Apply the option this way:

```
find / -mount -path */doc/* -a -path */bash/*
```

When you add the second search condition, the `find` utility shows a short list of documents. You can find the right `README` file easily there.

There are other ways to search for the documentation file. Suppose that you know its name. Then you can specify it together with an assumed path. You will get the `find` call like this:

```
find / -path */doc/* -name README
```

Figure 2-16 shows the result of this command.

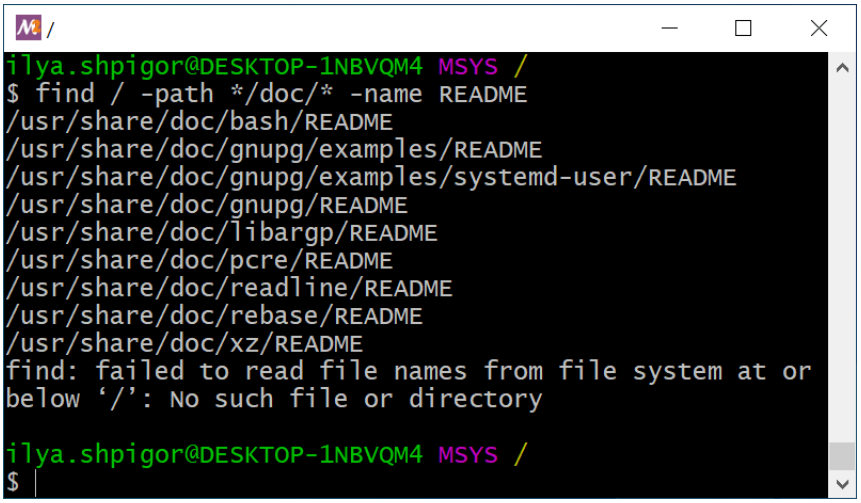



**Figure 2-16. The output of the `find` utility**

Again you got a short list of files. It is easy to locate the right file there.

You can group the conditions of the `find` utility. Do it using the escaped[191] parentheses. Here is an example of using them. Let's write the `find` call that searches `README` files with path `*/doc/*` or `LICENSE` files with an arbitrary path. This call looks like this:

```
find / \( -path */doc/* -name README \) -o -name LICENSE
```

Why should you apply backslashes to escape brackets here? The parentheses are part of the Bash syntax. Therefore, Bash treats them like language constructs. When Bash meets parentheses in a command, it performs an **expansion**. The expansion is the replacement of a part of the command with something else. When you escape parentheses, you force Bash to ignore them. Thus, Bash does not perform the expansion and passes all search conditions to the `find` utility as it is.

The `find` utility can process the found objects. You can specify an action to apply as an extra option. The utility will apply this action to each found object.

Table 2-4 shows the `find` options that specify actions.

**Table 2-4. Options for specifying actions on found objects**

| Option | Meaning | Example |
|---|---|---|
| `-exec command {} \;` | Execute the specified command on each found object. | `find -name README -type f -exec cp {} ~ \;` |
| `-exec command {} +` | Execute the specified command once over all found objects. The command receives all these objects on the input. | `find -type d -exec cp -t ~ {} +` |
| `-delete` | Delete each of the found files. The utility deletes empty directories only. | `find -name README -type f -delete` |

[191] https://en.wikipedia.org/wiki/Escape_character

Table 2-4 shows that there are two variants of the `-exec` action. They differ by the characters at the end. It can be an escaped semicolon `\;` or a plus sign `+`. Use the plus sign variant only if the called command handles several input parameters. You will make a mistake if the command accepts one parameter only. It will process the first found object and skip the rest.

Let's apply the `-exec` action in practice. Suppose that you want to copy files with the Bash documentation into the home directory. You are interested in the HTML files only.

The first step is preparing the correct `find` call for searching the files. You should apply two conditions here. The first one checks the directory of the Bash documentation. The second condition checks the file extensions. If you combine these conditions, you get the following `find` call:

```
find / -path "*/doc/bash/*" -name "*.html"
```

When you pass the glob pattern to the `find` utility, always enclose it in double-quotes. The quotes do the same as the backslash before parentheses. They prevent Bash from expanding the patterns. Instead, Bash passes them to the `find` utility.

Figure 2-17 shows the result of our `find` call. You can see that it found HTML files correctly.

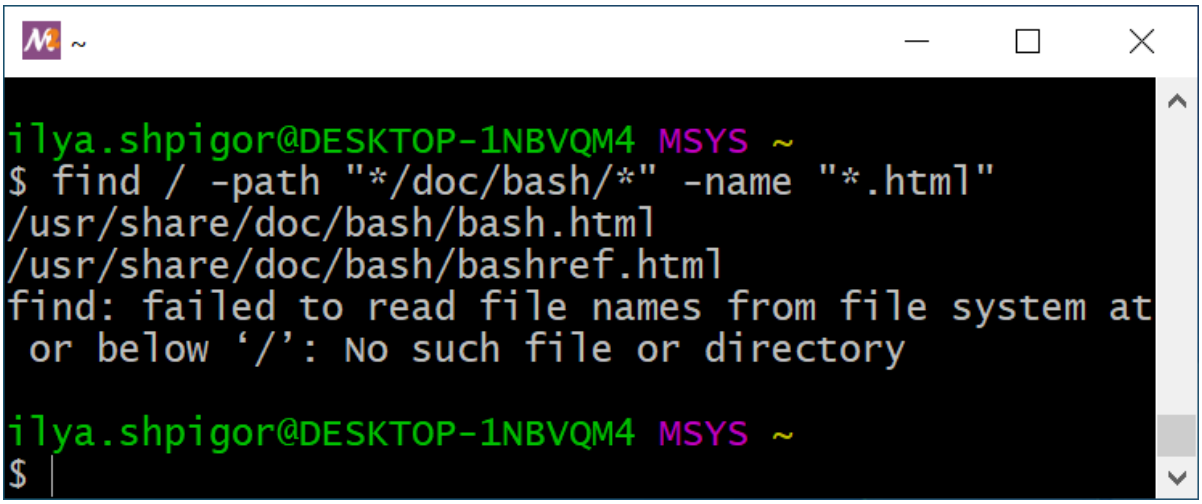



**Figure 2-17. The output of the `find` utility**

The second step for solving your task is adding the `-exec` action. The action should call the `cp` utility. This utility copies files and directories to the specified path. It takes two parameters. The first one is the source object to copy. The second parameter is the target path. When you apply the `-exec` action, you get the following `find` call:

```
find / -path "*/doc/bash/*" -name "*.html" -exec cp {} ~ \;
```

Run this command. It prints an error about the mount point. Despite the error, the command did its job. It copied the HTML files into the home directory.

How does the command work in detail? It calls the `cp` utility for each HTML file it found. When calling the utility, `find` inserts each found object instead of curly braces {}. Therefore, two `cp` calls happen here. They look like this:

```
cp ./usr/share/doc/bash/bash.html ~
cp ./usr/share/doc/bash/bashref.html ~
```

Each `cp` call copies one HTML file to the home directory.

Good job! You just wrote your first program in the language of the `find` utility. The program works according to the following algorithm:

1. Find HTML files starting from the root directory. Their paths match the `*/doc/bash/*` pattern.
2. Copy each found file into the home directory.

The program is quite simple and consists of two steps only. However, it is a scalable solution for finding and copying files. The program processes two or dozens of HTML files with the same speed.

You can combine the `-exec` actions in the same way as the search conditions. For example, let's print the contents of each found HTML file and count the number of its lines. You should call the `cat` utility to print the file contents. The `wc` utility counts the lines. It takes the filename as an input parameter. If you combine `cat` and `wc` calls, you get the following `find` command:

```
find / -path "*/doc/bash/*" -name "*.html" -exec cat {} \; -exec wc -l {} \;
```

There is no logical operation between the `-exec` actions. The `find` utility inserts logical AND by default. This has a consequence in our case. If the `cat` utility fails, `find` does not call the `wc` utility. It means that `find` executes the second action only if the first one succeeds. You can apply the logical OR explicitly. Then `find` always calls `wc`. Here is the command with logical OR:

```
find / -path "*/doc/bash/*" -name "*.html" -exec cat {} \; -o -exec wc -l {} \;
```

You can group the `-exec` actions with escaped parentheses `\(` and `\)`. It works the same way as grouping search conditions.

**Exercise 2-3. Searching for files with the `find` utility**

---

```
Write a find call to search for text files in a Unix environment.
Extend the command to print the total number of lines in these files.
```

---

## Logic Expressions

The search conditions of the `find` utility are **Boolean expressions**[192]. A Boolean expression is a programming language statement. It produces a Boolean value when evaluated. This value equals either "true" or "false".

The `find` condition is a statement of the utility's language. It produces the "true" value if the found object meets its requirement. Otherwise, the condition produces "false". If there are several conditions in the `find` call, they make a single compound Boolean expression.

When we have considered the binary numeral system, we already met Boolean algebra. This section of mathematics studies **logical operators**[193]. They differ from the arithmetic operations: addition, subtraction, multiplication, and division.

You can apply a logical operator to Boolean values or expressions. Using an arithmetic operation does not make sense in this case. Addition or subtraction is trivial for Boolean values. It yields nothing. When you apply a logical operator, you get a condition with strict evaluation rules. This way, you wrote search conditions for the `find` utility. When you combine several conditions, you get a program with complex behavior.

An **operand** is an object of a logical operator. Boolean values and expressions can be operands.

Let's consider Boolean expressions using an example. The example is not related to the `find` utility or Bash for simplicity. Imagine that you are programming a robot for a warehouse. Its job is to move boxes from point A to point B. You can write the following straightforward algorithm for the robot:

1. Move to point A.
2. Pick up the box at point A.
3. Move to point B.
4. Put the box at point B.

This algorithm does not have any conditions. It means that the robot performs each step independently of external events.

Now imagine that an obstacle happens in the robot's way. For example, another robot stuck there. Executing your algorithm leads to the collision of the robots in this case. You should add a condition in the algorithm to prevent the collision. For example, it can look like this:

1. Move to point A.
2. Pick up the box at point A.
3. If there is no obstacle, move to point B. Otherwise, stop.
4. Put the box at point B.

[192] https://en.wikipedia.org/wiki/Boolean_expression
[193] https://en.wikipedia.org/wiki/Logical_connective

The third step of the algorithm is called **conditional statement**[194]. All modern programming languages have such a statement.

The conditional statement works according to the following algorithm:

1. Evaluate the Boolean expression in the condition.
2. If the expression produces "true", perform the first action.
3. If the expression produces "false", perform the second action.

The robot evaluates the value of the Boolean expression "there is no obstacle" in our example. If there is an obstacle, the expression produces "false" and the robot stops. Otherwise, the robot moves to point B.

When writing the conditional statement, you can combine several Boolean expressions using logical operators. Here is an example. Suppose that the robot tries to pick up a box at point A, but there is no box. Then there is no reason for him to move to point B. You can check this situation in the conditional statement. Add the new Boolean expression there using **logical AND**[195] (conjunction). Then the robot's algorithm becomes like this:

1. Move to point A.
2. Pick up the box at point A.
3. If there is a box AND no obstacle, move to point B. Otherwise, stop.
4. Put the box at point B.

Logical operators produce Boolean values when evaluated. The result of a logical AND equals "true" when both operands are "true". In our example, it happens when the robot has a box and there is no obstacle on its way. Otherwise, the result of logical AND equals "false". It forces the robot to stop.

You have used two more logical operators when learning the `find` utility. These operators are **OR**[196] (disjunction) and **NOT**[197] (negation).

Actually, you have already applied logical NOT in the robot's algorithm. It stays implicitly in the expression "there is no obstacle". It equals the following negation: "there is NOT an obstacle". You can specify the logical NOT in the algorithm explicitly this way:

1. Move to point A.
2. Pick up the box at point A.
3. If there is a box AND there is NOT an obstacle, move to point B. Otherwise, stop.
4. Put the box at point B.

[194] https://en.wikipedia.org/wiki/Conditional_(computer_programming)
[195] https://en.wikipedia.org/wiki/Logical_conjunction
[196] https://en.wikipedia.org/wiki/Logical_disjunction
[197] https://en.wikipedia.org/wiki/Negation

You can always replace logical AND by OR with some extra changes. Let's do it for our example but keep the robot's behavior the same. You should add the negation to the first Boolean expression and remove it from the second one. Also, you have to change the order of actions in the conditional statement. If the condition produces "true", the robot stops. If it produces "false", the robot moves to point B. The new algorithm looks this way:

1. Move to point A.
2. Pick up the box at point A.
3. If there is NOT a box OR there is an obstacle, stop. Otherwise, move to point B.
4. Put the box at point B.

Read the new conditional statement carefully. The robot follows the same decisions as before. It stops if it has no box or if there is an obstacle on its way. However, you have exchanged the logical AND to OR. This trick helps you to keep your conditional statements clear. Choose between logical AND and OR depending on your Boolean expressions. Pick one that fits your case better.

You wrote the Boolean expressions as sentences in English in our example. Such a sentence sounds unnatural. You have to read it several times to understand it. This happens because the natural humans' language is not suitable for writing Boolean expressions. This language is not accurate enough. Boolean algebra uses mathematical notation for that reason.

We have considered logical AND, OR and NOT. You will deal with three more operators in programming often:

- Equivalence
- Non-equivalence
- Exclusive OR

Table 2-5 explains them.

**Table 2-5. Logical operators**

| Operator | Evaluation Rule |
|---|---|
| AND | It produces "true" when both operands are "true". |
| OR | It produces "true" when any of the operands is "true". It produces "false" when all operands are "false". |
| NOT | It produces "true" when the operand is "false" and vice versa. |
| Exclusive OR (XOR) | It produces "true" when the operands have different values (true-false or false-true). It produces "false" when the operands are the same (true-true, false-false). |
| Equivalence | It produces "true" when the operands have the same values. |
| Non-equivalence | It produces "true" when the values of the operands differ. |

Try to memorize this table. It is simple to reach when you use logical operators often.

## grep

The GNU utilities have one more searching tool besides `find`. It is called `grep`. This utility checks file contents when searching.

How to choose the proper utility for searching? Use `find` for searching a file or directory by its name, path or **metadata**[198]. Metadata is extra information about an object. Examples of the file metadata are size, time of creation and last modification, permissions. Use the `grep` utility to find a file when you know nothing about it except its contents.

Here is an example. It shows you how to choose the right utility for searching. Suppose that you are looking for a documentation file. You know that it contains the phrase "free software". If you apply the `find` utility, the searching algorithm looks like this:

1. Call `find` to list all the files with the `README` name.
2. Open each file in a text editor and check if it has the phrase "free software".

Using a text editor for checking dozens of files takes too much effort and time. You should perform several operations with each file manually: open it, activate the editor's searching mode, type the "free software" phrase. The `grep` utility automates this task. For example, the following command finds all lines with the "free software" phrase in the specified `README` file:

```
grep "free software" /usr/share/doc/bash/README
```

The first parameter of the utility is a string for searching. Always enclose it in the double-quotes. This way, you prevent Bash expansions and guarantee that the utility receives the string unchanged. Without the quotes, Bash splits the phrase into two separate parameters. This mechanism of splitting strings into words is called **word splitting**[199].

The second parameter of `grep` is a relative or absolute path to the file. If you specify a list of files separated by spaces, the utility processes them all. In the example, we passed the `README` file path only.

Figure 2-18 shows the result of the `grep` call.

[198] https://en.wikipedia.org/wiki/Metadata
[199] http://mywiki.wooledge.org/WordSplitting

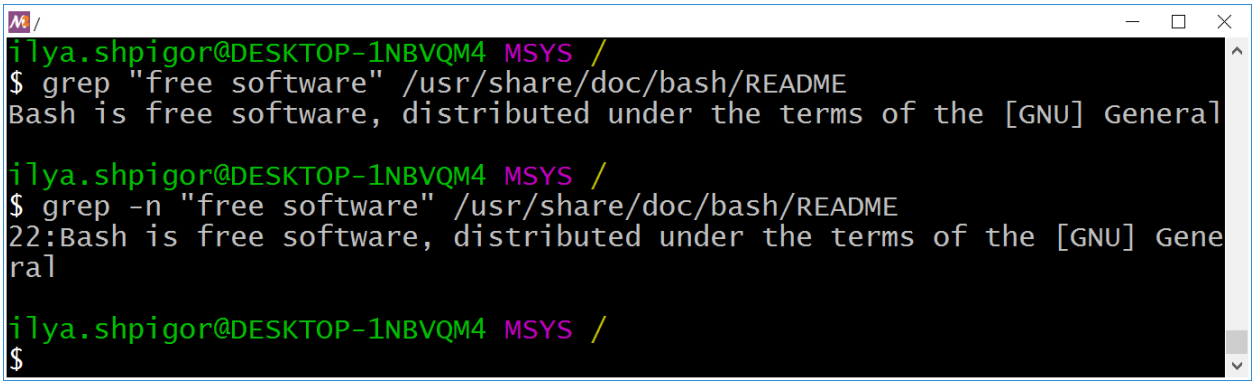



**Figure 2-18. The output of the `grep` utility**

You see all lines of the file where the utility found the specified phrase. The `-n` option adds the line numbers to the `grep` output. It can help you to check big text files. Add the option before the first parameter when calling the utility. Figure 2-18 shows the output in this case.

We have learned how to use `grep` to find a string in the specified files. Now let's apply the utility to solve our task. You are looking for the documentation files with the phrase "free software". There are two ways to find them with the `grep` utility:

- Use Bash glob patterns.
- Use the file search mechanism of the `grep` utility.

The first method works well when you have all files for checking in the same directory. Suppose that you found two `README` files: one for Bash and one for the `xz` utility. You have copied them to the home directory with the names `bash.txt` and `xz.txt`. The following two commands find the file that contains the phrase "free software":

```
1 cd ~
2 grep "free software" *
```

The first command changes the current directory to the user's home. The second command calls the `grep` utility.

When calling `grep`, we have specified the asterisk for the target file path. This wildcard means any string. Bash expands all wildcards in the command before launching it. In our example, Bash replaces the asterisk with all files of the home directory. The resulting `grep` call looks like this:

```
grep "free software" bash.txt xz.txt
```

Launch both versions of the `grep` call: with the `*` pattern and with a list of two files. The utility prints the same result for both cases.

You can search for the phrase in a single command. Just exclude the `cd` call. Then add the home directory to the search pattern. You will get the following `grep` call:

```
grep "free software" ~/*
```

This command does not handle subdirectories. It means that the `grep` call does not check the files in the `~/tmp` directory, for example.

There is an option to check how the Bash expands a glob pattern. Use the `echo` command for that. Here are `echo` calls for checking our patterns:

```
1 echo *
2 echo ~/*
```

Run these commands. The first one lists files in the current directory. The second command does the same for the home directory.

Do not enclose search patterns in double-quotes. Here is an example of the wrong command:

```
grep "free software" "*"
```

Quotes prevent the Bash expansion. Therefore, Bash does not insert the filenames to the command but passes the asterisk to the `grep` utility. The utility cannot handle the glob pattern properly as `find` does. Thus, you will get an error like Figure 2-19 shows.

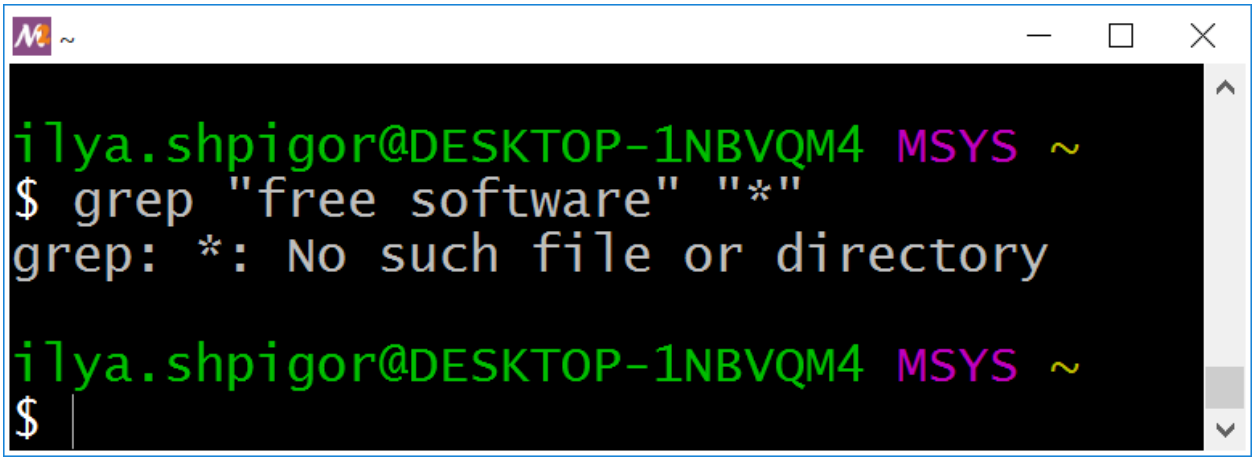



**Figure 2-19. The result of processing a search pattern by `grep`**

When expanding the `*` pattern, Bash ignores hidden files and directories. Therefore, the `grep` utility ignores them too in our example. Add the dot before the asterisk to get the glob pattern for hidden objects. It looks like `.*`. If you want to check all files at once, specify two patterns separated by the space. Here is an example `grep` call:

```
grep "free software" * .*
```

The second approach to search files with `grep` is using its built-in mechanism. It traverses the directories recursively and checks all files there. The `-r` option enables this mechanism. When using this option, specify the search directory in the second utility's parameter.

Here is an example of using the `-r` option:

```
grep -r "free software" .
```

This command finds the "free software" phrase in the files of the current directory. It processes the hidden objects too. If you work on Linux or macOS, prefer the `-R` option instead of `-r`. It forces `grep` to follow **symbol links**[200] when searching. Here is an example:

```
grep -R "free software" .
```



A symbolic link is a file of a special type. Instead of data, it contains a pointer to another file or directory.

You can specify the starting directory for searching by a relative or absolute path. Here are the examples for both cases:

```
1 grep -R "free software" ilya.shpigor/tmp
2 grep -R "free software" /home/ilya.shpigor/tmp
```

Suppose that you are interested in a list of files that contain a phrase. You do not need all occurrences of the phrase in each file. The `-l` option switches the `grep` utility in the mode you need. Here is an example of using it:

```
grep -Rl "free software" .
```

Figure 2-20 shows the result of this command.

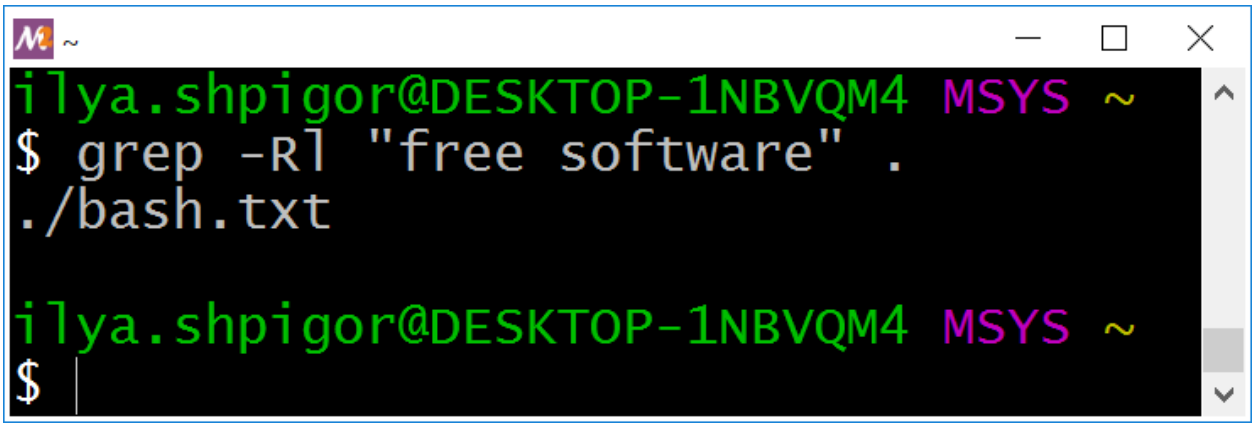



**Figure 2-20. The `grep` outputs filenames only**

[200] https://en.wikipedia.org/wiki/Symbolic_link

You see a list of files where the phrase “free software” occurs at least once. Suppose that you need the opposite result: a list of files without the phrase. Use the `-L` option for finding them. Here is an example:

```
grep -RL "free software" .
```

The `grep` utility processes the text files only. Therefore, it deals well with the source code files. You can use the utility as an add-on to your code editor or IDE.

You may have liked the `grep` utility. You want to process PDF[201] and MS Office documents with it. Unfortunately, this approach does not work. The contents of these files are not text. It is encoded. You need another utility to process such files. Table 2-6 shows `grep` alternatives for non-text files.

**Table 2-6. Utilities for text searching in PDF and MS Office files**

| Utility | Features |
|---|---|
| pdftotext[202] | It converts a PDF file into text format. |
| pdfgrep[203] | It searches PDF files by their contents. |
| antiword[204] | It converts an MS Office document into text format. |
| catdoc[205] | It converts an MS Office document into text format. |
| xdoc2txt[206] | It converts PDF and MS Office files into text format. |

Some of these utilities are available in the MSYS2 environment. Use the `pacman` package manager for installing them. The last chapter of the book describes how to use it.

**Exercise 2-4. Searching for files with the `grep` utility**

```
Write a grep call to find system utilities with a free license.
Here are widespread licenses for open-source software:

1. GNU General Public License
2. MIT license
3. Apache license
4. BSD license
```

[201] https://en.wikipedia.org/wiki/PDF
[202] http://www.xpdfreader.com
[203] https://pdfgrep.org
[204] http://www.winfield.demon.nl
[205] https://www.wagner.pp.ru/~vitus/software/catdoc
[206] https://documentation.help/xdoc2txt/xdoc2txt_en.html

# Command Information

We got acquainted with commands for navigating the file system. Each command has several options and parameters. We have covered the most common ones only. What if you need a rare feature that is missing in this book? You would need official documentation in this case.

All modern OSes and applications have documentation. However, you rarely need it when using the graphical interface. It happens because graphic elements are self-explanatory in most cases. Therefore, most PC users do not care about documentation.

When working with the CLI, the only way to know about available features of the software is by reading documentation. Besides that, you do not have anything that gives you a quick hint. When using CLI utility, it is crucial to know its basics. The negligence can lead to loss or corruption of your data.

The first versions of Unix had paper documentation. Using it was inconvenient and time-consuming. Soon it became even worse because the documentation volume grew rapidly. It exceeded the size of a single book. The Unix developers introduced the system called man page[207] to solve the issue with documentation. Using this software, you can quickly find the required topic. It contains information about OS features and all installed applications.

The man page system is a centralized place to access documentation. Besides it, every program in Unix provides brief information about itself. For example, the Bash interpreter has its own documentation system. It is called `help`.

Suppose that you want to get a list of all Bash built-ins. Launch the `help` command without parameters. Figure 2-21 shows its output.

[207] https://en.wikipedia.org/wiki/Man_page

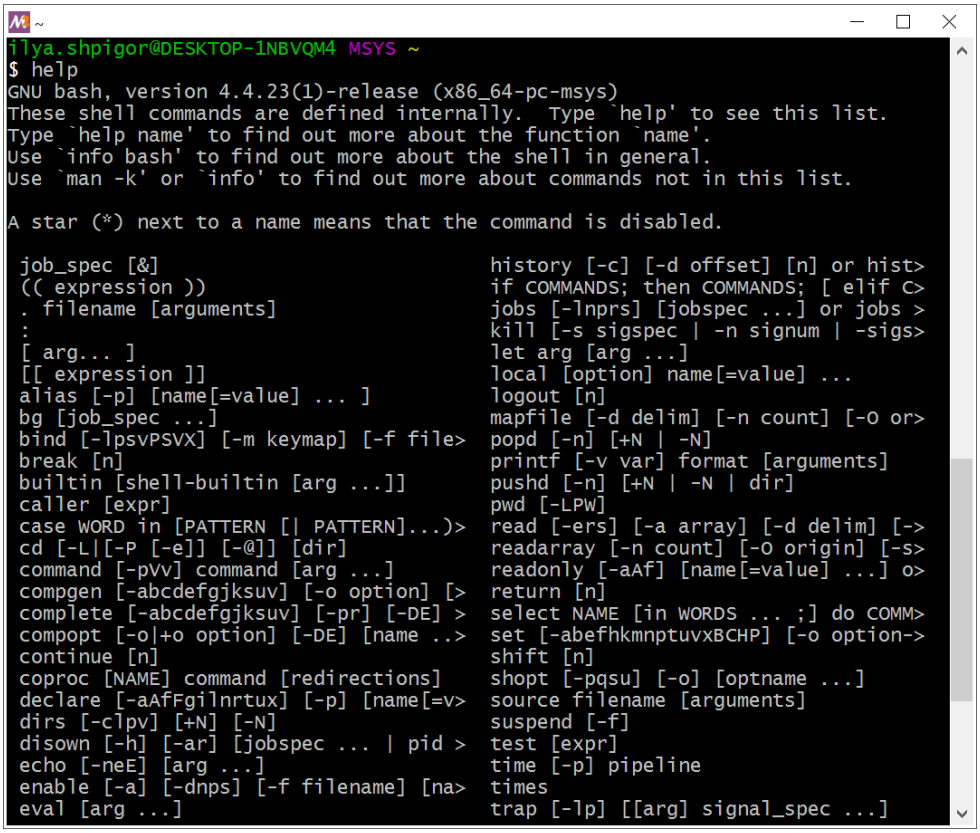



**Figure 2-21. The output of the `help` command**

You see a list of all commands that Bash executes on its own. If some command is missing in this list, Bash calls a GNU utility or another program to execute it.

Here is an example. The `cd` command presents in the `help` list. It means that Bash executes it without calling another program. Now suppose you type the `find` command. It is missing in the `help` list. Therefore, Bash looks for an executable file with the `find` name on the disk drive. If it succeeds, Bash launches this file.

Where does Bash look for files that execute your commands? Bash has a list of paths where it searches utilities and programs. The **environment variable** called `PATH` stores this list. The **variable**[208] is a named area of memory. If you write a program in machine code and want to access the memory area, you should specify its address. A variable is a mechanism of a programming language. It allows you to use the variable name instead of the memory address. Therefore, you do not need to remember addresses, which are long numbers.

Bash stores about a hundred environment variables. They hold data that affect the interpreter's behavior. Most of these data are system settings. We will consider Bash variables in the next chapter.



Call the `env` utility without parameters. It shows you all the environment variables that Bash uses at the moment.

[208] https://en.wikipedia.org/wiki/Variable_(computer_science)

You can imagine the variable as a value that has a name. For example, you can say: "The time now is 12 hours". "Time now" is the variable name. Its value equals "12 hours". The computer stores it in memory at some address. You do not know the address. However, you can ask a computer the value of the "time now" variable. It returns you "12 hours". This is how the variables work.

The `echo` command prints strings. It can also show you the value of a variable. For example, the following `echo` call prints the `PATH` variable:

```
echo "$PATH"
```

Why do we need the dollar sign `$` before the variable name? The `echo` command receives the string on input and outputs it. For example, this `echo` call prints the text "123":

```
echo 123
```

The dollar sign before a word tells Bash that it is a variable name. The interpreter handles it differently than a regular word. When Bash encounters a variable name in a command, it checks its variable list. If the name presents there, Bash inserts the variable value into the command. Otherwise, the interpreter places an empty string there.



Enclosing variable names in double-quotes is a good practice[209]. This way, you avoid potential errors. There is an example. Bash replaces the variable name by its value. The value contains **control characters**[210]. The interpreter handles these characters. As a result, the inserted variable's value differs from one that is stored in memory. This effect leads to incorrect behavior of the program. Double-quotes prevent Bash from processing strings.

Let's come back to the `echo` command that prints the `PATH` variable. Figure 2-22 shows this output.

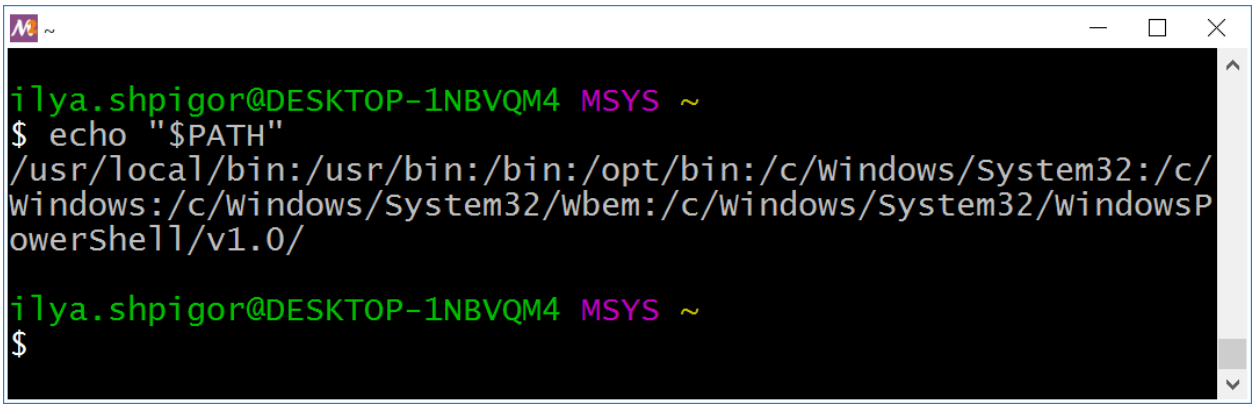



**Figure 2-22. The value of the `PATH` variable**

What does this line mean? It is a list of paths separated by colons. If you write each path on a new line, you get the following list:

[209] https://www.tldp.org/LDP/abs/html/quotingvar.html
[210] https://en.wikipedia.org/wiki/Control_character

```
/usr/local/bin
/usr/bin
/bin
/opt/bin
/c/Windows/System32
/c/Windows
/c/Windows/System32/Wbem
/c/Windows/System32/WindowsPowerShell/v1.0/
```

The format of the `PATH` variable raises questions. Why does Bash use colons as delimiters instead of line breaks[211]? Line breaks make it easy to read the list. The reason is the specific behavior of Bash and some utilities when handling line breaks. Colons allow developers to avoid potential problems.

Suppose that you want to locate an executable file of some program on the disk. The `PATH` variable gives you a hint of where to look. Then you can apply the `find` utility and locate the file. For example, the following command searches the executable of the `find` utility:

```
find / -name find
```

The command shows you two locations of the `find` file:

- `/bin`
- `/usr/bin`

Both locations present in the `PATH` variable.

There is a much faster way to locate an executable on the disk. The `type` Bash built-in does it. Call the command and give it a program name. You will get the absolute path to the program's executable. Figure 2-23 shows how it works.

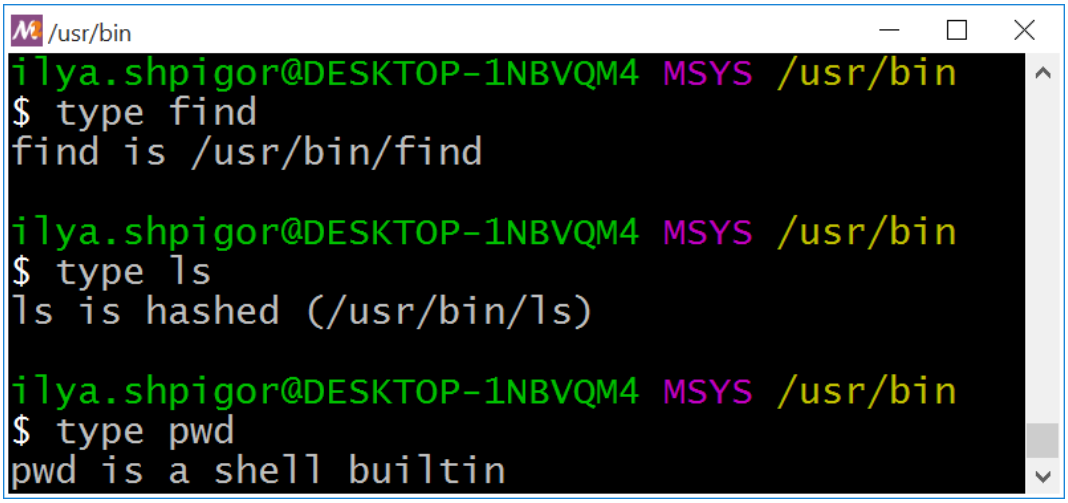



**Figure 2-23. The output of the `type` command**

[211] https://en.wikipedia.org/wiki/Newline

You see that the `/usr/bin` directory stores the executables of `find` and `ls` utilities. The `ls` utility is marked as **hashed**. It means that Bash has remembered its path. When you call `ls`, the interpreter does not search the executable on the disk. Bash uses the stored path and calls the utility directly. If you move the hashed executable, Bash cannot find it anymore.

You can call the `type` command and pass a Bash built-in there. Then `type` tells you that Bash executes this command. Figure 2-23 shows an example of such output for the `pwd` command.

Suppose that you found the executable of the required utility. How do you know the parameters it accepts? Call the utility with the `--help` option. The option prints a brief help. Figure 2-24 shows this help for the `cat` utility.

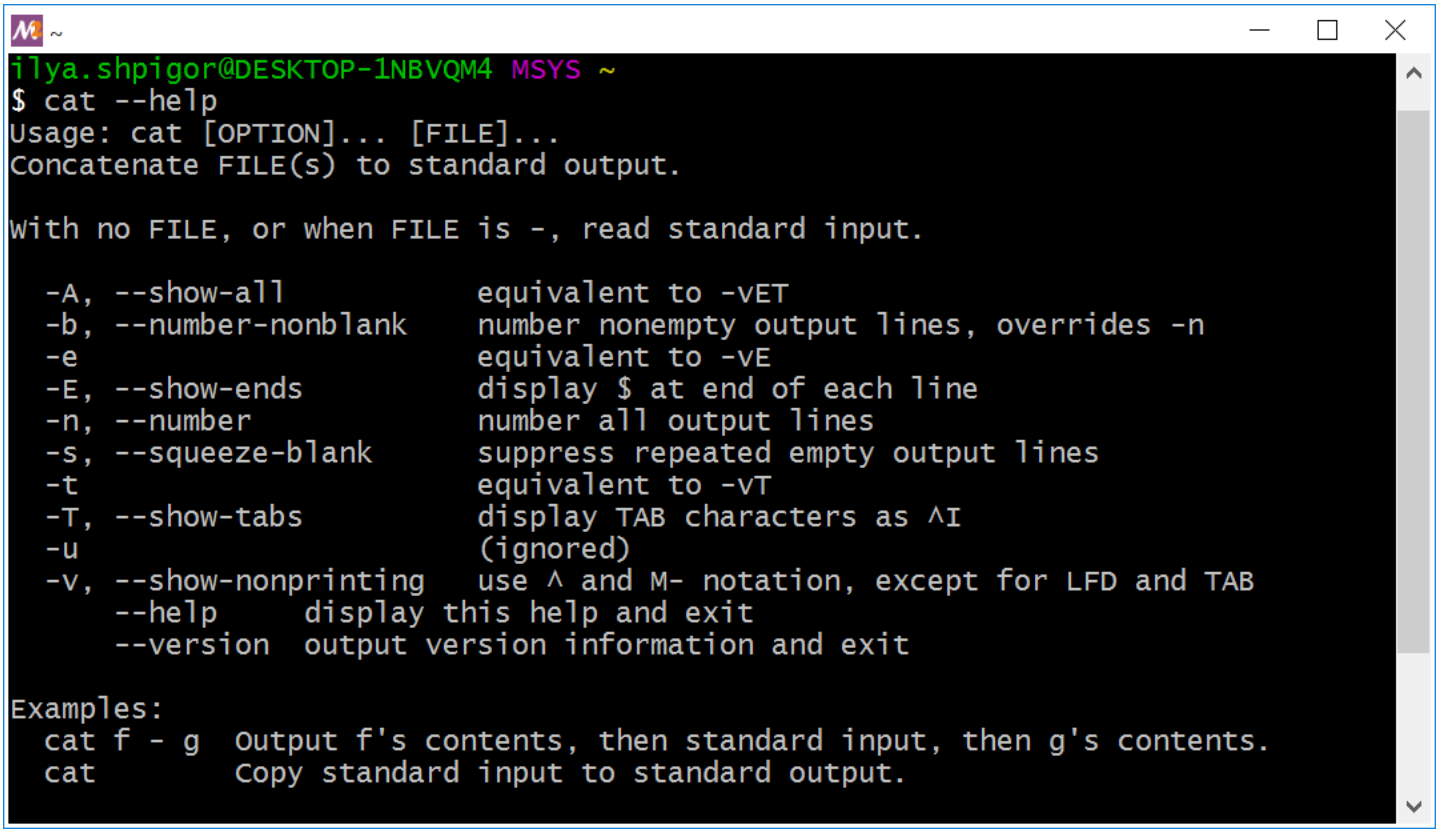



**Figure 2-24. The brief help for the `cat` utility**

If the brief help is not enough, refer to the documentation system called `info`. Suppose you need examples of how to use the `cat` utility. The following command shows them:

```
info cat
```

Figure 2-25 shows the result of the command.

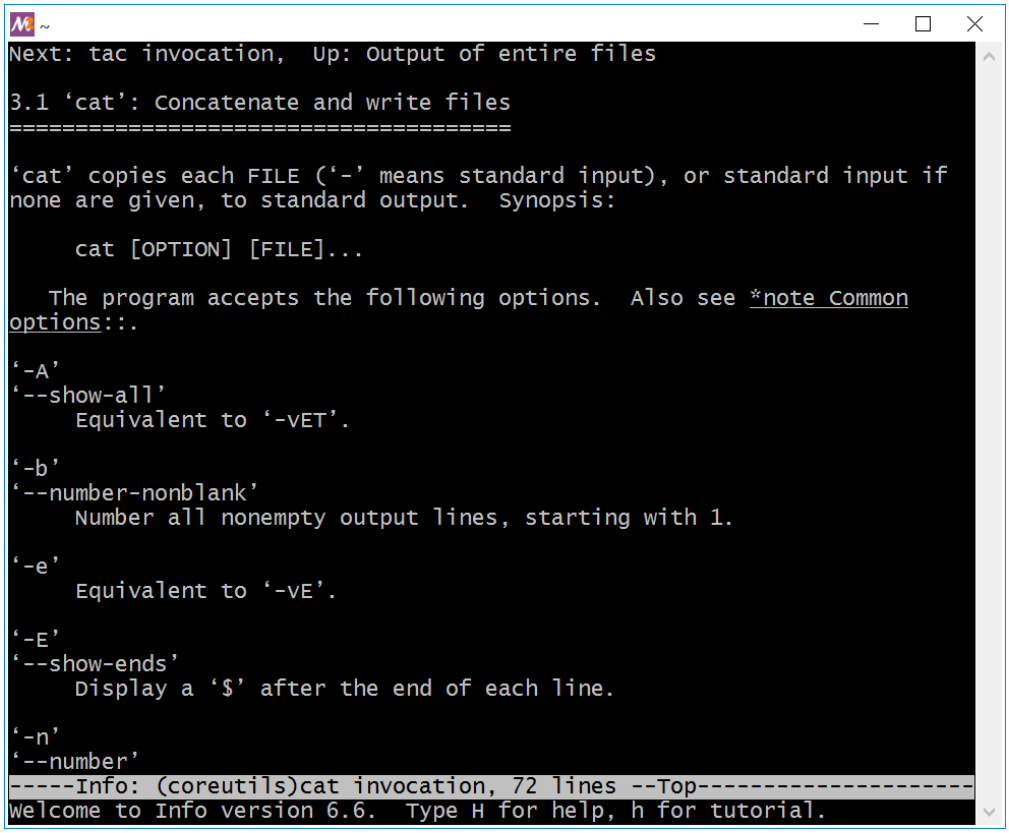



**Figure 2-25. The `info` page for the `cat` utility**

You see a program for reading text documents. Use the arrow keys, PageUp and PageDown to scroll the text. Press the Q key to end the program.

Developers of GNU utilities have created the `info` system. Before that, all Unix distributions used the `man` system. The capabilities of `info` and `man` are similar. The MSYS2 environment uses the `info` system, which is more modern.

Your Linux distribution may use `man` instead of `info`. Use it in the same way as `info`. For example, the following `man` call shows you help for the `cat` utility:

```
man cat
```

When you know which utility solves your task, it is easy to get help. What would you do if you don't know how to solve the task? The best approach is to look for the answer on the Internet. You will find tips there. They are more concise than the manuals for GUI programs. You don't need screenshots and videos that explain each action. Instead, you will find a couple of lines with command calls that do everything you need.

**Exercise 2-5. The documentation system**

```
Find documentation for each of the built-in commands and utilities in Table 2-1.
Check the parameters of the ls and find utilities that we did not consider.
```

# Actions on Files and Directories

You have learned how to find a file or directory on the disk. Now let's discuss what you can do with it. If you have an experience with Windows GUI, you know the following actions with a file system object:

- Create
- Delete
- Copy
- Move or rename

Each of these actions has a corresponding GNU utility. Call them to manage the file system objects. Table 2-7 describes these utilities.

**Table 2-7. Utilities for operating files and directories**

| Utility | Feature | Examples |
|---|---|---|
| `mkdir` | It creates the directory with the specified name and path. | `mkdir /tmp/docs`<br>`mkdir -p tmp/docs/report` |
| `rm` | It deletes the specified file or directory | `rm readme.txt`<br>`rm -rf ~/tmp` |
| `cp` | It copies a file or directory. The first parameter is the current path. The second parameter is the target path. | `cp readme.txt tmp/readme.txt`<br>`cp -r /tmp ~/tmp` |
| `mv` | It moves or renames the file or directory specified by the first parameter. | `mv readme.txt documentation.txt.`<br>`mv ~/tmp ~/backup` |

Each of these utilities has the `--help` option. It displays a brief help. Please read it before using the utility the first time. You will find there some modes that this book misses. Refer to the `info` or `man` system if you need more details.

It is time to consider the utilities of Table 2-7.

## mkdir

The `mkdir` utility creates a new directory. Specify its target path in the first parameter of the command. Here is an example `mkdir` call for creating the `docs` directory:

```
mkdir ~/docs
```

We specified the absolute path to the `docs` directory. You can pass the relative path instead. There are two steps to take it:

1. Navigate the home directory.
2. Call the `mkdir` utility there.

Here are the corresponding commands:

```
cd ~
mkdir docs
```

The utility has an option `-p`. It creates the nested directories. Here is an example of when to use it. Suppose you want to move the documents into the `~/docs/reports/2019` path. However, the `docs` and `reports` directories do not exist yet. If you use `mkdir` in the default mode, you should call it three times to create each of the nested directories. Another option is to call `mkdir` once with the `-p` option like this:

```
mkdir -p ~/docs/reports/2019
```

This command succeeds even if the `docs` and `reports` directories already exist. It creates only the missing `2019` directory in this case.

## rm

The `rm` utility deletes files and directories. Specify the object to delete by its absolute or relative path. Here are examples of `rm` calls:

```
rm report.txt
rm ~/docs/reports/2019/report.txt
```

The first call deletes the `report.txt` file in the current directory. The second one deletes it in the `~/docs/reports/2019` path.

The `rm` utility can remove several files at once. Specify a list of filenames separated by spaces in this case. Here is an example:

```
rm report.txt ~/docs/reports/2019/report.txt
```

If you want to delete dozens of files, listing them all is inconvenient. Use a Bash glob pattern in this case. For example, you need to delete all text files whose names begin with the word “report”. The following `rm` call does it:

```
rm ~/docs/reports/2019/report*.txt
```

When removing a write-protected file, the `rm` utility shows you a warning. You can see how it looks like in Figure 2-26.

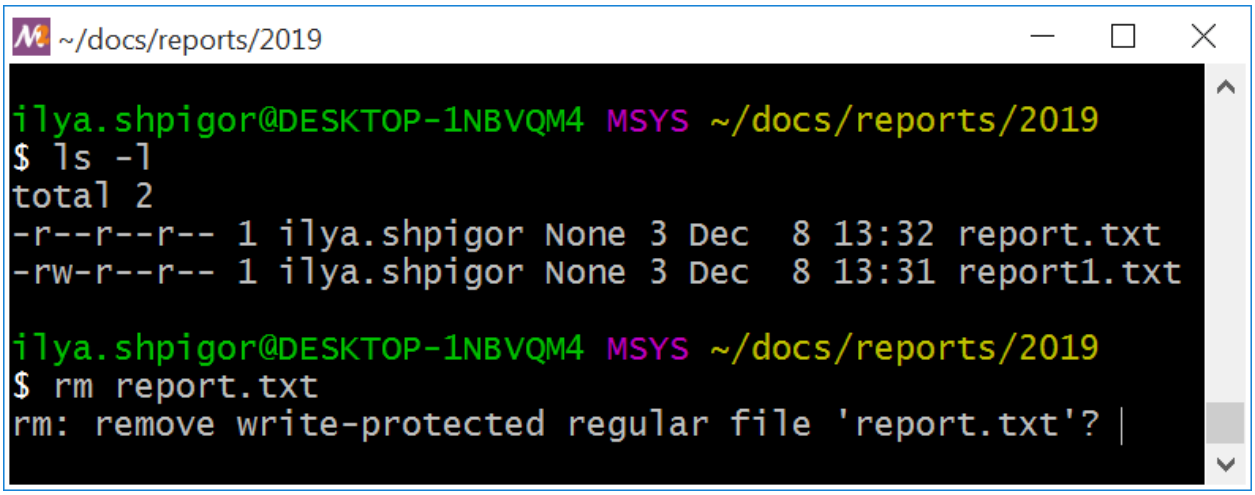



**Figure 2-26. The warning when deleting a write-protected file**

When you see such a warning, there are two options. You can press the Y (short for yes) and Enter. Then the `rm` utility removes the file. Another option is to press N (no) and Enter. It cancels the operation.

If you want to suppress any `rm` warnings, use the `-f` or `--force` option. The utility removes files without confirmation in this case. Here is an example call:

```
rm -f ~/docs/reports/2019/report*.txt
```



The GNU utility options have two forms. The short form consists of one letter and begins with a dash `-`. The full form is a word preceded by a double dash `--`. This option format is recommended by the POSIX standard[212] and its GNU extension[213].

The `rm` utility cannot remove a directory unless you pass one of two possible options there. The first option is `-d` or `--dir`. Use it for removing an empty directory. Here is an example:

[212] https://www.gnu.org/software/libc/manual/html_node/Argument-Syntax.html
[213] https://www.gnu.org/prep/standards/html_node/Command_002dLine-Interfaces.html

```
rm -d ~/docs
```

If the directory contains files or subdirectories, use the `-r` or `--recursive` option to remove it. Such a call looks like this:

```
rm -r ~/docs
```

The `-r` option removes empty directories too. Therefore, you can always use the `-r` option when calling `rm` for a directory.

## cp and mv

The `cp` and `mv` utilities copy and move file system objects. Their interfaces are almost the same. Specify the target file or directory in the first parameter. Pass the new path for the object in the second parameter.

Here is an example. You want to copy the `report.txt` file. First, you should come to its directory. Second, call the `cp` utility this way:

```
cp report.txt report-2019.txt
```

This command creates the new file `report-2019.txt` in the current directory. Both `report-2019.txt` and `report.txt` files have the same contents.

Suppose that you do not need the old file `report.txt`. You can remove it with the `rm` utility after copying. The second option is to combine copying and removing in a single command. The `mv` utility does that:

```
mv report.txt report-2019.txt
```

This command does two things. First, it copies the `report.txt` file with the new name `report-2019.txt`. Second, it removes the old file `report.txt`.

Both `cp` and `mv` utilities accept relative and absolute paths. For example, let's copy a file from the home directory to the `~/docs/reports/2019` path. Here is the command for that:

```
cp ~/report.txt ~/docs/reports/2019
```

This command copies the `report.txt` file into the `~/docs/reports/2019` directory. The copy has the same name as the original file.

You can repeat the copying command with relative paths. Come to the home directory and call the `cp` utility there. The following commands do it:

```
cd ~
cp report.txt docs/reports/2019
```

When copying a file between directories, you can specify the copy name. Here is an example:

```
cp ~/report.txt ~/docs/reports/2019/report-2019.txt
```

This command creates a file copy with the `report-2019.txt` name.

Moving files works the same way as copying. For example, the following command moves the `report.txt` file:

```
mv ~/report.txt ~/docs/reports/2019
```

The following command moves and renames the file at once:

```
mv ~/report.txt ~/docs/reports/2019/report-2019.txt
```

You can rename a directory using the `mv` utility too. Here is an example:

```
mv ~/tmp ~/backup
```

This command changes the name of the `tmp` directory to `backup`.

The `cp` utility cannot copy a directory when you call it in the default mode. Here is an example. Suppose you want to copy the directory `/tmp` with the temporary files to the home directory. You call `cp` this way:

```
cp /tmp ~
```

This command fails.

You must add the `-r` or `--recursive` option when copying directories. Then the `cp` utility can handle them. This is the correct command for our example:

```
cp -r /tmp ~
```

Suppose you copy or move a file. If the target directory already has the file with the same name, the `cp` and `mv` utilities ask you to confirm the operation. If you press the Y and Enter keys, utilities overwrite the existing file.

There is an option to suppress the confirmation when copying and moving files. Use the `-f` or `--force` option. It forces `cp` and `mv` utilities to overwrite the existing files. Here are examples:

```
cp -f ~/report.txt ~/tmp
mv -f ~/report.txt ~/tmp
```

Both commands overwrite the existing `report.txt` file in the `tmp` directory. You do not need to confirm these operations.

**Exercise 2-6. Operations with files and directories**

```
Handle your photos from the past three months using the GNU utilities.
Make a backup before you start.
Separate all photos by year and month.
You should get a directory structure like this:

~/
  photo/
        2019/
             11/
             12/
        2020/
             01/
```

## File System Permissions

Each utility of Table 2-7 checks the **file system permissions**[214] before acting. These permissions define if you are allowed to operate the target object. Let's consider this file system mechanism in detail.

Permissions restrict the user actions on the file system. The OS tracks these actions and checks their allowance. Each user can access only his files and directories, thanks to this feature. It also restricts access to the OS components.

The permissions allow several people to share one computer. This workflow was widespread in the 1960s before appearing PCs. Hardware resources were expensive at that time. Therefore, several users have to operate with one computer.

Today most users have their own PC or laptop. However, the file system permissions are still relevant. They protect your Linux or macOS system from unauthorized access and malware.

Have a look at Figure 2-26 again. There you see the output of the `ls` utility with the `-l` option. It is the table. Each row corresponds to a file or directory. The columns have the following meaning:

1. Permissions to the object.
2. The number of hard links to the file or directory.
3. Owner.

[214] https://en.wikipedia.org/wiki/File-system_permissions

4. Owner's group.
5. The object's size in bytes.
6. Date and time of the last change.
7. File or directory name.

The permissions to the file `report.txt` equal the "-r–r–r–" string. What does it mean?

Unix stores permissions to a file object as a **bitmask**[215]. The bitmask is a positive integer. When you store it in computer memory, the integer becomes a sequence of zeros and ones. Each bit of the mask keeps a value that is independent of the other bits. Therefore, you can pack several values into a single bitmask.

What values can you store in a bitmask? This is a set of object's properties, for example. Each bit of the mask corresponds to one property. If it is present, the corresponding bit equals one. Otherwise, the bit equals zero.

Let's come back to the file access rights. We can represent these rights as the following three attributes:

1. Read permission.
2. Write permission.
3. Permission to execute.

If you apply a mask of three bits, you can encode these attributes there. Suppose a user has full access to the file. He can read, change, copy, remove or execute it. It means that the user has read, write, and execute permissions to the file. The writing permission allows changing the file and removing it. Therefore, the file permissions mask looks like this:

```
111
```

Suppose the user cannot read or execute the file. The first bit of the mask corresponds to the read access. The third bit is execution permission. When you set both these bits to zero, you restrict the file access. Then you get the following mask:

```
010
```

You should know the meaning of each bit in the mask if you want to operate it properly. The mask itself does not provide this information.

Our mask with three bits is a simplified example of file permissions. The permissions in Unix follow the same idea. However, bitmasks there have more bits. The `ls` utility prints these access rights to the `report.txt` file:

[215] https://en.wikipedia.org/wiki/Mask_(computing)

```
-r--r--r--
```

This string is the bitmask. Here dashes correspond to zeroed bits. Latin letters match the set bits. If you follow this notation, you can convert the "-r–r–r–" string to the 0100100100 mask. If all bits of the mask equal ones, the `ls` prints it like the "drwxrwxrwx" string.

The Unix permissions string has four parts. Table 2-8 explains their meaning.

**Table 2-8. Parts of the permissions string in Unix**

| d | rwx | rwx | rwx |
|---|---|---|---|
| The directory attribute. | The permissions of the object's owner. The owner is a user who has created the object. | The permissions of the user group that is attached to the object. By default, it is the group to which the owner belongs. | The permissions of all other users except the owner and the group attached to the object. |



The `groups` utility prints the list of groups that the current user belongs.

You can imagine the Unix permissions as four separate bitmasks. Each of them corresponds to one part of Table 2-8. All bitmasks have a size of four bits. Using this approach, you can represent the "-r–r–r–" string this way:

```
0000 0100 0100 0100
```

The Latin letters in the Unix permissions have special meaning. First of all, they match bits that are set to one. The position of each bit defines the allowed action to the object. You do not need to remember the meaning of each position. The Latin letters give you a hint. For example, "r" means read access. Table 2-9 explains the rest letters.

**Table 2-9. Letters in the Unix permissions string**

| Letter | Meaning for a file | Meaning for a directory |
|---|---|---|
| d | If the first character is a dash instead of `d`, the permissions correspond to a file. | The permissions correspond to a directory. |
| r | Access for reading. | Access for listing the directory contents. |
| w | Access for writing. | Access for creating, renaming or deleting objects in the directory. |
| x | Access for executing. | Access for navigating to the directory and accessing its nested objects. |

**Table 2-9. Letters in the Unix permissions string**

| Letter | Meaning for a file | Meaning for a directory |
|---|---|---|
| — | The corresponding action is prohibited. | The corresponding action is prohibited. |

Suppose that all users of the system have full access to the file. Then its permissions look like this:

```
-rwxrwxrwx
```

If all users have full access to a directory, the permissions look this way:

```
drwxrwxrwx
```

The only difference is the first character. It is `d` instead of the dash.

Now you know everything to read the permissions of Figure 2-26. It shows two files: `report.txt` and `report1.txt`. All users can read the first one. Nobody can modify or execute it. All users can read the `report1.txt` file. Only the owner can change it. Nobody can execute it.

We have considered commands and utilities for operating the file system. When you call each of them, you specify the target object. You should have appropriate permissions to the object. Otherwise, your command fails. Table 2-10 shows the required permissions.

**Table 2-10. Commands and required file system permissions for them**

| Command | Required Bitmask | Required Permissions | Comment |
|---|---|---|---|
| `ls` | `r--` | Reading | Applied for directories only. |
| `cd` | `--x` | Executing | Applied for directories only. |
| `mkdir` | `-wx` | Writing and executing | Applied for directories only. |
| `rm` | `-w-` | Writing | Specify the `-r` option for the directories. |
| `cp` | `r--` | Reading | The target directory should have writing and executing permissions. |
| `mv` | `r--` | Reading | The target directory should have writing and executing permissions. |
| Execution | `r-x` | Reading and executing. | Applied for files only. |

## Files Execution

Windows has strict rules for executable files. The file extension defines its type. The Windows loader runs only files with the EXE and COM extensions. These are compiled executable of programs. Besides them, you can run scripts. The script's extension defines the interpreter that launches it. Windows cannot run the script if there is no installed interpreter for it. The possible extensions of the scripts are BAT, JS, PY, RB, etc.

Unix rules for executing files differ from Windows ones. Here you can run any file if it has permissions for reading and executing. Its extension does not matter, unlike Windows. For example, the file called `report.txt` can be executable.

There is no convention for extensions of the executable files in Unix. Therefore, you cannot deduce the file type from its name. Use the `file` utility to get it. The command receives the file path on input and prints its type. Here is an example of calling `file`:

```
file /usr/bin/ls
```

If you launch the command in the MSYS2 environment, it prints the following information:

```
/usr/bin/ls: PE32+ executable (console) x86-64 (stripped to external PDB), for MS Wi\
ndows
```

The output says that the `/usr/bin/ls` file has the PE32[216] type. Files of this type are executable and contain machine code. The Windows loader can run them. The output also shows the bitness of the file "x86-64". It means that this version of `ls` utility works on 64-bit Windows only.

If you run the same `file` command on Linux, it gives another output. For example, it might look like this:

```
/bin/ls: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked, in\
terpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 2.6.32, BuildID[sha1]=d0bc0fb9b\
3f60f72bbad3c5a1d24c9e2a1fde775, stripped
```

This is the executable file with machine code. It has the ELF[217] type. Linux loader can run it. The file bitness "x86-64" is the same as in MSYS2.

We have learned to distinguish executable and non-executable files in the Unix environment. Now let's find out where you can find them.

GNU utilities are part of OS. Therefore, they are available right after installing the system. You do not need to install them separately. Their executable files are located in the `/bin` and `/usr/bin`

[216] https://en.wikipedia.org/wiki/Portable_Executable
[217] https://en.wikipedia.org/wiki/Executable_and_Linkable_Format

directories. The Bash variable `PATH` stores these paths. Now the question is, where can you find newly installed applications?

Windows installs new applications in the `Program Files` and `Program Files (x86)` directories on the system drive. Each application has its own subdirectory. For example, it can be `C:\Program Files (x86)\Notepad++`. The installer program copies executables, libraries, configuration and resource files into that subdirectory. The application requires all these files to work properly. You can specify another installation directory than `Program Files` and `Program Files (x86)`. Then the installer program creates the application subdirectory there.

There are two approaches to install applications in the Unix environment. The first one resembles the Windows way. There is the system directory `/opt`. The installer program creates an application subdirectory with all its files there.

Here is an example. Suppose that you are installing the TeamViewer application. Its installer creates the `/opt/teamviewer` subdirectory. You can find the `TeamViewer` executable there. Developers of proprietary applications prefer this way of installing programs.

Developers of open-source programs follow another approach. An application requires files of various types. Each file type has a separate system directory in Unix. It means that the executable files of all applications occupy the same directory. The documentation for them is in another directory and so on. The POSIX standard dictates the purposes of all system directories.

Table 2-11 explains the purposes of Unix system directories.

**Table 2-11. Unix system directories**

| Directory | Purpose |
|---|---|
| `/bin` | It stores executable files of system utilities. |
| `/etc` | It stores configuration files of applications and system utilities. |
| `/lib` | It stores libraries of system utilities. |
| `/usr/bin` | It stores executable files of user applications. |
| `/usr/lib` | It stores libraries of user applications. |
| `/usr/local` | It stores applications that the user compiled on his own. |
| `/usr/share` | It stores architecture-independent resource files of user applications (e.g. icons). |
| `/var` | It stores files created by applications and utilities while running (e.g. log files). |

Copying all files of the same type into one directory sounds like a controversial solution. Its

disadvantage is the complexity of maintenance. Suppose that the application updates to the next version. It should update all its files in all system directories. If the application misses one of the files, it cannot run anymore.

However, the Unix system directories have an advantage. When you install an application in Windows, it brings all files it needs. There are libraries with subroutines among these files. Some applications require the same libraries to run. When each application has its own copy of the same library, it causes the file system overhead.

The Unix way gets rid of library copies. Suppose that all applications respect the agreement and install their files in the proper system directories. Then applications can locate files of each other. Therefore, they can use the same library if they require it. A single instance of each library is enough for supporting all dependent applications.

Suppose that you have installed a new application (e.g., a browser). Its executable file (for example, `firefox`) comes to the `/usr/bin` path according to Table 2-11. How to run this application in Bash? There are several ways for that:

1. By the name of the executable file.
2. By the absolute path.
3. By the relative path.

Let's consider each way in detail.

You have used the first approach when calling GNU utilities. For example, the following command runs the `find` utility:

```
find --help
```

It launches the `/usr/bin/find` executable file.

Use a similar command to run a newly installed application. Here is an example for the Firefox browser:

```
firefox
```

Why does this command work? The executable file `firefox` is located in the `/usr/bin` system directory. The Bash variable `PATH` stores this path. When Bash receives the "firefox" command, it searches the executable with that name. The shell takes searching paths from the `PATH` variable. This way, Bash finds the `/usr/bin/firefox` file and launches it.

The paths have a specific order in the `PATH` variable. Bash follows this order when searching for an executable. There is an example. Suppose that both `/usr/local/bin` and `/usr/bin` directories contain the `firefox` executable. If the path `/usr/local/bin` comes first in the `PATH` list, Bash always runs the file from there. Otherwise, Bash calls the `/usr/bin/firefox` executable.

The second way to run an application reminds the first one. Instead of the executable filename, you type its absolute path. For example, the following command runs the Firefox browser:

```
/usr/bin/firefox
```

You would need this approach when launching proprietary applications. They are installed in the `/opt` system directory. The `PATH` variable does not contain this path by default. Therefore, Bash cannot find executables there. You can help Bash by specifying an absolute path to the program.

The third approach to run an application is something in between the first and second ways. You use a relative executable path instead of the absolute one. Here is an example for the Firefox browser:

```
cd /usr
bin/firefox
```

The first command navigates to the `/usr` directory. Then the second command launches the browser by its relative path.

Now let's change the first command. Suppose that you navigate the `/opt/firefox/bin` directory. The following try to launch the browser fails:

```
cd /opt/firefox/bin
firefox
```

Bash reports that it cannot find the `firefox` file. It happens because you are launching the application by the executable filename here. It is the first way to run applications. Bash looks for the `firefox` executable in the paths of the `PATH` variable. However, the application is located in the `/opt` directory, which is not there.

You should specify the relative path to the executable instead of its filename. If the current directory contains the file, mention it in the relative path. The dot symbol indicates the current directory. Thus, the following commands run the browser properly:

```
cd /opt/firefox/bin
./firefox
```

Now Bash follows your hint and searches the executable in the current directory.

Suppose that you have installed a new application. You are going to use it in your daily work frequently. Add its installation path to the `PATH` variable in this case. The following steps explain how to do it for the `/opt/firefox/bin` directory:

1. Navigate the home directory:

```
cd ~
```

2. Print its Windows path:

```
pwd -W
```

3. Open the file `~/.bash_profile` in the text editor (for example, Notepad).
4. Add the following line at the end of the file:

```
PATH="/opt/firefox/bin:${PATH}"
```

You have redefined the `PATH` variable this way. The next chapter considers Bash variables in detail. There you will know how to operate them.

Restart the MSYS2 terminal for applying changes. Now you can run the browser by the name. Bash finds the corresponding executable in the `/opt/firefox/bin` path correctly.

# Extra Bash Features

We have learned the built-in Bash commands and GNU utilities for operating on the file system. Now you can run a program and copy a file in the command-line. The graphical interface provides the same features. If you are solving such simple tasks, the interface does not matter.

The Bash interpreter provides several features that the GUI does not have. Thanks to them, some tasks are faster to perform in the command-line interface. If you have such a task, knowing Bash saves you time thanks to automation.

We will take a look at the following Bash features:

1. I/O redirection.
2. Pipelines.
3. Logical operators.

## Unix Philosophy

Douglas McIlroy[218] is one of the Unix developers. He summarized the philosophy of this OS in three points:

1. Write programs that do one thing and do it well.
2. Write programs to work together.
3. Write programs to handle text streams, because that is a universal interface.

[218] https://en.wikipedia.org/wiki/Douglas_McIlroy

The plain text format is the cornerstone of the Unix philosophy. It is the foundation for the first two points.

The text format allows programs to share data easily. Suppose that two developers independently of each other wrote two utilities. Both utilities accept text data on input. They print results in text format too. You have a task. If you combine two utilities, you solve it. Because of the text interface, it is quite simple to do. You have to pass the output of the first utility to the input of the second one.

When it is easy for programs to interact, there is no need to overload them with extra features. For example, you write a program to copy files. It does the job well. But eventually, you realize that it lacks a search function. With this feature, it would be faster to find and copy the files at once. Then you decide that it is convenient to create directories and copy files by one program. This example shows that the requirements for a self-contained application grow rapidly.

When applications work together, each solves a single task only. If you need an extra feature, you call the corresponding special utility. You don't have to add this feature to your application. The feature is already available and well tested. Just call an external utility that does what you need.

## I/O Redirection

GNU utilities follow the Unix philosophy. They use a text format for input and output. Therefore, they are just as easy to combine as Unix utilities.

When combining GNU utilities, there is a task to pass text data between them. The task has several solutions.

Suppose the output of a utility fits one line. You need to pass it to another utility. Using the clipboard is the simplest solution here. Here are the steps to take it:

1. Select the utility output with the mouse.
2. Type the command to call another utility.
3. Paste the contents from the clipboard at the end.
4. Launch the command.

This simple method does not work for copying multiple lines. When you paste them, Bash handles line breaks as commands delimiter. It executes the command right after receiving the delimiter. Because of this, the shell loses some copied lines.

Another solution is to use the file system. Create a temporary file to save the utility output. Then pass the file name to another utility. It will read its contents and retrieve the data. This approach is more convenient than the clipboard for two reasons:

1. There is no limit on the number of lines to transfer.
2. There are no manual operations with the clipboard.

Bash has a mechanism that redirects the command output to the file. The same mechanism redirects data from the file to the command input. It means that your application does need a feature for interacting with the file system. Instead, it should support the text data format on input and output. Bash does all the other work of redirecting that data.

Let's look at an example. Suppose that you are looking for the files on the disk. The search result has to be saved into a file. To solve this task, use the `find` utility and the redirect operator `1>`. Then the utility call looks like this:

```
find / -path */doc/* -name README 1> readme_list.txt
```

The command creates the `readme_list.txt` file in the current directory. It contains the output of the `find` utility. The output looks the same in the file as it is printed on the screen. If the current directory has the `readme_list.txt` file already, the command overwrites it.

What does the `1>` operator mean? It is redirection[219] of standard output stream. There are three standard streams[220] in the Unix environment. Table 2-12 explains them.

**Table 2-12. POSIX standard streams**

| Number | Name | Purpose |
|---|---|---|
| 0 | Standard input stream (stdin). | A program receives input data from this stream. By default, it comes from an input device like a keyboard. |
| 1 | Standard output stream (stdout). | A program outputs data there. The terminal window prints it by default. |
| 2 | Standard error stream (stderr). | A program outputs the error messages there. The terminal window prints it by default. |

A program operates in the software environment that the OS provides. A thread is a communication channel between the program and the environment.

Early Unix systems have used physical channels for input and output data. The input was tied to the keyboard. The same way the output was tied to the monitor. Streams were introduced as abstraction[221] over these channels. The abstraction makes it possible to work with different objects using the same algorithm. It allows replacing input from a real device with input from a file. Similarly, printing to the screen can be replaced by output to a file. At the same time, the same OS code handles these I/O operations.

The purpose of the input and output streams is clear. But the error stream causes questions. Why do we need it? Imagine that you run the `find` utility to search for files. You do not have access to some directories. When the `find` utility reads their contents, it is unavailable. The utility prints an error message in this case.

[219] https://en.wikipedia.org/wiki/Redirection_(computing)
[220] https://en.wikipedia.org/wiki/Standard_streams
[221] https://en.wikipedia.org/wiki/Abstraction_layer

Now imagine that the utility found many files. It is easy to miss error messages in a huge file list. Separating the output and error streams helps in this case. If you redirect the output stream to the file, the utility prints error messages only on the screen.

Use the `2>` operator to redirect the standard error stream. Here is an example of the `find` utility call with this operator:

```
find / -path */doc/* -name README 2> errors.txt
```

The number before the angle bracket in the operator means the number of the redirected stream. For example, the `2>` operator redirects stream number two.

Use the `0<` operator to redirect the standard input stream. Here is an example for searching the "Bash" pattern in the `README.txt` file:

```
grep "Bash" 0< README.txt
```

This example is not entirely correct. The command uses the `grep` utility interface that handles the standard input stream. But `grep` can read the contents of a specified file. It is always better to pass the filename to the utility. Here is an example:

```
grep "Bash" README.txt
```

Let's take a more complicated example. Some Bash manuals recommend the `echo` command to print the contents of the file. For example, the command for printing the`README.txt` file looks like this:

```
echo $( 0< README.txt )
```

Here `echo` receives the output of the following command:

```
0< README.txt
```

**Command substitution** is substituting the command result where it was called. When the interpreter encounters the `$(` and `)` characters, it executes the command enclosed between them and substitutes its output.

Because of the command substitution, Bash executes our `echo` call in two steps:

1. Pass the contents of the `README.txt` file to the standard input stream.
2. Print data from the standard input stream with the `echo` command.

Please take into account the execution order of the substituted commands. The interpreter executes commands and inserts their results in the order they follow. Only when all substitutions are done, Bash executes the resulting command in its entirety.

The next `find` call leads to the error due to a commands order mistake:

```
$ find / -path */doc/* -name README -exec echo $(0< {}) \;
```

It prints the following error message:

```
bash: {}: No such file or directory
```

This command should print the contents of all the files that the `find` utility found. But we get an error from Bash instead.

The problem happens because the "`0<` to the standard input stream. But there is no file with such a name. We expect the `find` utility substitutes the brackets " command.

If you replace the `echo` command by the `cat` utility, it solves the problem. The `find` call would look like this:

```
find / -path */doc/* -name README -exec cat {} \;
```

This command prints the contents of all found files.

The redirection operators of standard input and output streams are used frequently. Therefore, developers have added short forms for them:

- The `<` operator redirects the input stream.
- The `>` operator redirects the output stream.

The `find` call with the short form of stream redirection looks like this:

```
find / -path */doc/* -name README > readme_list.txt
```

Here is an example of the `echo` call with the short redirection form:

```
echo $( < README.txt )
```

Suppose that you redirect the standard output stream to a file. This file already exists and its contents should stay. In that case, append the command output to the end of the file. The `>>` operator does it.

For example, your computer has installed applications in the `/usr` and `/opt` system paths. Then the following two calls find their README files:

```
1 find /usr -path */doc/* -name README > readme_list.txt
2 find /opt -name README >> readme_list.txt
```

The first `find` call creates the `readme_list.txt` file and writes the result there. If the file already exists, the command overwrites its contents. The second `find` call appends its result to the end of `readme_list.txt`. If the file does not exist, the `>>` operator creates it.

The full form of the `>>` operator looks like `1>>`. If you want to redirect the error stream without overwriting the file, use the `2>>` operator.

Sometimes you need to redirect both the output and the error streams to the same file. The `&>` and `&>>` operators do it. The first operator overwrites an existing file. The second one appends data to its end. Here is an example:

```
find / -path */doc/* -name README &> result_and_errors.txt
```

This command works in Bash properly. But it can fail in other shells. If you want to use the POSIX standard features only, use the `2>&1` operator. Here is an example:

```
find / -path */doc/* -name README > result_and_errors.txt 2>&1
```

This redirection is called **stream duplicating**. Use it for redirecting both output and error streams to the same target.

Be careful when using streams duplicating. It's easy to make a mistake and mix up the operator order in a command. If you work in Bash, always prefer the `&>` and `&>>` operators.

Here is an example of the error with duplicating streams:

```
find / -path */doc/* -name README 2>&1 > result_and_errors.txt
```

The command outputs the error stream data on the screen. However, we expect that these data appear in the `result_and_errors.txt` file. The problem happens because the `2>&1` operator has the wrong position in the command.

The POSIX standard has the concept of **file descriptor**[222]. The descriptor is a pointer to a file or communication channel. The descriptor works as an abstraction that simplifies handling of the streams.

When you start a program, descriptors of output and error streams point to the terminal window. You can change it and associate them with the file. The streams descriptors point to that file in this case. The BashGuide article[223] describes this mechanism in detail.

Let's go back to our `find` utility call. Bash applies redirection operators one by one from left to right. Table 2-13 shows this order for our example.

[222] https://en.wikipedia.org/wiki/File_descriptor
[223] http://mywiki.wooledge.org/BashFAQ/055

**Table 2-13. The order for applying redirection operators**

| Number | Operation | Result |
|---|---|---|
| 1 | `2>&1` | Now the error stream points to the same target as the output stream. In our case, the target is the terminal window. |
| 2 | `> result_and_errors.txt` | Now the output stream points to the file `result_and_errors.txt`. But the error stream is still associated with the terminal window. |

Let's fix the mistake in the `find` call. Changing the order of the redirection operators does it. The redirection of the output stream should come first. Then the duplicating streams should take place. Here is the resulting command:

```
find / -path */doc/* -name README > result_and_errors.txt 2>&1
```

The output stream points to a file in this command. The error stream points to the same file.

You can specify redirection operators one after another if the output and error streams should point to different files. Here is an example:

```
find / -path */doc/* -name README > result.txt 2> errors.txt
```

## Pipelines

It is inconvenient to create temporary files for sharing data between programs. Managing these files takes extra effort. You should remember their paths and removing them after usage.

Unix environment provides an alternative solution. It is called **pipeline**[224]. This mechanism is more convenient than temporary files. It shares data between programs by passing messages without using the file system.

Let us consider an example. Suppose that you need information about the Bash license[225]. You can find this information in the Bash documentation. The following `grep` utility call does it:

```
grep -R "GNU" /usr/share/doc/bash
```

Another solution is to search license information on the `info` help page. The pipeline allows you to connect the output of one program with the input of another one. If we apply the pipeline, we can call the `grep` utility to process the `info` page. The following command does it:

```
info bash | grep -n "GNU"
```

[224] https://en.wikipedia.org/wiki/Pipeline_(Unix)
[225] https://en.wikipedia.org/wiki/Software_license

The `info` utility sends its result to the output stream. Then there is the vertical bar `|` symbol. It means the pipeline. The pipeline transfers the command's output on the left side to the command's input on the right side. This way, the `grep` utility receives the Bash `info` page. The utility searches the lines with the word "GNU" there. The nonempty command output means that Bash has the GNU GPL[226] license.

There is the `-n` option in the `grep` call. The option adds the line numbers in the `grep` output. It helps to find a particular place in the file.

## du

Let's take a more complex example with pipelines. The `du` utility evaluates disk space usage. Run it without parameters in the current directory. The utility **recursively** passes through all subdirectories and prints the occupied space by each of them.

Recursive traversal means repeating visiting of subdirectories. The traversal algorithm looks like this:

1. Check the contents of the current directory.
2. If there is an unvisited subdirectory, go to it and start from the 1st step of the algorithm.
3. If all subdirectories are visited, go to the parent directory and start from the 1st step of the algorithm.
4. If it is impossible to go to the parent directory, finish the algorithm.

Following this algorithm, we bypass all subdirectories from the selected file system point. It is a universal traversal algorithm. You can add any action to it for processing each subdirectory. The action of the `du` utility is calculating disk space usage.

The algorithm of the `du` utility looks like this:

1. Check the contents of the current directory.
2. If there is an unvisited subdirectory, go to it and start from the 1st step of the algorithm.
3. If all subdirectories are visited:
   3.1 Calculate and display the disk space occupied by the current directory.
   3.2 Go to the parent directory.
   3.3 Start from the 1st step of the algorithm.
4. If it is impossible to go to the parent directory, finish the algorithm.

The `du` utility takes a path to the file or directory on input. In the case of the file, the utility prints its size. For the directory, the utility traverses its subdirectories.

Here is the `du` call for the system path `/usr/share`:

[226] https://en.wikipedia.org/wiki/GNU_General_Public_License

```
du /usr/share
```

Here is a clipped example of the command output:

```
261     /usr/share/aclocal
58      /usr/share/awk
3623    /usr/share/bash-completion/completions
5       /usr/share/bash-completion/helpers
3700    /usr/share/bash-completion
2       /usr/share/cmake/bash-completion
2       /usr/share/cmake
8       /usr/share/cygwin
1692    /usr/share/doc/bash
85      /usr/share/doc/flex
...
```

We get a table that has two columns. The right column shows the subdirectory. The left column shows the number of occupied bites.

If you want to add information about files in the `du` output, use the `-a` option. Here is an example:

```
du /usr/share -a
```

The `-h` option makes the output of the `du` utility clearer. The option converts the number of bytes into kilobytes, megabytes, and gigabytes.

Suppose that we should evaluate all HTML files in the `/usr/share` path. The following command prints that statistics:

```
du /usr/share -a -h | grep "\.html"
```

Here the pipeline transfers the `du` output to the `grep` input. Then `grep` prints the lines that contain the ".html" pattern.

The backslash \ escapes the dot in the ".html" pattern. The dot means a single occurrence of any character. If you specify the ".html" pattern, the `grep` utility finds extra files (like `pod2html.1perl.gz`) and subdirectories (like `/usr/share/doc/pcre/html`). When you escape the dot, the `grep` utility treats it as a dot character.

The pipeline combines two commands in our example with `du` and `grep`. But the number of combined commands is not limited. For example, you can sort the found HTML files in descending order. The `sort` utility does this job. Here is the example command:

```
du /usr/share -a -h | grep "\.html" | sort -h -r
```

By default, the `sort` utility sorts the strings in ascending **lexicographic order**[227]. For example, there is a file that contains the following strings:

```
abc
aaaa
aaa
dca
bcd
dec
```

If you call the `sort` utility without options, it places the file's lines in the following order:

```
aaa
aaaa
abc
bcd
dca
dec
```

The `-r` option reverts the order. If you apply this option, the utility prints these lines:

```
dec
dca
bcd
abc
aaaa
aaa
```

The `du` utility prints file sizes in the first column. The `sort` utility operates this column. The file sizes are numbers. Therefore, we cannot place them in order properly with lexicographic sorting. An example will help us to understand the issue.

Suppose there is a file with the following numbers:

```
3
100
2
```

Lexicographical sort of the file produces the following result:

[227] https://en.wikipedia.org/wiki/Lexicographic_order

```
100
2
3
```

The algorithm put the number 100 before 2 and 3. It assumes that 100 is less than other numbers. It happens because the ASCII code of character 1 is smaller than the codes of characters 2 and 3. Use the `-n` option for sorting integers by their values.

We have an example command with two pipelines. The `du` utility has the `-h` option there. The option converts bytes into larger memory units. The `sort` utility can process them only if you call it with the same `-h` option.

You can combine pipelines with stream redirection. Let's save the filtered and sorted output of the `du` utility to the file. The following command does it:

```
du /usr/share -a -h | grep "\.html" | sort -h -r > result.txt
```

The command writes its result into the `result.txt` file.

Suppose you are combining pipelines and stream redirection. You want to write the output stream into the file and pass it to another utility simultaneously. How can you do that? Bash does not have a built-in mechanism for this task. But the special `tee` utility does it. Here is an example:

```
du /usr/share -a -h | tee result.txt
```

The command prints the `du` utility result on the screen. It writes the same result into the `result.txt` file. It means that the `tee` utility duplicates its input stream to the specified file and the output stream. The utility overwrites the contents of `result.txt`. Use the `-a` option if you need to append data to the file instead of overwriting it.

Sometimes you want to check the data flow between commands in a pipeline. The `tee` utility helps you in this case. Just call the utility between the commands in the pipeline. Here is an example:

```
du /usr/share -a -h | tee du.txt | grep "\.html" | tee grep.txt | sort -h -r > resul\
t.txt
```

It stores the output of each command in the pipeline to the corresponding file. These intermediate results are useful for debugging. The `result.txt` file still contains the final result of the whole command.

## xargs

The `find` utility has the `-exec` parameter. It calls a command for each found object. This behavior resembles a pipeline: `find` passes its result to another program. These mechanisms look similar but their internals differs. Choose an appropriate mechanism depending on your task.

Let's look at how the `find` utility performs the `-exec` action. Some program performs this action. The built-in `find` interpreter runs it. The interpreter passes to the program whatever the `find` utility has found. Note that the Bash interpreter is not involved in the `-exec` call. Therefore, you cannot use the following Bash features in the call:

- built-in Bash commands
- functions
- pipelines
- stream redirection
- conditional statements
- loops

Try to run the following command:

```
find ~ -type f -exec echo {} \;
```

The `find` utility calls the `echo` Bash built-in here. It works correctly. Why? Actually, `find` calls another utility with the same name `echo` as the Bash command. Unix environment provides several utilities that duplicate Bash built-ins. You can find them in the `/bin` system path. For example, there is the `/bin/echo` file there.

Some tasks require the Bash features in the `-exec` action. There is a trick to access them in this case. Run the interpreter explicitly and pass a command to it. Here is an example:

```
find ~ -type f -exec bash -c 'echo {}' \;
```

This command does the same as the previous one that calls the `echo` utility. It prints the results of the `find` search.

You can apply the pipeline and redirect the `find` output to another command. In this case, you pass the text through the pipeline rather than filenames and directories. Here is an example of such passing:

```
find ~ -type f | grep "bash"
```

The command prints the output like this:

```
/home/ilya.shpigor/.bashrc
/home/ilya.shpigor/.bash_history
/home/ilya.shpigor/.bash_logout
/home/ilya.shpigor/.bash_profile
```

The pipeline passes the `find` output to the `grep` utility input. Then `grep` prints only the filenames where the pattern "bash" occurs.

The `-exec` action behaves in another way. When `find` passes its results to the `-exec` command, it constructs a utility call. The utility receives the names of found files and directories. You can get the same behavior when using the pipeline. Just apply the `xargs` utility for that.

Let's change our command with `find` and `grep`. Now we pass the text to the input stream of the `grep` utility. Then it filters found filenames. Instead of that, we want to filter the contents of the files. In this case, the `grep` utility should receive filenames via command-line parameters. The `xargs` utility does this job:

```
find ~ -type f | xargs grep "bash"
```



This command cannot handle files whose names contain spaces and line breaks. The next section considers how to solve this problem.

Here is the output of the command:

```
/home/ilya.shpigor/.bashrc:# ~/.bashrc: executed by bash(1) for interactive shells.
/home/ilya.shpigor/.bashrc:# The copy in your home directory (~/.bashrc) is yours, p\
lease
/home/ilya.shpigor/.bashrc:# User dependent .bashrc file
/home/ilya.shpigor/.bashrc:# See man bash for more options...
/home/ilya.shpigor/.bashrc:# Make bash append rather than overwrite the history on d\
isk
/home/ilya.shpigor/.bashrc:# When changing directory small typos can be ignored by b\
ash
...
```

The `xargs` utility constructs a command from parameters that it receives on the input stream. The utility takes two things on the input: parameters and text from the stream. The parameters come in the first place in the constructing command. Then all data from the input stream follows.

Let's come back to our example. Suppose the first file that the `find` gets is `~/.bashrc`. Here is the `xargs` call that receives the file via the pipeline:

```
xargs grep "bash"
```

The utility receives two command-line parameters in this call: `grep` and "bash". Therefore, the constructed command starts with these two words:

```
grep "bash"
```

Then `xargs` appends the text from the input stream to the command. There is the filename `~/.bashrc` in the stream. Therefore, the constructed command looks like this:

```
grep "bash" ~/.bashrc
```

The `xargs` utility does not call Bash for executing the constructed command. It means that the command has the same restrictions as the `-exec` action of the `find` utility. No Bash features are allowed there.

The `xargs` utility puts all data from the input stream at the end of the constructed command. In some cases, you want to change the position of these data. For example, you want to put them at the beginning of the command. The `-I` parameter of `xargs` does that.

Here is an example. Suppose that you want to copy the found files to the user's home directory. The `cp` utility does copying. But you should construct its call properly by the `xargs` utility. The following command does it:

```
find /usr/share/doc/bash -type f -name "*.html" | xargs -I % cp % ~
```

The `-I` parameter changes the place where the `xargs` utility puts the filenames. In our case, the parameter points to the position of the percent sign `%` for that.

The `xargs` utility calls `cp` for each line that it receives via the pipeline. Thus, it constructs the following two commands:

```
1 cp /usr/share/doc/bash/bash.html /home/ilya.shpigor
2 cp /usr/share/doc/bash/bashref.html /home/ilya.shpigor
```

The `-t` option of the `xargs` utility displays the constructed commands before executing them. It can help you with debugging. Here is an example of using the option:

```
find /usr/share/doc/bash -type f -name "*.html" | xargs -t -I % cp % ~
```

We have considered how to combine the `find` utility and pipelines. These examples are for educational purposes only. Do not apply them in your Bash scripts. Use the `-exec` action of the `find` utility instead of pipelines. This way, you avoid issues with processing filenames with spaces and line breaks.

There are few cases when a combination of `find` and pipeline makes sense. One of these cases is the parallel processing of found files.

Here is an example. When you call the `cp` utility in the `-exec` action, it copies files one by one. It is inefficient if your hard disk has a high access speed. You can speed up the operation by running it

in several parallel processes. The `-P` parameter of the `xargs` utility does that. Specify the number of parallel processes in this parameter. They will execute the constructed command.

Suppose your computer processor has four cores. Then you can copy files in four parallel processes. Here is the command to do that:

```
find /usr/share/doc/bash -type f -name "*.html" | xargs -P 4 -I % cp % ~
```

The command copies four files at once. As soon as one of the parallel processes finishes, it handles the next file. This approach can speed up the execution of the command considerably. The gain depends on the configuration of your CPU and hard drive.

There are several GNU utilities for processing text data on the input stream. They work well in pipelines. Table 2-14 shows the most commonly used of these utilities.

**Table 2-14. Utilities for processing the input stream**

| Utility | Description | Examples |
|---|---|---|
| `xargs` | It constructs a command using command-line parameters and data from the input stream. | `find . -type f -print0 \| xargs -0 cp -t ~` |
| `grep` | It searches for text that matches the specified pattern. | `grep -A 3 -B 3 "GNU" file.txt`<br>`du /usr/share -a \| grep "\.html"` |
| `tee` | It redirects the input stream to the output stream and file at the same time. | `grep "GNU" file.txt \| tee result.txt` |
| `sort` | It sorts strings from the input stream in forward and reverse order (`-r`). | `sort file.txt`<br>`du /usr/share \| sort -n -r` |
| `wc` | It count the number of lines (`-l`), words (`-w`), letters (`-m`) and bytes (`-c`) in a specified file or input stream. | `wc -l file.txt`<br>`info find \| wc -m` |
| `head` | It outputs the first bytes (`-c`) or lines (`-n`) of a file or text from the input stream. | `head -n 10 file.txt`<br>`du /usr/share \| sort -n -r \| head -10` |
| `tail` | It outputs the last bytes (`-c`) or lines (`-n`) of a file or text from the input stream. | `tail -n 10 file.txt`<br>`du /usr/share \| sort -n -r \| tail -10` |
| `less` | It is the utility for navigating through text from the standard input stream. Press the Q key to exit. | `less /usr/share/doc/bash/README`<br>`du \| less` |

## Pipelines Pitfalls

Pipelines are a popular feature of the Unix environment. Users apply them frequently. Unfortunately, it is quite simple to make a mistake using pipelines. Let's consider the common mistakes by examples.

You can expect that the following two commands give the same result:

```
find /usr/share/doc/bash -name "*.html"
ls /usr/share/doc/bash | grep "\.html"
```

The commands' results differ in some cases. There is no problem in processing the search pattern differently by the `find` and `grep` utilities. The problem happens when you pass the filenames through the pipeline.

The POSIX standard allows all printable characters in filenames. It means that spaces and line breaks are allowed too. The only forbidden character is **null character**[228] (NULL). This rule can lead to unexpected results.

Here is an example. Create a file in the user's home directory. The filename should contain the line break. This control character matches the `\n` **escape sequence**[229] in ASCII encoding. You can add escape sequences in filenames with the `touch` utility. Call it this way:

```
touch ~/$'test\nfile.txt'
```

The `touch` utility updates the modification time of the file. It is a primary task of the utility. If the file does not exist, `touch` creates it. It is a **side effect**[230] feature of the utility.

Create extra two files for our example: `test1.txt` and `file1.txt`. The following command does that:

```
touch ~/test1.txt ~/file1.txt
```

Now call the `ls` utility in the user's home directory. Pass its output to `grep` via pipeline. Here are the examples:

```
ls ~ | grep test
ls ~ | grep file
```

Figure 2-27 shows the output of these commands.

[228] https://en.wikipedia.org/wiki/Null_character
[229] https://en.wikipedia.org/wiki/Escape_sequence
[230] https://en.wikipedia.org/wiki/Side_effect_(computer_science)

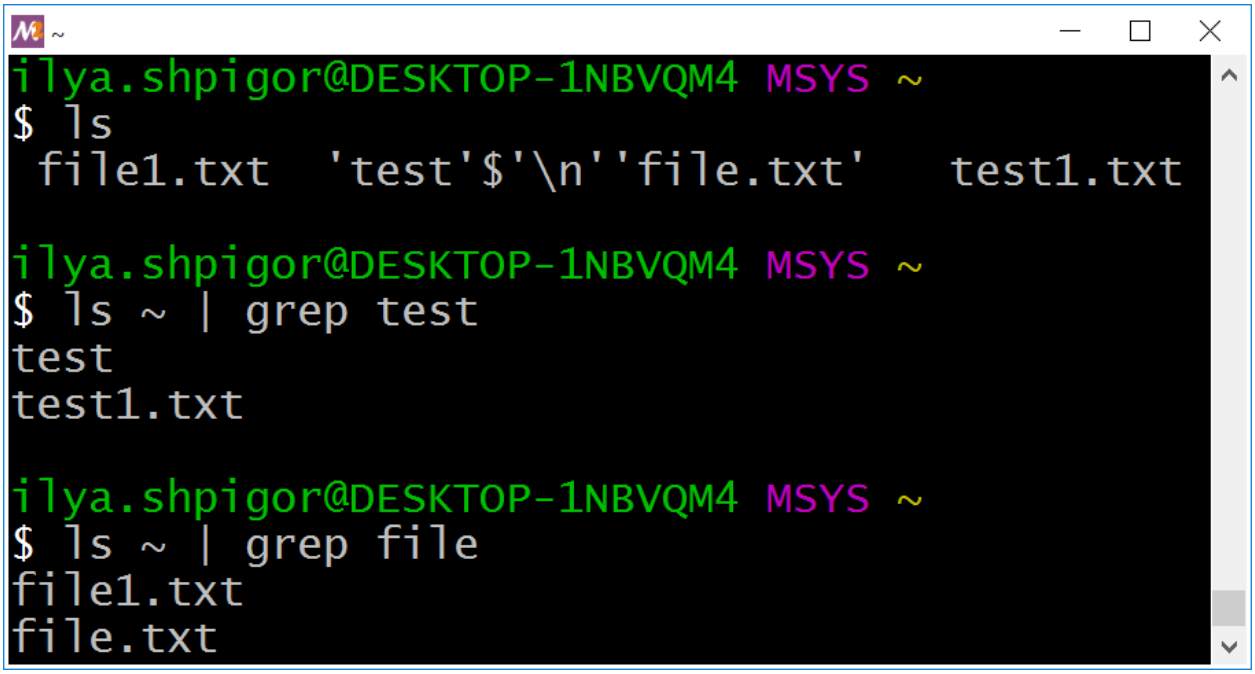



**Figure 2-27. The result of combining the `ls` and `grep` utilities**

Both commands truncate the `test\nfile.txt` filename. Remove the grep calls in the commands. You see that the `ls` utility prints the filename properly in this way: 'test'$'\n''file.txt'. When you pass it via the pipeline, the escaping sequence `\n` is replaced by the line break. It leads to splitting the filename into two parts. Then `grep` handles the parts separately as two different filenames.

There is another potential problem. Suppose you search and copy the file. Its name has space (for example, "test file.txt"). Then the following command fails:

```
ls ~ | xargs cp -t ~/tmp
```

In this case, `xargs` constructs the following call of the `cp` utility:

```
cp -t ~/tmp test file.txt
```

The command copies the `test` and `file.txt` files to the `~/tmp` path. But none of these files exists. The reason for the error is the word splitting mechanism of Bash. It splits lines in words by spaces. You can disable the mechanism by double-quotes. Here is an example for our command:

```
ls ~ | xargs -I % cp -t ~/tmp "%"
```

It copies the "test file.txt" file properly.

Double-quotes do not help if the filename has a line break. The only solution here is not to use `ls`. The `find` utility with the `-exec` action does this job right. Here is an example:

```
find . -name "*.txt" -exec cp -t tmp {} \;
```

It would be great not to use pipelines with filenames at all. However, it is required for solving some tasks. In this case, you can combine the `find` and `xargs` utilities. This approach works fine if you call `find` with the `-print0` option. Here is an example:

```
find . -type f -print0 | xargs -0 -I % bsdtar -cf %.tar %
```

The `-print0` option changes the `find` output format. It separates the paths to found objects by the null character. Without the option, the separator is a line break.

We changed the `find` output format. Then we should notify the `xargs` utility about it. By default, the utility separates the strings on the input stream by line breaks. The `-0` option changes this behavior. With the option, `xargs` applies the null character as the separator. In this way, we have reconciled the output and input formats of the utilities.

You can change the output format of the `grep`utility in the same manner. It allows you to pass its output through the pipeline. The `-Z` option does that. The option separates filenames with the null character. Here is an example:

```
grep -RlZ "GNU" . | xargs -0 -I % bsdtar -cf %.tar %
```

The command searches files that contain the "GNU" pattern. Then it passes their names to the `xargs` utility. The utility constructs the `bsdtar` call for archiving the files.

Here are the general advices for using pipelines:

1. Be aware of spaces and line breaks when passing filenames through the pipeline.
2. Never process the `ls` output. Use the `find` utility with the `-exec` action instead.
3. Always use `-0` option when process filenames by `xargs`. Pass null separated names to the utility.

**Exercise 2-7. Pipelines and I/O streams redirection**

---

```
Write a command to archive photos with the bsdtar utility.
If you are a Linux or macOS user, use the tar utility instead.
The photos are stored in the directory structure from exercises 2-6:

~/
  photo/
        2019/
             11/
             12/
        2020/
             01/
```

```
The photos of the same month should come into the same archive.
Your command should provide the following result:

~/
  photo/
        2019/
             11.tar
             12.tar
        2020/
             01.tar
```

## Logical Operators

Pipelines allow you to combine several commands. Together they make an algorithm with **linear sequence**[231]. The computer executes actions of such an algorithm one by one without any checks.

Suppose we implement a more complex algorithm. There the result of the first command determines the next step. If the command succeeds, the computer does one action. Otherwise, it does another action. Such a dependency is known as **conditional algorithm**[232]. The pipeline does not work in this case.

Here is an example of the conditional algorithm. We want to write a command to copy the directory. Then we should write the operation result to the log file. The "OK" line matches successful copying. The "Error" line means error.

We can write the following command using a pipeline:

```
cp -R ~/docs ~/docs-backup | echo "OK" > result.log
```

The command does not work properly. It writes the "OK" line to the `result.log` file regardless of the copying result. Even if the `docs` directory does not exist, the log file's message says that the operation succeeds.

The `cp` utility result should define the `echo` command output. The operator `&&` can provide such behavior. Here is an example:

```
cp -R ~/docs ~/docs-backup && echo "OK" > result.log
```

Now the command prints the "OK" line when the `cp` utility succeeds. Otherwise, there is no output to the log file.

[231] https://www.cs.utexas.edu/users/mitra/csSpring2017/cs303/lectures/algo.html
[232] https://en.wikipedia.org/wiki/Conditional_(computer_programming)

What is the && operator? It is a logical AND operation. Here its operands are Bash commands (actions) instead of Boolean expressions (conditions). Let's have a look at how the logical operation works in this case.

The POSIX standard requires each running program to provide **exit status**[233] when it finishes. The zero code means that the program completed successfully. Otherwise, the code takes a value from 1 to 255.

When you apply a logical operator to a command, the operator handles its exit status. First, Bash executes the command. Then its exit status is used as the Boolean expression in the logical operator.

Let's go back to our example:

```
cp -R ~/docs ~/docs-backup && echo "OK" > result.log
```

Suppose the `cp` utility completes successfully. It returns the zero code in this case. The zero code matches the value "true" in Bash. Therefore, the left part of the && operator equals "true". This information is not enough to deduce the result of the whole expression. It can be "true" or "false" depending on the right operand. Then the && operator has to execute the `echo` command. It always succeeds and returns the zero code. Thus, the result of the && operator equals "true".

It is not clear how do we use the result of the && operator in our example. The answer is we do not. Logical operators are needed to calculate Boolean expressions. But they are often used for their side effect in Bash. This side effect is a strict order of operands evaluation.

Let's consider the case when the `cp` utility finishes with an error in our example. Then it returns a non-zero exit status. It is equivalent to the "false" value for Bash. In this case, the && operator can already calculate the value of the whole Boolean expression. It does not need to calculate the right operand. Because If at least one operand of the logical AND is "false", the entire expression is "false". Thus, the exit status of the `echo` command is not required. Then the && operator does not execute it. In this case, there is no "OK" output in the log file.

Now you get the **short-circuit evaluation**[234]. It means calculation only those operands that are sufficient to deduce the value of the whole Boolean expression.



Bash stores the exit status of the last executed command in the environment variable. Its name is the question mark. The `echo` command displays its value like this:

```
echo $?
```

We have done only the first part of our task. Now the command prints the "OK" line in the log file when copying succeeds. But we should handle the false case. The "Error" line should appear in the log file in this case. We can do it with the logical OR operator. It is called || in Bash.

With the OR operator, our command looks like this:

[233] https://en.wikipedia.org/wiki/Exit_status
[234] https://en.wikipedia.org/wiki/Short-circuit_evaluation

```
cp -R ~/docs ~/docs-backup && echo "OK" > result.log || echo "Error" > result.log
```

This command implements the conditional algorithm that we need. If the cp utility finishes successfully, the command writes "OK" in the log file. Otherwise, it writes "Error". To understand how it works, let's consider the priority of the operation first.

First, we would make our command simpler for reading. Let's denote all operands by Latin letters. The "A" letter matches the cp call. The "B" letter marks the first echo call with the "OK" line. The "C" letter is the second echo call. Then we can rewrite our command this way:

```
A && B || C
```

The && and || operators have the same priorities in Bash. The Boolean expression is calculated from the left to the right side. The operators are called **left-associative**[235] in this case. Given this, we can rewrite our expression this way:

```
(A && B) || C
```

Adding parentheses does not change anything. First, Bash evaluates the expression (A && B). Then, it calculates the "C" operand if it is necessary.

What happens if "A" equals "true"? The && operator calculates its right operand "B" in this case. It leads to printing the "OK" line to the log file. Next, Bash processes the || operator. The interpreter already knows the value of its left operand (A && B). It equals "true". The OR operator's value equals "true" when at least one of its operands is "true". Therefore, the right operand's value does not affect the expression result. Bash skips it. It leads that the "Error" line absents in the log file.

If the "A" value is "false", the expression (A && B) equals "false" too. In this case, Bash skips the operand "B". It leads to the missing "OK" output in the log file. Then Bash handles the next || operator. The interpreter already knows that its left operand equals "false". Thus, it should evaluate the right operand for deducing the whole expression. It leads to the execution of the second echo command. Then the "Error" line comes to the log file.

The principle of short-circuits evaluation is not obvious. You would need some time to figure it out. Please do your best for that. Every modern programming language supports Boolean expressions. Therefore, understanding the rules of their evaluation is essential.

We already know how to combine Bash commands with pipelines and logical operators. There is a third way to do that. You can use a semicolon. When two commands have the semicolon in between, Bash executes them one by one without any conditions. You get the linear sequence algorithm in this case.

Here is an example. Suppose that you want to copy two directories to different target paths. The single cp call cannot do it at once. But you can combine two calls into one command like this:

[235] https://en.wikipedia.org/wiki/Operator_associativity

```
cp -R ~/docs ~/docs-backup ; cp -R ~/photo ~/photo-backup
```

The command calls the `cp` utility twice. The second call does not depend on the result of copying the `docs` directory. Even if it fails, Bash copies the `photo` directory.

Can we do the same with the pipeline? Yes. Bash executes both `cp` calls in this case too. It means that we get the same linear sequence algorithm in both cases. Here is the command with the pipeline:

```
cp -R ~/docs ~/docs-backup | cp -R ~/photo ~/photo-backup
```

However, semicolon and pipeline behave differently in general. When you use a semicolon, two commands do not depend on each other completely. When you use a pipeline, there is dependency. The output stream of the first command is connected to the input stream of the second command. In some cases, it changes the behavior of the entire algorithm.

Compare the following two commands:

```
1 ls /usr/share/doc/bash | grep "README" * -
2 ls /usr/share/doc/bash ; grep "README" * -
```

The `-` option of `grep` appends data from the input stream to the utility parameters.

Figure 2-28 shows the results of both commands.

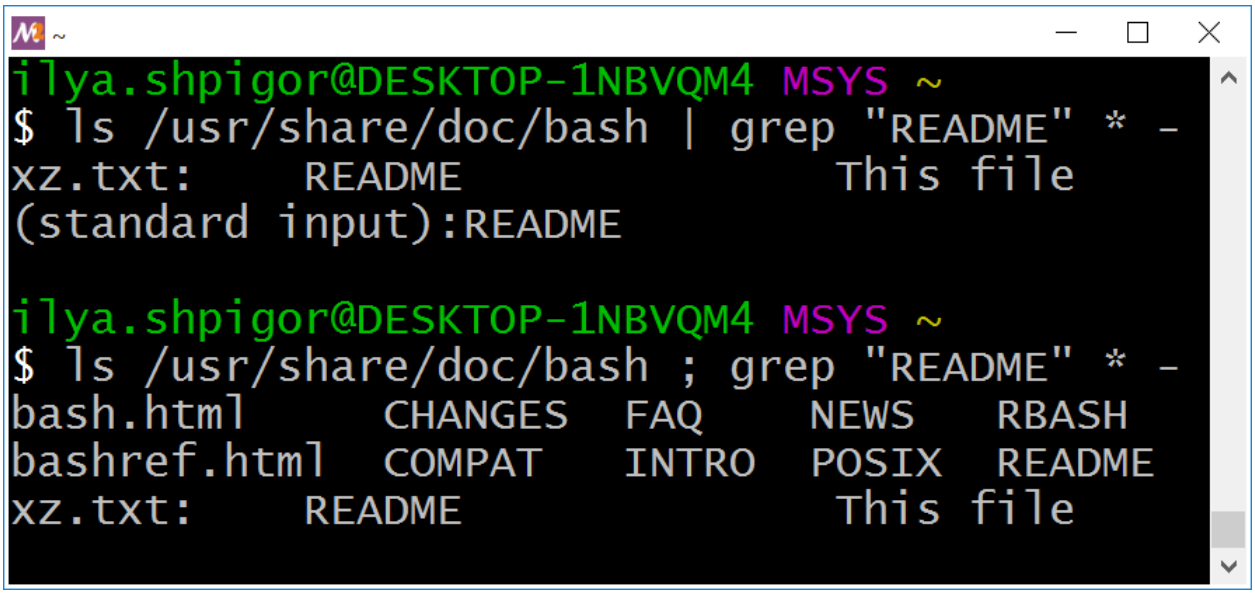



**Figure 2-28. Results of commands with pipeline and semicolon**

Even the behavior of the `ls` utility differs in these two commands. With the pipeline, `ls` prints on the screen nothing. Instead, it redirects the output to the `grep` utility input.

Let's consider the output of the commands. The second parameter of `grep` is the "*" pattern. Because of it, the utility processes all files in the current directory. It founds the "README" word in the `xz.txt` file. Then it prints this line on the screen:

```
xz.txt: README This file
```

On the next step, `grep` processes the `ls` output that it receives on the input stream. This data also contains the "README" word. Then `grep` prints the following line:

```
(standard input):README
```

This way, the `grep` utility processed two things at once:

- Files of the current directory.
- Data on the input stream.

When you combine the commands with the semicolon, the `ls` utility prints its result on the screen. Then Bash calls the `grep` utility. It processes all files in the current directory. Next, `grep` checks its input stream. But there is no data there. This way, `grep` finds the "README" word in the `xz.txt` file only.

**Exercise 2-8. Logical operators**

```
Write a command that implements the following algorithm:

1. Copy the README file with the Bash documentation to the user's home directory.

2. Archive the copied ~/README file.

3. Delete the copied ~/README file.

Each step takes place only if the previous one succeeds.
Write the result of each step to the log file result.txt.
```

# Bash Scripts

We have learned the basics of how to operate the file system in Bash. Now we do the next step. We come from the standalone shell commands to programs. When you write a program in Bash, it is called **script**. Let's learn how to write scripts.

## Development Tools

In the previous chapter, we typed Bash commands in the terminal window. Then the memory of the terminal process stores these commands. It means that physically the computer's RAM stores them. RAM is a place for the temporary storage of information. Whenever you shut down the computer, this memory is cleared.

The terminal window is not sufficient for program development. You would need a convenient source code editor[236]. This application allows you to create, edit and save source code files on your hard drive. The hard disk stores information for the long-term.

### Source Code Editor

You can write Bash scripts in any text editor. Even the standard Windows application Notepad[237] fits. But this application is inconvenient for editing the source code. Notepad does not have features for that. Such features can increase your productivity. Try several special editors and choose the one you like.

Here there is a list of three popular source code editors. If none of the three fits you, please look for alternatives on the Internet. There are many editors of this kind.



MS Office and its open-source counterpart LibreOffice[238] are not suitable for writing source code. They save files in binary or XML-like[239] formats. Interpreters and compilers cannot read them. They require text files. Text data is the format of source code editors.

**Notepad++**[240] is a fast and minimalistic open-source editor. It runs on Windows only. If you use macOS or Linux, it is better to consider other editors. You can download the latest version of Notepad++ on the official website[241].

[236] https://en.wikipedia.org/wiki/Source-code_editor
[237] https://en.wikipedia.org/wiki/Microsoft_Notepad
[238] https://en.wikipedia.org/wiki/LibreOffice
[239] https://en.wikipedia.org/wiki/XML
[240] https://en.wikipedia.org/wiki/Notepad%2B%2B
[241] https://notepad-plus-plus.org/downloads/

**Sublime Text**[242] is a proprietary **cross-platform**[243] source code editor. Cross-platform means that the program runs on several OSes and hardware configurations. You can use Sublime Text for free without activation or buying a license. Download it on the official website[244].

*Visual Studio Code*[245] is a free cross-platform source code editor from Microsoft. It is open-source software. It means that the program is available for free and without a license. Download the editor on the official website[246].

All three editors have the following features for working with source code:

- Syntax highlighting[247].
- Auto-complete[248].
- Support of commonly used character encodings[249].

It is possible to edit the source code without these features. But they speed up your work, make it easier to edit the program and find bugs in it. They also help you to get used to the Bash syntax.

## Launching the Editor

You can launch the source code editor through the graphical interface of the OS. In the case of Windows, do it via the Start menu or the icon on the desktop.

There is another possibility. You can launch the editor from the command-line interface. This approach is more convenient in some cases. For example, when the Bash command returns a filename, you can quickly open it via command-line.

There are three ways to run the application in Bash:

1. By absolute path.
2. By relative path.
3. By executable file name.

The last approach is the fastest and most commonly used. If you want to apply it, you should add the program's installation path to the `PATH` variable.

Let's consider how to run the Notepad++ editor by the executable name. The editor has the following installation path by default:

[242] https://en.wikipedia.org/wiki/Sublime_Text
[243] https://en.wikipedia.org/wiki/Cross-platform_software
[244] https://www.sublimetext.com/
[245] https://en.wikipedia.org/wiki/Visual_Studio_Code
[246] https://code.visualstudio.com/
[247] https://en.wikipedia.org/wiki/Syntax_highlighting
[248] https://en.wikipedia.org/wiki/Autocomplete
[249] https://en.wikipedia.org/wiki/Character_encoding

```
C:\Program Files (86)\Notepad++
```



You can clarify the installation path of the program in the properties of the desktop icon or the Start menu item.

In the MSYS2 environment, the same installation path looks like this:

```
/c/Program Files (x86)/Notepad++
```

First, let's run Notepad++ by this absolute path. If you try to do that, Bash prints the error message as Figure 3-1 shows.

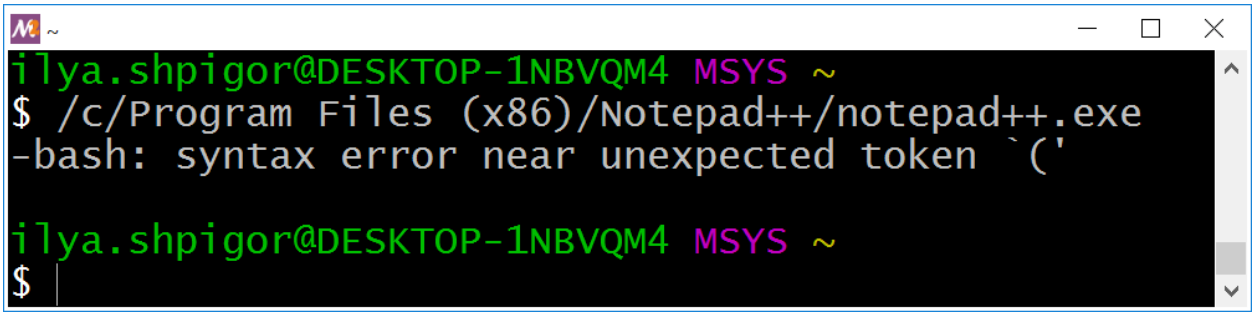



**Figure 3-1. Result of launching Notepad++**

This command has several problems. We will consider them one by one. First, try to launch the following `cd` command:

```
cd /c/Program Files
```

Figure 3-2 shows the result.

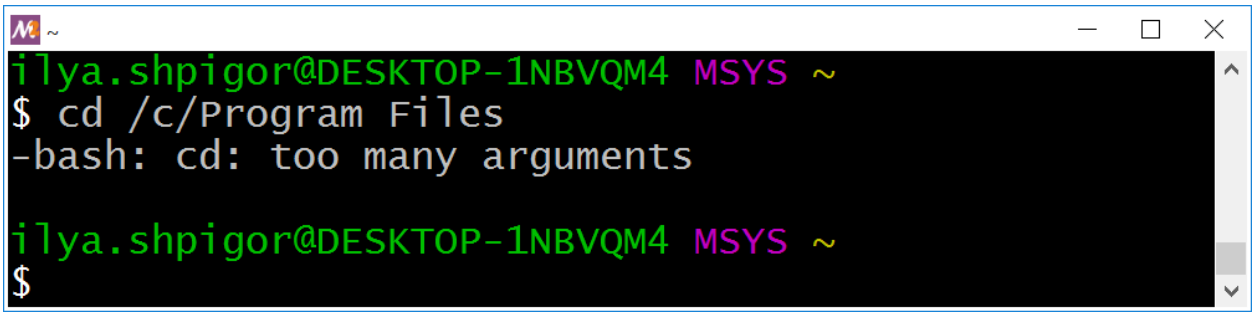



**Figure 3-2. Result of the `cd` command**

Bash complains that you pass more parameters to `cd` than it requires. This command accepts one path on input only. But you pass two paths here. The mistake happens because of word splitting. Bash

handles the space as a separator between two words. Therefore, it splits the single path "/c/Program Files" into two paths "/c/Program" and "Files".

There are two ways to solve such errors:

1. Enclose the path in double-quotes:

```
cd "/c/Program Files"
```

2. Escape all spaces with backslashes:

```
cd /c/Program\ Files
```

Bash executes each of these commands correctly.

Now let's try to navigate the `/c/Program Files (x86)` path. Here is a command for that:

```
cd /c/Program Files (x86)
```

We found out that Bash handles spaces on its own. Therefore, we should prevent it and escape them with backslashes. Then we get the following command:

```
cd /c/Program\ Files\ (x86)
```

This command still fails. Figure 3-3 shows its error message.

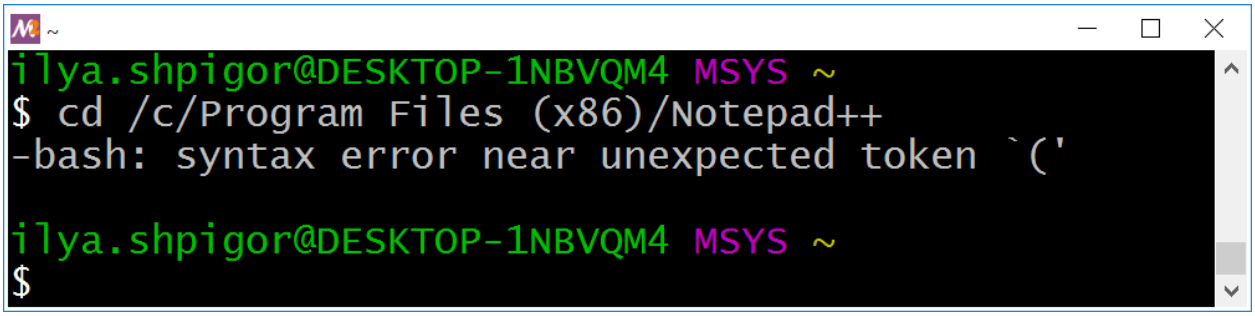



**Figure 3-3. Result of the `cd` command**

This message looks the same as the error in Figure 3-1 when we tried to launch Notepad++. The problem happens because the parentheses are part of the Bash syntax. Therefore, the interpreter treats them as a language construct. We met this problem before when grouping conditions of the `find` utility. Escaping or double-quotes solves this issue. Here is an example:

```
cd /c/Program\ Files\ \(x86\)
cd "/c/Program Files (x86)"
```

If we choose the approach with double-quotes, the command to launch Notepad++ looks like this:

```
"/c/Program Files (x86)/Notepad++/notepad++.exe"
```

The absolute path of the editor is too long. Typing it every time is inconvenient. It is better to launch Notepad++ via the executable name. Let's add its installation path to the `PATH` Bash variable for making that work.

Add the following line to the end of the `~/.bash_profile` file:

```
PATH="/c/Program Files (x86)/Notepad++:${PATH}"
```

Close and reopen the terminal window. Now you can start Notepad++ by the following command:

```
notepad++.exe
```

There is an alternative solution. Instead of changing the `PATH` variable, you can declare an alias. The alias mechanism replaces the entered command with another one. It allows you to abbreviate long lines in the terminal window.

Let's declare an alias with the `notepad++` name for the following command:

```
"/c/Program Files (x86)/Notepad++/notepad++.exe"
```

The following call of `alias` Bash built-in does that:

```
alias notepad++="/c/Program\ Files\ \(x86\)/Notepad++/notepad++.exe"
```

After declaring the alias, Bash translates the command "notepad++" into the absolute path of the editor's executable. This solution has one problem. You should declare the alias whenever launching the terminal window. There is a way to automate this declaration. Just add the `alias` command at the end of the `~/.bashrc` file. Bash executes all commands of this file at every terminal startup.

Now you can open the source code files in Notepad++ via command-line. To do that, pass the filename as the first parameter to the `notepad++` executable. Here is an example:

```
notepad++ test.txt
```

If the `test.txt` file does not exist, Notepad++ shows the dialog to create it.

Suppose that you run a GUI application in the terminal window. After that, you cannot enter the commands in this window. The GUI program controls it and prints the diagnostic messages there. The terminal window starts working in normal mode after finishing the application.

If you want to continue working with the terminal window, run the GUI application in the **background mode**. To do that, add the ampersand & at the end of the command. Here is an example:

```
notepad++ test.txt &
```

This command starts the Notepad++. The editor still prints error messages to the terminal window. But at the same time, you can type and execute commands there.

There is an option to separate the terminal window and the running GUI application completely. Run the application in the background mode. Then call the `disown` Bash built-in with the `-a` option. Here is an example:

```
notepad++ test.txt &
disown -a
```

Notepad++ stops printing its messages to the terminal after these commands. Also, if you close the terminal window, the editor continues its work.

You can combine Notepad++ and `disown` calls into one command. Here is an example:

```
notepad++ test.txt & disown -a
```

The `-a` option of the `disown` command detaches all programs that work in the background. You can detach the specific application instead. There is the Bash environment variable called `$!`. It stores the **process identifier**[250] (PID) of the last launched application. PID is a unique number that OS assigns to each process. You can manipulate the process by this number. Also, you can pass the PID to the `disown` command for detaching the specific application. Here is an example of doing that:

```
notepad++ test.txt & disown $!
```

If you want to list all applications that work in the background, use the `jobs` Bash built-in. It has the `-l` for that. Here is an example:

```
jobs -l
```

The command shows the PIDs of all background processes you have launched in the current terminal window.

# Why Do We Need Scripts?

We learned how to write complex Bash commands using pipelines and logical operators. The pipeline combines several commands into one. You get a linear sequence algorithm this way. Logical operators add conditions to the algorithm. You get a complete program in the result.

Why isn't the shell enough for Bash programming? Bash scripts are programs that the hard drive stores. Let's figure out why users need them.

[250] https://en.wikipedia.org/wiki/Process_identifier

## Backup Command

To understand Bash scripts better, we would write the command. It makes a backup of photos on the external hard drive[251]. The command consists of two actions: archiving and copying.

Suppose that the `~/photo` directory keeps all photos. The mount point of the external drive is `/d`. Then the backup command can be like this:

```
bsdtar -cjf ~/photo.tar.bz2 ~/photo && cp -f ~/photo.tar.bz2 /d
```

Because of the logical AND, copying happens when the archiving step succeeds. If the `bsdtar` utility returns an error, there is no `cp` call.



In our example, the backup operation happens in two steps: archiving and copying. This separation of actions is necessary for educational purposes. You can do the same by the single `bsdtar` call:

```
bsdtar -cjf /d/photo.tar.bz2 ~/photo
```

Suppose that our backup command runs automatically. For example, it launches every day by scheduling. In this case, you cannot read error messages of utilities if they fail. Then you need a log file to store the results of utilities. Here is the `bsdtar` call with the debug output:

```
1 bsdtar -cjf ~/photo.tar.bz2 ~/photo &&
2 echo "bsdtar - OK" > results.txt ||
3 echo "bsdtar - FAILS" > results.txt
```

You can split a Bash command into multiple lines. There are two ways for doing that:

1. Line break immediately after the logical operator (&& or ||).
2. Line break after backslash .

We applied the first option in the last `bsdtar` call. The second option looks like this:

```
1 bsdtar -cjf ~/photo.tar.bz2 ~/photo \
2 && echo "bsdtar - OK" > results.txt \
3 || echo "bsdtar - FAILS" > results.txt
```

Here is the command to print the `cp` utility result in the log file:

[251] https://en.wikipedia.org/wiki/Hard_disk_drive#EXTERNAL

```
cp -f ~/photo.tar.bz2 /d &&
echo "cp - OK" >> results.txt ||
echo "cp - FAILS" >> results.txt
```

The single command should perform the backup operation. Therefore, we can combine the `bsdtar` and `cp` calls with the logical AND. Here is the result:

```
bsdtar -cjf ~/photo.tar.bz2 ~/photo &&
  echo "bsdtar - OK" > results.txt ||
  echo "bsdtar - FAILS" > results.txt &&
cp -f ~/photo.tar.bz2 /d &&
  echo "cp - OK" >> results.txt ||
  echo "cp - FAILS" >> results.txt
```

Let's consider how this command works. For convenience, we will rewrite it in the form of a Boolean expression. The Latin letter replaces each command and utility call. Then we get the following result:

```
B && O1 || F1 && C && O2 || F2
```

The letters "B" and "C" represent the `bsdtar` and `cp` calls. "O1" and "F1" are the commands for printing the `bsdtar` result. The "O1" command prints the "bsdtar - OK" line into the log file. The "F1" command prints the "bsdtar - FAIL" line. Similarly, "O2" and "F2" are the commands for logging the `cp` result.

If the `bsdtar` call succeeds, the "B" operand equals "true". Then Bash performs the following steps sequence:

1. B
2. O1
3. C
4. O2 or F2

If the `bsdtar` fails, the "B" operand equals false. Then Bash does the following actions:

1. B
2. F1
3. C
4. O2 or F2

The copy operation does not make sense if the archiving step fails. The `bsdtar` utility makes things even more confusing. It creates an empty archive if you pass to it an unavailable file or directory. Then the `cp` utility copies the empty archive successfully. These operations lead to the following output to the log file:

```
bsdtar - FAILS
cp - OK
```

Such output confuses the user instead of clarifying the issue.

Let's come back to our expression:

```
B && O1 || F1 && C && O2 || F2
```

Why does Bash call the `cp` utility after an error in `bsdtar`? It happens because the `echo` command always succeeds. It returns zero code, which means "true". Thus, the "O1", "F1", "O2" and "F2" operands are always "true".

Now we focus on the `bsdtar` call and corresponding `echo` commands. They match the following part of the Boolean expression:

```
B && O1 || F1
```

We can put its left side into brackets without changing the result. Then it looks like this:

```
(B && O1) || F1
```

Here is a logical OR for the operands (B && O1) and "F1". The "F1" operand always equals "true". Therefore, the whole expression is always "true".

We can solve the problem by inverting the result of "F1". The logical NOT operator does that. This way, we get the following expression:

```
B && O1 || ! F1 && C && O2 || F2
```

Now, if the "B" operand fails, Bash evaluates "F1". It always equals false because of negation. Then Bash skips the "C" and "O2" command. It happens because there is a logical AND between them and "F1". The "F2" command is the next one that Bash performs. The shell needs its result because it has a logical OR in front of it.

The following parentheses would make the expression clearer:

```
(B && O1 || ! F1 && C && O2) || F2
```

It is evident that Bash executes the "F2" action when the parenthesized expression equals "false". Otherwise, it cannot evaluate the value of the entire expression.

This command writes the following output into the log file:

```
bsdtar - FAILS
cp - FAILS
```

This output is better than the previous one. Now the `cp` utility does not copy an empty archive.

However, our current result is not good enough. Imagine that the backup command contains 100 actions. If an error occurs at the 50th action, all the remaining operations print their failed results into the log file. Such output prevents you from finding the problem. The best solution here is to terminate the command after the first error. To do that, we can group calls of utilities and their output with parentheses. Here is the result:

```
(B && O1 || ! F1) && (C && O2 || F2)
```

Let's check what happens if the "B" operand is false. Then Bash executes the "F1" action. The negation inverts "F1" result. Therefore, the entire left side of the expression equals "false". Here is the left side:

```
(B && O1 || ! F1)
```

Then short-circuit evaluation happens. Because of it, Bash does not calculate the right operand of the logical AND. It means that the shell skips all actions on the right side of the expression. Here is the right side:

```
(C && O2 || F2)
```

It is the behavior we wanted.

We can add one more improvement. The "F2" operand should be inverted. Then the whole expression equals "false" if "C" is "false". It means that the backup command fails if the `cp` utility fails. It sounds reasonable. Also, you would need such behavior for integrating the backup command with other commands.

Here is the final version of our expression:

```
(B && O1 || ! F1) && (C && O2 || ! F2)
```

Let's come back to the real Bash code. The backup command looks like this now:

```
(bsdtar -cjf ~/photo.tar.bz2 ~/photo &&
  echo "bsdtar - OK" > results.txt ||
  ! echo "bsdtar - FAILS" > results.txt) &&
(cp -f ~/photo.tar.bz2 /d &&
  echo "cp - OK" >> results.txt ||
  ! echo "cp - FAILS" >> results.txt)
```

We wrote the command quickly. But it is hard for reading and understanding. It is often happening in programming. Because of it, you should pay more efforts to make your code clean and obvious. Code cleanliness is more important than a high speed of writing it.

## Poor Technical Solution

We have written the long and complicated backup command. If you run it regularly, you should store it somewhere. Otherwise, you have to type the command in the terminal window each time. Typing is a bad idea because you can make a mistake easily.

The Bash history file saves all commands that you execute in the terminal. Each user has this file in his home directory. Its path is `~/.bash_history`. When you press Ctrl+R keys in the terminal window, Bash calls the quick search over the history file. You can quickly find the required command this way.

Can we use the history file for storing the backup command permanently? There you can find and execute the command every time you need it. This solution seems to be reliable and convenient. But don't jump to conclusions. Let's take a look at its possible problems.

First, the history file has a limited size. By default, the file saves only the 500 most recently executed commands. If this number is exceeded, each new command overwrites the oldest ones in the file. This way, you can accidentally lose the backup command.

It is possible to increase the maximum size of the history file. But what size would be enough? Whatever size we choose, there is a risk of exceeding it. There is an option to remove the size limitation at all. Then the history file saves all commands without overwriting the old ones.

It seems we find a way to store the backup command effectively. The Bash history file with unlimited size does it. Could this solution have any problems?

Suppose you use Bash for a couple of years. All commands you entered during this time come to the `.bash_history` file. If you execute the same command twice, it appears in the file twice too. Therefore, the file size will reach hundreds of megabytes in two years. You do not need most of these commands. Instead, only a small part of them are needed for regular usage. It leads to inefficient use of hard disk space.

You might argue that storing the extra two hundred megabytes is not a problem for modern computers. Yes, it is true. But there is an overhead that you miss. When you press Ctrl+R, Bash searches the command in the entire `.bash_history` file. The larger it is, the longer the search takes. Over time, you will wait several seconds even on a powerful computer.

When the history file grows, the search time increases. There are two reasons for that. First, Bash should process more lines to find your request. Second, the file has many commands that have the same first letters. It forces you to type more letters after pressing Ctrl+R to find the right command. At some point, the history file search becomes inconvenient. That is the second problem with our solution.

What else could go wrong? Suppose that you get new photo albums. Their path differs from `~/photo`. It is `~/Documents/official_photo` for example. Our backup command works with the `~/photo` path only. It cannot copy photos from another path. You should write a new command for doing that. Thus, the complexity of extending features is the third problem of our solution.

You may already have several backup commands. The first one copies photos. The second one copies documents. It would be hard to combine them. You have to write the third command that includes all actions of the existing ones.

We can conclude that a history file is a bad option for the long-term storage of commands. There is the same reason for all our problems. We misuse the history file mechanism. It was not intended as a permanent storage place. As a result, we came up with the poor technical solution.

Everybody can make a poor technical solution. Professionals with extensive experience often come to them too. It happens for various reasons. The lack of knowledge played a role in our case. We got how to work with Bash in shell mode. Then we applied this experience to the new task. But we did not take into account all the requirements.

Here is the complete list of the requirements for our task:

1. The backup command should have long-term storage.
2. The way to call the command quickly should be available.
3. It should be a possibility to extend the command by new features.
4. The command should be able to combine with other commands.

First, let's evaluate our knowledge of Bash. They are not enough to meet all these requirements. All the mechanisms we know don't fit here. Maybe a Bash script could help us? I propose to explore its features. Then we can check if it is suitable for our task.

## Bash Script

Let's create a Bash script with our backup command. Here are the steps for that:

1. Open the source code editor and create a new file. If you have integrated Notepad++ into Bash, run the following command:

```
notepad++ ~/photo-backup.sh
```

2. Copy the command to the file:

```
(bsdtar -cjf ~/photo.tar.bz2 ~/photo &&
  echo "bsdtar - OK" > results.txt ||
  ! echo "bsdtar - FAILS" > results.txt) &&
(cp -f ~/photo.tar.bz2 /d &&
  echo "cp - OK" >> results.txt ||
  ! echo "cp - FAILS" >> results.txt)
```

3. Save the file in the user's home directory. It should have the `photo-backup.sh` name.
4. Close the editor.

Now we have the Bash script file. Call the Bash interpreter and pass the script there as the first parameter. Here is an example of such a command:

```
bash photo-backup.sh
```

We have run our first script. The script itself is a sequence of Bash commands. The file on some drive stores them. When Bash runs a script, it reads the file line by line. The interpreter executes lines in this order. Conditional and loop statements can change the order of execution.

It is inconvenient to run a script with an explicit call of the Bash interpreter. You can run it by a relative or absolute path. To do that, change the permissions of the script. Here are the steps:

1. Run the following command in the terminal window:

```
chmod +x ~/photo-backup.sh
```

2. Open the script file in an editor.
3. Add the following line at the beginning of the script:

```
#!/bin/bash
```

4. Save the modified file.
5. Close the editor.

Now you can run the script by a relative or absolute path. Here are examples of these commands:

```
./photo-backup.sh
~/photo-backup.sh
```

Let's consider our steps for launching the script. The first thing that prevents it from running is permissions. When you create a new file, it gets the following permissions by default:

```
-rw-rw-r--
```

It means that the owner and his group can read and modify the file. Everyone else can only read it. No one can run the file.

The `chmod` utility changes the permissions of the specified file. We call it with the `+x` option. The option changes the file permissions to this:

```
-rwxrwxr-x
```

It means that everyone can execute the file now.

If you run the script by a relative or absolute path, your shell tries to interpret its lines. It works if your shell is Bash.

Suppose that your shell is not Bash. It is the Csh[252] for example. Then the script fails. It happens because the syntax of Bash and Csh differs. It means that they use different language constructions for the same things. You wrote the script in Bash language. Therefore, the Bash interpreter only can execute it.

There is an option to specify the interpreter that should execute the script. To do that, add the **shebang**[253] at the beginning of the script file. Shebang is a combination of the number sign and exclamation mark. It looks like this:

```
#!
```

The absolute path to the interpreter comes after the shebang. In our case, we get the following line:

```
#!/bin/bash
```

Now Bash always executes the script. It happens even when the user works in another shell.

The `file` utility prints the type of the specified file. If the script does not have the shebang, the utility defines it as a regular text file. Here is an example output:

```
~/photo-backup.sh: ASCII text
```

If you add the shebang, the utility defines the same file as a Bash script:

```
~/photo-backup.sh: Bourne-Again shell script, ASCII text executable
```

Most Linux systems have the same path to the Bash interpreter. It equals `/bin/bash`. However, this path differs on some Unix systems (for example, FreeBSD). It can be a reason why your script does not work there. You can solve this problem by the following shebang:

[252] https://ru.wikipedia.org/wiki/Csh
[253] https://en.wikipedia.org/wiki/Shebang_(Unix)

```
#!/usr/bin/env bash
```

Here we call the `env` utility. It searches the Bash executable in the paths of the `PATH` variable.

## Commands Sequence

Listing 3-1 demonstrates the current version of our script.

**Listing 3-1. The script for making the photos backup**

```
#!/bin/bash
(bsdtar -cjf ~/photo.tar.bz2 ~/photo &&
  echo "bsdtar - OK" > results.txt ||
  ! echo "bsdtar - FAILS" > results.txt) &&
(cp -f ~/photo.tar.bz2 /d &&
  echo "cp - OK" >> results.txt ||
  ! echo "cp - FAILS" >> results.txt)
```

Now the script contains one command. The command is too long. Therefore, it isn't easy to read and modify it. We can split the command into two parts. Listing 3-2 shows the result.

**Listing 3-2. The script with two commands**

```
#!/bin/bash

bsdtar -cjf ~/photo.tar.bz2 ~/photo &&
  echo "bsdtar - OK" > results.txt ||
  ! echo "bsdtar - FAILS" > results.txt

cp -f ~/photo.tar.bz2 /d &&
  echo "cp - OK" >> results.txt ||
  ! echo "cp - FAILS" >> results.txt
```

Unfortunately, the behavior of the script has changed. Now the logical AND does not take place between the commands. Therefore, Bash executes the `cp` utility regardless of the `bsdtar` result. This behavior is wrong.

The script should finish if the `bsdtar` utility fails. We can apply the `exit` Bash built-in to terminate the script. The command receives the exit status as the parameter. The script returns this code on termination.

Listing 3-3 shows the script with the `exit` call.

**Listing 3-3. The script with the `exit` call**

```
#!/bin/bash

bsdtar -cjf ~/photo.tar.bz2 ~/photo &&
  echo "bsdtar - OK" > results.txt ||
  (echo "bsdtar - FAILS" > results.txt ; exit 1)

cp -f ~/photo.tar.bz2 /d &&
  echo "cp - OK" >> results.txt ||
  ! echo "cp - FAILS" >> results.txt
```

We changed the command that calls the `bsdtar` utility. First, it looks like this:

```
B && O1 || ! F1
```

It becomes like this after adding the `exit` call:

```
B && O1 || (F1 ; E)
```

The "E" letter means the `exit` command here.

If `bsdtar` returns an error, Bash evaluates the right operand of the logical OR. It is equal to "(F1; E)". We removed the negation of the `echo` command. Its result is not important anymore. Then Bash calls `exit`. We expect that the call terminates the script.

The current version of the script does not solve the problem. The `exit` call does not terminate the script. It happens because parentheses create a child process[254]. The child process is called **subshell**. It executes the commands specified in parentheses. When the commands are done, Bash continues executing the parent process. The parent process is one that spawned the subshell.

The `exit` command means finishing the subshell in our script. Then Bash calls the `cp` utility. To solve this problem, we should replace the parentheses with braces. Bash executes the commands in braces in the current process. The subshell is not spawned in this case.

Listing 3-4 shows the fixed version of the script.

[254] https://en.wikipedia.org/wiki/Child_process

**Listing 3-4. The fixed script with the `exit` call**

```
#!/bin/bash

bsdtar -cjf ~/photo.tar.bz2 ~/photo &&
  echo "bsdtar - OK" > results.txt ||
  { echo "bsdtar - FAILS" > results.txt ; exit 1 ; }

cp -f ~/photo.tar.bz2 /d &&
  echo "cp - OK" >> results.txt ||
  ! echo "cp - FAILS" >> results.txt
```

Notice the semicolon before the closing brace. The semicolon is mandatory here. Also, spaces after the opening brace and before closing one are mandatory too.

There is another solution for our problem. It is more elegant than calling the `exit` command. Suppose you want to terminate the script after the first failed command. The `set` Bash built-in does that. It changes the parameters of the interpreter. Call the command with the `-e` option like this:

```
set -e
```

You can specify the same option when starting the Bash. Here is an example:

```
bash -e
```

The `-e` option has several problems[255]. The option changes the behavior of the current Bash process only. The subshells it spawns work as usual.

Bash executes each command of a pipeline or logical operator in a separate subshell. Therefore, the `-e` option does not affect these commands. It means that the option does not fit in our case.

## Changing Parameters

Suppose you have moved photos from the `~/photo` path to `~/Documents/Photo`. Then you should change the backup script. Listing 3-5 shows the script with the fixed path.

[255] http://mywiki.wooledge.org/BashFAQ/105

**Listing 3-5. The script with the new path**

```
#!/bin/bash

bsdtar -cjf ~/photo.tar.bz2 ~/Documents/Photo &&
  echo "bsdtar - OK" > results.txt ||
  { echo "bsdtar - FAILS" > results.txt ; exit 1 ; }

cp -f ~/photo.tar.bz2 /d &&
  echo "cp - OK" >> results.txt ||
  ! echo "cp - FAILS" >> results.txt
```

Every time you change the path of photos, you have to fix the script. It is inconvenient. The better solution is to make a universal script. Such a script should receive the path of photos as an input parameter.

When you run a Bash script, you can pass command-line parameters there. It works the same as for any GNU utility. Specify the parameters separated by a space after the script name. Then Bash passes it to the script. Here is an example:

```
./photo-backup.sh ~/Documents/Photo
```

This command runs our script. Bash passes there the path `~/Documents/Photo` as an input parameter. You can access the path in the script via the variable called `$1`. If the script receives more parameters, they are available through the variables `$2`, `$3`, `$4`, etc. The variable name matches the number of the parameter. They are called **positional parameters**.

There is a special positional parameter `$0`. It stores the path to the launched script. In our case, it equals `./Photo-backup.sh`.

Let's improve our script. It will read the path to the photos in the first positional parameter. Listing 3-6 shows the new source code.

**Listing 3-6. The script uses the positional parameter**

```
#!/bin/bash

bsdtar -cjf ~/photo.tar.bz2 "$1" &&
  echo "bsdtar - OK" > results.txt ||
  { echo "bsdtar - FAILS" > results.txt ; exit 1 ; }

cp -f ~/photo.tar.bz2 /d &&
  echo "cp - OK" >> results.txt ||
  ! echo "cp - FAILS" >> results.txt
```

The `$1` variable stores the path to the photos. We use it in the `bsdtar` call. There are quotes around the variable name. They prevent the word splitting mechanism.

Suppose you want to backup photos from the `~/photo album` path. Here is a command to call our script for that:

```
./photo-backup.sh "~/photo album"
```

If you skip quotes around the variable name in the script, the `bsdtar` call looks like this:

```
bsdtar -cjf ~/photo.tar.bz2 ~/photo album &&
  echo "bsdtar - OK" > results.txt ||
  { echo "bsdtar - FAILS" > results.txt ; exit 1 ; }
```

In this case, the `bsdtar` utility receives the "/photo album" string in parts. Instead of one parameter, there are two: "/photo" and "album". There are no such directories. Therefore, the script fails.

Our example showed that it is not enough to quote parameters when calling a script. You should quote all occurrences of the variable name in the script. It happens because of the way how the Bash runs scripts. When you call a script from the shell, Bash spawns a child process. This process executes the script. The child process does not receive quotes from the command-line. Bash removes them. Therefore, you need quotes inside the script.

We added the parameter to our script. What are the benefits of this solution? This way, we get a universal script for making backups. It processes any input files: documents, photos, videos, source code, etc.

Adding a parameter to our script leads to the problem. Suppose you call it twice for making backups of photos and documents:

```
1 ./photo-backup.sh ~/photo
2 ./photo-backup.sh ~/Documents
```

These commands create the `~/photo.tar.bz2` archive. Then they copy the archive to the D disk. When the second command copies it, it overwrites the existing `/d/photo.tar.bz2` file. This file is a result of the first command. So, we lose it.

To solve this problem, we should use the script's parameter for naming the archive. This way, we avoid filename conflicts. Listing 3-7 shows the new version of the script.

**Listing 3-7. The script with the unique archive name**

```
#!/bin/bash

bsdtar -cjf "$1".tar.bz2 "$1" &&
  echo "bsdtar - OK" > results.txt ||
  { echo "bsdtar - FAILS" > results.txt ; exit 1 ; }

cp -f "$1".tar.bz2 /d &&
  echo "cp - OK" >> results.txt ||
  ! echo "cp - FAILS" >> results.txt
```

Now the script creates the archive in the target path. For example, we call it this way:

```
./photo-backup.sh ~/Documents
```

The command creates an archive in the `~/Documents.tar.bz2` path. Then it copies the file to the D disk. In this case, the filename does not conflict with the photo archive `/d/photo.tar.bz2`.

There is the last improvement of the script. We can call the `mv` utility instead of `cp`. It deletes the temporary archive. Listing 3-8 shows the result.

**Listing 3-8. The script with removing the temporary archive**

```
#!/bin/bash

bsdtar -cjf "$1".tar.bz2 "$1" &&
  echo "bsdtar - OK" > results.txt ||
  { echo "bsdtar - FAILS" > results.txt ; exit 1 ; }

mv -f "$1".tar.bz2 /d &&
  echo "cp - OK" >> results.txt ||
  ! echo "cp - FAILS" >> results.txt
```

Now we get the universal backup script. Its old name `photo-backup.sh` does not fit anymore. The new version of the script can copy any data. Let's rename it to `make-backup.sh`.

## Combination with Other Commands

Now you can call the script by an absolute or relative path. If you integrate it into Bash, you can call the script by name. Then it will be more convenient to combine it with other programs.

We learned two ways how to integrate an application with Bash. There is also the third way. Here is a complete list of these options:

1. Add the script's path to the `PATH` variable. To do that, edit the `~/.bash_profile` file.
2. Define an alias with an absolute path to the script. Do that in the file `~/.bashrc`.
3. Copy the script to the `/usr/local/bin` directory. The `PATH` variable contains its path by default. If there is no such directory in your MSYS2 environment, create it.



Call the `unalias` Bash command to remove a declared alias. Here is an example:

```
unalias make-backup.sh
```

Suppose that you have integrated the backup script with Bash in one of three ways. Now you can launch it by name like this:

```
make-backup.sh ~/photo
```

You can combine the script with other commands in pipelines and logical operators. It works the same way as for any Bash built-in or GNU utility.

Here is an example. Suppose we want to backup all PDF documents of the `~/Documents` directory. The `find` utility can find them by the following call:

```
find ~/Documents -type f -name "*.pdf"
```

We can apply our script to archive and copy each found file. Here is the command for that:

```
find ~/Documents -type f -name "*.pdf" -exec make-backup.sh {} \;
```



There is the escaped semicolon at the end of the `find` call. It means that action is executed on each found file separately.

The command copies archives to disk D. Each archive matches one PDF file. This approach is inconvenient. It would be better to collect all PDF files into one archive. Let's try the following command for that:

```
find ~/Documents -type f -name *.pdf -exec make-backup.sh {} +
```

The command should processes all found files in the single `make-backup.sh` call. However, it produces an archive with the first found PDF file only. Where are the rest of the documents? Let's take a look at the `bsdtar` call inside our script. For simplicity, we omit all echo outputs there. The call looks like this:

```
bsdtar -cjf "$1".tar.bz2 "$1"
```

The problem happens because we process the first positional parameter only. The `$1` variables stores it. The `bsdtar` call ignores other parameters in variables `$2`, `$3`, etc. But they contain all results of the `find` utility. Therefore, we cut off all results except the first one.

We should use the `$@` variable instead of `$1` to solve the problem. Bash stores all script parameters into `$@`. Here is the fixed `bsdtar` call:

```
bsdtar -cjf "$1".tar.bz2 "$@"
```

Now the `bsdtar` utility processes all script's parameters. Note that we still use the first parameter `$1` as the archive name. It should be a single word. Otherwise, `bsdtar` fails.

Listing 3-9 shows the fixed version of the backup script. It handles an arbitrary number of input parameters.

**Listing 3-9. The script with an arbitrary number of input parameters**

```
#!/bin/bash

bsdtar -cjf "$1".tar.bz2 "$@" &&
  echo "bsdtar - OK" > results.txt ||
  { echo "bsdtar - FAILS" > results.txt ; exit 1 ; }

mv -f "$1".tar.bz2 /d &&
  echo "cp - OK" >> results.txt ||
  ! echo "cp - FAILS" >> results.txt
```

Bash has an alternative variable for `$@`. It is called `$*`. If you put the `$*` variable in double-quotes, Bash interprets its value as a single word. It interprets the `$@` variable as a set of words in the same case.

Here is an example. Suppose you call the backup script this way:

```
make-backup.sh "one two three"
```

Then substitution of the "$*" value gives the following result:

```
"one two three"
```

Here is the result for the "$@" substitution:

```
"one" "two" "three"
```



Always prefer to use `$@` instead of `$*`. The only exception is when a single word must represent all input parameters.

## Scripts Features

When solving the backup task, we considered the features of the Bash scripts.

Here are the requirements for the task:

1. The backup command should have long-term storage.
2. The command has to be called quickly.
3. It must be possible to extend it.
4. It should be possible to combine it with other commands.

The final version of the `make-backup.sh` script meets all these requirements. Let's check each of them:

1. The script is stored on the hard disk. It is long-term memory.
2. The script is easy to integrate with Bash. Then you can quickly call it.
3. The script is a sequence of commands. Each one starts on a new line. It is easy to read and edit. Thanks to parameterization, it is easy to generalize it for solving tasks of the same type.
4. Due to integration with Bash, the script is easy to combine with other commands. You can do it with pipelines and logical operators.

If your task requires any of these features, write a Bash script for that.

# Variables and Parameters

Variables in Bash have already been mentioned several times in this book. We have learned the list of system paths in the `PATH` variable. We have used positional parameters in the backup script. It is time to get a good grasp on the topic.

First, let's find out what the term "variable" means in programming. The variable is an area of memory where some value is stored. In most cases, this is short-term memory (RAM, CPU cache and registers).

There was a generation of the first programming languages (for example **assembler**[256]). When using such a language, you should refer to a variable by its address. If you want to read or write its value, you have to specify its memory address.

[256] https://en.wikipedia.org/wiki/Assembly_language

Suppose you work on a computer with 32-bit processors. Then a memory address has a length of 4 bytes. It is the number from 0 to 4294967295. This number is twice larger for 64-bit processors. It is inconvenient to remember and operate with such big numbers. That is why modern programming languages allow you to replace a variable address with its name. This name is translated into memory address while compiling or interpreting the program. Thus, the compiler or interpreter takes care of "remembering" large numbers.

Why do we need variables? Our experience with `PATH` and positional parameters has shown that variables store some data. It is needed for one of the following purposes:

1. To transfer information from one part of a program or system to another.
2. To store the intermediate result of a calculation for later use.
3. Save the current state of the program or system. This state may determine future behavior.
4. Set a constant value to be used repeatedly later.

A programming language has a special type of variable for each of these purposes. The Bash language has it too.

## Classification of variables

The Bash interpreter has two modes of operation: interactive (shell) and non-interactive (scripting). In each mode, variables solve similar tasks. But the contexts of these tasks are different. Therefore, there are more features to classify variables in Bash than in other interpreted languages.

Let's simplify the terminology for convenience. It is not quite right, but it helps to avoid confusion. When we talk about scripts, we use the term "variable". When we talk about shell and command-line arguments, we use the term "parameter". These terms are often used synonymously.

There are four attributes for classifying variables in Bash. Table 3-1 shows them.

**Table 3-1. Variable Types in Bash**

| Classification Attribute | Variable Types | Definition | Examples |
|---|---|---|---|
| Declaration mechanism | User-defined variables | The user sets them. | `filename="README.txt" ; echo "$filename"` |
| | Internal variables | The interpreter sets them. It needs them to work correctly. | `echo "$PATH"` |
| | Special parameters | The interpreter sets them for the user. The user can read them but not write. | `echo "$?"` |

**Table 3-1. Variable Types in Bash**

| Classification Attribute | Variable Types | Definition | Examples |
|---|---|---|---|
| Scope[257] | Environment or global variables | They are available in any instance of the interpreter. The `env` utility lists them. | `echo "$PATH"` |
| | Local variables | They are available in a particular instance of the interpreter only. | `filename="README.txt" ; echo "$filename"` |
| Content type | String | It stores a string. | `filename="README.txt"` |
| | Integer | It stores an integer. | `declare -i number=10/2 ; echo "$number"` |
| | Indexed array | It stores a numbered list of lines. | `cities=("London" "New York" "Berlin") ; echo "${cities[1]}" cities[0]="London" ; cities[1]="New York" ; cities[2]="Berlin" ; echo "${cities[1]}"` |
| | **Associative array**[258] | It is a data structure with elements that are key-value pairs. Each key and value are strings. | `declare -A cities=( ["Alice"]="London" ["Bob"]="New York" ["Eve"]="Berlin" ) ; echo "${cities[Bob]}"` |
| Changeability | Constants | The user cannot delete them. They store values that cannot be changed. | `readonly CONSTANT="ABC" ; echo "$CONSTANT" declare -r CONSTANT="ABC" ; echo "$CONSTANT"` |
| | Variables | The user can delete them. They store values that can be changed. | `filename="README.txt"` |

Let's consider each type of variable.

[257] https://en.wikipedia.org/wiki/Scope_(computer_science)
[258] https://en.wikipedia.org/wiki/Associative_array

# Declaration Mechanism

## User-Defined Variables

The purpose of user variables is obvious from their name. The user declares them for his purposes. Such variables usually store intermediate results of the script, its state and frequently used constants.

To declare the user-defined variable, specify its name, put an equal sign, and type its value.

Here is an example. We declare a variable called `filename`. Its value equals the filename `README.txt`. The declaration of the variable looks like this:

```
filename="README.txt".
```

Spaces before and after the equal sign are not allowed. Other programming languages allow them, but Bash does not. It leads to an error when Bash handles the following declaration:

```
filename = "README.txt"
```

Bash misinterprets this line. It assumes that you call the command with the `filename` name. Then you pass there two parameters: `=` and `"README.txt"`.

Only Latin characters, numbers and the underscore are allowed in variable names. The name must not start with a number. Letter case is important. It means that `filename` and `FILENAME` are two different variables.

Suppose we have declared a variable `filename`. Then Bash allocates the memory area for that. It writes the `README.txt` string there. You can read this value back by specifying the variable name. But when you do that, Bash should understand your intention. If you put a dollar sign before the variable name, it would be a hint for Bash. Then it treats the word `filename` as the variable name.

When you reference the variable in a command or script, it should look like this:

```
$filename
```

Bash handles words with a dollar sign in a special way. When it encounters such a word, it runs the **parameter expansion** mechanism. The mechanism replaces all occurrences of a variable name by its value. Here is the command for example:

```
cp $filename ~
```

After parameter expansion, the command looks like this:

```
cp README.txt ~
```

There are nine kinds of expansions that Bash does. There is a strict order in which the interpreter performs them. The order is important. If you miss it, errors can occur. Let's consider an example of such an error.

Suppose we manipulate the "my file.txt" file in the script. For the sake of convenience, we put the filename into a variable. Its declaration looks like this:

```
filename="my file.txt"
```

Then we use the variable in the `cp` call. Here is the command:

```
cp $filename ~
```

Bash does word splitting after parameter expansion. They are two different expansion mechanisms. When both of them are done, the `cp` call looks like this:

```
cp my file.txt ~
```

This command leads to an error. Bash passes two parameters to the `cp` utility: "my" and "file.txt". There are no such files.

Another error happens if the variable's value contains a special character. Here is an example:

```
1 filename="*file.txt"
2 rm $filename
```

The `rm` utility deletes all files ending in `file.txt`. The globbing mechanism causes such behavior. It happens because Bash does globbing after parameter expansion too. Then it substitutes files of the current directory whose names match the "*file.txt" pattern. It leads to unexpected results. Here is an example of the possible command:

```
rm report_file.txt myfile.txt msg_file.txt
```

When referencing variables, always apply double-quotes. They prevent unwanted Bash expansions. Here are the examples:

```
filename1="my file.txt"
cp "$filename1" ~

filename2="*file.txt"
rm "$filename2"
```

Thanks to the quotes, Bash inserts the variables' values as it is:

```
cp "my file.txt" ~
rm "*file.txt"
```

We already know several Bash expansions[259]. Table 3-2 gives their complete list and order of execution.

**Table 3-2. Bash expansions**

| Order of Execution | Expansion | Description | Example |
|---|---|---|---|
| 1 | Brace Expansion[260] | It generates a set of strings by the specified pattern with braces. | `echo a{d,c,b}e` |
| 2 | Tilde Expansion[261] | Bash replaces the tilde by the value of the `HOME` variable. | `cd ~` |
| 3 | Parameter Expansion[262] | Bash replaces parameters and variables by their values. | `echo "$PATH"` |
| 4 | Arithmetic Expansion[263] | Bash replaces arithmetic expressions by their results. | `echo $((4+3))` |
| 5 | Command Substitution[264] | Bash replaces the command with its output. | `echo $(< README.txt)` |

[259] http://www.gnu.org/savannah-checkouts/gnu/bash/manual/bash.html#Shell-Expansions
[260] https://www.gnu.org/software/bash/manual/html_node/Brace-Expansion.html
[261] https://www.gnu.org/software/bash/manual/html_node/Tilde-Expansion.html
[262] https://www.gnu.org/software/bash/manual/html_node/Shell-Parameter-Expansion.html
[263] https://www.gnu.org/software/bash/manual/html_node/Arithmetic-Expansion.html
[264] https://www.gnu.org/software/bash/manual/html_node/Command-Substitution.html

**Table 3-2. Bash expansions**

| Order of Execution | Expansion | Description | Example |
|---|---|---|---|
| 6 | Process Substitution[265] | Bash replaces the command with its output. Unlike Command Substitution, it is done asynchronously[266]. The command's input and output are bound to a temporary file. | `diff <(sort file1.txt) <(sort file2.txt)` |
| 7 | Word Splitting[267] | Bash splits command-line arguments into words and passes them as separate parameters. | `cp file1.txt file2.txt ~` |
| 8 | Filename Expansion[268] (globbing) | Bash replaces patterns with filenames. | `rm ~/delete/*` |
| 9 | Quote Removal | Bash removes all unshielded \, ‘ and “ characters that were not derived from one of the expansions. | `cp "my file.txt" ~` |

**Exercise 3-1. Testing the Bash expansions**

```
Run the example of each Bash expansion from Table 3-2 in the terminal.
Figure out how the final command turned out.
Come up with your own examples.
```

The dollar sign before a variable name is a shortened form of parameter expansion. Its full form looks this way:

```
${filename}
```

Use the full form to avoid ambiguity. Here is an example when the text follows the variable name:

[265] https://www.gnu.org/software/bash/manual/html_node/Process-Substitution.html
[266] https://en.wikipedia.org/wiki/Asynchrony_(computer_programming)
[267] https://www.gnu.org/software/bash/manual/html_node/Word-Splitting.html
[268] https://www.gnu.org/software/bash/manual/html_node/Filename-Expansion.html

```
prefix="my"
name="file.txt"
cp "$prefix_$name" ~
```

In this case, Bash looks for a variable named "prefix_". It happens because the interpreter appends the underscore to the variable name. The full form of the parameter expansion solves this problem:

```
cp "${prefix}_${name}" ~
```

There is an alternative solution. Enclose each variable name in double-quotes. Here is an example:

```
cp "$prefix"_"$name" ~
```

The full form of parameter expansion helps when the variable has not been defined. In that case, you can insert the specified value instead. Do it like this:

```
cp file.txt "${directory:-~}"
```

Here Bash checks if the `directory` variable is defined and has a non-empty value. If it is, Bash performs a regular parameter expansion. Otherwise, it inserts the value that follows the minus character. It is the user's home directory in our example.

The full form of parameter expansion has several variations. Table 3-3 shows all of them.

**Table 3-3. The full form of parameter expansion**

| Variation | Description |
|---|---|
| `${parameter:-word}` | If the "parameter" variable is not declared or has an empty value, Bash inserts the specified "word" value instead. Otherwise, it inserts the variable's value. |
| `${parameter:=word}` | If a variable is not declared or has an empty value, Bash assigns it the specified value. Then it inserts this value. Otherwise, Bash inserts the variable's value. You cannot override positional and special parameters in this way. |
| `${parameter:?word}` | If the variable is not declared or has an empty value, Bash prints the specified value on the error stream. Then, it terminates the script with a non-zero exit status. Otherwise, Bash inserts the variable's value. |
| `${parameter:+word}` | If the variable is not declared or has an empty value, Bash skips the expansion. Otherwise, it inserts the specified value. |

**Exercise 3-2. The full form of parameter expansion**

```
Write a script that searches for files with TXT extension in the current directory.
The script ignores subdirectories.
Copy or move all found files to the user's home directory.
When calling the script, you can choose whether to copy or move the files.
If no action is specified, the script chooses to copy the files.
```

## Internal variables

The user can declare variables. Bash also can do that. In this case, they are called **internal** or **shell variables**. The interpreter assigns the default values to them. The user can change some shell variables.

Internal variables have two functions:

1. Passing information from the shell to the application it runs.
2. Storing the current state of the interpreter itself.

The variables are divided into two groups:

1. Bourne Shell variables.
2. Bash variables.

The first group of variables comes from Bourne Shell. Bash needs it for POSIX compatibility. Table 3-4 shows the frequently used of these variables.

**Table 3-4. Bourne Shell variables**

| Name | Value |
|---|---|
| `HOME` | The home directory of the current user. Bash uses this variable for doing tilde expansion and processing the `cd` call without parameters. |
| `IFS`[269] | It contains a list of delimiter characters. The word splitting mechanism uses these characters to split the strings into words. The default delimiters are space, tab[270] and a line break. |
| `PATH` | It contains a list of paths. Bash uses the list to look for utilities and programs when the user runs them. Colons separate the paths in the list. |
| `PS1` | It is a command prompt. The prompt can include control characters[271]. Bash replaces them with specific values (for example, the current user's name). |
| `SHELLOPTS` | A list of shell options[272]. They change the operating mode of interpreter. Colons separate the options in the list. |

[269] http://mywiki.wooledge.org/IFS
[270] https://en.wikipedia.org/wiki/Tab_key#Tab_characters
[271] https://www.gnu.org/software/bash/manual/html_node/Controlling-the-Prompt.html#Controlling-the-Prompt
[272] https://www.gnu.org/software/bash/manual/html_node/The-Shopt-Builtin.html#The-Shopt-Builtin

In addition to the inherited Bourne Shell variables, Bash introduces new ones. Table 3-5 shows them. The list is incomplete. There are some extra variables, but they are rarely used.

**Table 3-5. Bash variables**

| Name | Value |
|---|---|
| `BASH` | The full path to the Bash executable file. This file corresponds to the current Bash process. |
| `BASHOPTS` | A list of Bash-specific shell options[273]. They change the operating mode of Bash. Colons separate the options in the list. |
| `BASH_VERSION` | The version of the running Bash interpreter. |
| `GROUPS` | A list of groups to which the current user belongs. |
| `HISTCMD` | The number of the current command in the command history. It shows you how many items are in history. |
| `HISTFILE` | The path to the file that stores the command history. The default value is `~/.bash_history`. |
| `HISTFILESIZE` | The maximum allowed number of lines in the command history. The default value is 500. |
| `HISTSIZE` | The maximum allowed amount of items in the command history. The default value is 500. |
| `HOSTNAME` | The name of the current computer as a node on the computer network. |
| `HOSTTYPE` | A string describing the hardware platform on which Bash is running. |
| `LANG` | Locale settings[274] for the user interface. They define the user's language, region and some special characters. Some settings are overridden by variables `LC_ALL`, `LC_COLLATE`, `LC_CTYPE`, `LC_MESSAGES`, `LC_NUMERIC`, `LC_TYPE`. |
| `MACHTYPE` | A string describing the system on which Bash is running. It includes information from the `HOSTTYPE` and `OSTYPE` variables. |
| `OLDPWD` | The previous working directory, which was set by the `cd` command. |
| `OSTYPE` | A string describing the OS on which Bash is running. |
| `POSIXLY_CORRECT` | If this variable is defined, Bash runs in the POSIX compatible mode[275]. |
| `PWD` | The current directory that the `cd` command has set. |

[273] https://www.gnu.org/software/bash/manual/html_node/The-Shopt-Builtin.html#The-Shopt-Builtin
[274] https://en.wikipedia.org/wiki/Locale_(computer_software)
[275] https://www.gnu.org/software/bash/manual/html_node/Bash-POSIX-Mode.html#Bash-POSIX-Mode

**Table 3-5. Bash variables**

| Name | Value |
|---|---|
| `RANDOM` | Each time the user reads this variable, Bash returns a random number between 0 and 32767. When the user writes the variable, Bash assigns a new initializing number (seed[276]) to the pseudorandom number generator[277]. |
| `SECONDS` | The number of seconds elapsed since the current Bash process started. |
| `SHELL` | The path to the shell executable file for the current user. Each user can use his own shell program. |
| `SHLVL` | The nesting level of the current Bash instance. This variable is incremented by one each time you start Bash from itself. |
| `UID` | The ID number of the current user. |

The internal variables are divided into three groups depending on the allowed actions with them. These are the groups:

1. Bash assigns a value to a variable at startup. It remains unchanged throughout the session. The user can read it, but changing is prohibited. Examples: `BASHOPTS`, `GROUPS`, `SHELLOPTS`, `UID`.
2. Bash assigns a default value to a variable at startup. User actions or other events change this value. Some variables can be explicitly re-assigned, but this can disrupt the interpreter. Examples: `HISTCMD`, `OLDPWD`, `PWD`, `SECONDS`, `SHLVL`.
3. Bash assigns a default value to the variable at startup. The user can change it. Examples: `HISTFILESIZE`, `HISTSIZE`.

## Special Parameters

Bash declare special parameters and assign values to them. It works in the same way as for shell variables.

Special parameters pass information from a shell to a launching application and vice versa. We have already considered positional parameters. All of them are special parameters.

Table 3-6 shows frequently used special parameters.

[276] https://en.wikipedia.org/wiki/Random_seed
[277] https://en.wikipedia.org/wiki/Pseudorandom_number_generator

**Table 3-6. Bash Special Parameters**

| Name | Value |
|---|---|
| `$*` | It contains all positional parameters passed to the script. Parameters start with the `$1` variable but not with `$0`. If you skip double-quotes (`$*`), Bash inserts each positional parameter as a separate word. With double-quotes ("`$*`"), Bash handles it as a single quoted string. The string contains all the parameters separated by the first character of the internal variable `IFS`. |
| `$@` | The array that contains all positional parameters passed to the script. Parameters start with the `$1` variable. If you skip double-quotes (`$@`), Bash handles each array's element as an unquoted string. Word splitting happens in this case. With double-quotes ("$@"), Bash handles each element as a quoted string without word splitting. |
| `$#` | The number of positional parameters passed to the script. |
| `$1`, `$2`... | It contains the value of the corresponding positional parameter. `$1` matches the first parameter. `$2` matches the second one, etc. The numbers are given in the decimal system. |
| `$?` | The exit status of the last executed command in the foreground mode. If a pipeline was executed, the parameter stores the exit status of the last command in the pipeline. |
| `$-` | It contains options for the current interpreter instance. |
| `$$` | The process ID of the current interpreter instance. If you use it in the subshell, Bash inserts the PID of the parent process. |
| `$!` | The process ID of the last command that was launched in the background mode. |
| `$0` | The name of the shell or script that is currently running. |

You cannot change special Bash parameters directly. For example, the following redeclaration of `$1` does not work:

```
1="new value"
```

If you want to redeclare positional parameters, use the `set` command. It redeclares all parameters at once. There is no option to change a single parameter only. Here is the form of the `set` call:

```
set -- NEW_VALUE_OF_$1 NEW_VALUE_OF_$2 NEW_VALUE_OF_$3...
```

What to do if you need to change a single positional parameter? Suppose you call the script with four parameters. For example, like this:

```
./my_script.sh arg1 arg2 arg3 arg4
```

You want to replace the third parameter `arg3` with the `new_arg3` value. Here is the `set` call for that:

```
set -- "${@:1:2}" "new_arg3" "${@:4}"
```

Let's consider this call in detail. Bash replaces the first argument "$" by the first two elements of the `$@` array. It leads that `$1` and `$2` parameters get their previous values. Then there is the new value for the parameter `$3`. Now it equals "new_arg3". Then there is the "$" value. Here Bash inserts all elements of the `$@` array starting from `$4`. It means that all these parameters get their previous values.

All special parameters from Table 3-6 are available in the POSIX-compatible mode.

## Scope

### Environment Variables

Variables are divided into **scopes** (scope) in any software system. A scope is a part of a program or system where the variable name remains associated with its value. In other words, you can convert the variable name into its address in the scope of that variable only. Outside the scope, the same name can point to another variable.

A scope is called **global** if it spreads to the whole system. For example, the variable called `filename` is in the global scope. Then you can access it by its name from any part of the system.

Bash keeps all its internal variables in the global scope. They are called **environment variables**. It means that all internal variables are environment variables. The user can declare his variable in the global scope too. Then it becomes an environment variable.

Why does Bash store variables in the global scope? It happens because Unix has a special set of settings. They affect the behavior of the applications the user runs. An example is locale settings. According to them, each running application adapts its interface. Applications share such kinds of settings through environment variables.

Suppose one process spawns a child process. The child process inherits all environment variables of the parent. Thus, all utilities and applications launched from the shell inherit its environment variables. This way, global Unix settings are spread to all user programs.

The child process can change its environment variables. Then it spawns another process. This new process inherits the changed variables. However, when the child changes its environment variables, it does not affect the corresponding variables of the parent process.

The `export` built-in command declares an environment variable. Here is an example of doing that:

```
export BROWSER_PATH="/opt/firefox/bin"
```

You can declare the variable and then add it to the global scope. Call the `export` command in this way:

```
1 BROWSER_PATH="/opt/firefox/bin"
2 export BROWSER_PATH
```

Sometimes you want to declare the environment variables for the specific application only. Then list the variables and their values before the application call. Here is an example:

```
MOZ_WEBRENDER=1 LANG="en_US.UTF-8" /opt/firefox/bin/firefox
```

Here the Firefox browser receives the specified `MOZ_WEBRENDER` and `LANG` variables. They can differ from the system-wide settings.

Suppose that you use another interpreter as the shell. An example is Bourne Shell. Then you should apply the `env` utility. It declares environment variables for the launching application. Here is an example:

```
env MOZ_WEBRENDER=1 LANG="en_US.UTF-8" /opt/firefox/bin/firefox
```

If you call the `env` utility without parameters, it prints all declared environment variables for the current interpreter process. Try to get this output in your terminal:

```
env
```

The `export` command and the `env` utility print the same thing if called without parameters. Prefer to use `export` instead of `env`. There are two reasons for that. First, the `export` output is sorted. Second, all variable values are enclosed in double-quotes. They prevent you from making a mistake if there is a line break in some value.

Environment variable names have uppercase letters only. Therefore, it is considered good practice to name local variables in lower case. It prevents you from accidentally using one variable instead of another.

## Local Variables

We know about the user-defined variables. The user can declare them in several ways. Depending on his choice, the new variable comes to the **local scope** or global scope (environment).

There are two ways to declare the global scope variable:

1. Add the `export` command to the variable declaration.

2. Pass the variable when launching the program. You can do it with the `env` utility when using a shell other than Bash.

If you do not apply any of these options, your variable comes to the local scope. It is called the **local variable**. It is available in the current instance of the interpreter. Child processes (except subshell) does not inherit it.

Here is an example. Suppose that you declare a variable in the terminal window like this:

```
filename="README.txt"
```

Now you can output its value in the same terminal window. Call the following `echo` command for that:

```
echo "$filename"
```

The same command works well in a subshell. Just add parentheses to spawn a subshell for the specific command. It looks like this:

```
(echo "$filename")
```

However, if you read the variable from a child process, you get an empty value. You can start a child process by calling the Bash explicitly in the terminal window. Here is an example:

```
bash -c 'echo "$filename"'
```

The `-c` parameter passes a command to be executed by the Bash child process. A similar Bash call occurs implicitly when running the script from a shell.

We enclose the `echo` call in the single-quotes when passing it to the `bash` command. The quotes disable all Bash expansions for the string inside. This behavior differs from the double-quotes. They disable all expansions except command substitution and parameter expansion. If we use double-quotes in our `bash` call, the parameter expansion happens. Then Bash inserts the variable's value in the call. The call would look like this:

```
bash -c "echo README.txt"
```

We are not interested in such a call. Instead, we want to check how the child process reads the local variable. The parent process should not put its value into the `bash` call.

If you change a local variable in the subshell, the parent process keeps its old value. For example, the following `echo` command prints the "README.txt" filename:

```
filename="README.txt"
(filename="CHANGELOG.txt")
echo "$filename"
```

This output confirms that changing the local variable in the subshell does not affect the parent process.

When you declare a local variable, it comes in the shell variables list. The list includes all local and environment variables that are available in the current interpreter process. The `set` command prints this list when called without parameters. Here is an example of how to find the `filename` variable there:

```
set | grep filename=
```

There is the following line in the command's output:

```
filename=README.txt
```

It means that the `filename` variable is in the list of shell variables.

# Variable Content Type

## Variable Types

In compiled programming languages (such as C), it is common to use **static type system**[278]. When using this system, you decide how to store the variable in memory. You should specify the variable type when declaring it. Then the compiler allocates the memory and chooses an appropriate format for this type of variable.

Here is an example of how the static type system works. Suppose we declare a variable called `number`. We should specify its type in the declaration. We choose the unsigned integer type, which has a size of two bytes. Then the compiler allocates exactly two bytes of memory for this variable.

When the application starts, we assign the 203 value to the variable. It is equal to 0xCB in hexadecimal. Then the variable looks this way in the memory:

```
00 CB
```



A modern computer uses the binary form to store information in the memory. Here we use the hexadecimal format instead for clarity.

[278] https://en.wikipedia.org/wiki/Type_system#Static_type_checking

One byte is enough to store the 203 value. But we force the compiler to reserve two bytes for that. The unused byte stays zero. No one can use it in the scope of the `number` variable. If the variable has the global scope, the byte is reserved and unused while the application works.

Suppose that we assign the 14037 value to the variable. It is equal to 0x36D5 in hexadecimal. Then it looks like this in the memory:

```
36 D5
```



The CPU determines the byte order when storing data in the computer memory. This byte order is called **endianness**[279]. It is **big-endian** in our example. The alternative order is **little-endian**.

Now we want to store the value 107981 (0x1A5CD) in the variable. This number does not fit into two bytes. But the variable's size is defined in the declaration. The compiler cannot extend it automatically afterward. Therefore, the 107981 value will be truncated to two bytes. It looks like this in the memory:

```
A5 CD
```

The first digit of the value has been discarded. If you read the `number` variable, you get 42445 (0xA5CD). It means that the original 107981 is lost. You cannot recover it anymore. This problem is called **integer overflow**[280].

Here is another example of the static type system. Suppose we want to store the username in a variable called `username`. We declare this variable of the string type. When doing that, we should specify the maximum length of the string. Let's choose the length of the ten characters.

Now we write the name "Alice" to the variable. If you use the C compiler, the string looks like this in memory:

```
41 6C 69 63 65 00 00 00 00 00
```



The C compiler uses the ASCII encoding for characters by default. You can clarify the hexadecimal code of each letter in the ASCII table[281].

Six bytes are enough to store the string "Alice". The first five bytes store characters. The last sixth byte stores the null character (00). It marks the end of the string. However, we have reserved ten bytes for the variable. Therefore, the compiler fills the unused memory with zeros or random values.

[279] https://en.wikipedia.org/wiki/Endianness
[280] https://en.wikipedia.org/wiki/Integer_overflow
[281] http://www.asciitable.com

**Dynamic type system**[282] is an alternative to the static system. Here there is another way to choose how to store a variable in memory. This choice happens whenever you assign the new value to the variable. Together with the value, the variable gets new **metadata**[283]. The metadata defines the variable type. They can change during the application work. Thus, the variable's representation in memory changes too. Most interpreted programming languages use the dynamic type system (for example, Python).



Metadata is additional information about some object or data. The library catalog[284] is an example of metadata. It has a card for each book. The card contains the author, title of the work, publisher, year of publication, and the number of pages. Thus, The card contains metadata about the book.

Strictly speaking, Bash does not have the type system. It is not a language with the static or dynamic type system. Bash stores all **scalar variables**[285] as strings in memory.

The scalar variable stores data of **primitive type**[286]. These data are the minimal building blocks to construct more complex **composite types**[287]. The scalar variable is just a name for the memory address where its value is stored.

Here is how Bash represents scalar variables in memory. There is the following declaration:

```
declare -i number=42
```

Bash stores the `number` variable in memory as the following string:

```
34 32 00
```

The language with the type system needs one byte to store this number. But Bash needs three bytes. The first two bytes store each character of the number: `4` and `2`. The third byte stores the null character.

The Bourne Shell language has the scalar variables only. Bash introduces two new composite types: **indexed array**[288] and **associative array**[289].

The indexed array is a numbered set of strings. There each string corresponds to the sequence number. Bash stores such an array as **linked list**[290] in memory. A linked list is a data structure that consists of nodes. Each node contains data and the memory address of the next node. Node data are strings in this case.

[282] https://en.wikipedia.org/wiki/Type_system#Dynamic_type_checking_and_runtime_type_information
[283] https://en.wikipedia.org/wiki/Metadata
[284] https://en.wikipedia.org/wiki/Library_catalog
[285] https://en.wikipedia.org/wiki/Variable_(computer_science)
[286] https://en.wikipedia.org/wiki/Primitive_data_type
[287] https://en.wikipedia.org/wiki/Composite_data_type
[288] https://en.wikipedia.org/wiki/Array_data_structure
[289] https://en.wikipedia.org/wiki/Associative_array
[290] https://en.wikipedia.org/wiki/Linked_list

The associative array is a more complicated thing. It is a set of elements. Each element consists of two strings. The first one is called the "key". The second is "value". When you want to access the array's element, you should specify its key. It works the same as for the indexed array, where you should specify the element's index. The keys are unique. It means that two elements with the same keys are not allowed. Bash stores associative array as **hash-table**[291] in memory.

Why are Bash "arrays" called arrays? Actually, they are linked lists and hash tables. A real array is the data structure whose elements are stored in memory one after another. Each element has a sequential number called an **index** or identifier. Bash "arrays" do not store their elements sequentially in memory. Thus, they are not arrays by definition.

Here is an example of how a real array stores its elements in memory. Suppose we have an array with numbers from five to nine. Each element takes one byte. Then the size of the array is five bytes. The array looks like this in memory:

```
05 06 07 08 09
```

The indexing of arrays' elements starts with zero. It means that the first element has the `0` index. The second one has the `1` index and so on. In our example, the first element with the `0` index equals the number `5`. The second element equals `6`. Elements follow each other in memory. Their indexes match the memory offset from the beginning of the array. Thus, the element with the `3` index has three bytes offset and equals the number `8`.

Let's come back to the question about naming the Bash "arrays". Only the authors of the language can answer it. However, we can guess. The name "array" gives the user a hint on how to work with them. When the user has experience in another language, he knows how to operate with a regular array. This way, he can start using Bash "arrays" immediately. The user does not need knowledge on how Bash stores these "arrays" internally.

## Attributes

The Bash language does not have the type system. It stores all scalar variables in memory as strings. But Bash has composite types that are arrays. The Bash array is the combination of strings.

When you declare the Bash variable, you should choose if it is scalar or composite. You can do that by specifying metadata for the variable. Such metadata is called **attributes** in Bash. The variable's attributes also define its constancy and scope.

The `declare` built-in command specifies the variable's attributes. The command prints all local and environment variables when calling without parameters. The `set` command prints the same output.

The `declare` command has the `-p` option. It adds variable attributes to the output.

If you want information on a particular variable, pass its name to the `declare` command. Here is an example for the `PATH` variable:

[291] https://en.wikipedia.org/wiki/Hash_table

```
declare -p PATH
```

The `declare` command also prints information about declared **subroutines**[292]. They are called **functions** in Bash. A function is a program fragment or an independent block of code that performs a certain task.

Suppose you are interested in declarations of the function but not in variables. Then use the `-f` option of the `declare` command. It filters out variables from the output. The `declare` call looks the following in this case:

```
declare -f
```

You can specify the function name right after the `-f` option. Then the `declare` command prints information about this function only. Here is an example for the function `quote`:

```
declare -f quote
```

This command displays the declaration of the function.

The `quote` function receives a string on the input. It encloses the string in single-quotes. If the string already contains the single-quotes, the function escapes them. You can call the function in the same way as a built-in Bash command. Here is an example:

```
quote "this is a 'test' string"
```

The `declare` command without the `-p` option does not print the function declaration. It means that the following call shows nothing:

```
declare quote
```

The `declare` command shows information about already declared variables and functions. Also, the command sets attributes for new variables.

Table 3-7 shows the frequently used options of the `declare` command.

[292] https://en.wikipedia.org/wiki/Subroutine

**Table 3-7. The `declare` command options and the corresponding variables' attributes**

| Option | Definition |
|---|---|
| `-a` | The declared variable is an indexed array. |
| `-A` | The declared variable is an associative array. |
| `-g` | It declares a variable in the global scope of the script. The variable does not come to the environment. |
| `-i` | It declares an integer variable. When you assign it a value, Bash treats it as an arithmetic expression. |
| `-r` | It declares a constant. The constant cannot change its value after declaration. |
| `-x` | It declares an environment variable. |

Here are examples of declaration variables with attributes. First, let's compare integer and string variables. Execute the following two commands in the terminal window:

```
declare -i sum=11+2
text=11+2
```

We declared two variables named `sum` and `text`. The `sum` variable has the integer attribute. Therefore, its value equals 13 that is the sum of 11 and 2. The `text` variable is equal to the "11+2" string.

Bash stores both variables as strings in memory. The `-i` option does not specify the variable's type. Instead, it limits the allowed values of the variable.

Try to assign a string to the `sum` variable. Here are a couple of possible ways for doing that:

```
declare -i sum="test"
sum="test"
```

Each of these commands set the `sum` value to zero.

Suppose you have declared an integer variable. Then you do not need a Bash expansion for arithmetic operations on it. The following commands do correct calculations:

```
sum=sum+1       # 13 + 1 = 14
sum+=1          # 14 + 1 = 15
sum+=sum+1      # 15 + 15 + 1 = 31
```

The calculation results come after the hash symbol. Bash ignores everything after this symbol. Such lines are called **comments**.

Now execute the same commands with the string variable. Their results differ:

```
text=text+1     # "text+1"
text+=1         # "text+1" + "1" = "text+11"
text+=text+1    # "text+11" + "text" + "1" = "text+11text+1"
```

Here Bash does string concatenation instead of arithmetic operations on numbers. You should apply the arithmetic expansion. Then Bash does arithmetic calculations instead. Here is an example:

```
text=11
text=$(($text + 2)) # 11 + 2 = 13
```

The `-r` option of the `declare` command makes a constant. The call looks like this:

```
declare -r filename="README.txt"
```

Whenever you change the value of the `filename` constant or delete it, Bash prints an error message. Therefore, both following commands fail:

```
filename="123.txt"
unset filename
```



Use the `unset` command for removing variables. The command cannot remove constants.

The `declare` command with the `-x` option declares an environment variable. It has the same effect as the `export` command in the declaration. Thus, the following two commands are equivalent:

```
export BROWSER_PATH="/opt/firefox/bin"
declare -x BROWSER_PATH="/opt/firefox/bin"
```

The good practice is to use the `export` command instead of `declare` with the `-x` option. Such a decision improves the code readability. You don't need to remember what the `-x` option means. For the same reason, prefer the `readonly` command instead of `declare` with the `-r` option. Both commands declare a constant. But `readonly` is easier to remember.

The `readonly` command declares a variable in the global scope of the script. The `declare` command with the `-r` option has another result. If you call `declare` in the body of a function, you declare a local variable. It is not available outside the function. Use the `-g` option to get the same behavior as `readonly`. Here is an example:

```
declare -gr filename="README.txt"
```

## Indexed Arrays

Bourne Shell has scalar variables only. The interpreter stores them as strings in memory. Such variables were not enough for users. Therefore, Bash developers have added arrays. When do you need an array?

Strings have a serious limitation. When you write a value to the scalar variable, it is a single unit. For example, you save a list of files in the variable called `files`. You separate filenames by spaces in this list. However, the `files` variable stores a single string from the Bash point of view. This fact leads to errors.

Here is an example. The POSIX standard allows any characters in filenames except the null character (NULL). NULL means the end of a filename. The same character means the end of a string in Bash. Therefore, a string variable can contain NULL at the end only. It turns out that you have no reliable way to separate filenames in a string. You cannot use NULL. But any other delimiter character can occur in these names.

The delimiter problem prevents processing of the `ls` utility output reliably. The utility cannot separate its output units with NULL. It leads to a recommendation to avoid parsing of the `ls` output. Also, do not use `ls` in variable declarations like this:

```
files=$(ls Documents/*.txt)
```

This declaration writes all TXT files of the `Documents` directory to the `files` variable. If there are spaces or line breaks in the filenames, it will be hard to restore them.

Bash arrays solve this problem. An array stores a list of separate units. It is simple to read them in their original form. Therefore, use arrays to store filenames instead of strings with the `ls` output. Here is a better declaration of the `files` variable:

```
declare -a files=(Documents/*.txt)
```

This command declares and **initializes the array**. Initializing means assigning values to the array's elements. You can do that in the declaration or afterward.

Bash can deduce the array type of the variable by itself. This mechanism works when you skip the `declare` command in a variable declaration. Bash adds the appropriate attribute automatically. Here is an example:

```
files=(Documents/*.txt)
```

The command declares the indexed array `files`.

Suppose that you know the array elements in advance. In this case, you can assign them explicitly in the declaration. It looks like this:

```
files=("/usr/share/doc/bash/README" "/usr/share/doc/flex/README.md" "/usr/share/doc/\
xz/README")
```

When assigning elements of an array, you can read them from other variables. Here is an example:

```
1 bash_doc="/usr/share/doc/bash/README"
2 flex_doc="/usr/share/doc/flex/README.md"
3 xz_doc="/usr/share/doc/xz/README"
4 files=("$bash_doc" "$flex_doc" "$xz_doc")
```

This command writes values of the variables `bash_doc`, `flex_doc` and `xz_doc` to the `files` array. If you change these variables after this declaration, it does not affect the array.

When declaring an array, you can explicitly specify an index for each element. Do it like this:

```
1 bash_doc="/usr/share/doc/bash/README"
2 flex_doc="/usr/share/doc/flex/README.md"
3 xz_doc="/usr/share/doc/xz/README"
4 files=([0]="$bash_doc" [1]="$flex_doc" [5]="/usr/share/doc/xz/README")
```

Here there are no spaces before and after each equal sign. Remember this rule: when you declare any variable in Bash, there are no spaces before or after the equal sign.

Instead of initializing the entire array at once, you can assign its elements separately. Here is an example:

```
1 files[0]="$bash_doc"
2 files[1]="$flex_doc"
3 files[5]="/usr/share/doc/xz/README
```

There are gaps in the array indexes in the last two examples. It is not a mistake. Bash allows arrays with such gaps. They are called **sparse arrays**.

Suppose that we have declared the array. Now there is a question of how to read its elements. The following expansion prints all of them:

```
$ echo "${files[@]}"
/usr/share/doc/bash/README /usr/share/doc/flex/README.md /usr/share/doc/xz/README
```

You see the `echo` command at the first line. There is its output in the second line.



Here, the `$` character means the command-line prompt. The `echo` command follows it. There is its output on the next line.

It can be useful to print indexes of elements instead of their values. For doing that, add an exclamation mark in front of the array name in the parameter expansion. Do it like this:

```
$ echo "${!files[@]}"
0 1 5
```

You can calculate the element index with a formula when accessing it. Specify the formula in square brackets. Here are examples for reading and writing the fifth element:

```
echo "${files[4+1]}"
files[4+1]="/usr/share/doc/xz/README
```

You can use variables in the formula. Bash accepts both integer and string variables there. Here is another way to access the fifth element of the array:

```
i=4
echo "${files[i+1]}"
files[i+1]="/usr/share/doc/xz/README
```

Bash can insert the sequential array elements at once. To do that, specify the starting index, colon and the number of elements. Here is an example:

```
$ echo "${files[@]:1:2}"
/usr/share/doc/flex/README.md /usr/share/doc/xz/README
```

This `echo` call prints two elements, starting from the first. The elements' indexes are not important in this case. We get the filenames with indexes 1 and 5.

Starting with version 4, Bash provides the `readarray` command. It is also known as `mapfile`. The command reads the contents of a text file into an indexed array. Let's see how to use it.

Suppose we have the file named `names.txt`. It contains names of some persons:

```
Alice
Bob
Eve
Mallory
```

We want to create an array with strings of this file. The following command does that:

```
readarray -t names_array < names.txt
```

The command writes the `names.txt` file contents to the `names_array` array.

**Exercise 3-3. Declaration of arrays**

Try all the following variants of the array declarations:

1. Using the declare command.

2. Without the declare command.

3. The globbing mechanism provides values for array elements.

4. Specify all array elements in the declaration.

5. Specify the elements separately after the array declaration.

6. Assign to array elements the values of the existing variables.

7. Read the array elements from a text file.

Print the array contents using the echo command for each case.

We have learned how to declare and initialize indexed arrays. Here are some more examples of using them. Suppose the `files` array contains a list of filenames. You want to copy the first file in the list. The following `cp` call does that:

```
cp "${files[0]}" ~/Documents
```



Most programming languages number the elements of arrays and characters of strings from zero, but not from one. Bash follows this rule too.

When reading an array element, we apply the full form of the parameter expansion with curly brackets. Put the index of the element in square brackets after the variable name.

When you use the @ symbol instead of the element's index, Bash inserts all array elements. Here is an example:

```
cp "${files[@]}" ~/Documents
```

You can get the size of the array. Put the # character in front of its name. Then specify the @ character as the element's index. You get the following parameter expansion:

```
echo "${#files[@]}"
```

When reading array elements, always use double-quotes. They prevent word splitting.

Use the `unset` command to remove an array element. Here is an example of removing the fourth element:

```
unset 'files[3]'
```

Do not forget about numbering array's elements from zero. Also, the single-quotes are mandatory here. They turn off all Bash expansions.

The `unset` command can clear the whole array. Here is an example:

```
unset files
```

## Associative Arrays

We have considered the indexed arrays. Their elements are strings. Each element has an index that is a positive integer. The indexed array gives you a string by its index.

The developers introduced associative arrays in the 4th version of Bash. The elements' indexes are not numbers but strings in such arrays. This kind of string-index is called **key**. The associative array gives you a string-value by its string-index. When is this feature useful?

Here is an example. Suppose we need a script that stores the list of contacts. The script adds a person's name, email or phone number to the list. We can omit the person's last name for simplicity. When you need these data back, the script prints it on the screen.

We can solve the task using the indexed array. But this solution is inefficient for searching for a contact. The script should traverse over all array elements. It compares each element with the person's name that you are looking for. The script prints the corresponding contacts if it finds the person.

An associative array makes searching for contacts faster. The script should not pass through all array elements in this case. Instead, it gives the key to the array and gets the corresponding value back.

Here is a possible way to declare and initialize the associative array with contacts:

```
declare -A contacts=(["Alice"]="alice@gmail.com" ["Bob"]="(697) 955-5984" ["Eve"]="(\
245) 317-0117" ["Mallory"]="mallory@hotmail.com")
```

There is only one way to declare an associative array. For doing that, you should use the `declare` command with the `-A` option. Bash cannot deduce the array type, even if you specify string-indexes. Therefore, the following command declares the indexed array:

```
contacts=(["Alice"]="alice@gmail.com" ["Bob"]="(697) 955-5984" ["Eve"]="(245) 317-01\
17" ["Mallory"]="mallory@hotmail.com")
```

The following `declare` call prints this `contacts` variable:

```
$ declare -p contacts
declare -a contacts='([0]="mallory@hotmail.com")'
```

It is an indexed array with one element. It happens because Bash converts all string-indexes to zero value. Then every next contact in the initialization list overwrites the previous one. This way, the zero-index element contains the contacts of the last person in the initialization list.

You can specify elements of the array separately. Here is an example:

```
declare -A contacts
contacts["Alice"]="alice@gmail.com"
contacts["Bob"]="(697) 955-5984"
contacts["Eve"]="(245) 317-0117"
contact["Mallory"]="mallory@hotmail.com"
```

So, we have declared an associative array. Its elements are accessible by keys. The key is the name of a person in our case. Here is an example of reading the contacts by the person's name:

```
$ echo "${contacts["Bob"]}"
(697) 955-5984
```

Use the @ character as the key for printing all array's elements:

```
$ echo "${contacts[@]}"
(697) 955-5984 mallory@hotmail.com alice@gmail.com (245) 317-0117
```

If you add the exclamation mark before the array name, you get the list of all keys. In our case, you get the list of persons in the contacts. Here is an example:

```
$ echo "${!contacts[@]}"
Bob Mallory Alice Eve
```

Add the # character before the array name to get the size of the contacts list:

```
$ echo "${#contacts[@]}"
4
```

Let's put the associative array with contacts into the script. There you can pass the name of the person via the command-line parameter. The script processes it and prints the email or phone number of that person.

Listing 3-10 shows the script for managing the contacts.

**Listing 3-10. The script for managing the contacts**

```
#!/bin/bash

declare -A contacts=(
  ["Alice"]="alice@gmail.com"
  ["Bob"]="(697) 955-5984"
  ["Eve"]="(245) 317-0117"
  ["Mallory"]="mallory@hotmail.com")

echo "${contacts["$1"]}"
```

To edit contacts, change the array's initialization in the script.

The `unset` command deletes an associative array or its element. It works like this

```
unset contacts
unset 'contacts[Bob]'
```

Bash inserts several elements of an associative array in the same way as it does for an indexed array. Here is an example:

```
$ echo "${contacts[@]:Bob:2}"
(697) 955-5984 mallory@hotmail.com
```

Bash inserts two elements in this case:

- The one that corresponds to the `Bob` key.
- The next one in memory.

There is one problem with such an approach. The order of the associative array's elements in memory does not match their initialization order. The **hash function**[293] calculates each element's numerical index in memory. The function takes a key on input and returns a unique integer on output. Because of this feature, inserting several elements of the associative array is a bad practice.

[293] https://en.wikipedia.org/wiki/Hash_function

# Conditional Statements

We have met the conditional statements when learning the `find` utility. Then we found out that Bash has its own logical operators AND (&&) and OR (||). The Bash language has other options to make branches.

We will consider the `if` and `case` operators in this section of the book. The scripts use them often. Both operators provide similar behavior. However, each of them fits better for some specific tasks.

## If Statement

Imagine you are writing a one-line command. Such a command is called **one-liner**. You are trying to make it as compact as possible because a short command is faster to type. Also, there is less chance to make a mistake when typing.

Now imagine that you are writing a script. Your hard drive stores it. You call the script regularly and change it rarely. The code compactness is not important here. You follow another goal to make the script easy to read and modify.

The && and || operators fit well one-liners. But Bash has better options for scripts. Actually, it depends on the particular case. Sometimes you can use these operators in the script and keep its code clean. But they lead to hard-to-read code in most cases. Therefore, it is better to replace them with the `if` and `case` statements.

Here is an example. Have a look at Listing 3-9 again. There is a backup script there. It calls the `bsdtar` utility this way:

```
1  bsdtar -cjf "$1".tar.bz2 "$@" &&
2    echo "bsdtar - OK" > results.txt ||
3    { echo "bsdtar - FAILS" > results.txt ; exit 1 ; }
```

We have tried to make this script better to read. We split calls of the `bsdtar` and `mv` utilities. However, it is not enough. The `bsdtar` call is still too long and complicated for reading. Therefore, it is easy to make a mistake when modifying it. Such error-prone code is called **fragile**. A poor technical solution introduces it in most cases.

Let's improve the `bsdtar` call. First, we should write its algorithm step by step. It looks like this:

1. Read a list of files and directories from the `$@` variable. Archive and compress them.
2. If the archiving and compression were successful, write the line "bsdtar - OK" into the log file.
3. If an error occurred, write the line "bsdtar - FAILS" into the log file. Then terminate the script.

The last step is the most confusing. When `bsdtar` completes successfully, the script does one action only. When an error happens, there are two actions. These actions are combined into the single **command block**[294] by curly brackets[295]. This code block looks confusing.

The `if` statement is a solution for executing command blocks on specific conditions. We can write the statement in the general form like this:

```
if CONDITION
then
  ACTION
fi
```

You can write this statement in one line too. For doing that, add a semicolon before the `then` and `fi` words like this:

```
if CONDITION; then ACTION; fi
```

Both the CONDITION and ACTION here are a single command or a block of commands. If the CONDITION completes successfully with the zeroed exit status, Bash executes the ACTION.

Here is an example of the `if` statement:

```
if cmp file1.txt file2.txt &> /dev/null
then
  echo "Files file1.txt and file2.txt are identical"
fi
```

The CONDITION here is the `cmp` utility call. The utility compares the contents of two files. If they differ, `cmp` prints the position of the first character that is different. The exit status is non-zero in this case. If the files are the same, the utility returns the zero status.

When we call a utility in the `if` condition, its exit status matters only. Therefore, we redirect the `cmp` output to the `/dev/null`[296] file. It is a special system file. Writing there always succeeds. OS deletes all data that you write there.

If the contents of the `file1.txt` and `file2.txt` files match, the `cmp` utility returns the zero status. Then the `if` condition equals "true". In this case, the `echo` command prints the message.

We have considered a simple case. When the condition is met, the script does an action. But there are cases where a condition selects one of two possible actions. The `if-else` statement works in this way. Here is the statement in the general form:

[294] https://en.wikipedia.org/wiki/Block_(programming)
[295] https://www.gnu.org/software/bash/manual/html_node/Command-Grouping.html
[296] https://en.wikipedia.org/wiki/Null_device

```
if CONDITION
then
  ACTION_1
else
  ACTION_2
fi
```

The same `if-else` written in one line looks like this:

```
if CONDITION; then ACTION_1; else ACTION_2; fi
```

Bash executes ACTION_2 if CONDITION returns the non-zero exit status. The condition is "false" in this case. Otherwise, Bash executes the ACTION_1.

You can extend the `if-else` statement by the `elif` blocks. Such a block adds an extra condition and the corresponding action. Bash executes the action if the condition equals "true".

Here is an example. Suppose you choose one of three actions depending on the value of a variable. The following `if` statement provides this behavior:

```
If CONDITION_1
then
  ACTION_1
elif CONDITION_2
then
  ACTION_2
else
  ACTION_3
fi
```

There is no limitation for the number of `elif` blocks in the statement. You can add as many of them as you need.

Let's extend our example of file comparison. We want to print the message in both cases: when the files match and when they do not. The `if-else` statement does the job. It looks like this:

```
if cmp file1.txt file2.txt &> /dev/null
then
  echo "Files file1.txt and file2.txt are the same."
else
  echo "Files file1.txt and file2.txt differ."
fi
```

It is time to come back to our backup script. The script has a block of commands. The result of the `bsdtar` utility decides if Bash should execute this block. When we meet the code block and condition, it is a hint to apply the `if` statement here.

We apply the `if-else` statement to the `bsdtar` call and handling its result. Then we get the following code:

```
if bsdtar -cjf "$1".tar.bz2 "$@"
then
  echo "bsdtar - OK" > results.txt
else
  echo "bsdtar - FAILS" > results.txt
  exit 1
fi
```

Do you agree that it is easier to read the code now? We can simplify it even more. The **early return**[297] pattern does that. Replace the `if-else` statement with `if` like this:

```
if ! bsdtar -cjf "$1".tar.bz2 "$@"
then
  echo "bsdtar - FAILS" > results.txt
  exit 1
fi

echo "bsdtar - OK" > results.txt
```

The code behaves the same as with the `if-else` statement. The logical negation inverts the `bsdtar` utility result. Now, if it fails, the condition of the `if` statement becomes "true". Then the script prints the "bsdtar - FAILS" message to the log file and terminates. Otherwise, the script skips the command block of the `if` statement. The further `echo` call prints the "bsdtar - OK" message to the log file.

The early return pattern is a useful technique that makes your code cleaner and easier to read. The idea behind it is to terminate the program as early as possible when an error appears. This solution allows you to avoid the nested `if` statements.

Here is an example. Imagine the algorithm that does five actions. Each action depends on the result of the previous one. If the previous action fails, the algorithm stops. We can implement this algorithm with the following nested `if` statements:

[297] https://medium.com/swlh/return-early-pattern-3d18a41bba8

```
if ACTION_1
then
  if ACTION_2
  then
    if ACTION_3
    then
      if ACTION_4
      then
        ACTION_5
      fi
    fi
  fi
fi
```

These nested statements look confusing. If you add the `else` blocks for handling errors, this code becomes even harder to read.

The nested `if` statements make the code hard to read. It is a serious problem. The early return pattern solves it. Let's apply the pattern to our algorithm. Then we get the following code:

```
if ! ACTION_1
then
  # error handling
fi

if ! ACTION_2
then
  # error handling
fi

if ! ACTION_3
then
  # error handling
fi

if ! ACTION_4
then
  # error handling
fi

ACTION_5
```

We got the same algorithm. Its behavior did not change. There are still five actions. If any of them fails, the algorithm stops. But the early return pattern made the code simpler and clearer.

We use **comments**[298] in the last example. They look like this: "# error handling". A comment is a string or part of a string that the interpreter ignores. In Bash, a comment is anything that comes after the hash symbol.



The usefulness of comments is the subject of endless debates in the programming community. They are needed to explain the code. However, some people consider that comments are a sign of incomprehensible, poorly written code. If you just started learning to program, comment your code without doubts. Explain the complex constructions in your scripts. You can forget what do they mean afterward. Thus, you save your time in the future when you come back to the old code.

Assume that each action of the algorithm corresponds to one short command. The exit command handles all errors. There is no output to the log file. In this case, the || operator can replace the `if` statement. Then the code remains simple and clear. It will look like this:

```
ACTION_1 || exit 1
ACTION_2 || exit 1
ACTION_3 || exit 1
ACTION_4 || exit 1
ACTION_5
```

There is only one case when the && and || operators are more expressive than the `if` statement. The case is short commands do actions and error handling.

Let's rewrite the backup script using the `if` statement. Listing 3-11 shows the result.

**Listing 3-11. The backup script with the early return pattern**

```
#!/bin/bash

if ! bsdtar -cjf "$1".tar.bz2 "$@"
then
  echo "bsdtar - FAILS" > results.txt
  exit 1
fi

echo "bsdtar - OK" > results.txt

mv -f "$1".tar.bz2 /d &&
  echo "cp - OK" >> results.txt ||
  ! echo "cp - FAILS" >> results.txt
```

[298] https://en.wikipedia.org/wiki/Comment_(computer_programming)

We replaced the && and || operators in the `bsdtar` call with the if statement. The behavior of the script has not changed.

Logical operators and the `if` statement are not equivalent in general. Here is an example. Suppose there is an expression of three commands A, B and C:

```
A && B || C
```

It might seem that the following `if-else` statement gives the same behavior:

```
if A
then
  B
else
  C
fi
```

If A is “true”, then Bash executes B. Otherwise, it executes C. But there is another behavior in the expression with logical operators. Here, if A is “true”, then Bash executes B. Then C execution depends on the result of B. If B is “true”, Bash skips C. If B is “false”, it executes C. Thus, execution of C depends on both A and B. There is no such dependence in the `if-else` statement.

**Exercise 3-4. The `if` statement**

---

```
Here is the Bash command:
( grep -RlZ "123" target | xargs -0 cp -t . && echo "cp - OK" || ! echo "cp - FAILS"\
 ) && ( grep -RLZ "123" target | xargs -0 rm && echo "rm - OK" || echo "rm - FAILS" \
)

It looks for the string "123" in the files of the directory named "target".
If the file contains the string, it is copied to the current directory.
If there is no string in the file, it is removed from the target directory.

Make the script from this command.
Replace the && and || operators with the if statements.
```

---

## Operator [[

We got acquainted with the `if` statement. It calls a built-in Bash command or utility in the condition.

For example, let’s solve a task. We should check if the text file contains a phrase. When the phrase present, our script prints the message in the log file.

We can solve the task by combining the `if` statement and the `grep` utility. Put the `grep` call in the `if` condition. Then if the utility succeeds, it returns zero exit status. In this case, the `if` condition equals "true" and the script prints the message.

The `grep` utility prints its results on the output stream. We do not need this output when calling the utility in the `if` condition. The `-q` option disables the `grep` output. Finally, we get the following `if` statement:

```
if grep -q -R "General Public License" /usr/share/doc/bash
then
  echo "Bash has the GPL license"
fi
```

Now suppose that the `if` condition compares two strings or numbers. Bash has a special operator [[ for this purpose. The double square brackets are **reserved word**[299] of the interpreter. Bash handles the brackets on its own without calling a utility.



The Bourne shell does not have the [[ operator. The POSIX standard does not have it too. Therefore, if you should follow the standard, use the obsolete `test`[300] operator or its synonym [. Never use `test` in Bash. It has limited capabilities comparing with the operator [[. Also, it has error-prone syntax.

Let's start with a simple example of using the [[. We need to compare two strings. In this case, the `if` condition looks like this:

```
if [[ "abc" = "abc" ]]
then
  echo "The strings are equal"
fi
```

Run this code. You see the message that the strings are equal. A check of this kind is not really useful. Usually, you want to compare some variable with a string. The [[ operator compares them this way:

```
if [[ "$var" = "abc" ]]
then
  echo "The variable equals the \"abc\" string"
fi
```

Double-quotes are optional in this condition. Bash skips globbing and word splitting when it substitutes a variable in the [[ operator. The interpreter uses a variable as it is in this operator. The quotes prevent problems if the string on the right side has spaces. Here is an example of such a string:

[299] https://en.wikipedia.org/wiki/Reserved_word
[300] http://mywiki.wooledge.org/BashFAQ/031

```
if [[ "$var" = abc def ]]
then
  echo "The variable equals the \"abc def\" string"
fi
```

Bash cannot execute the condition of this `if` statement because of word splitting. Always use quotes when working with strings. This way, you avoid such problems. Here is the fixed `if` condition:

```
if [[ "$var" = "abc def" ]]
then
  echo "The variable equals the \"abc def\" string"
fi
```

The [[ operator can compare two variables with each other. It looks like this:

```
if [[ "$var" = "$filename" ]]
then
  echo "The variables are equal"
fi
```

Table 3-8 shows all string comparisons that the [[ operator allows.

**Table 3-8. String comparisons in the [[ operator**

| Operation | Description | Example |
|---|---|---|
| > | The string on the left side is larger than the string on the right side in the **lexicographic order**[301]. | [[ “bb” > “aa” ]] && echo “The ”bb” string is larger than ”aa”” |
| < | The string on the left side is smaller than the string on the right side in the lexicographic order. | [[ “ab” < “ac” ]] && echo “The ”ab” string is smaller than ”ac”” |
| = or == | The strings are equal. | [[ “abc” = “abc” ]] && echo “The strings are equal” |
| != | The strings are not equal. | [[ “abc” != “ab” ]] && echo “The strings are not equal” |
| -z | The string is empty. | [[ -z “$var” ]] && echo “The string is empty” |
| -n | The string is not empty. | [[ -n “$var” ]] && echo “The string is not empty” |

[301] https://en.wikipedia.org/wiki/Lexicographic_order

**Table 3-8. String comparisons in the [[ operator**

| Operation | Description | Example |
|---|---|---|
| -v | The variable is set to any value. | [[ -v var ]] && echo “The string is set” |
| = or == | Search the pattern on the right side in the string on the left side. Put the pattern without quotes here. | [[ “$filename” = READ* ]] && echo “The filename starts with ”READ”” |
| != | Check that the pattern on the right side does not occur in the string on the left side. Put the pattern without quotes here. | [[ “$filename” != READ* ]] && echo “The filename does not start with ”READ”” |
| =∼ | Search the **regular expression**[302] on the right side in the string on the left side. | [[ “$filename” =∼ ^READ.* ]] && echo “The filename starts with ”READ”” |

You can use the logical operations AND, OR and NOT in the [[ operator. They combine several expressions into a single condition. Table 3-9 gives examples of such conditions.

**Table 3-9. Logical operations in the [[ operator**

| Operation | Description | Example |
|---|---|---|
| && | Logical AND. | [[ -n “$var” && “$var” < “abc” ]] && echo “The string is not empty and it is smaller than ”abc”” |
| \|\| | Logical OR. | [[ “abc” < “$var” \|\| -z “$var” ]] && echo “The string is larger than ”abc” or it is empty” |
| ! | Logical NOT. | [[ ! “abc” < “$var” ]] && echo “The string is not larger than ”abc”” |

You can group expressions using parentheses in the [[ operator. Here is an example:

```
[[ (-n "$var" && "$var" < "abc") || -z "$var" ]] && echo "The string is not empty an\
d less than \"abc\" or the string is empty"
```

Comparing the strings is one feature of the [[ operator. Also, it can check files and directories for various conditions. Table 3-10 shows operations for doing that.

[302] https://mywiki.wooledge.org/RegularExpression

**Table 3-10. Operations for checking files and directories in the [[ operator**

| Operation | Description | Example |
|---|---|---|
| -e | The file exists. | [[ -e “$filename” ]] && echo “The $filename file exists” |
| -f | The specified object is a regular file. It is not a directory or **device file**[303]. | [[ -f “~/README.txt” ]] && echo “The README.txt is a regular file” |
| -d | The specified object is a directory. | [[ -f “/usr/bin” ]] && echo “The /usr/bin is a directory” |
| -s | The file is not empty. | [[ -s “$filename” ]] && echo “The $filename file is not empty” |
| -r | The specified file exists. The user who runs the script can read the file. | [[ -r “$filename” ]] && echo “The $filename file exists. You can read it” |
| -w | The specified file exists. The user who runs the script can write into the file. | [[ -w “$filename” ]] && echo “The $filename file exists. You can write into it” |
| -x | The specified file exists. The user who runs the script can execute the file. | [[ -x “$filename” ]] && echo “The $filename file exists. You can execute it” |
| -N | The file exists. It was modified since you read it last time. | [[ -N “$filename” ]] && echo “The $filename file exists. It was modified” |
| -nt | The file on the left side is newer than the file on the right side. Either the file on the left side exists and the file on the right side does not. | [[ “$file1” -nt “$file2” ]] && echo “The $file1 file is newer than $file2” |
| -ot | The file on the left side is older than the file on the right side. Either the file on the right side exists and the file on the left side does not. | [[ “$file1” -ot “$file2” ]] && echo “The $file1 file is older than $file2” |
| -ef | There are paths to the same file on the left and right sides. If your file system supports **hard links**, it can be the links to the same file on the left and right sides. | [[ “$file1” -ef “$file2” ]] && echo “The $file1 and $file2 files match” |

The [[ operator can compare integers. Table 3-11 shows operations for doing that.

[303] https://en.wikipedia.org/wiki/Device_file

**Table 3-11. Integer comparisons in the [[ operator**

| Operation | Description | Example |
|---|---|---|
| -eq | The number on the left side equals the number on the right side. | [[ “$var” -eq 5 ]] && echo “The variable equals 5” |
| -ne | The numbers are not equal. | [[ “$var” -ne 5 ]] && echo “The variable is not equal to 5” |
| -gt | Greater (>). | [[ “$var” -gt 5 ]] && echo “The variable is greater than 5” |
| -ge | Greater or equal. | [[ “$var” -ge 5 ]] && echo “The variable is greater than or equal to 5” |
| -lt | Less (<). | [[ “$var” -lt 5 ]] && echo “The variable is less than 5” |
| -le | Less or equal. | [[ “$var” -le 5 ]] && echo “The variable is less than or equal to 5” |

Table 3-11 raises questions. The operations there are harder to remember than the usual number comparisons: <, >, and =. Why aren’t comparison signs used in the [[ operator? To answer this question, let’s look at the operator’s history.

The [[ operator came into Bash to replace its obsolete `test` analog. The utility has implemented the `test` command in the first version of the Bourne shell in 1979. This command becomes the built-in starting with the System III shell version in 1981. But this change did not affect the `test` syntax. The reason for that is backward compatibility. Software developers have written a lot of shell code by 1981. This code has used the old syntax. Therefore, the new System III shell version had to support it.

Let’s take a look at the syntax of the `test` operator. When it was a utility, the format of its input parameters had to follow Bourne shell rules. For example, here is a typical `test` call to compare the `var` variable with the number five:

```
test "$var" -eq 5
```

This command does not raise any questions. Here we pass three parameters to the `test` utility:

1. The value of the `var` variable.
2. The `-eq` option.
3. The number 5.

We can use this call in the `if` condition this way:

```
if test "$var" -eq 5
then
  echo "The variable equals 5"
fi
```

The Bourne shell introduces the [ synonym for the `test` operator. The only difference between them is the presence of the closing parenthesis ]. The `test` operator does not need it but the synonym does.

Using the [ synonym, we rewrite the `if` condition this way:

```
if [ "$var" -eq 5 ]
then
  echo "The variable equals 5"
fi
```

The synonym [ improves readability of the code. That was an idea behind it. Thanks to the synonym, the `if` statement in Bourne shell looks the same as in other programming languages (e.g., C). The problem is that the [ and `test` operators are equivalent. It is easy to lose sight of this fact. Mostly it happens when you have experience in other languages. This mismatch between expected and real behavior leads to errors.

For example, programmers often forget the space between the bracket [ and the following character. This way, they get a condition like this:

```
if ["$var" -eq 5]
then
  echo "The variable equals 5"
fi
```

Replace the bracket [ with the `test` call in the condition. Then the error becomes obvious:

```
if test"$var" -eq 5
then
  echo "The variable equals 5"
fi
```

There must be a space between the command name and its parameters.

Let's come back to the question about comparison signs for numbers. Imagine the following `test` call:

```
test "$var" > 5
```

As you remember, the > symbol is a short form of the redirect operator `1>`. Thus, our `test` call does the following steps:

1. Calls the built-in `test` command and pass the `var` variable to it.
2. Redirects the `test` output to a file named `5` in the current directory.

We expect another behavior. Such an error is easy to make and hard to detect. To prevent it, shell developers introduced the two-letter comparison operations. The Bash operator [[ inherited these operations. It was done for backward compatibility.

Imagine that the [[ operator replaces two-letter operations with comparison signs. Then you have the legacy code written on Bourne shell. You want to port it on Bash. There is the following `if` statement in the legacy code:

```
1 if [ "$var1" -gt 5 -o 4 -lt "$var2" ]
2 then
3   echo "The var1 variable is greater than 5 or var2 is less than 4"
4 fi
```

Here you should replace the `-gt` operation to > and `-lt` to <. It is easy to make a mistake while doing that. Putting an extra parenthesis at the beginning and end of an expression is much simpler. This idea answers our question.

You can use comparison signs for string only when working with the [[ operator. Why there is no backward compatibility issue there? The first version of the `test` utility did not support the lexicographic comparison of strings. Therefore, the utility did not have comparison signs < and >. They appeared in the extension of POSIX standard later. The standard allows comparison signs for strings only. It was too late to add them for numbers because of the legacy code amount. According to the standard, you should escape comparison signs like this: `/<` and `/>`. The operator [[ took them and dropped the escaping.

**Exercise 3-5. The [[ operator**

```
Write a script to compare two directories named "dir1" and "dir2".
The script should print all files from one directory that absent in another one.
```

## Case Statement

Programs often choose their actions depending on some values. If the variable has one value, the program does one thing. When the value differs, it does another thing. The condition statements provide such a behavior.

We have considered the `if` statement. There is an alternative `case` statement in Bash. It is more convenient than `if` in some cases.

Let's look at an example. Suppose you are writing a script for archiving documents. The script has three operating modes:

1. Archiving without compression.
2. Archiving with compression.
3. Unarchiving.

You can choose the mode by the command-line option. Table 3-12 shows an example of possible options.

**Table 3-12. Options of the archiving script**

| Option | Operating mode |
|---|---|
| `-a` | Archiving without compression |
| `-c` | Archiving with compression |
| `-x` | Unarchiving |



Always follow the POSIX agreement[304] and its GNU extension[305] when choosing scripts' options and parameters format.

You can check the script option in the `if` statement. Listing 3-12 shows an example of doing that.

**Listing 3-12. The script for archiving documents**

```
#!/bin/bash

operation="$1"

if [[ "$operation" == "-a" ]]
then
    bsdtar -c -f documents.tar ~/Documents
elif [[ "$operation" == "-c" ]]
then
    bsdtar -c -j -f documents.tar.bz2 ~/Documents
elif [[ "$operation" == "-x" ]]
then
    bsdtar -x -f documents.tar*
else
    echo "Invalid option"
```

[304] https://www.gnu.org/software/libc/manual/html_node/Argument-Syntax.html
[305] https://www.gnu.org/prep/standards/html_node/Command_002dLine-Interfaces.html

```
    exit 1
fi
```

The `$1` position parameter keeps the script option. We write it to the `operation` variable for convenience. Then depending on this variable, we choose parameters for the `bsdtar` call. The `if` statement with two `elif` blocks checks the `operation` value.

Now we replace the `if` statement with the `case` one. Listing 3-13 shows the result.

**Listing 3-13. The script for archiving documents**

```
#!/bin/bash

operation="$1"

case "$operation" in
  "-a")
    bsdtar -c -f documents.tar ~/Documents
    ;;

  "-c")
    bsdtar -c -j -f documents.tar.bz2 ~/Documents
    ;;

  "-x")
    bsdtar -x -f documents.tar*
    ;;

  *)
    echo "Invalid option"
    exit 1
    ;;
esac
```

Let's call our script `archiving-case.sh`. Then we can launch it in one of the following ways:

```
./archiving-case.sh -a
./archiving-case.sh -c
./archiving-case.sh -x
```

If you pass any other parameters to the script, it prints the error message and terminates.



Always call the `exit` command when handling errors in a script. It should return the non-zero exit status in this case.

The `case` statement compares a string with a list of patterns. Each pattern has a corresponding code block. When the string matches the pattern, Bash executes the corresponding code block.

Each `case` block consists of the following elements:

1. A pattern or a list of patterns separated by vertical bars.
2. Right parenthesis.
3. A code block.
4. Two semicolons. They mark the end of the code block.

Bash checks patterns of the `case` blocks one by one. If the string matches the first pattern, Bash executes its code block. Then it skips other patterns. Instead, Bash executes the command that follows the `case` statement.

The `*` pattern without quotes matches any string. It is usually placed at the end of the list. The corresponding code block handles cases when none of the patterns match the string. It usually means an error.

At first sight, it may seem that the `if` and `case` statements are equivalent. They are not. They allow you to achieve the same behavior.

Let's compare the statements from Listings 3-12 and 3-13. First, we write them in a general form. Here is the result for the `if` statement:

```
if CONDITION_1
then
  ACTION_1
elif CONDITION_2
then
  ACTION_2
elif CONDITION_3
then
  ACTION_3
else
  ACTION_4
fi
```

The `case` statement looks like this:

```
case STRING in
  PATTERN_1)
    ACTION_1
     ;;

  PATTERN_2)
    ACTION_2
    ;;

  PATTERN_3)
    ACTION_3
    ;;

  PATTERN_4)
    ACTION_4
    ;;
esac
```

The difference between the constructs is evident now. First, `if` checks the results of Boolean expressions. The `case` statement compares the string with the patterns. Therefore, it makes no sense to pass a Boolean expression to the `case` condition. Doing that, you handle two cases only: when the expression is "true" and "false". The `if` statement is more convenient for such checking.

The second difference between `if` and `case` is the number of conditions. Each branch of the `if` statement checks a separate Boolean expression. In general, these expressions are independent of each other. In our example, they check the same variable, but that is a particular case. The `case` statement checks one string that you pass to it.

The `if` and `case` statements are fundamentally different. They are not interchangeable. In each case, use the statement depending on the nature of the check. The following questions will help you to make the right choice:

- How many conditions should you check? Use `if` for checking several conditions.
- Would it be enough to check one string only? Use `case` when the answer is yes.
- Do you need compound Boolean expressions? Use `if` when the answer is yes.

There are two possible delimiters between case blocks:

1. Two semicolons `;;`.
2. Semicolons and ampersand `;&`.

The ampersand delimiter is allowed in Bash, but it is not part of the POSIX standard. When Bash meets this delimiter, it executes the next block's code without checking its pattern. It can be useful

when you want to start executing an algorithm from a specific step. Also, you can avoid code duplication in some cases with the ampersand delimiter.

Here is an example of a code duplication problem. We write a script that archives PDF documents and copies the resulting file. The script receives an option to choose the action to do. For example, the `-a` option means archiving and `-c` means copying. Suppose that the script always has to do the copying after archiving. In this case, we get code duplication.

Listing 3-14 shows the `case` statement where the `cp` call is duplicated.

**Listing 3-14. The script for archiving and copying PDF documents**

```
#!/bin/bash

operation="$1"

case "$operation" in
  "-a")
    find Documents -name "*.pdf" -type f -print0 | xargs -0 bsdtar -c -j -f document\
s.tar.bz2
    cp documents.tar.bz2 ~/backup
    ;;

  "-c")
    cp documents.tar.bz2 ~/backup
    ;;

  *)
    echo "Invalid option"
    exit 1
    ;;
esac
```

We can avoid code duplication by adding the `;&` separator between the `-a` and `-c` blocks. Listing 3-15 shows the changed script.

**Listing 3-15. The script for archiving and copying PDF documents**

```
#!/bin/bash

operation="$1"

case "$operation" in
  "-a")
    find Documents -name "*.pdf" -type f -print0 | xargs -0 bsdtar -c -j -f document\
s.tar.bz2
    ;&

  "-c")
    cp documents.tar.bz2 ~/backup
    ;;

  *)
    echo "Invalid option"
    exit 1
    ;;
esac
```

The `;&` delimiter is useful in some cases. However, use it carefully. You can easily confuse the delimiters when reading. This way, you misread `;;` instead of `;&` and misunderstand the code.

## Alternative to the Case Statement

The case statement and the associative array solve similar tasks. The array makes the relationship between data (key-value). The case statement does the same between data and commands (value-action).

Usually, it is more convenient to work with data than with code. It is easier to modify data and check them for correctness. Therefore, it is worth replacing the `case` statement with an associative array in some cases. Converting data into code is easy in Bash comparing with other programming languages.

Here is an example. We want to write a wrapper script for the archiving utilities. It receives command-line options. They define if the script calls the `bsdtar` or `tar` utility.

Listing 3-16 shows the script. It handles the command-line options in the `case` statement.

**Listing 3-16. The wrapper script for the archiving utilities**

```
#!/bin/bash

utility="$1"

case "$utility" in
  "-b"|"--bsdtar")
    bsdtar "${@:2}"
    ;;

  "-t"|"--tar")
    tar "${@:2}"
    ;;

  *)
    echo "Invalid option"
    exit 1
    ;;
esac
```

Here we use the pattern list for the first two `case` blocks. The script executes the first block's commands when the `utility` variable matches the string `-b` or `--bsdtar`. Likewise, the script executes the second block when the variable matches `-t` or `--tar`.

For example, you can launch the script this way:

```
./tar-wrapper.sh --tar -cvf documents.tar.bz2 Documents
```

The script calls the `tar` utility to archive the `Documents` directory. If you want to use the `bsdtar` utility instead, replace the `--tar` option with `-b` or `--bsdtar`. Do that like this:

```
./tar-wrapper.sh -b -cvf documents.tar.bz2 Documents
```

The script handles the first positional parameter on its own. It passes all the following parameters to the archiving utility. We use the `$@` parameter for doing that. It is not an array. But it supports the array-like syntax for accessing several elements. The archiving utility receives all elements of the `$@` parameter starting from the second one.

Let's rewrite our wrapper script using an associative array.

First, let's consider the Bash mechanisms for converting data into commands. For doing that, you should store the command and its parameters into the variable. Bash must insert the variable's value somewhere in the script for executing the command. You should check that Bash executes the resulting command correctly.

Here is an example of how to convert data into a command. For the first time, we will do it in the shell but not in the script. The first step is declaring the variable like this:

```
ls_command="ls"
```

Now the `ls_command` variable stores the command to call the `ls` utility. After such a declaration, we can use the variable name for calling the utility. It looks the following way:

```
$ls_command
```

This command calls the `ls` utility without parameters. How does it happen? Bash inserts the value of the `ls_command` variable. Then the command becomes like this:

```
ls
```

After parameter expansion, Bash executes the resulting command.

Why don't we use double-quotes when inserting the `ls_command` variable? One small change would help to answer this question. Let's add an option to the `ls` utility call. Here is an example of such `ls_command` variable:

```
ls_command="ls -l"
```

In this case, parameter expansion with double-quotes leads to the error:

```
$ "$ls_command"
ls -l: command not found
```

Double-quotes cause the problem because they prevent word splitting. Therefore, the command looks like this after parameter expansion:

```
"ls -l"
```

This command means that Bash must call a built-in or utility named "ls -l". As you remember, the POSIX standard allows spaces in filenames. Therefore, "ls -l" is the correct executable filename. Removing the quotes solves the problem. We meet one of the rare cases when you do not need double-quotes for parameter expansion.

It can happen that you still need double-quotes when reading the command from the variable. This task has the solution. Use the `eval` built-in in this case. It constructs a command from its input parameters. Then Bash does word splitting for this command regardless of double-quotes.

Here is an example of the `eval` call for processing the `ls_command` variable:

```
eval "$ls_command"
```



Many Bash manuals claim that it is bad practice to use `eval` and store commands in variables. It can lead to serious errors and vulnerabilities. In general, this idea is true. Be careful and never pass user-entered data to eval.

Now we can rewrite our wrapper script using an associative array. Listing 3-17 shows the result.

**Listing 3-17. The wrapper script for the archiving utilities**

```
#!/bin/bash

option="$1"

declare -A utils=(
  ["-b"]="bsdtar"
  ["--bsdtar"]="bsdtar"
  ["-t"]="tar"
  ["--tar"]="tar")

if [[ -z "$option" || ! -v utils["$option"] ]]
then
  echo "Invalid option"
  exit 1
fi

${utils["$option"]} "${@:2}"
```

Here the `utils` array stores matching between the script's options and utility names. Using the array, we construct the utility calls easily.

The utility call looks like this in the script:

```
${utils["$option"]} "${@:2}"
```

Bash reads the utility name from the `utils` array. The `option` variable provides the element's key. If you pass the wrong option to the script, the corresponding key does not present in `utils`. Then Bash inserts an empty string after parameter expansion. It leads to an error. The `if` statement checks the `option` variable and prevents this error.

The `if` statement checks two Boolean expressions:

1. The `option` variable, which matches the `$1` parameter, is not empty.

2. The `utils` array has a key that equals the `option` variable.

The second expression uses the `-v` option of the [[ operator. It checks if the variable has been declared. If you have declared it and assigned an empty string, the check passes too.

Our example has shown that replacing the `case` statement with an associative array makes your code cleaner in some cases. Always consider if this option fits your case when writing scripts.

**Exercise 3-6. The `case` statement**

```
There are two configuration files in the user's home directory:
".bashrc-home" and ".bashrc-work".
Write a script to switch between them.
You can do that by copying one of the files to the path "~/.bashrc" or
creating a symbolic link.
Solve the task with the "case" statement first.
Then replace the "case" statement with an associative array.
```

# Arithmetic Expressions

Bash can calculate integers. It does simple arithmetic operations: addition, subtraction, multiplication and division. Also, Bash does bitwise and logical operations. They are often used in programming. We consider them in detail now.

Bash does not support floating-point arithmetic. For doing that, use the bc[306] or dc[307] calculator.

## Integer Representation

Let's have a look at how a computer represents integers in its memory. Knowing it, you understand how mathematical operations work in Bash.

Mathematical integers[308] can be positive or negative. They match the data type with the same name **integer**[309].

If an integer type variable accepts positive values only, it is called **unsigned**. If it allows both positive and negative values, the variable is called **signed**.

There are three most common ways of representing integers in computer memory:

- **Sign-Magnitude representation**[310] (SMR)
- **Ones' complement**[311]
- **Two's complement**[312]

[306] https://en.wikipedia.org/wiki/Bc_(programming_language)
[307] https://en.wikipedia.org/wiki/Dc_(computer_program)
[308] https://en.wikipedia.org/wiki/Integer
[309] https://en.wikipedia.org/wiki/Integer_(computer_science)
[310] https://en.wikipedia.org/wiki/Signed_number_representations#Sign-and-magnitude_method
[311] https://en.wikipedia.org/wiki/Signed_number_representations#Ones'_complement
[312] https://en.wikipedia.org/wiki/Signed_number_representations#Two's_complement

## Sign-Magnitude Representation

All numbers in computer memory are represented in binary form. It means that the computer stores any number as a sequence of zeros and ones. Number representation defines what those zeros and ones mean.

First, we consider the simplest numbers representation that is the sign-magnitude representation or SMR. There are two options to use it:

1. To store only positive integers (unsigned).
2. To store both positive and negative integers (signed).

The computer allocates a fixed block of memory for any number. When you choose the first option of SMR, all bits of the allocated memory are used in the same way. They store the value of the number. Table 3-13 shows how it looks like.

**Table 3-13. Sign-magnitude representation of the unsigned integers**

| Decimal | Hexadecimal | SMR |
|---|---|---|
| 0 | 0 | 0000 0000 |
| 5 | 5 | 0000 0101 |
| 60 | 3C | 0011 1100 |
| 110 | 6E | 0110 1110 |
| 255 | FF | 1111 1111 |

Suppose that the computer allocates one byte of memory for a number. Then you can store unsigned integers from 0 to 255 there using SMR.

The second option of SMR allows you to store signed integers. In this case, the highest bit of the number stores its sign. Therefore, there are fewer bits left for the value of the number. Suppose that there is one byte to keep the number. The number's sign takes one bit. Then only seven bits can store the value of the number.

Table 3-14 shows the sign-magnitude representation of the signed integers.

**Table 3-14. The sign-magnitude representation of the signed integers**

| Decimal | Hexadecimal | SMR |
|---|---|---|
| -127 | FF | 1111 1111 |
| -110 | EE | 1110 1110 |
| -60 | BC | 1011 1100 |
| -5 | 85 | 1000 0101 |
| -0 | 80 | 1000 0000 |
| 0 | 0 | 0000 0000 |
| 5 | 5 | 0000 0101 |
| 60 | 3C | 0011 1100 |

**Table 3-14. The sign-magnitude representation of the signed integers**

| Decimal | Hexadecimal | SMR |
|---|---|---|
| 110 | 6E | 0110 1110 |
| 127 | 7F | 0111 1111 |

The highest (leftmost) bit of all negative numbers equals one. It equals zero for positive numbers. Because of the sign, it is impossible to store numbers greater than 127 in one byte. For the same reason, the minimum allowed negative number is -127.

There are two reasons why SMR is not widespread nowadays:

1. Arithmetic operations on negative numbers complicate the processor architecture. A processor module for adding positive numbers is not suitable for negative numbers.
2. There are two representations of zero: positive (0000 0000) and negative (1000 0000). It complicates the comparison operation because these values are not equal in memory.

Take your time and try to understand SMR. Without getting it, you won't understand the other two ways of representing integers.

## Ones' Complement

SMR has two disadvantages. They have led to technical issues when using the representation in practice. Therefore, the engineers have looked for an alternative approach to store numbers in memory. This way, they came to ones' complement representation.

The first problem of SMR is related to operations on negative numbers. The ones' complement solves it. Let's consider the root cause of this problem.

Here is an example. We want to add the numbers 10 and -5. First, we should write them in SMR. Assume that each number occupies one byte in computer memory. Then we get the following result:

```
10 = 0000 1010
-5 = 1000 0101
```

Now the question arises. How does the processor add these numbers? Any modern processor has a standard module called **adder**[313]. It adds two numbers in a bitwise manner. If we apply it in our task, we get the following result:

```
10 + (-5) = 0000 1010 + 1000 0101 = 1000 1111 = -15
```

The result is wrong. It means that the adder cannot add numbers in SMR. The calculation mistake happens because the adder does not consider the highest bit of the number. This bit stores the sign.

There are two ways for solving the problem:

[313] https://en.wikipedia.org/wiki/Adder_(electronics)

1. Add a special module to the processor. It should process operations on negative integers.
2. Change the integer representation in memory. The representation should fit the adder logic when it adds negative integers.

The development of computer technology followed the second way. It is cheaper than complicating the processor architecture.

The ones' complement reminds SMR. The sign of the number occupies the highest bit. The remaining bits store the value. The difference is all bits of the value are inverted for negative numbers. It means zeros become ones and ones become zeros. Bits of positive numbers are not inverted.

Table 3-15 shows the ones' complement representation of some numbers.

**Table 3-15. The ones' complement of the signed integers**

| Decimal | Hexadecimal | Ones' Complement |
|---|---|---|
| -127 | 80 | 1000 0000 |
| -110 | 91 | 1001 0001 |
| -60 | C3 | 1100 0011 |
| -5 | FA | 1111 1010 |
| -0 | FF | 1111 1111 |
| 0 | 0 | 0000 0000 |
| 5 | 5 | 0000 0101 |
| 60 | 3C | 0011 1100 |
| 110 | 6E | 0110 1110 |
| 127 | 7F | 0111 1111 |

The memory capacity when using SMR and the ones' complement is the same. One byte can store numbers from -127 to 127 in both cases.

What is the effect of inverting the value bits for negative numbers? Let's have a look at how the addition of negative numbers works now. First, we will write 10 and -5 in the ones' complement. Then add them using the adder CPU module.

Here is how the numbers look like in memory:

```
10 = 0000 1010
-5 = 1111 1010
```

Their addition gives the following result:

```
10 + (-5) = 0000 1010 + 1111 1010 = 1 0000 0100
```

The addition caused an overflow. The highest one does not fit into one byte. In this case, it is discarded. Then the result of the addition becomes like this:

```
0000 0100
```

The discarded one affects the final result. We need a second calculation step to take it into account. In this step, we add the discarded one as a number to the result. It looks like this:

```
0000 0100 + 0000 0001 = 0000 0101 = 5
```

We got the correct result of adding the numbers 10 and -5.

If the addition results in a negative number, the second calculation step is unnecessary. As an example, add the numbers -7 and 2. First, write them in the ones' complement:

```
-7 = 1111 1000
2 = 0000 0010
```

Then do the first step of addition:

```
-7 + 2 = 1111 1000 + 0000 0010 = 1111 1010
```

The high bit equals one. It means that we got a negative number. Therefore, we should skip the second step of addition.

Let's check if the result is correct. For convenience, convert the number from ones' complement to SMR. Invert all bits of the number value for doing that. The sign bit should stay unchanged. Here is the result:

```
1111 1010 -> 1000 0101 = -5
```

We got the right result again.

The ones' complement solved one problem. Now the adder CPU module can add any signed integers. There is one disadvantage of this solution. Addition requires two steps in some cases. It slows down computations.

SMR has the second problem. It represents zero in two ways. Ones' complement did not solve it.

## Two's Complement

The two's complement solves both problems of SMR. First, it allows the standard adder to add negative numbers. In the ones' complement, this operation requires two steps. In two's complement, one step is sufficient. Second, there is only one way to represent zero.

Positive integers in the two's complement look the same as in SMR. The highest bit equals zero there. The remaining bits store the value of the number. Negative integers have the highest bit equal to one. The value bits are inverted the same way as in the ones' complement. The result is increased by one.

Table 3-16 shows the two's complement representation of some numbers.

**Table 3-16. The two's complement of the signed integers**

| Decimal | Hexadecimal | Two's Complement |
|---|---|---|
| -127 | 81 | 1000 0001 |
| -110 | 92 | 1001 0010 |
| -60 | C4 | 1100 0100 |
| -5 | FB | 1111 1011 |
| 0 | 0 | 0000 0000 |
| 5 | 5 | 0000 0101 |
| 60 | 3C | 0011 1100 |
| 110 | 6E | 0110 1110 |
| 127 | 7F | 0111 1111 |

The memory capacity stays the same when using the two's complement. One byte can store the numbers from -127 to 127.

Let's consider adding negative numbers in the two's complement. For example, we add 14 and -8. First, write them in the two's complement. Here is the result:

```
14 = 0000 1110
-8 = 1111 1000
```

Now add these number like this:

```
14 + (-8) = 0000 1110 + 1111 1000 = 1 0000 0110
```

The addition leads to the overflow. The highest one did not fit into one byte. We should discard it. Then the final result looks like this:

```
0000 0110 = 6
```

When addition gives a negative result, you should not discard the highest bit. For example, we want to add the numbers -25 and 10. When we write them in two's complement, they look like this:

```
-25 = 1110 0111
10 = 0000 1010
```

The addition of these numbers gives the following result:

```
-25 + 10 = 1110 0111 0000 1010 = 1111 0001
```

Let's convert the result from two's complement to ones' complement. Then do one more step and get it in SMR. Here are the conversions:

```
1111 0001 - 1 = 1111 0000 -> 1000 1111 = -15
```

When converting from ones' complement to SMR, we invert all bits except the highest one. This way, we got the correct result of adding -25 and 10.

Two's complement allowed the standard adder of the CPU to add negative numbers. Moreover, the adder does this calculation in a single step. Therefore, there is no performance loss, unlike the ones' complement case.

Two's complement solved the problem of zero representation. There is only one way to represent it. It is the number with all bits zeroed. Therefore, there are no issues with comparing numbers.

All modern computers represent integers in two's complement.

**Exercise 3-7. Arithmetic operations with numbers in the two's complement representation**

```
Represent the following integers in two's complement and add them:

* 79 + (-46)
* -97 + 96

Represent the following two-byte integers in two's complement and add them:

* 12868 + (-1219)
```

## Converting Numbers

We learned how a computer represents numbers in memory. When would you need it in practice?

Modern programming languages take care of converting numbers into the correct format. For example, you declare a signed integer variable in decimal notation. You do not need to worry about how the computer stores it in memory. If the variable's value becomes negative, the computer makes two's complement representation without your involvement.

In some cases, you want to treat a variable as a set of bits. Then declare it as a positive integer. Do all operations on it in hexadecimal. Do not convert the variable's value to decimal. This way, you avoid the problems of converting numbers.

The issue arises when you want to read data from some device. Such a task often occurs in **system software development**[314]. This domain includes the development of device drivers, OS kernels and modules, system libraries and network protocol stacks.

Here is an example. Suppose that you write a driver for a peripheral device. The device periodically sends data to the CPU. For example, it is the results of some measurements. Interpret them correctly is your task. The computer cannot do it for you in most cases. It happens because the computer and the device represent the numbers differently. You know this difference. Thus, you should apply your knowledge about numbers representation and convert them properly.

[314] https://en.wikipedia.org/wiki/System_software

There is another task that every programmer faces. It is debugging the program. For example, there is an integer overflow in the arithmetic expression. Knowing numbers representation helps you find and solve the problem.

## Operator ((

Bash performs integer arithmetic in **math context**. Its syntax resembles the C language.

Suppose that you want to store the result of adding two numbers in a variable. Declare it with the `-i` integer attribute. Then assign its value in the declaration. Here is an example:

```
declare -i var=12+7
```

Bash assigned the number 19 to the variable but not the "12+7" string. When you add the `-i` attribute to the declaration, Bash calculates the assigned value in the mathematical context. It happened in our example.

You can declare the mathematical context explicitly. The `let` built-in command does that.

Suppose you declare the variable without the `-i` attribute. Then the `let` command allows you to assign an arithmetic expression result to the variable. Here is an example:

```
let text=5*7
```

The `text` variable equals 35 in the result.

When declaring the variable with the `-i` attribute, you do not need the `let` command. It looks like this:

```
declare -i var
var=5*7
```

Now the `var` variable equals 35 too.

Declaring a variable with the `-i` attribute creates the mathematical context implicitly. It can lead to errors. Therefore, try to avoid using the `-i` attribute. It does not affect the way how the Bash stores the variable in memory. Instead, converting a string to a number and back takes place every time you assign the variable.

The `let` command allows you to treat a string variable as an integer variable. This way, you can do the following assignments:

```
let var=12+7
let var="12 + 7"
let "var = 12 + 7"
let 'var = 12 + 7'
```

All four commands give the same result. They set the variable's value to 19.

The `let` command takes parameters on input. Each of them must be a valid arithmetic expression. If there are spaces in the expression, Bash splits it into parts because of word splitting. In this case, `let` computes each part of the expression separately. It can lead to an error.

The following command demonstrates the issue:

```
let var=12 + 7
```

Here the `let` command receives three expressions after word splitting. These are the expressions:

- var=12
- +
- 7

When calculating the second one, `let` reports about an error. The plus means an arithmetic addition. It requires two operands. But in our case, there are no operands at all.

Suppose that you pass correct expressions to the `let` command. Then the command evaluates them one by one. Here are the examples:

```
let a=1+1 b=5+1
let "a = 1 + 1" "b = 5 + 1"
let 'a = 1 + 1' 'b = 5 + 1'
```

All three commands give the same result. The variable `a` gets the value of 2. The variable `b` gets the value of 6.

You can prevent word splitting of the `let` parameters. Use single or double-quotes for that.

The `let` command has a synonym that is the (( operator. Bash skips word splitting inside the operator. Therefore, you can skip quotes there. Always use the (( operator instead of the `let` command. This way, you will avoid mistakes.



The relationship of the (( operator and the `let` command resembles that of `test` and the [[ operator. In both cases, use operators rather than commands.

The (( operator has two forms. The first one is called **arithmetic evaluation**[315]. It is a synonym for the `let` command. The arithmetic evaluation looks like this:

[315] https://wiki.bash-hackers.org/syntax/ccmd/arithmetic_eval

```
((var = 12 + 7))
```

Here we use opening double-parentheses instead of the `let` command. There are closing double-parentheses at the end. This form of the (( operator returns exit status zero if it succeeds. It returns exit status one if it fails. After calculating the expression, Bash replaces it with its exit status.

The second form of the (( operator is called **arithmetic expansion**[316]. It looks like this:

```
var=$((12 + 7))
```

Here we put a dollar sign before the (( operator. In this case, Bash calculates the value of the expression. Then Bash inserts this value in place of the expression. This behavior differs from the first form of the (( operator. Bash inserts the exit status there.



The second form of the (( operator is part of the POSIX standard. Use it for writing portable code. The first form of the operator is available in Bash, ksh and zsh interpreters only.

You can skip the dollar sign before variables names in the (( operator. Bash still inserts their values correctly in this case. For example, here are two equivalent expressions for calculating the `result` variable:

```
1 a=5 b=10
2 result=$(($a + $b))
3 result=$((a + b))
```

Both expressions assign the value of 15 to the `result` variable.

Do not use the dollar sign in the (( operator. It makes your code clearer.



Bash has an obsolete form of arithmetic expansion that is the "$[ ]" operator. Never use it. There is a GNU utility called `expr` for calculating arithmetic expressions. Bash needs it for launching old Bourne Shell scripts. Never use `expr` when developing new scripts.

Table 3-17 shows the operations that Bash allows in arithmetic expressions.

[316] https://wiki.bash-hackers.org/syntax/expansion/arith

**Table 3-17. The operations in arithmetic expressions**

| Operation | Description | Example |
|---|---|---|
| | **Calculations** | |
| `*` | Multiplication | `echo "$((2 * 9)) = 18"` |
| `/` | Division | `echo "$((25 / 5)) = 5"` |
| `%` | The remainder of division | `echo "$((8 % 3)) = 2"` |
| `+` | Addition | `echo "$((7 + 3)) = 10"` |
| `-` | Subtraction | `echo "$((8 - 5)) = 3"` |
| `**` | Exponentiation | `echo "$((4**3)) = 64"` |
| | **Bitwise operations** | |
| `~` | Bitwise NOT | `echo "$((~5)) = -6"` |
| `<<` | Bitwise left shift | `echo "$((5 << 1)) = 10"` |
| `>>` | Bitwise right shift | `echo "$((5 >> 1)) = 2"` |
| `&` | Bitwise AND | `echo "$((5 & 4)) = 4"` |
| `\|` | Bitwise OR | `echo "$((5 \| 2)) = 7"` |
| `^` | Bitwise XOR[317] | `echo "$((5 ^ 4)) = 1"` |
| | **Assignments** | |
| `=` | Ordinary assignment | `echo "$((num = 5)) = 5"` |
| `*=` | Multiply and assign the result | `echo "$((num *= 2)) = 10"` |
| `/=` | Divide and assign the result | `echo "$((num /= 2)) = 5"` |
| `%=` | Get the remainder of the division and assign it | `echo "$((num %= 2)) = 1"` |
| `+=` | Add and assign the result | `echo "$((num += 7)) = 8"` |

[317] https://en.wikipedia.org/wiki/Exclusive_or

**Table 3-17. The operations in arithmetic expressions**

| Operation | Description | Example |
|---|---|---|
| `-=` | Subtract and assign the result | `echo "$((num -= 3)) = 5"` |
| `<<=` | Bitwise left shift and assign the result | `echo "$((num <<= 1)) = 10` |
| `>>=` | Bitwise right shift and assign the result | `echo "$((num >>= 2)) = 2"` |
| `&=` | Bitwise AND and assign the result | `echo "$((num &= 3)) = 2"` |
| `^=` | Bitwise XOR and assign the result | `echo "$((num^=7)) = 5"` |
| `\|=` | Bitwise OR and assign the result | `echo "$((num \|= 7)) = 7"` |
| | **Comparisons** | |
| `<` | Less than | `((num < 5)) && echo "The \"num\" variable is less than 5"` |
| `>` | Greater than | `((num > 5)) && echo "The \"num\" variable is greater than 3"` |
| `<=` | Less than or equal | `((num <= 20)) && echo "The \"num\" variable is less or equal 20"` |
| `>=` | Greater than or equal | `((num >= 15)) && echo "The \"num\" variable is greater or equal 15"` |
| `==` | Equal | `((num == 3)) && echo "The \"num\" variable is equal to 3"` |
| `!=` | Not equal | `((num != 3)) && echo "The \"num\" variable is not equal to 3"` |
| | **Logical operations** | |

**Table 3-17. The operations in arithmetic expressions**

| Operation | Description | Example |
|---|---|---|
| `!` | Logical NOT | `(( ! num )) && echo "The \"num\" variable is FALSE"` |
| `&&` | Logical AND | `(( 3 < num && num < 5 )) && echo "The \"num\" variable is greater than 3 but less than 5"` |
| `\|\|` | Logical OR | `(( num < 3 \|\| 5 < num )) && echo "The \"num\" variable is less than 3 or greater than 5"` |
| | **Other operations** | |
| `num++` | Postfix increment | `echo "$((num++))"` |
| `num--` | Postfix decrement | `echo "$((num--))"` |
| `++num` | Prefix increment | `echo "$((++num))"` |
| `--num` | Prefix decrement | `echo "$((--num))"` |
| `+num` | Unary plus or multiplication of a number by 1 | `a=$((+num))"` |
| `-num` | Unary minus or multiplication of a number by -1 | `a=$((-num))"` |
| `CONDITION ? ACTION_1 : ACTION_2` | Ternary operator[318] | `a=$(( b < c ? b : c ))` |
| `ACTION_1, ACTION_2` | The list of expressions | `((a = 4 + 5, b = 16 - 7))` |
| `( ACTION_1 )` | Grouping of expressions (subexpression) | `a=$(( (4 + 5) * 2 ))` |

Bash performs all operations in order of their priorities. The operations with a higher priority come first.

Table 3-18 shows the priority of operations.

[318] https://en.wikipedia.org/wiki/%3F:

**Table 3-18. Priority of operations in arithmetic expressions**

| Priority | Operation | Description |
|---|---|---|
| 1 | `( ACTION_1 )` | Grouping of expressions |
| 2 | `num++, num--` | Postfix increment and decrement |
| 3 | `++num, --num` | Prefix increment and decrement |
| 4 | `+num, -num` | Unary plus and minus |
| 5 | `~, !` | Bitwise and logical NOT |
| 6 | `**` | Exponentiation |
| 7 | `*, /, %` | Multiplication, division and the remainder of division |
| 8 | `+, -` | Addition and subtraction |
| 9 | `<<, >>` | Bitwise shifts |
| 10 | `<, <=, >, >=` | Comparisons |
| 11 | `==, !=` | Equal and not equal |
| 12 | `&` | Bitwise AND |
| 13 | `^` | Bitwise XOR |
| 14 | `\|` | Bitwise OR |
| 15 | `&&` | Logical AND |
| 16 | `\|\|` | Logical OR |
| 17 | `CONDITION ? ACTION_1 : ACTION_2` | Ternary operator |
| 18 | `=, *=, /=, %=, +=, -=, <<=, >>=, &=, ^=, \|=` | Assignments |
| 19 | `ACTION_1, ACTION_2` | The list of expressions |

You can change the order of execution using parentheses "( )". Their contents are called **subexpression**. Bash calculates subexpressions first. If there is more than one subexpression, Bash calculates them in the left-to-right order.

Suppose your code uses a numeric constant. You can specify its value in any numeral system. Use a prefix to select a numeral system. Table 3-19 shows the list of allowable prefixes.

**Table 3-19. The prefixes for numeral systems**

| Prefix | Numeral System | Example] |
|---|---|---|
| `0` | Octal | `echo "$((071)) = 57"` |
| `0x` | Hexadecimal | `echo "$((0xFF)) = 255"` |
| `0X` | Hexadecimal | `echo "$((0XFF)) = 255"` |
| `<base>#` | The numeral system with a base from 2 to 64 | `echo "$((16#FF)) = 255"` |
| | | `echo "$((2#101)) = 5"` |

When printing a number to the screen or file, Bash always converts it in decimal. The `printf` command changes the format of the number output. You can use it this way:

```
printf "%x\n" 250
```

This command prints the number 250 in hexadecimal.

You can format the variable's value in the same way:

```
printf "%x\n" $var
```

## Arithmetic Operations

Let's start with the simplest mathematical operations that are arithmetic operations. Programming languages use usual symbols to denote them:

- `+` addition
- `-` subtraction
- `/` division
- `*` multiplication

Two more operations are often used in programming. These are exponentiation and division with remainder[319].

Suppose that you want to raise the `a` number to the power of `b`. You can write it this way: $a^b$. Here `a` is the base and `b` is the exponent. For example, raising two to the power of seven is written as $2^7$. You can write this operation with two asterisks in Bash:

```
2**7
```

[319] https://en.wikipedia.org/wiki/Euclidean_division

Calculating the remainder of the division is a complex but essential operation in programming. Let's take a closer look at it. Suppose we have divided one integer number by another. A result is a fractional number. In this case, we say that the division produced a **remainder**[320].

For example, let's divide 10 (the dividend) by 3 (the divisor). If we round the result, we get 3.33333 (the quotient). In this case, the remainder of the division is 1. To find it, we should multiply the divisor 3 by the integer part of the quotient 3 (the incomplete quotient). Then subtract the result from the dividend 10. It gives us the remainder, which is equal to 1.

Let's write our calculations in formulas. We can introduce the following notation for that:

- `a` is a dividend
- `b` is a divisor
- `q` is an incomplete quotient
- `r` is a remainder

Using the notation, we can write the formula for calculating the dividend:

```
a = b * q + r
```

Then we derive the formula for finding the remainder:

```
r = a - b * q
```

The choice of an incomplete quotient `q` raises questions. Sometimes several numbers fit this role. The restriction helps to choose the right one. The `q` quotient must be such that the absolute value of the `r` remainder is less than the `b` divisor. In other words, you should fulfill the following inequality:

```
|r| < |b|
```

The percent sign denotes the operation of finding the remainder in Bash. Some languages use the same symbol for the **modulo operation**[321]. These two operations are not the same[322]. They give the same results only when the signs of the dividend and the divisor match.

For example, let's calculate the remainder and modulo when dividing 19 by 12 and -19 by -12. We get these results:

[320] https://en.wikipedia.org/wiki/Remainder
[321] https://en.wikipedia.org/wiki/Modulo_operation
[322] https://rob.conery.io/2018/08/21/mod-and-remainder-are-not-the-same

```
19 % 12 = 19 - 12 * 1 = 7
19 modulo 12 = 19 - 12 * 1 = 7

-19 % -12 = -19 - (-12) * 1 = -7
-19 modulo -12 = -19 - (-12) * 1 = -7
```

Now consider cases when the signs of the dividend and the divisor differ. Here are the results:

```
19 % -12 = 19 - (-12) * (-1) = 7
19 modulo -12 = 19 - (-12) * (-2) = -5

-19 % 12 = -19 - 12 * (-1) = -7
-19 modulo 12 = -19 - 12 * (-2) = 5
```

The reminder and the modulo are different for the same pairs of numbers.

The same formula calculates the remainder and modulo. But the choice of the `q` incomplete quotient differs. For calculating the reminder, you get `q` this way:

```
q = a / b
```

You should round the result to the lowest absolute number[323]. It means discarding all decimal places.

Calculation of the incomplete quotient for finding the modulo depends on the signs of `a` and `b`. If the signs are the same, the formula for the quotient stays the same:

```
q = a / b
```

If the signs differ, here is another formula:

```
q = (a / b) + 1
```

In both cases, you should round the result to the lowest absolute number.

When one speaks of the remainder of division `r`, he usually assumes that both the divisor a and the divisor b are positive. That is why reference books often mention the following condition for `r`:

```
0 ≤ r < |b|
```

[323] https://en.wikipedia.org/wiki/Absolute_value

However, you can get a negative remainder when dividing numbers with different signs. Remember a simple rule: the `r` remainder always has the same sign as the `a` dividend. If the signs of `r` and `a` differ, you have found the modulo but not the reminder.

Always keep in mind the difference between the remainder and the modulo. Some programming languages calculate the remainder in the `%` operator, while others calculate it in the `modulo` operator. It leads to confusion.

If in doubt about your calculations, check them. The `%` Bash operator always computes the remainder of a division. Suppose you want to find the remainder of dividing 32 by -7. This command shows it to you:

```
echo $((32 % -7))
```

The remainder of the division is four.

Now find the modulo for the same pair of numbers. Use the online calculator[324] for that. Enter the dividend32 in the "Expression" field. Then enter the divisor 7 in the "Modulus" field. Click the "CALCULATE" button. You will get two results:

- The "Result" field shows 4.
- The "Symmetric representation" field shows -3.

The second answer -3 is the modulo. The first one is the reminder.

When do you need the remainder of a division in programming? The most common task is to check a number for parity. For example, it helps to control the integrity of transmitted data in computer networks. This approach is called **the parity bit**[325].



To check a number for parity, you should find the remainder of its division by 2. If the remainder is zero, then the number is even. Otherwise, it is odd.

There is another task where you need to calculate the remainder. It is the converting of time units. Suppose you want to convert 128 seconds into minutes. Then you should count the number of minutes in 128 seconds. The next step is to add the remainder to the result.

The first step is calculating the number of minutes. Divide 128 by 60 to do that. The result is an incomplete quotient of 2. It means that 128 seconds contains 2 minutes. Calculate the remainder of dividing 128 by 60 to find the remaining seconds. Here is the result:

```
r = 128 - 60 * 2 = 8
```

[324] https://planetcalc.com/8326
[325] https://en.wikipedia.org/wiki/Parity_bit

The remainder equals 8. It means that 128 seconds equals two minutes and eight seconds.

Calculating the remainder is useful when working with loops. Suppose that you want to act on every 10th iteration of the loop. Then you need to check the remainder of dividing the loop counter by 10. If the remainder is zero, then the current iteration is a multiple of 10. Thus, you should act on this iteration.

The modulo operation is widely used in **cryptography**[326].

**Exercise 3-8. Modulo and the remainder of a division**

```
Calculate the remainder of a division and modulo:

* 1697 % 13
* 1697 modulo 13

* 772 % -45
* 772 modulo -45

* -568 % 12
* -568 modulo 12

* -5437 % -17
* -5437 modulo -17
```

## Bitwise operations

**Bitwise operations**[327] are another type of mathematical operations. Software developers use them often. The operations get their name because they operate each bit of a number individually.

### Bitwise negation

We start with the simplest bitwise operation that is the negation. It is also called bitwise NOT. The tilde symbol indicates this operation in Bash.

Swap the value of each bit of a number to perform bitwise negation. It means that you replace each one to zero and vice versa.

Here is an example of doing bitwise NOT of number 5:

```
5 = 101
~5 = 010
```

[326] https://en.wikipedia.org/wiki/Modulo_operation#Properties_(identities)
[327] https://en.wikipedia.org/wiki/Bitwise_operation

The bitwise NOT is a simple operation when we are talking about mathematics. However, using it in programming causes difficulties. First, you should know how many bytes the number occupies.

Suppose that the two-byte variable stores the number 5 in our example. Then it looks like this in memory:

```
00000000 00000101
```

Bitwise NOT for these bits gives us the following result:

```
11111111 11111010
```

What does this result mean? If the variable is an unsigned integer, the result equals the number 65530 in SMR. If the variable is a signed integer, it is the two's complement value. The result equals -6 in this case.

Bash commands and operators represent integers in different ways. For example, `echo` always outputs numbers as signed integers. The `printf` command allows you to specify the output format: signed or unsigned integer.

There are no types in the Bash language. Bash stores
all scalar variables as strings. Therefore, it interprets integers when it inserts them into arithmetic expressions. The interpretation (signed or unsigned) depends on the context.

Bash allocates 64 bits of memory space for each integer regardless of its sign. Table 3-20 shows maximum and minimum allowed integers in Bash.

**Table 3-20. Maximum and minimum allowed integers in Bash**

| Integer | Hexadecimal | Decimal |
|---|---|---|
| Maximum positive signed | 7FFFFFFFFFFFFFFF | 9223372036854775807 |
| Minimum negative signed | 8000000000000000 | -9223372036854775808 |
| Maximum unsigned | FFFFFFFFFFFFFFFF | 18446744073709551615 |

The following examples show how Bash interprets integers in `echo`, `printf`, and the (( operator:

```
$ echo $((16#FFFFFFFFFFFFFFFF))
-1

$ printf "%llu\n" $((16#FFFFFFFFFFFFFFFF))
18446744073709551615

$ if ((18446744073709551615 == 16#FFFFFFFFFFFFFFFF)); then echo "ok"; fi
ok

```

```
$ if ((-1 == 16#FFFFFFFFFFFFFFFF)); then echo "ok"; fi
ok

$ if ((18446744073709551615 == -1)); then echo "ok"; fi
ok
```

The last example of comparing the numbers 18446744073709551615 and -1 shows that Bash stores signed and unsigned integers in memory the same way. But their interpretation depends on the context.

Let's come back to the bitwise negation of the number 5. Bash gave us the result 0xFFFFFFFFFFFFFFFA in hexadecimal. You can print this 64-bit number as a positive or negative integer this way:

```
$ echo $((~5))
-6

$ printf "%llu\n" $((~5))
18446744073709551610
```

The numbers 18446744073709551610 and -6 are equal in terms of Bash. It happens because all their bits in memory are the same.

**Exercise 3-9. Bitwise NOT**

Apply bitwise NOT for the following unsigned two-byte integers:

* 56
* 1018
* 58362

Repeat the calculations for the case when these integers are signed.

## Bitwise AND, OR and XOR

The bitwise AND operation resembles the logical AND. The result of the logical AND is "true" when both operands are "true". Any other values of operands lead to the "false" result.

Operands of the bitwise AND are two numbers. You can do this operation in three steps:

1. Represent numbers in the two's complement.
2. If one number has fewer bits than another, add zeros to its left side.
3. Apply the logical AND operation to each pair of numbers' bits. The pair means two bits from each number that have the same position.

Here is an example. We want to calculate the bitwise AND for numbers 5 and 3. First, we should represent them in the two's complement like this:

```
5 = 101
3 = 11
```

Number 3 has fewer bits than 5. Thus, we add an extra zero on its left side. This way, we get the following representation:

```
3 = 011
```

Now we apply the logical AND for bit pairs of the numbers. For convenience, let's write the numbers in columns like this:

```
101
011
---
001
```

The result is 001. We can translate it in decimal:

```
001 = 1
```

It means that the bitwise AND operation with numbers 5 and 3 gives 1.

The ampersand sign denotes the bitwise AND operation in Bash. Here is the command to repeat our calculations and print the result:

```
echo $((5 & 3))
```

The bitwise OR operation works similarly as bitwise AND. But instead of the logical AND, you should perform the logical OR on bit pairs of the numbers.

For example, we want to calculate the bitwise OR for the numbers 10 and 6. First, write them in the two's complement like this:

```
10 = 1010
6 = 110
```

One bit is missing in the number 6. Let's extend it by one zero to get four bits:

```
6 = 0110
```

Now perform the logical OR on bit pairs of the numbers:

```
1010
0110
----
1110
```

The last step is converting the result in decimal:

```
1110 = 14
```

We got the number 14.

The vertical bar denotes the bitwise OR in Bash. Here is the command to check our calculations:

```
echo $((10 | 6))
```

The bitwise exclusive OR (XOR) operation is similar to the bitwise OR. Here the logical OR is replaced by the **exclusive OR**. The exclusive OR returns "false" only if both operands are the same. In other cases, the result equals "true".

Let's calculate the exclusive OR for the numbers 12 and 5. Here is their representation in the two's complement:

```
12 = 1100
5 = 101
```

We should supplement the number 5 to four bits:

```
5 = 0101
```

Perform a bitwise exclusive OR for each pair of bits:

```
1100
0101
----
1001
```

Convert the result to the decimal:

```
1001 = 9
```

The caret symbol denotes the exclusive OR in Bash. The following command repeats our calculations:

```
echo $((12 ^ 5))
```

**Exercise 3-10. Bitwise AND, OR and XOR**

```
Perform bitwise AND, OR and XOR for the following unsigned two-byte integers:

* 1122 and 908
* 49608 and 33036
```

## Bit Shifts

A bit shift is a changing of the bit positions in a number.

There are three types of bit shifts:

1. Logical
2. Arithmetic
3. Circular

The simplest shift is the logical one. Let's take a look at it first.

Any bit shift operation takes two operands. The first one is the integer. The operation shifts its bits. The second operand is the number of bits to shift.

You should represent the integer in the two's complement for doing the logical bit shift. Suppose that you do a shift in the right direction by two bits. Then you discard the two rightmost bits of the integer. Instead of them, add two zeros on the left side.

You can do the shift to the left in the same way. Discard two leftmost bits of the integer. Then add two zeroes on the right side.

Here is an example. Perform a logical shift of unsigned single-byte integer 58 to the right by three bits. First, represent the number in the two's complement:

```
58 = 0011 1010
```

Then discard the three bits on the right side like this:

```
0011 1010 >> 3 = 0011 1
```

Then add zeros to the left side of the result:

```
0011 1 = 0000 0111 = 7
```

The result of the shift is the number 7.

Now shift the number 58 to the left by three bits. We get the following result:

```
0011 1010 << 3 = 1 1010 = 1101 0000 = 208
```

Here we follow the same algorithm as for the right shift. First, discard the outermost bits on the left side. Then add zeros to the right side.

Now let's consider the second type of shift that is the arithmetic shift. If you do it to the left, you follow the logical shift algorithm. The steps are entirely the same.

The arithmetic shift to the right differs from the logical shift to the right. When performing it, you should discard the bits on the right side. Then complete the result with the bits on the left side. Their value should match the highest bit of the integer. If it equals one, add ones to the right. Otherwise, add zeros. This way, we keep the sign of the integer unchanged after the shifting.

For example, let's do an arithmetic shift of the signed one-byte integer -105 to the right by two bits.

First, we represent the number in the two's complement like this:

```
-105 = 1001 0111
```

Now do the arithmetic shift to the right by two bits. We get:

```
1001 0111 >> 2 -> 1001 01 -> 1110 0101
```

The highest bit of the integer equals one in our case. Thus, we complement the result with two ones on the left side.

This way, we got a negative number in the two's complement. Let's convert it to SMR and get the decimal like this:

```
1110 0101 = 1001 1011 = -27
```

The result of the shift is the number -27.

Bash has operators << and >>. They do arithmetic shifts. The following commands repeat our calculations:

```
$ echo $((58 >> 3))
7

$ echo $((58 << 3))
464

$ echo $((-105 >> 2))
-27
```

Bash gives another result for shifting 58 to the left by three bits. We got 208 in this case. It happens because Bash always operates with eight-byte integers.

The last type of bit shift is circular. You can rarely meet it when programming. Therefore, most programming languages do not have built-in operators for circular shifts.

In the cyclic shift, the discarded bits appear in the vacated place at the other side of the number.

Here is an example of the circular shift of the number 58 to the right by three bits:

```
0011 1010 >> 3 = 010 0011 1 = 0100 0111 = 71
```

We have discarded bits 010 on the right side. Then they appeared on the left side of the result.

**Exercise 3-11. Bit shifts**

```
Perform arithmetic bit shifts of the following signed two-byte integers:

* 25649 >> 3
* 25649 << 2
* -9154 >> 4
* -9154 << 3
```

## Using Bitwise Operations

Bitwise operations are widely used in system programming. It happens because working with computer networks and peripheral devices requires translating data from one format to another.

Here is an example. Suppose you are working with a peripheral device. The byte order on the device is big-endian. Your computer uses another order, which is little-endian.

Now suppose that the device sends an unsigned integer to the computer. It equals 0xAABB in hexadecimal. Because of the different byte orders, your computer cannot receive the integer as it is. You should convert the integer to 0xBBAA. Then the computer reads it correctly.

Here are the steps for converting the 0xAABB integer into the computer's byte order:

1. Read the lowest (rightmost) byte of the integer and shift it to the left by eight bits, i.e. one byte. The following Bash command does that:

```
little=$(((0xAABB & 0x00FF) << 8))
```

2. Read the highest (leftmost) byte of the number and shift it to the right by eight bits. Here is the command:

```
big=$(((0xAABB & 0xFF00) >> 8))
```

3. Connect the highest and lowest bytes with the bitwise OR this way:

```
result=$((little | big))
```

Bash wrote the converting result to the `result` variable. It is equal to 0xBBAA.

The following command does all steps of our calculation at once:

```
value=0xAABB
result=$(( ((value & 0x00FF) << 8) | ((value & 0xFF00) >> 8) ))
```

Here is another example of using bitwise operations. They are essential for computing bitmasks. We are already familiar with file permission masks in the Unix environment. Suppose that a file has the permissions "-rw-r–r–". This mask looks like this in binary:

```
0000 0110 0100 0100
```

We want to check if the owner of the file has the right to execute it. We can do that by calculating the bitwise AND for the file's permissions and the mask 0000 0001 0000 0000 0000. Here is the calculation:

```
0000 0110 0100 0100 & 0000 0001 0000 0000 = 0000 0000 0000 0000 = 0
```

The result is zero. It means that the owner cannot execute the file.

The bitwise OR adds bits to the bitmask. We can add the execution permission to the file owner. The calculation looks like this:

```
0000 0110 0100 0100 | 0000 0001 0000 0000 = 0000 0111 0100 0100 = -rwxr--r--
```



The numbering of bits in an integer starts at zero. It usually goes from right to left.

We performed the bitwise OR for the file's permissions and the mask 0000 0001 0000 0000 0000. The only eighth bit of the mask equals one. We use it to change the eighth bit of the permissions. This bit can have any value. It does not matter in this calculation. The bitwise OR sets it to one regardless of its current value. If we do not change the permissions bit, the corresponding bit in the mask equals zero.

The bitwise AND removes bits from the bitmask. For example, let's remove the file owner's permission to write. Here is the calculation:

```
0000 0111 0100 0100 & 1111 1101 1111 1111 = 0000 0101 0100 0100 = -r-xr--r--
```

We set the ninth bit of the permissions to zero. To do that, we calculate the bitwise AND for the permissions and the mask 1111 1101 1111 1111 1111. The ninth bit of the mask equals zero and all other bits are ones. Therefore, the bitwise AND changes the ninth bit of the permissions only.

The OS performs mask operations every time you access a file. This way, it checks your access rights.

Here is the last example of using bitwise operations. Until recently, software developers used bit shifts as an alternative to multiplication and division by a power of two. For example, the bit shift to the left by two bits corresponds to multiplication by $2^2$ (i.e. four). Let's check it with the following Bash command:

```
$ echo $((3 << 2))
12
```

The bit shift gave a correct result. Multiplying 3 by 4 is equal to 12.

Such tricks reduce the number of processor clock cycles to perform multiplication and division. These optimizations are now unnecessary due to the development of compilers and processors. Compilers automatically select the fastest assembly instructions when generating code. Processors execute these instructions in parallel with several threads[328]. Therefore, today software developers tend to write code that is easier to read and understand. They do not care about optimizations as they do it before. Multiplication and division operations are better for reading the code than bit shifts.

Cryptography and computer graphics algorithms use bit operations a lot.

## Logical Operations

The [[ operator is inconvenient for comparing integers in the `if` statement. This operator uses two-letter abbreviations for expressing the relations between numbers. For example, the `-gt` abbreviation means greater. The (( operator fits the `if` condition better. You can use usual number comparison symbols (>, <, =) there.

Here is an example. Suppose that you want to compare the variable with 5. The following `if` construction does that:

```
if ((var < 5))
then
  echo "The variable is less than 5"
fi
```

Here we use the (( operator in the arithmetic evaluation form. You can replace it by the `let` command. It provides the same result:

[328] https://en.wikipedia.org/wiki/Hyper-threading

```
if let "var < 5"
then
  echo "The variable is less than 5"
fi
```

However, you should always prefer to use the (( operator.

There is an important difference between arithmetic evaluation and expansion. According to the POSIX standard, any program or command returns the zero exit status when executed successfully. It returns the status between 1 and 255 in case of an error. The shell interprets the exit status like this: zero means "true" and nonzero means "false". If you apply this rule, the logical result of the arithmetic expansion is inverted. There is no inversion for the arithmetic evaluation result.

Arithmetic evaluation is synonymous with the `let` command. Therefore, it follows the POSIX standard just like any other command. The shell executes the arithmetic expansion in the context of another command. Thus, its result depends on the interpreter's implementation.

Suppose that you use the (( operator in the arithmetic expansion form. Then Bash interprets its result this way: if the condition in the (( operator equals "true", it returns one. Otherwise, the operator returns zero. The C language deduces Boolean expressions in the same way.

Here is an example for comparing the results of arithmetic expansion and evaluation. There is a Bash command that compares a variable with a number. It looks like this:

```
((var < 5)) && echo "The variable is less than 5"
```

There is an arithmetic evaluation in this command. Therefore, if the variable is less than 5, the (( operator succeeds. It returns the zero exit status according to the POSIX standard.

When you use the operator (( in the form of an arithmetic expansion, it gives another result. The following command makes the same comparison:

```
echo "$((var < 5))"
```

When the condition is true, the `echo` command prints the number one. If you are familiar with the C language, you expect the same result.

Logical operations are used in the arithmetic evaluation form of the (( operator in most cases. They work in the same way as the Bash logical operators.

Here is an example of applying the logical operation. The following `if` condition compares the variable with two numbers:

```
if ((1 < var && var < 5))
then
  echo "The variable is less than 5 but greater than 1"
fi
```

This condition is true when both expressions are true.

The logical OR works similarly:

```
if ((var < 1 || 5 < var))
then
  echo "The variable is less than 1 or greater than 5"
fi
```

The condition is true if at least one of the expressions is true.

The logical NOT is rarely applied to numbers themselves. It negates an expression in most cases. If you apply the logical NOT to a number, its output corresponds to the POSIX standard. In other words, zero means "true" and nonzero means "false". Here is an example:

```
if ((! var))
then
  echo "The vraiable equals true or zero"
fi
```

This condition is true if the variable is zero.

## Increment and Decrement

The increment and decrement operations first appeared in the programming language B[329]. Ken Thompson and Dennis Ritchie developed it in 1969 while working at Bell Labs. Dennis Ritchie later transferred these operations to his new language called C. Bash copied them from C.

First, let's consider the assignment operations. It helps to get the meaning of increment and decrement. A regular assignment in arithmetic evaluation looks like this:

```
((var = 5))
```

This command assigns the integer 5 to the variable.

Bash allows you to combine an assignment with arithmetic or bitwise operation. Here is an example of simultaneous addition and assignment:

[329] https://en.wikipedia.org/wiki/B_(programming_language)

```
((var += 5))
```

The command performs two actions:

1. It adds the integer 5 to the current value of the `var` variable.
2. It writes the result to the variable `var`.

All other assignment operations work the same way. First, they do a mathematical or bitwise operation. Second, they assign the result to the variable. The assignments' syntax makes the code shorter and clearer.

Now we are ready to consider the increment and decrement operations. They have two forms: postfix and prefix. They are written in different ways. The ++ and – signs come after the variable name in the postfix form. They come before the variable name in the prefix form.

Here is an example of the prefix increment:

```
((++var))
```

This command provides the same result as the following assignment operation:

```
((var+=1))
```

The increment operation increases the variable's value by one. The decrement operation decreases it by one.

Why does it make sense to introduce separate operations for adding and subtracting one? The Bash language has similar assignments like += and -=.

The most probable reason to add increment and decrement to the programming language is a loop counter[330]. This counter keeps the number of loop iterations. When you want to interrupt the loop, you check its counter in a condition. The result defines if you should interrupt the loop or not.

Increment and decrement make it easier to serve the loop counter. Besides that, modern processors perform these operations at the hardware level. Therefore, they work faster than addition and subtraction combined with the assignment.

What is the difference between prefix and postfix forms of increment? If the expression consists only of an increment operation, it gives the same result for both forms.

For example, the following two commands increase the variable's value by one:

[330] https://en.wikipedia.org/wiki/For_loop#Loop_counters

```
((++var))
((var++))
```

The difference between the increment forms appears when assigning the result to a variable. Here is an example:

```
var=1
((result = ++var))
```

After these two commands, both variables `result` and `var` store the integer `2`. It means that the prefix increment first adds one and then returns the result.

If you break the prefix increment into steps, you get the following commands:

```
var=1
((var = var + 1))
((result = var))
```

The postfix increment behaves differently. Here we change the increment's form in our example:

```
var=1
((result = var++))
```

These commands write the integer `1` to the `result` variable and the integer `2` to the `var` variable. Thus, postfix increment returns the value first. Then it adds one.

If you break the postfix increment into steps, you get the following commands:

```
var=1
((tmp = var))
((var = var + 1))
((result = tmp))
```

Note the order of steps in the postfix increment. First, it increments the `var` variable by one. Then it returns the past value of `var`. Therefore, the increment requires the temporary variable `tmp` to store the past value.

The postfix and prefix forms of decrement work similarly to increment.

Always use the prefix increment and decrement instead of the postfix form. First, the CPU performs them faster. The reason is it does not need to save the past value of the variable in the registers. Second, it is easier to make an error using the postfix form. It happens because of the non-obvious assignment order.

## Ternary Operator

The ternary operator is also known as the conditional operator and **ternary if**. It first appeared in the programming language ALGOL[331]. The operator turned out to be convenient and in demand by programmers. Therefore, the languages of the next generation (BCPL[332] and C) inherited ternary if. Then it comes to almost all modern languages: C++, C#, Java, Python, PHP, etc.

A ternary operator is a compact form of the `if` statement.

Here is an example. Suppose that there is the following `if` statement in the script:

```
if ((var < 10))
then
  ((result = 0))
else
  ((result = var))
fi
```

Here the `result` variable gets the zero value if `var` is less than 10. Otherwise, `result` gets the value of `var`.

The ternary operator can provide the same behavior as our `if` statement. The operator looks like this:

```
((result = var < 10 ? 0 : var))
```

Here we replaced six lines of the `if` statement with one line.

A ternary operator consists of a conditional expression and two actions. It looks like this in the general form:

```
(( CONDITION ? ACTION_1 : ACTION_2 ))
```

If the CONDITION is true, Bash executes the ACTION_1. Otherwise, it executes the ACTION_2. This behavior matches the `if` statement. Let's write it down in the general form too:

[331] https://en.wikipedia.org/wiki/ALGOL
[332] https://en.wikipedia.org/wiki/BCPL

```
if CONDITION
then
  ACTION_1
else
  ACTION_2
fi
```

Compare the ternary operator and the `if` statement.

Unfortunately, Bash only allows the ternary operator in arithmetic evaluation and expansion. It means that the operator accepts only arithmetic expressions as actions. You cannot call commands and utilities as it happens in the code blocks of the `if` statement. There is no such restriction in other programming languages.

Use the ternary operator as often as possible. It is considered a good practice. The operator makes your code compact and easy to read. The less code has less room for potential errors.

# Loop Constructs

Conditional statements manage the **control flow**[333] of a program. The control flow is the order in which the operators and commands of a program are executed.

The conditional operator chooses a branch of execution depending on the Boolean expression. This operator is not enough sometimes. You want additional features to manage the control flow. **Loop constructs**[334] solve tasks that the conditional operator cannot handle.

The loop construct repeats the same block of commands multiple times. The single execution of this block is called the **loop iteration**. At each iteration, the loop checks its condition. The check result defines to perform the next iteration or not.

## Repetition of Commands

Why do you need to repeat the same block of commands in a program? Several examples will help us to answer this question.

We are already familiar with the `find` utility. It looks for files and directories on the hard disk. If you add the `-exec` option to the `find` call, you can specify an action. The utility performs this action on each object found.

For example, the following command deletes all PDF documents in the `~/Documents` directory:

[333] https://en.wikipedia.org/wiki/Control_flow
[334] https://en.wikipedia.org/wiki/Control_flow#Loops

```
find ~/Documents -name "*.pdf" -exec rm {} \;
```

In this case, `find` calls the `rm` utility several times. It passes the next found file on each call. It means that the `find` utility executes a loop. The loop ends when all the files found have been processed.

The `du` utility is another example of the repetition of commands. The utility estimates the amount of disk space used on the disks. It has an optional parameter. The parameter sets the path where the estimation starts.

You can call `du` like this, for example:

```
du ~/Documents
```

Here the utility recursively traverses all `~/Documents` subdirectories. It adds the size of each file found to the final result. This way, incrementing the result value repeats over and over again.

The `du` utility has an internal loop. It traverses over all files and subdirectories. There are the same actions on each loop iteration. The only difference is the checked file system object.

Repetition of operations happens in mathematical calculations often. A canonical example is the calculation of **factorial**[335]. The factorial of the number N is a multiplication of natural numbers from 1 to N inclusive.

Here is an example of calculating the factorial of number 4:

```
4! = 1 * 2 * 3 * 4 = 24
```

You can calculate the factorial easily by using the loop operator. The loop should pass through the integers from 1 to N in sequence. Multiply each integer to the final result on the loop iteration. This way, you repeat the multiplication operation several times.

Here is the last example of repetition actions in a computer system. Repetition helps to manage some events.

Suppose that you write a program. It downloads files to your computer from the Internet. First, the program establishes a connection to a server. If the server doesn't respond, the program has two options to do. The first one is to terminate with a non-zero exit status. The second option is to wait for the server's response. This option is preferable. There are many reasons why the server's response delays. There is an overload of the network or the server, for example. A couple of seconds waiting is enough to get a response. Then our program can continue to work.

Now the question arises: how can you wait for the event to occur in the program? The easiest way is to use the loop operator. Its condition should check if the event occurs. In this case, the operator stops.

[335] https://en.wikipedia.org/wiki/Factorial

Let's come back to our example. The loop should stop when the program receives a response from the server. While it does not happen, the loop continues. You do not need any actions on each loop's iteration. Thus, you can leave this code block empty. This technique is called **busy waiting**[336].

You can optimize the busy waiting. Replace the loop's empty code block with a command that stops the program for a short while. This way, OS gets a chance to execute another task while your program is waiting.

We have considered several examples where the program repeats the same actions. Let's write down the tasks that we have solved in each example. Here is the list:

1. Process of multiple entities monotonously. The `find` utility processes the search results this way.
2. Apply the intermediate data to accumulate the final result. The `du` utility does it for collecting statistics.
3. Mathematical calculations. You can calculate factorial using the loop operator.
4. Wait for some event to happen. You can wait for the server's response in a busy waiting loop.

The list is far from being complete. These are just the most common programming tasks that require a loop operator.

## While Statement

There are two loop operators in Bash: `while` and `for`. First, we will consider the `while` statement. It works simpler than `for`.

The `while` syntax resembles the `if` statement. It looks like this in general:

```
while CONDITION
do
  ACTION
done
```

You can write the `while` statement in one line:

```
while CONDITION; do ACTION; done
```

The CONDITION is a single command or block of commands. The same is true for the ACTION. It resembles the `if` statement again. The ACTION is called the **loop body**.

Bash checks the CONDITION of the `while` statement first. If the CONDITION command returns null exist status, it equals "true". In this case, Bash executes the ACTION. Then it checks the

[336] https://en.wikipedia.org/wiki/Busy_waiting

CONDITION again. If it is still true, the ACTION is performed again. The loop execution stops when the CONDITION becomes "false".

Use the while loop when you do not know the number of iterations beforehand. The example is busy waiting for some event.

Let's write a script with the `while` statement. It should check if the server is available on the Internet. The simplest way for that is by sending a request to the server. When the server replies, the script displays a message and stops.

We can call the `ping`[337] utility to send a request to the server. The utility uses the ICMP[338] **protocol**. The protocol is an agreement for the format of the messages between the computers on the network. The ICMP protocol describes the format of the messages to serve the network. You need them, for example, to check if some computer is available.

The `ping` utility takes one mandatory input parameter. It is URL[339] or IP address[340] of the target **host**. A host is any computer or device connected to the network.

Here is the command to call the `ping` utility:

```
ping google.com
```

We have specified the Google server as the target host. The utility sends ICMP messages to it. The server replies to each of them. The output of the utility looks like this:

```
1 PING google.com (172.217.21.238) 56(84) bytes of data.
2 64 bytes from fra16s13-in-f14.1e100.net (172.217.21.238): icmp_seq=1 ttl=51 time=17.\
3 8 ms
4 64 bytes from fra16s13-in-f14.1e100.net (172.217.21.238): icmp_seq=2 ttl=51 time=18.\
5 5 ms
```

You see information about each ICMP message sent by the utility. The "time" field means the delay between sending the request and receiving the server's response.

The utility runs in an infinite loop by default. You can stop it by pressing Ctrl+C.

You do not need to send several requests to check the availability of a server. It is sufficient to send a single ICMP message instead. The `-c` option of the `ping` utility specifies the number of messages to send. Here is an example of how to use it:

```
ping -c 1 google.com
```

[337] https://en.wikipedia.org/wiki/Ping_(networking_utility)
[338] https://en.wikipedia.org/wiki/Internet_Control_Message_Protocol
[339] https://en.wikipedia.org/wiki/URL
[340] https://en.wikipedia.org/wiki/IP_address

If the `google.com` server is available, the utility returns the zero exit status. Otherwise, it returns non-zero.

The `ping` utility waits for the server's response until you do not interrupt it. The `-W` option limits the waiting time to one second. Using the option, we get the following command:

```
ping -c 1 -W 1 google.com
```

Now we have the condition for the `while` statement. Let's write the whole statement like this:

```
1 while ! ping -c 1 -W 1 google.com &> /dev/null
2 do
3   sleep 1
4 done
```

The output of the `ping` utility is not interested in our case. Therefore, we redirect it to the `/dev/null` file.

We invert the `ping` result in the `while` condition. Therefore, Bash executes the loop's body as long as the utility returns a non-zero exit status. It means that the loop continues as long as the server is unavailable.

We call the `sleep` utility in the loop's body. It stops the script for the specified number of seconds. The stop lasts for one second in our case.



You can specify a suffix for the parameter when using the `sleep` utility. It defines units of time. The suffix "s" matches seconds. It is "m" for minutes, "h" for hours and "d" for days.

Listing 3-18 shows a complete script for checking server availability.

**Listing 3-18. Script for checking server availability**

```
1 #!/bin/bash
2
3 while ! ping -c 1 -W 1 google.com &> /dev/null
4 do
5   sleep 1
6 done
7
8 echo "The google.com server is available"
```

The `while` statement has an alternative form called `until`. Here the ACTION is executed as long as the CONDITION is "false". It means that the loop continues as long as the CONDITION returns a non-zero exit status. Use the `until` statement when you want to invert the condition of the `while` loop.

Here is the `until` statement in general:

```
until CONDITION
do
  ACTION
done
```

You can write it in one line the same way as `while`:

```
until CONDITION; do ACTION; done
```

Let's replace the `while` statement with `until` in Listing 3-18. You should remove the negation of the `ping` utility result for that. Listing 3-19 shows the resulting script.

**Listing 3-19. Script for checking server availability**

```
#!/bin/bash

until ping -c 1 -W 1 google.com &> /dev/null
do
  sleep 1
done

echo "The google.com server is available"
```

The scripts in Listing 3-18 and Listing 3-19 behave the same.

Choose the `while` or `until` statement depending on the loop condition. Try to avoid negations in conditions. Negations make the code harder to read.

## Infinite Loop

The `while` statement fits well to implement **infinite loops**[341]. This kind of loop continues as long as the program is running.

You can find infinite loops in system software that runs until the computer is powered off. Examples are OS or microcontroller firmware. Computer games and monitor programs for collecting statistics also use such loops.

The `while` loop becomes infinite if its condition is always true. The easiest way to set such a condition is to call the `true` command. Here is an example of using it:

[341] https://en.wikipedia.org/wiki/Infinite_loop

```
while true
do
  sleep 1
done
```

The `true` command always returns the "true" value. It means that it returns zero exit status. There is the symmetric command called `false`. It always returns exit status one that matches the "false" value.



Words "true" and "false" are **literals**[342] in most programming languages. Literals are reserved words for representing fixed values. In our case, they represent the "true" and "false" values.

You can replace the `true` command in the `while` condition with a colon. This way, you get the following:

```
while :
do
  sleep 1
done
```

The colon is synonymous with the `true` command. The synonymous solves the compatibility task[343] with the Bourne shell. This shell does not have `true` and `false` commands. Therefore, Bourne shell scripts use colons and Bash should support them.

The POSIX standard includes all three commands: colon, `true`, and `false`. However, avoid using a colon in your scripts. It is a deprecated syntax that makes your code harder to understand.

Here is an example of an infinite loop. We want to write a script that displays statistics of disk space usage. The `df` utility can help us in this case. It prints the following when called without parameters:

```
$ df
Filesystem     1K-blocks      Used Available Use% Mounted on
C:/msys64       41940988  24666880  17274108  59% /
Z:             195059116 110151748  84907368  57% /z
```

The utility shows "Used" and "Available" disk space in bytes. We can add the `-h` option to the utility call. Then it shows kilobytes, megabytes, gigabytes and terabytes instead of bytes. Also, we add an option `-T`. It shows the file system type for each disk. This way, we get the following output:

[342] https://en.wikipedia.org/wiki/Literal_(computer_programming)
[343] https://stackoverflow.com/questions/3224878/what-is-the-purpose-of-the-colon-gnu-bash-builtin

```
$ df -hT
Filesystem     Type  Size  Used Avail Use% Mounted on
C:/msys64      ntfs   40G   24G   17G  59% /
Z:             hgfs  187G  106G   81G  57% /z
```

If you want to get information about all mount points, add the `-a` option.

Now let's write an infinite loop. It calls the `df` utility on each iteration. This way, we get a simple script to monitor the file system. Listing 3-20 shows the script.

**Listing 3-20. The script to monitor the file system**

```
#!/bin/bash

while :
do
  clear
  df -hT
  sleep 2
done
```

The first action of the cycle iteration is the `clear` utility call. It clears the terminal window of text. Thanks to this step, the terminal shows the output of our script only.

Executing a command in a cycle is a common task that arises when working with Bash. The `watch` utility solves this task. The utility is a part of the `procps` package. The following command installs this package to the MSYS2 environment:

```
pacman -S procps
```

Now you can replace the script from listing 3-20 with a single command. It looks like this:

```
watch -n 2 "df -hT"
```

The `-n` option of the `watch` utility specifies the interval between command calls. The command to execute follows all utility options.

The `-d` utility option highlights the difference in the command's output at the current iteration and the last iteration. This way, it is easier to keep track of changes that have occurred.

## Reading a Standard Input Stream

The `while` loop fits well for handling an input stream. Here is an example of such a task. We want to write a script that reads a text file. The script makes an associative array from the file's content.

Listing 3-10 shows the script for managing the list of contacts. The script stores contacts in the format of the Bash array declaration. It makes adding a new person to the list inconvenient. The user must know the Bash syntax. Otherwise, he can make a mistake when initializing an array element. It will break the script.

We can solve the problem of editing the contacts list. Let's put the list in a separate text file. Our script should read it at startup. This way, we separate data and code. It is a well-known and good practice in software development.

Listing 3-21 shows a possible format of the file with contacts.

**Listing 3-21. The file with contacts `contacts.txt`**

```
Alice=alice@gmail.com
Bob=(697) 955-5984
Eve=(245) 317-0117
Mallory=mallory@hotmail.com
```

Let's write a script to read this file. It is convenient to read the list of contacts directly into the associative array. This way, we keep the searching mechanism over the list as effective as before.

When reading the file, we should process its lines in the same manner. It means that we will repeat our actions. Therefore, we need a loop statement. At the beginning of the loop, we don't know the size of the file. Thus, we do not know the number of iterations to do. The `while` statement fits this case perfectly.

Why do we not know the number of iterations in advance? It happens because the script reads the file line by line. It cannot count the lines before it reads them all. We can make two loops. The first one counts the lines. The second loop processes them. However, this solution works slower and less ineffective.

We can use the `read` built-in command for reading lines of the file. The command receives a string from the standard input stream. Then it writes the string into the specified variable. You can pass the variable's name as a parameter. Here is an example of doing that:

```
read var
```

Run this command. Then type the string and press Enter. The `read` command writes your string into the `var` variable. You can call `read` without parameters. It writes the string into the reserved variable `REPLY` in this case.

When `read` receives the string, it removes backslashes \ there. They escape special characters. Therefore, the `read` command considers the backslashes unnecessary. The `-r` option disables this feature. Use it always to prevent losing characters of the input string.

You can pass several variable names to the `read` command. Then it divides the input text into parts. The command uses the delimiters from the reserved variable `IFS` in this case. Default delimiters are spaces, tabs and line breaks.

Here is an example of multiple variables for the `read` command. Suppose that we want to store the input string into two variables. They are called `path` and `file`. The following command reads them:

```
read -r path file
```

The user types the following string for this command:

```
~/Documents report.txt
```

Then the `read` command writes the `~/Documents` path into the `path` variable. The filename `report.txt` comes into the `file` variable.

If the path or filename contains spaces, an error occurs. Suppose the user type the following string:

```
~/My Documents report.txt
```

Then the command writes the `~/My` string into the `path` variable. The `file` variable stores the rest part of the input: `Documents report.txt`. This is a wrong result. Don't forget about such behavior when using the `read` command.

We can solve the problem of splitting the input string. This can be done by redefining the `IFS` variable. Here is an example to specify comma as only one possible delimiter:

```
IFS=$',' read -r path file
```

Here we have applied the Bash-specific type of quotes[344] `$'...'`. Bash does not perform any expansions inside them. At the same time, you can place some control sequences there: `\n` (new line), `\\` (escaped backslash), `\t` (tabulation) and `\xnn` (bytes in hexadecimal).

The new `IFS` declaration allows to process the following input string properly:

```
1 ~/My Documents,report.txt
```

The comma separates the path and filename. Therefore, the `read` command writes the `~/My Documents` string into the `path` variable. The `report.txt` string comes into the `file` variable.

The `read` command receives data from the standard input stream. It means that you can redirect the file contents to the command.

Here is an example to read the first line of the `contacts.txt` file from Listing 3-21. The following command does it:

[344] http://mywiki.wooledge.org/Quotes

```
read -r contact < contacts.txt
```

This command writes the "Alice=alice@gmail.com" string into the `contact` variable.

We can write the name and contact information into two different variables. Let's define the equal sign as a delimiter to do that. Then our `read` command looks like this:

```
IFS=$'=' read -r name contact < contacts.txt
```

Now the `name` variable gets the "Alice" name. The e-mail address comes into the `contact` variable.

Let's try the following `while` loop for reading the entire `contacts.txt` file:

```
1 while IFS=$'=' read -r name contact < "contacts.txt"
2 do
3   echo "$name = $contact"
4 done
```

Unfortunately, it does not work. Here we got an infinite loop accidentally. It happens because the `read` command always reads only the first line of the file. Then the command returns the zero exit status. The zero status leads to the loop body execution. It happens over and over again.

We should force the `while` loop to pass through all lines of the file. The following form of the loop does it:

```
1 while CONDITION
2 do
3   ACTION
4 done < FILE
```

You can handle the input from the keyboard this way. Specify the `/dev/tty` file in this case. The loop will read keystrokes until you press Ctrl+D.

Here is the right `while` loop to read the `contacts.txt` file:

```
1 while IFS=$'=' read -r name contact
2 do
3   echo "$name = $contact"
4 done < "contacts.txt"
```

This loop prints the entire contents of the contact file.

There is the last step left to finish our task. We should write the `name` and `contact` variables to the array on each iteration. The `name` variable is the key and `contact` is the value.

Listing 3-22 shows the final version of the script for reading the contacts from the file.

**Listing 3-22. The script for managing the contacts**

```
#!/bin/bash

declare -A array

while IFS=$'=' read -r name contact
do
  array[$name]=$contact
done < "contacts.txt"

echo "${array["$1"]}"
```

This script behaves the same way as one in Listing 3-10.

## For Statement

There is another loop statement in Bash called `for`. Unlike `while`, use it when you know the number of iterations in advance.

The `for` statement has two forms. The first one processes words in a string sequentially. The second form applies an arithmetic expression in the loop's condition.

### The First Form of For

Let's start with the first form of the `for` statement. It looks like this in general:

```
for VARIABLE in STRING
do
  ACTION
done
```

You can write the same construction in a single line like this:

```
for VARIABLE in STRING; do ACTION; done
```

The ACTION in the `for` statement is a single command or a block of commands. It is the same as in the `while` statement.

Bash performs all expansions in the `for` condition before starting the first iteration of the loop. What does it mean? Suppose you specified the command instead of a STRING. Then Bash executes this command and replaces it with its output. Also, you can specify a pattern instead of STRING. Then Bash expands it before starting the loop.

BASH splits the STRING into words when there are no commands or patterns left in the `for` condition. It takes separators for splitting from the `IFS` variable.

Then Bash executes the first iteration of the loop. The first word of the STRING is available via VARIABLE inside the loop body on the first iteration. Then Bash writes the second word of the STRING to the VARIABLE and starts the second iteration. It happens again and again until we pass all words of the STRING.

Here is an example of the `for` loop. We want to write a script to print words in a string one by one. The script receives the string via the first parameter.

Listing 3-23 shows the script.

**Listing 3-23. The script for printing words of a string**

```
#!/bin/bash

for word in $1
do
  echo "$word"
done
```

Here you should not enclose the position parameter `$1` in quotes. Quotes prevent word splitting, but we want it in this case. Otherwise, Bash passes the whole string to the first iteration of the `for` loop. Then the loop finishes. We do not want this behavior. The script should process each word of the string separately.

When you call the script, you should enclose the input string in double-quotes. Then the whole string comes into the `$1` parameter. Here is an example of calling the script:

There is a way to get rid of the double-quotes when calling the script. Replace the `$1` parameter in the `for` condition with `$@`. Then the loop statement becomes like this:

```
for word in $@
do
  echo "$word"
done
```

Now both following script calls work properly:

```
./for-string.sh this is a string
./for-string.sh "this is a string"
```

The `for` loop condition has a short form. Use it when you want to pass through all input parameters of the script. This short form looks like this:

```
for word
do
  echo "$word"
done
```

It does the same as the script in Listing 3-23. We just dropped the "in $@" part in the condition. It did not change the loop behavior.

Let's make the task a bit more complicated. Suppose the script receives a list of paths on input. Commas separate them. The paths may contain spaces. We should redefine the `IFS` variable to process such input correctly.

Listing 3-24 shows the `for` loop to print the list of paths.

**Listing 3-24. The script for printing the list of paths**

```
#!/bin/bash

IFS=$','
for path in $1
do
  echo "$path"
done
```

We have specified only one allowable delimiter in the `IFS` variable. The delimiter is the comma. Therefore, the `for` loop ignores spaces when splitting the input string.

You can call the script this way:

```
./for-path.sh "~/My Documents/file1.pdf,~/My Documents/report2.txt"
```

Here double-quotes for the input string are mandatory. You cannot replace the `$1` parameter with `$@` in the `for` condition and omit quotes. This will lead to an error. The error happens because Bash does word splitting when calling the script. This word splitting applies spaces as delimiters. It happens before our redeclaration of the `IFS` variable. Thus, Bash ignores our change of the variable in this case

If there is a comma in one of the paths, it leads to an error.

The `for` loop can pass through the elements of an indexed array. It works the same way as processing words in a string. Listing 3-25 shows an example of doing that.

**Listing 3-25. The script for printing all elements of the array**

```
#!/bin/bash

array=(Alice Bob Eve Mallory)

for element in "${array[@]}"
do
    echo "$element"
done
```

Suppose you need the first three elements. Then you should expand only the elements you need in the loop condition. Listing 3-26 shows how to do that.

**Listing 3-26. The script for printing the first three elements of the array**

```
#!/bin/bash

array=(Alice Bob Eve Mallory)

for element in "${array[@]:0:2}"
do
    echo "$element"
done
```

There is another option to pass through the array. You can iterate over the indexes instead of the array's elements. Write the string with indexes of the elements you need. Spaces should separate them. Put the string into the `for` condition. Then the loop gives you an index on each iteration. The loop looks like this:

```
array=(Alice Bob Eve Mallory)

for i in 0 1 2
do
  echo "${array[i]}"
done
```

This loop passes only through elements with indexes 0, 1 and 2.

You can apply the brace expansion to specify the indexes list. Here is an example:

```
array=(Alice Bob Eve Mallory)

for i in {0..2}
do
  echo "${array[i]}"
done
```

The loop behaves the same way. It prints the first three elements of the array.

Do not iterate over the element's indexes when processing arrays with gaps. Expand the array's elements in the loop condition instead. Listing 3-25 and Listing 3-26 show how to do that.

## Files Processing

The `for` loop fits well for processing a list of files. When solving this task, you should compose the loop condition correctly. There are several common mistakes here. Let's consider them by examples.

The first example is a script that prints types of files in the current directory. We can do it by calling the `file` utility for each file.

The most common mistake when composing the `for` loop condition is neglecting patterns (globbing). Users often call the `ls` or `find` utility to get the STRING. It happens this way:

```
for filename in $(ls)
for filename in $(find . - type f)
```

This is wrong. Such a solution leads to the following problems:

1. Word splitting breaks names of files and directories with spaces.
2. If the filename contains an asterisk, Bash performs globbing before starting the loop. Then it writes the expansion result to the `filename` variable. This way, you lose the actual filename.
3. The output of the `ls` utility depends on the regional settings. Therefore, you can get question marks instead of the national alphabet characters in filenames. Then the `for` loop cannot process these files.

Always use patterns in the `for` loop to enumerate filenames. It is the only correct solution for this task.

We should write the following `for` loop condition in our case:

```
for filename in *
```

Listing 3-27 shows the complete script.

**Listing 3-27. The script for printing the file types**

```
#!/bin/bash

for filename in *
do
  file "$filename"
done
```

Do not forget to use double-quotes when accessing the `filename` variable. They prevent word splitting of filenames with spaces.

You can still use the pattern in the `for` loop condition if you want to process files from a specific directory. Here is an example of such a pattern:

```
for filename in /usr/share/doc/bash/*
```

A pattern can filter out files with a specific extension or name. It looks like this:

```
for filename in ~/Documents/*.pdf
```

There is a new feature for patterns in Bash version 4. You can pass through directories recursively. Here is an example:

```
shopt -s globstar

for filename in **
```

This feature is disabled by default. Activate it by enabling the `globstar` interpreter option with the `shopt` command.

When Bash meets the `**` pattern, it inserts a list of all subdirectories and their files starting from the current directory. You can combine this mechanism with regular patterns.

For example, let's process all files with the PDF extension from the user's home directory. The following `for` loop condition does that:

```
shopt -s globstar

for filename in ~/**/*.pdf
```

There is another common mistake when using the `for` loop. Sometimes you just do not need it. For example, you can replace the script in Listing 3-27 with the following `find` call:

```
find . -maxdepth 1 -exec file {} \;
```

This command is more efficient than the `for` loop. It is compact and works faster because of fewer operations to do.

When should you use the `for` loop instead of the `find` utility? Use `find` when one short command processes found files. If you need a conditional statement or block of commands for this job, use the `for` loop.

There are cases when patterns are not enough in the `for` loop condition. You want to do a complex search with checking file types, for example. In this case, use the `while` loop instead of `for`.

Let's replace the `for` loop in Listing 3-27 with `while`. The `find` utility will provide us a list of files. But we should call it with the `-print0` option. This way, we avoid word splitting issues. Listing 3-28 shows how to combine the `find` utility and `while` loop properly.

**Listing 3-28. The script for printing the file types**

```
#!/bin/bash

while IFS= read -r -d '' filename
do
  file "$filename"
done < <(find . -maxdepth 1 -print0)
```

There are several tricky solutions in this script. Let's take a closer look at them. The first question is why we need to assign an empty value to the `IFS` variable? If we keep the variable unchanged, Bash splits the `find` output by default delimiters (spaces, tabs and line breaks). It can break filenames with these characters.

The second solution is applying the `-d` option of the `read` command. The option defines a delimiter character for splitting the input text. When using it, the `filename` variable gets the part of the string that comes before the next delimiter.

The `-d` option specifies the empty delimiter. It means a NULL character. You can also specify it explicitly. Do it like this:

```
while IFS= read -r -d $'\0' filename
```

Thanks to the `-d` option, the read command handles the `find` output correctly. There is the `-print0` option in the utility call. It means that `find` separates found files by a NULL character. This way, we reconcile the `read` input format and the `find` output.

Note that you cannot specify a NULL character as a delimiter using the `IFS` variable. In other words, the following solution does not work:

```
while IFS=$'\0' read -r filename
```

The problem comes from the peculiarity when interpreting the `IFS` variable[345]. If the variable is empty, Bash does not do word splitting at all. When you assign a NULL character to the variable, it means an empty value for Bash.

There is the last tricky solution in Listing 3-28. We use process substitution for passing the `find` output to the `while` loop. Why did we not use the command substitution instead? We can do it like this:

```
1 while IFS= read -r -d '' filename
2 do
3   file "$filename"
4 done < $(find . -maxdepth 1 -print0)
```

Unfortunately, this redirection does not work. The `<` operator couples the input stream and the specified file descriptor. But there is no file descriptor when using the command substitution. Bash calls the `find` utility and inserts its output instead of `$(...)`. When you use process substitution, Bash writes the `find` output to a temporary file. This file has a descriptor. Therefore, the stream redirection works fine.

There is only one issue with process substitution. It is not part of the POSIX standard. If you need to follow the standard, use a pipeline instead. Listing 3-29 demonstrates how to do it.

**Listing 3-29. The script for printing the file types**

```
1 #!/bin/bash
2
3 find . -maxdepth 1 -print0 |
4 while IFS= read -r -d '' filename
5 do
6   file "$filename"
7 done
```

Combine the `while` loop and `find` utility only when you have both following cases at the same time:

1. You need a conditional statement or code block to process files.
2. You have a complex condition for searching files.

When combining `while` and `find`, always use a NULL character as a delimiter. This way, you avoid the word splitting problems.

[345] https://mywiki.wooledge.org/IFS

## The Second Form of For

The second form of the `for` statement allows you to specify an arithmetic expression as a condition. Let's consider cases when do you need it.

Suppose we need a script to calculate the factorial. The solution for this task depends on the way we enter the data. The first option is we have a predefined integer. Then the first form of the `for` loop fits well. Listing 3-30 shows this solution.

**Listing 3-30. The script for calculating the factorial for integer 5**

```
#!/bin/bash

result=1

for i in {1..5}
do
  ((result *= $i))
done

echo "The factorial of 5 is $result"
```

The second option is to receive the integer as an input parameter of the script. We can try the following loop's condition to process the `$1` parameter:

```
for i in {1..$1}
```

We expect that Bash will do brace expansion for integers from one to the `$1` value. However, it does not work this way.

According to Table 3-2, the brace expansion happens before the parameter expansion. Thus, the loop condition gets the string "" instead of "1 2 3 4 5". Bash does not recognize the brace expansion because the upper bound of the range is not an integer. Then Bash writes the "" string to the `i` variable. Therefore, the following (( operator fails.

The `seq` utility can solve our problem. It generates a sequence of integers or fractions.

Table 3-21 shows the ways to call the `seq` utility.

**Table 3-21. The ways to call the `seq` utility**

| Number of parameters | Description | Example | Result |
|---|---|---|---|
| 1 | The parameter defines the last number in the generated sequence. The sequence starts with one. | `seq 5` | 1 2 3 4 5 |
| 2 | The parameters are the first and last numbers of the sequence. | `seq -3 3` | -2 -1 0 1 2 |

**Table 3-21. The ways to call the `seq` utility**

| Number of parameters | Description | Example | Result |
|---|---|---|---|
| 3 | The parameters are the first number, step and last numbers of the sequence. | `seq 1 2 5` | 1 3 5 |

The `seq` utility splits integers on the output by line breaks. The `-s` option allows you to specify another delimiter. The `IFS` variable contains the line break symbol. Therefore, you do not need the `-s` option in our case.

There are line breaks instead of spaces in the "Result" column of Table 3-21. This is done for convenience.

Let's apply the `seq` utility and write the script to calculate a factorial for any integer. Listing 3-31 shows this script.

**Listing 3-31. The script for calculating a factorial**

```
#!/bin/bash

result=1

for i in $(seq $1)
do
  ((result *= $i))
done

echo "The factorial of $1 is $result"
```

This solution works properly. However, it is ineffective. The performance overhead comes because of calling the external `seq` utility. It costs the same time as launching an application (for example, Windows Calculator). The OS kernel performs several complicated operations whenever Bash creates a new process. They take significant time on the processor's scale. Therefore, apply the built-in Bash commands whenever possible.

We need the second form of the `for` statement to solve the task effectively. This form looks like this in general:

```
for (( EXPRESSION_1; EXPRESSION_2; EXPRESSION_3 ))
do
  ACTION
done
```

You can write it in one line this way:

```
for (( EXPRESSION_1; EXPRESSION_2; EXPRESSION_3 )); do ACTION; done
```

Bash executes the `for` loop with an arithmetic condition this way:

1. Bash calculates the EXPRESSION_1 once before the first iteration of the loop.
2. The loop continues as long as EXPRESSION_2 remains true. The loop stops when it returns "false".
3. Bash calculates the EXPRESSION_3 at the end of each iteration.

Let's replace the `seq` utility call with the arithmetic expression in Listing 3-31. Listing 3-32 shows the result.

**Listing 3-32. The script for calculating a factorial**

```
#!/bin/bash

result=1

for (( i = 1; i <= $1; ++i ))
do
  ((result *= i))
done

echo "The factorial of $1 is $result"
```

This script works faster. It uses Bash built-in commands only. There is no need to create new processes here.

The `for` statement in the script follows this algorithm:

1. Declare the `i` variable before the first iteration of the loop. Assign it integer 1. The variable is a loop counter.
2. Compare the loop counter with the input parameter `$1`.
3. If the counter is smaller than the `$1` parameter, do the loop iteration.
4. If the counter is greater than the parameter, stop the loop.
5. Calculate the arithmetic expression "result *= i" in the loop's body. It multiplies the `result` variable by `i`.
6. When the loop iteration is done, calculate the "++i" expression of the `for` condition. It increments the `i` variable by one.
7. Go to the 2nd step of the algorithm.



In the general case, you do not need the dollar sign for variables names in the (( operator and `let` command. However, it is necessary for code in Listing 3-32. Without the sign, Bash confuses the `$1` parameter and literal 1.

We use the prefix increment form in the loop. It works faster than the postfix form.

Use the second form of the `for` whenever you should calculate the loop counter. There are no other effective solutions in this case.

## Controlling the Loop Execution

The loop stops according to its condition. There are additional ways to control the loop execution. They allow you to interrupt it or skip the current iteration. Let's consider them in detail.

### break

The `break` command stops the loop immediately. It is useful for handling an error or finishing an infinite loop.

Here is an example. We want to write the script that searches an array's element by value. When the script finds it, there is no reason to continue the loop. We can finish it immediately with the `break` command. Listing 3-33 shows how to do it.

**Listing 3-33. The script for searching an array's element**

```
#!/bin/bash

array=(Alice Bob Eve Mallory)
is_found="0"

for element in "${array[@]}"
do
  if [[ "$element" == "$1" ]]
  then
    is_found="1"
    break
  fi
done

if [[ "$is_found" -ne "0" ]]
then
  echo "The array contains the $1 element"
else
  echo "The array does not contain the $1 element"
fi
```

The script receives an element's value to search via the `$1` parameter.

The `is_found` stores the search result. The `if` statement checks the value of the current array's element. If it matches the `$1` parameter, we set the `is_found` variable to one. Then we interrupt the loop with the `break` command.

There is the `if` statement after the loop. It checks the `is_found` variable. Then the `echo` command prints if the searching has succeeded.

Use the `break` command to take as much as possible out of the loop body. When the loop body is short, it is easier to read and understand.

Here is an example. We can print the searching result right in the loop body in Listing 3-33. Then we do not need to store the result in the `is_found` variable. On the other hand, the processing of the found element can be complex. In this case, take it out of the loop body.

Sometimes it does not make sense to continue the script when interrupting the loop. Use the `exit` command instead of `break` in this case.

For example, it can be an error in the input data that we catch in the loop body. Then printing an error message and calling the `exit` command is the best choice to stop the script.

The `exit` command makes your code cleaner if you process the result in the loop body. Just call `exit` when you are done.

Let's replace the `break` command with `exit` in Listing 3-33. Listing 3-34 shows the result.

**Listing 3-34. The script for searching an array's element**

```
#!/bin/bash

array=(Alice Bob Eve Mallory)

for element in "${array[@]}"
do
  if [[ "$element" == "$1" ]]
  then
    echo "The array contains the $1 element"
    exit 0
  fi
done

echo "The array does not contain the $1 element"
```

Using the `exit` command, we handle the search result in the loop body. In this case, it has shortened our code and made it simpler. But you get the opposite effect if the result processing requires a block of commands.

The scripts in Listing 3-33 and Listing 3-34 give the same result.

## continue

The `continue` command skips the current loop iteration. The loop does not stop in this case. It starts the next iteration instead.

Here is an example. Suppose we want to calculate the sum of positive integers in an array. Thus, we should distinguish the signs of the integers. The `if` statement fits well for this task. If the integer's sign is positive, we add the integer to the result. Listing 3-35 shows the final script.

**Listing 3-35. The script for calculating the sum of positive integers in an array**

```
#!/bin/bash

array=(1 25 -5 4 -9 3)
sum=0

for element in "${array[@]}"
do
  if (( 0 < element ))
  then
    ((sum += element))
  fi
done

echo "The sum of the positive numbers is $sum"
```

If the `element` variable is greater than zero, we add it to the result `sum`.

Let's use the `continue` command to get the same behavior. Listing 3-36 shows the result.

**Listing 3-36. The script for calculating the sum of positive integers in an array**

```
#!/bin/bash

array=(1 25 -5 4 -9 3)
sum=0

for element in "${array[@]}"
do
  if (( element < 0))
  then
    continue
  fi

  ((sum += element))
done
```

```
15
16 echo "The sum of the positive numbers is $sum"
```

We have inverted the condition of the `if` statement. Now it is "true" for negative numbers. Bash calls the `continue` command in this case. The command interrupts the current loop iteration. It means that all further operations are ignored. Then the next iteration starts with the next array element.

We have applied the early return pattern in the context of the loop.

Use the `continue` command to handle errors. It is also helpful for conditions where it does not make sense to execute the loop body to the end. This way, you avoid the nested `if` statements. This solution makes your code cleaner.

**Exercise 3-12. Loop Constructs**

```
Write a game called "More or Fewer". The first participant chooses a number from
1 to 100.
The second participant tries to guess it in seven tries.

Your script chooses a number. The user enters his guess.
The script answers if the guess is more or less than the chosen number.
The user then tries to guess the number six more times.
```

# Functions

Bash is **procedural programming language**[346]. Procedural languages allow you to divide a program into logical parts called **subroutine**[347]. A subroutine is an independent block of code that solves a specific task. A program calls subroutines when it is necessary.

Subroutines are called **functions** in modern programming languages. We have already met functions when considering the `declare` command. Now let's study the structure and purposes of functions.

## Programming Paradigms

We should start with the terminology. It will help us to understand why functions have appeared and which tasks they solve.

What is procedural programming? It is one of **paradigms**[348] of software development. A paradigm is a set of ideas, methods and principles that define how to write programs.

There are two dominant paradigms today. Most modern programming languages follow one of them. The paradigms are the following:

[346] https://en.wikipedia.org/wiki/Procedural_programming
[347] https://en.wikipedia.org/wiki/Subroutine
[348] https://en.wikipedia.org/wiki/Programming_paradigm

1. **Imperative programming**[349]. The developer explicitly specifies to the computer how to change its state. In other words, he writes a complete algorithm for calculating the result.
2. **Declarative programming**[350]. The developer specifies the properties of the desired result but not the algorithm to calculate it.

Bash follows the first paradigm. It is an imperative language.

The imperative and declarative paradigms define general principles for writing a program. There are different methodologies (i.e. approaches) within the same paradigm. Each methodology offers specific programming techniques.

The imperative paradigm has two dominant methodologies:

1. Procedural programming.
2. **Object-oriented programming**[351].

These methodologies suggest structuring the source code of a program in different ways. Bash follows the first methodology.

Let's take a closer look at procedural programming. This methodology suggests features for combining the program's instruction sets into independent blocks of code. These blocks are called subroutines or functions.

You can call a function from any place of a program. The function can receive input parameters. This mechanism works similarly to passing command-line parameters to a script. This is a reason why a function is called "a program inside a program" sometimes.

The main task of the functions is to manage the complexity of a program. The larger the size of the source code, the harder it is to maintain. Repeating code fragments make this situation worse. They are scattered throughout the program and may contain errors. After fixing a mistake in one fragment, you have to find and fix all the rest. If you put the fragment into a function, it is enough to fix the error only there.

Here is an example of a repeating code fragment. Suppose that you are writing a large program. The program prints text messages to the error stream when handling errors. Then there will be many places in the source code where you call the `echo` command. These calls can look like this:

```
>&2 echo "The N error has happened"
```

At some point, you decide that it is better to write all errors in a log file. Then it will be easier to analyze them. Users of your program may redirect the error stream to the log file themselves. But let's assume that some of them do not know how to use redirection. Thus, the program must write messages into the log file by itself.

Let's change the program. You have to go through all places where it handles the errors. Then you should replace the `echo` calls there with the following one:

[349] https://en.wikipedia.org/wiki/Imperative_programming
[350] https://en.wikipedia.org/wiki/Declarative_programming
[351] https://en.wikipedia.org/wiki/Object-oriented_programming

```
echo "The N error has happened" >> debug.log
```

If you miss one `echo` call accidentally, its output does not come into the log file. But this specific output can be critical. Without it, you would not understand why the program fails for the user.

We have considered one of the difficulties of maintaining programs. It often occurs when you change the existing code. The root cause of the problem is a violation of the **don't repeat yourself**[352] or DRY development principle. The same error handling code was copied over and over again in different places of the program. You should not do that.

Functions solve the problem of code duplication. This solution somewhat resembles loops. The difference is that a loop executes a set of commands in one place of the program cyclically. In contrast to a loop, a function executes the same set of commands at different program places.

Using functions improves the readability of the program's code. It combines a set of commands into a single block. If you give the block a speaking name, the task it solves becomes obvious. You can call the function by its name. It makes the program easier to read. Instead of a dozen lines of the function's body, there will be just its name. It explains to the reader what is going on in the function.

## Using Functions in Shell

The functions are available in both Bash modes: the command interpreter and script execution. First, let's consider how they work in the command interpreter.

Here is the function's declaration in general:

```
1 FUNCTION_NAME()
2 {
3   ACTION
4 }
```

You can also declare the function in one line this way:

```
FUNCTION_NAME() { ACTION ; }
```

The semicolon before the closing curly bracket is mandatory here.

The ACTION is a single command or a block of commands. It is called the **function body**.

Function names follow the same restrictions as variable names in Bash. They allow Latin characters, numbers and the underscore character. The name must not begin with a number.

Let's look at how to declare and use functions in the shell. Suppose you need statistics about memory usage. This statistics are available via the special file system **proc**[353] or procfs. This file system provides the following information:

[352] https://en.wikipedia.org/wiki/Don't_repeat_yourself
[353] https://en.wikipedia.org/wiki/Procfs

- The list of running processes.
- The state of the OS.
- The state of the computer's hardware.

There are files in the `/proc` system path. You can read the required information from these files.

RAM usage statistics are available in the `/proc/meminfo` file. We can read it with the `cat` utility this way:

```
cat /proc/meminfo
```

The output of this command depends on your OS. The `/proc/meminfo` file contains less information for the MSYS2 environment and more for the Linux system.

Here is an example of the `meminfo` file contents for the MSYS2 environment:

```
MemTotal:        6811124 kB
MemFree:         3550692 kB
HighTotal:             0 kB
HighFree:              0 kB
LowTotal:        6811124 kB
LowFree:         3550692 kB
SwapTotal:       1769472 kB
SwapFree:        1636168 kB
```

Table 3-22 explains the meaning of each field of this file.

**Table 3-22. Fields of the `meminfo` file**

| Field | Description |
|---|---|
| MemTotal | The total amount of usable RAM in the system |
| MemFree | The amount of unused RAM at the moment. It is equal to sum of fields LowFree + HighFree. |
| HighTotal | The total amount of usable RAM in the high region (above 860 MB). |
| HighFree | The amount of unused RAM in the high region (above 860 MB). |
| LowTotal | The total amount of usable RAM in the non-high region. |
| LowFree | The amount of unused RAM in the non-high region. |
| SwapTotal | The total amount of physical **swap**[354] memory. |
| SwapFree | The amount of unused swap memory |

[354] https://en.wikipedia.org/wiki/Memory_paging#Unix_and_Unix-like_systems

This article[355] provides more details about fields of the `meminfo` file.

Typing the `cat` command for printing the `meminfo` file contents takes time. We can declare the function with a short name for that. Here is an example of this function:

```
mem() { cat /proc/meminfo; }
```

This is a one-line declaration of the `mem` function. Now you can call it the same way as any regular Bash command. Do it like this:

```
mem
```

The function prints statistics on memory usage.

The `unset` command removes the previously declared function. For example, the following call removes our `mem` function:

```
unset mem
```

Suppose a variable and a function are declared with the same name. Use the `-f` option to remove the function. Here is an example:

```
unset -f mem
```

You can add the function declaration to the `~/.bashrc` file. Then the function will be available every time you start the shell.

We declared the `mem` function in single-line format when using command-line. It is a convenient and fast way to type. But clarity is more important when declaring the function in the file `~/.bashrc`. Therefore, it is better to declare the `mem` function in a standard format there. Do it like this:

```
1 mem()
2 {
3   cat /proc/meminfo
4 }
```

## Difference Between Functions and Aliases

We have declared the `mem` function. It prints statistics on memory usage. The following alias does the same thing:

[355] https://www.thegeekdiary.com/understanding-proc-meminfo-file-analyzing-memory-utilization-in-linux/

```
alias mem="cat /proc/meminfo"
```

It looks like functions and aliases work the same way. What should you choose then?

Functions and aliases have one similar aspect only. They are built-in Bash mechanisms. From the user's point of view, they reduce the input of long commands. But these mechanisms work in completely different ways.

An alias replaces one text with another in a typed command. In other words, Bash finds text in the command that matches the alias name. Then it replaces that text with the alias value. Finally, Bash executes the resulting command.

Here is an example of an alias. Suppose you have declared an alias for the `cat` utility. It adds the `-n` option to the utility call. The option adds line numbers to the `cat` output. The alias declaration looks like this:

```
alias cat="cat -n"
```

Whenever you type a command that starts with the word "cat", Bash replaces it with the "cat -n". For example, you type this command:

```
cat ~/.bashrc
```

When Bash inserts an alias value, the command becomes like this:

```
cat -n ~/.bashrc
```

Bash has replaced the word "cat" with "cat -n". It did not change the parameter, i.e. the path to the file.



You can force Bash to insert an alias before executing the command. Type the command and press the Ctrl+Alt+E keystroke for doing that.

Now let's look at how functions work. Suppose that Bash meets the function name in the typed command. Unlike an alias, the shell does not replace the function's name with its body. Instead, Bash executes the function's body.

Here is an example to explain the difference. We want to write the function that behaves the same way as the `cat` alias. If Bash functions work as aliases, the following declaration solves our task:

```
cat() { cat -n; }
```

We expect that Bash will add the `-n` option to the following command:

```
cat ~/.bashrc
```

However, it would not work. Bash does not insert the body of the function in the command. The shell executes the body and inserts the result into the command.

In our example, Bash calls the `cat` utility with the `-n` option but without the `~/.bashrc` parameter. We do not want such behavior.

We can solve the problem by passing the filename to the function as a parameter. This works just like passing a parameter to a command or script. You can call the function and specify its parameters, separated by spaces.

Calling a function and passing parameters to it looks like this in general:

```
FUNCTION_NAME PARAMETER_1 PARAMETER_2 PARAMETER_3`
```

You can read parameters in the function's body via names `$1`, `$2`, `$3`, etc. The `$@` array stores all received parameters.

Let's correct the declaration of the `cat` function. We will pass all parameters of the function to the `cat` utility input. Then the declaration becomes like this:

```
cat() { cat -n $@; }
```

This function would not work either. The problem happens because of unintentional **recursion**. When the function calls itself, it is called recursion.

Bash checks the list of declared functions before executing the command "cat -n $@". There is the `cat` function in the list. Bash executes its body at the moment, but it does not change anything. Thus, the shell calls the `cat` function again instead of calling the `cat` utility. The call repeats over and over again. It leads to the infinite recursion, which is similar to an infinite loop.

Recursion is not a mistake of Bash's behavior. It is a powerful mechanism that simplifies complex algorithms. The example of such algorithms is the traversing a **graph**[356] or **tree**[357].

The mistake occurred in our declaration of the `cat` function. The recursive call happened by accident and led to a loop. There are two ways to solve this problem:

1. Use the `command` built-in.
2. Rename the function so that its name differs from the utility name.

Let's look at the first solution. The `command` built-in receives a command as parameters. If there are aliases and function names there, Bash ignores them. It does not insert the alias value instead of its name. It does not call a function. Instead, Bash executes the command as it is.

If we apply the `command` built-in in the `cat` function, we get the following result:

[356] https://en.wikipedia.org/wiki/Graph_(abstract_data_type)
[357] https://en.wikipedia.org/wiki/Tree_(data_structure)

```
cat() { command cat -n "$@"; }
```

Now Bash calls the `cat` utility instead of the `cat` function.

The second solution is renaming the function. For example, this version works well:

```
cat_func() { cat -n "$@"; }
```

Always be aware of the problem of unintentional recursion. Keep the names of the functions unique. They should not match the names of built-in Bash commands and GNU utilities.

Here is a summary of our comparison of functions and aliases. If you want to shorten a long command, use an alias.

You need a function in the following cases only:

1. You need a conditional statement, loop, or code block to perform a command.
2. The parameters are not at the end of the command.

The second case needs an example. Let's shorten the `find` utility call. It should search for files in the specified directory. When you search in the home directory, the call looks like this:

```
find ~ -type f
```

We cannot write an alias that takes the target path as a parameter. The following solution does not work:

```
alias="find -type f"
```

The target path should come before the `-type` option. This is a problem for the alias.

However, we can use the function in this case. We can choose the position to insert the parameter in the `find` call. The function declaration looks like this:

```
find_func() { find $1 -type f; }
```

Now we can call it for searching files in the home directory this way:

```
find_func ~
```

## Using Functions in Scripts

The declaration of functions in scripts is the same as in a shell. There are two options: the standard and one-line declarations.

For example, let's come back to the task of error handling in a large program. We can declare the following function for printing error messages:

```
print_error()
{
  >&2 echo "The error has happened: $@"
}
```

This function receives parameters. They should explain the root cause of the error. Suppose our program reads a file on the disk. The file becomes unavailable for some reason. Then the following `print_error` function call reports the problem:

```
print_error "the readme.txt file was not found"
```

Suppose that the requirements for the program have changed. Now the program should print error messages to a log file. It is enough to change only the declaration of the `print_error` function to meet the new requirement. The function's body looks like this after the change:

```
print_error()
{
  echo "The error has happened: $@" >> debug.log
}
```

This function prints all error messages to the `debug.log` file. There is no need to change places of the program where the function is called.

Sometimes you want to call one function from another. This is called **nested function call**. Bash allows it. In general, you can call a function from any point of the program.

Here is an example of nested function calls. Suppose you want to translate the program interface to another language. This task is called **localization**[358]. It is better to print error messages in a language the user understands. This requires duplicating all messages in all languages supported by the program. How to do this?

The simplest solution is to assign a unique code to each error. Using error codes is a common practice in system programming. Let's apply this approach to our program. Then the `print_error` function will receive error codes via the parameters.

We can write error codes to the log file as it is. But then the user will need information about the meaning of that codes. Therefore, it is more convenient to print the message text as we did it before. To do this, we should convert error codes to the text in a specific language. We need a separate function for doing this conversion. Here is an example of such a function:

[358] https://en.wikipedia.org/wiki/Internationalization_and_localization

```
code_to_error()
{
  case $1 in
    1)
      echo "File not found:"
      ;;
    2)
      echo "Permission to read the file denied:"
      ;;
  esac
}
```

Let's apply the `code_to_error` function when printing an error in `print_error`. We get the following result:

```
print_error()
{
  echo "$(code_to_error $1) $2" >> debug.log
}
```

Here is an example of the `print_error` function call from the program's code:

```
print_error 1 "readme.txt"
```

It prints the following line into the log file:

```
File not found: readme.txt
```

The first parameter of the `print_error` function is the error code. The second parameter is the name of the file that caused the error.

Using functions made the error handling in our program easier to maintain. Changing the requirements can demonstrate it. Suppose that our customer asked us to support German language. We can do it by declaring two following functions:

- `code_to_error_en` for messages in English.
- `code_to_error_de` for messages in German.

How can you choose the right function to convert error codes? The `LANGUAGE` Bash variable helps you in this case. It stores the current language of the user's system. Check this variable in the `print_error` function and convert error codes accordingly.



If the `LANGUAGE` variable is not available on your system, use the `LANG` variable instead.

Our solution with the error codes conversion is just an example for demonstration. Never does it in the real project. Bash has a special mechanism to localize scripts. It uses PO files with texts in different languages. Read more about this mechanism in the BashFAQ article[359].

**Exercise 3-13. Functions**

```
Write the following functions to display error messages in English and German:

* print_error
* code_to_error_en
* code_to_error_de

Write two versions of the "code_to_error" function:

* Using the case statement.
* Using an associative array.
```

## Returning a Function's Result

Procedural languages have a reserved word for returning the function's result. It is called `return` in most cases. Bash also has a command with this name. But it has another purpose. The `return` command in Bash does not return a value. Instead, it passes a function's exit status, which is an integer between 0 and 255.

The complete algorithm for calling and executing the function looks like this:

1. Bash meets the function name in the command.
2. The interpreter goes to the body of the function and executes it from the first command.
3. If Bash meets the `return` command in the function body, it stops executing it. The interpreter jumps to the place where the function is called. The special parameter `$?` stores the exit status of the function.
4. If there is no `return` command in the function's body, Bash executes it until the last command. Then, the interpreter jumps to the place where the function is called.

In procedural languages, the `return` command returns the variable of any type from a function. It can be a number, a string or an array. You need other mechanisms for doing that in Bash. There are three options:

1. Command Substitution.

[359] https://mywiki.wooledge.org/BashFAQ/098

2. A global variable.
3. The caller specifies a global variable.

Let's look at examples of using these approaches.

We wrote the functions `code_to_error` and `print_error` to print error messages in the last section. They look like this:

```
code_to_error()
{
  case $1 in
    1)
      echo "File not found:"
      ;;
    2)
      echo "Permission to read the file denied:"
      ;;
  esac
}

print_error()
{
  echo "$(code_to_error $1) $2" >> debug.log
}
```

Here we have used the first approach for returning the function's result. We put the `code_to_error` call into the command substitution statement. Thus, Bash inserts whatever the function prints to the console into the place of the call.

The `code_to_error` function prints the error message via the `echo` command in our example. Then Bash inserts this output into the body of the `print_error` function. There is only one `echo` call there. It consists of two parts:

1. Output of the `code_to_error` function. This is the error message.
2. The input parameter `$2` of the `print_error` function. This is the name of the file, which we tried to access but got an error.

The `echo` command of the `print_error` function accumulates all data and prints the final error message to the log file.

The second way to return a value from a function is to write it to a global variable. This kind of variable is available anywhere in the script. Thus, you can access it in the function's body and the place where it is called.



All variables declared in the script are global by default. There is one exception to this rule. We will discuss it later.

Let's apply the global variable approach to our case. We should rewrite the `code_to_error` and `print_error` functions. The first one will store its result in a global variable. Then `print_error` reads it. The resulting code looks like this:

```
code_to_error()
{
  case $1 in
    1)
      error_text="File not found:"
      ;;
    2)
      error_text="Permission to read the file denied:"
      ;;
  esac
}

print_error()
{
  code_to_error $1
  echo "$error_text $2" >> debug.log
}
```

The `code_to_error` function writes its result to the `error_text` global variable. Then the `print_error` function combines this variable with the `$2` parameter to make a final error message and print it to the log file.

Returning the function result via a global variable is the error-prone solution. It may cause a naming conflict. For example, assume that there is another variable called `error_text` in the script. It has nothing to do with the output to the log file. Then any `code_to_error` call will overwrite the value of that variable. This will cause errors in all places where `error_text` is used outside the `code_to_error` and `print_error` functions.

**Variable naming convention** can solve the problem of naming conflict. The convention is an agreement of how to name the variables in all parts of the project's code. This agreement is one of the clauses of **code style**[360] guide. Any large program project must have such a guide.

Here is an example of a variable naming convention:

> All global variables, which functions use to return values, should have an underscore sign prefix in their names.

[360] https://en.wikipedia.org/wiki/Programming_style

Let's follow this convention. Then we should rename the `error_text` variable to `_error_text`. We solved the problem in our specific case. But there are cases when the issue still can happen. Suppose one function calls another, i.e. there is a nested call. What happens if both functions use the variable called the same to return their values? We get the naming conflict again.

The third way to return a function's result solves the name conflict problem. The idea is to let the caller code a way to specify the global variable's name. Then the called function writes its result to that variable.

How to pass a variable name to the called function? We can do it via the function's input parameter. Next, the function calls the `eval` command. This command converts the specified text into a Bash command. It is required because we pass the variable's name as a text. Bash does not allow referring to the variable using text. So, `eval` resolves this obstacle.

Let's change the `code_to_error` function. We will pass two following parameters there:

1. The error code in the `$1` parameter.
2. The name of the global variable to store the result. We use the `$2` parameter for that.

This way, we get the following code:

```
code_to_error()
{
  local _result_variable=$2

  case $1 in
    1)
      eval $_result_variable="'File not found:'"
      ;;
    2)
      eval $_result_variable="'Permission to read the file denied:'"
      ;;
  esac
}

print_error()
{
  code_to_error $1 "error_text"
  echo "$error_text $2" >> debug.log
}
```

At first glance, the code is almost the same as it was before. But it is not. Now it behaves more flexibly. The `print_error` function chooses the global variable's name to get the `code_to_error` result. The caller code explicitly specifies this name. Therefore, it is easier to find and resolve naming conflicts.

## Variable Scope

Naming conflicts is a serious problem. It occurs in Bash when functions declare variables in the global scope. As a result, the names of two variables can match. Then two functions will access them at different moments. This leads to confusion and data loss.

Procedural languages suggest the solution for the naming conflicts problem. They provide features to restrict the scope of declared variables. Bash has these features too.

If you declare a variable with the `local` keyword in a function, its body limits the variable's scope. In other words, the variable becomes available inside the function only.

Here is our latest version of the `code_to_error` function:

```
code_to_error()
{
  local _result_variable=$2

  case $1 in
    1)
      eval $_result_variable="'File not found:'"
      ;;
    2)
      eval $_result_variable="'Permission to read the file denied:'"
      ;;
  esac
}
```

We have declared the `_result_variable` variable using the `local` keyword. Therefore, it becomes a local variable. You can read and write its value in `code_to_error` and any other functions that it calls.

Bash limits a local variable's scope by the execution time of the function where it is declared. Such a scope is called **dynamic**[361]. Modern languages prefer to use **lexical** scope. There the variable is available in the function's body only. If you have nested calls, the variable is not available in the called functions.

Local variables do not come into the global scope. This ensures that no function will overwrite them by accident.

**Exercise 3-14. Variable scope**

What text will the script in Listing 3-37 print to the console when it executes?

[361] https://en.wikipedia.org/wiki/Scope_(computer_science)#Lexical_scope_vs._dynamic_scope_2

**Listing 3-37. The script for testing the variable scope**

```
#!/bin/bash

bar()
{
    echo "bar1: var = $var"
    var="bar_value"
    echo "bar2: var = $var"
}

foo()
{
    local var="foo_value"

    echo "foo1: var = $var"
    bar
    echo "foo2: var = $var"
}

echo "main1: var = $var"
foo
echo "main2: var = $var"
```

Careless handling of local variables leads to errors. They happen because a local variable hides a global variable with the same name. Let's look at an example.

Suppose you write a function to handle a file. For example, it calls the `grep` utility to look for a pattern in the file. The function looks like this:

```
check_license()
{
  local filename="$1"
  grep "General Public License" "$filename"
}
```

Now suppose that you have declared the global variable named `filename` at the beginning of the script. Its declaration looks like this:

```
#!/bin/bash

filename="$1"
```

Will the `check_license` function run correctly? Yes. It happens thanks to **hiding a global variable**. This mechanism works in the following way. When Bash meets the `filename` variable in the function's body, it accesses the local variable instead of the global one. It happens because the local variable is declared later than the global one. Because of the hiding mechanism in the function's body, you cannot access the global variable `filename` there.

Accidental hiding of global variables leads to errors. Try to avoid any possibility of getting such a situation. To do this, add a prefix or postfix for local variable names. For example, it can be an underscore at the end of the name.

A global variable becomes unavailable in a function's body only after declaring the local variable with the same name. Consider the following variant of the `check_license` function:

```
#!/bin/bash

filename="$1"

check_license()
{
  local filename="$filename"
  grep "General Public License" "$filename"
}
```

Here we initialize the local variable `filename` by the value of a global variable with the same name. This assignment operation works as expected. It happens because Bash does parameter expansion before executing the assignment.

Suppose that you pass the `README` filename to the script. Then the assignment looks like this after parameter expansion:

```
  local filename="README"
```

Developers changed the default scopes of arrays in the 4.2 Bash version. If you declare an indexed or associative array in a function' body, it comes to the local scope. You should use the `-g` option of the `declare` command to make an array global.

Here is the declaration of the local array `files`:

```
check_license()
{
  declare files=(Documents/*.txt)
  grep "General Public License" "$files"
}
```

You should change the declaration this way to declare the global array:

```
check_license()
{
  declare -g files=(Documents/*.txt)
  grep "General Public License" "$files"
}
```

We have considered the functions in Bash. Here are general recommendations on how to use them:

1. Choose names for functions carefully. Each name should tell the reader of your code what the function does.
2. Declare only local variables inside functions. Use the naming convention for them. This solves potential conflicts of local and global variable names.
3. Do not use global variables in functions. Instead, pass the value of the global variable to the function via a parameter.
4. Do not use the `function` keyword when declaring functions. It is present in Bash but not in the POSIX standard.

Let's take a closer look at the last tip. Do not declare functions this way:

```
function check_license()
{
  declare files=(Documents/*.txt)
  grep "General Public License" "$files"
}
```

The `function` keyword is useful in only one case. It resolves the conflict between the function name and the alias.

For example, the following function declaration does not work without the `function` keyword:

```
alias check_license="grep 'General Public License'"

function check_license()
{
  declare files=(Documents/*.txt)
  grep "General Public License" "$files"
}
```

If you have such a declaration, you can call the function only by adding a slash before its name. Here is an example of this call:

```
\check_license
```

If you skip the slash, Bush inserts the alias value instead of calling the function. It means this command brings the alias:

```
check_license
```

There is a low probability that you get alias and function names conflict in the script. Each script runs in a separate Bash process. This process does not load aliases from the `.bashrc` file. Therefore, name conflicts can happen by mistake in the shell mode only.

# Package Manager

We are already familiar with the basic Bash built-in commands and the standard set of GNU utilities. These tools are installed in the Unix environment by default. It can happen that their features are not enough to solve your task. You can solve this problem by installing additional programs and utilities.

Installing software in the Unix environment is not the same as in Windows. Let's look at how to properly install and update software in any Unix environment or Linux distribution.

## Repository

Whenever you install software in the Unix environment, you should use **repository**[362]. A repository is a server that stores all available applications. **Maintainers** take open sources of these applications and build them. Most maintainers are volunteers and free software enthusiasts.

The repository stores each application as a separate file. This file has a special format. The format depends on the Linux distribution. Thus, each Linux distribution has its own software repository. Examples of the formats are deb, RPM, zst, etc. A file with an application is called a **package**. A package is a unit for installing software in a system.

The repository stores packages with applications and libraries. Besides, the repository has meta-information about all packages. One or more files store this meta-information. They are called **the package index**.

You can install packages in a Unix environment from several repositories at once. For example, one repository provides new versions of packages, and another offers special builds of them. Depending on your requirements, you can choose the repository for installing the package.

## Package Operating

The Unix environment provides a special program to work with the repository. It is called **package manager**[363].

Why does the Unix environment need a package manager? For example, Windows does not have one. Users of this OS download programs from the Internet and install them manually.

The package manager installs and removes packages in the Unix environment. Its main task is to keep track of **package dependencies**. Suppose a program from one package uses features of an application or library from another package. Then the first package depends on the second one.

[362] https://help.ubuntu.com/community/Repositories
[363] https://en.wikipedia.org/wiki/Package_manager

Package dependency prevents the same application or library from being installed multiple times on the system. Instead, the packages you need are installed once. All dependent programs know where to install them on disk and share them.

Package dependency prevents duplicating of applications and libraries in your system. It is enough to install all required packages once. Then all dependent programs know where to access files from these packages.

Install applications in a Unix environment or Linux system using the package manager only. This rule has one exception. If you need a proprietary program, you have to install it manually. Usually, such a program is distributed in a single package. It includes all dependencies (necessary libraries and applications). There is no need to track dependencies in this case. Therefore, you can install the program without the package manager.

Here is the algorithm to install a package from the repository:

1. Download a package index from the repository.
2. Find the required application or library in the package index.
3. Download the package with the application or library from the repository.
4. Install the downloaded package.

The package manager does all these steps. You need to know its interface and call it with the right parameters.

The MSYS2 environment uses the package manager `pacman`[364]. It is designed for the Arch Linux distribution. The `pacman` manager operates packages, which have a simple format. You do not need any special skills or experience to build applications and libraries into these packages.

Let's take the `pacman` manager as an example and look at the commands for working with the repository.

The following command downloads a package index from the repository:

```
pacman -Syy
```

This command finds the package by the keyword in the loaded index:

```
pacman -Ss KEYWORD
```

Suppose you are looking for a utility for accessing MS Word documents. Then the following command finds the right package for that:

[364] https://wiki.archlinux.org/index.php/Pacman

```
pacman -Ss word
```

The list of results will contain two packages:

- mingw-w64-i686-antiword
- mingw-w64-x86_64-antiword

These are builds of the `antiword` utility for 32-bit and 64-bit systems. The utility converts MS Word documents to text format.

Run the command to install the package:

```
pacman -S PACKAGE_NAME
```

This command installs the `antiword` utility:

```
pacman -S mingw-w64-x86_64-antiword
```

As a result, `pacman` will install `antiword` and all the packages that it needs for running.

Now you can launch the `antiword` utility by the following command:

```
antiword
```

You have installed a package in the system. If it becomes unnecessary, uninstall it. The package manager will uninstall all dependencies of the package if other applications do not use them. Here is the command to uninstall a package:

```
pacman -Rs PACKAGE_NAME
```

This command uninstalls the `antiword` utility:

```
pacman -Rs mingw-w64-x86_64-antiword
```

Suppose you have installed several packages in your system. After a while, their new versions appear in the repository. You decide to upgrade your packages to the new versions. The following command does it:

```
pacman -Syu
```

The command updates all installed packages to their actual versions in the repository.

We have considered the basic `pacman` commands. Other package managers work along the same lines. They follow the same steps as `pacman` to install and remove packages. The only differences are their name and their command-line parameters.

Table 4-1 shows the commands for working with packages in different Linux distributions.

**Table 4-1. The commands for working with packages**

| Command | MSYS2 and Arch Linux | Ubuntu | CentOS | Fedora |
|---|---|---|---|---|
| Download a package index | `pacman -Syy` | `apt-get update` | `yum check-update` | `dnf check-update` |
| Search for a package by the keyword | `pacman -Ss KEYWORD` | `apt-cache search KEYWORD` | `yum search KEYWORD` | `dnf search KEYWORD` |
| Install the package from the repository | `pacman -S PACKAGE_NAME` | `apt-get install PACKAGE_NAME` | `yum install PACKAGE_NAME` | `dnf install PACKAGE_NAME` |
| Install the package from the local file | `pacman -U FILENAME` | `dpkg -i FILENAME` | `yum install FILENAME` | `dnf install FILENAME` |
| Remove the installed package | `pacman -Rs PACKAGE_NAME` | `apt-get remove PACKAGE_NAME` | `yum remove PACKAGE_NAME` | `dnf erase PACKAGE_NAME` |
| Update all installed packages | `pacman -Syu` | `apt-get upgrade` | `yum update` | `dnf upgrade` |

# Finalwords

Here we finish our introduction to Bash. We have covered the basics of the language only. This book leaves many topics out of the scope. They are required for advanced Bash development. Most probably, you would not need them for daily work with a computer.

These advanced topics can be interesting for you if you want to go deeper in Bash:

- Manipulating Strings[365].
- Regular Expressions[366].
- The `sed` stream editor[367].
- The `awk` text processing language[368].

These topics are important. They are material for advanced study. You can skip them if you are using Bash for simple tasks and basic automation. Learn these topics if you are going to write complex applications in Bash.

Perhaps you enjoyed programming. You found it useful. What to do next after reading this book?

First of all, let's admit that Bash is not a general-purpose programming language. This term refers to languages for developing applications in various **domains**[369]. Such languages do not contain constructs that are suitable for one domain only and useless in another.

Bash is the **domain-specific language**[370]. Does that mean it is useless? No, it does not. Bash is a powerful auxiliary tool for each software developer. Today it is used for integrating large projects, testing, building software and automating routine tasks. You would not find a commercial project written in Bash only. This language copes well with the tasks for which it was created. But it is inferior to modern general-purpose languages in many application domains.

Nobody creates a programming language just for fun. Its author faced some applied tasks. The languages existing at that time were not suitable to solve them. Then this developer wrote the new language that is focused on specific tasks. Therefore, modern general-purpose languages have advantages in a few domains only. No language fits any type of task perfectly. This leads that the applied domain dictates you the language for software development.

So, you have read this book. Now it is time to choose an applied domain that interests you. Read articles on the Internet. Think about it: which area of software development is right for you? Only after answering this question, choose a programming language for further study.

[365] https://tldp.org/LDP/abs/html/string-manipulation.html
[366] https://tldp.org/LDP/abs/html/x17129.html
[367] https://tldp.org/LDP/abs/html/x23170.html
[368] https://tldp.org/LDP/abs/html/awk.html
[369] https://en.wikipedia.org/wiki/Domain_(software_engineering)
[370] https://en.wikipedia.org/wiki/Domain-specific_language

Table 5-1 is your starting point for getting to know the applied domains. It also lists the languages used in them.

**Table 5-1. Applied domains of software development**

| Domain | Programming language |
|---|---|
| Mobile applications[371] | Java, C, C++, HTML5, JavaScript |
| Web applications[372] (front end[373]) | JavaScript, PHP, HTML5, CSS, SQL |
| Web applications[374] (back end[375]) | JavaScript, PHP, Ryby, Perl, C#, Java, Go |
| High Load Systems[376] | C++, Python, Ruby, SQL |
| System administration[377] | Bash, Python, Perl, Ruby |
| Embedded systems[378] | C, C++, Ассемблер |
| Machine learning [379] and data analysis[380] | Python, Java, C++ |
| Information security[381] | C, C++, Python, Bash |
| Enterprise software[382] | Java, C#, C++, SQL |
| Video games[383] | C++ |

It is not enough to know a programming language to become a high-class developer. It is necessary to know the technologies that are used in the specific applied domain. For example, an information security expert must understand the architecture of computer networks and operating systems. As you grow professionally, you will come to understand what technologies you need to learn.

Suppose you have chosen your applied domain and programming language. Now it is time to enroll in a free online course. This book has introduced you to the basics of programming. So, learning a new language will go faster. The statements of Python or C++ will seem familiar to you from Bash. However, these languages have concepts that you will have to learn from scratch. Do not lose your motivation. Apply the new knowledge into practice and learn from your mistakes. This is the only way to get results.

[371] https://en.wikipedia.org/wiki/Mobile_app
[372] https://en.wikipedia.org/wiki/Web_application
[373] https://en.wikipedia.org/wiki/Front_end_and_back_end
[374] https://en.wikipedia.org/wiki/Web_application
[375] https://en.wikipedia.org/wiki/Front_end_and_back_end
[376] https://en.wikipedia.org/wiki/Server_(computing)
[377] https://en.wikipedia.org/wiki/System_administrator
[378] https://en.wikipedia.org/wiki/Embedded_system
[379] https://en.wikipedia.org/wiki/Machine_learning
[380] https://en.wikipedia.org/wiki/Data_analysis
[381] https://en.wikipedia.org/wiki/Information_security
[382] https://en.wikipedia.org/wiki/Enterprise_software
[383] https://en.wikipedia.org/wiki/Video_game

I hope you learned something new from this book and had an enjoyable time reading it. If you like the book, please share it with your friends. I would also appreciate it if you would take a few minutes to write a review on Goodreads[384].

If you have any questions or comments about the book, write me to petrsum@gmail.com[385]. Also, ask questions in the "Issues" section of the GitHub repository of the book[386].

Thank you for reading "Bash programming from scratch"!

[384] https://www.goodreads.com/book/show/57301128-bash-programming-from-scratch
[385] mailto:petrsum@gmail.com
[386] https://github.com/ellysh/bash-programming-from-scratch/issues

# Acknowledgements

Nobody writes a book alone. Several people helped me write this book. Some of them suggested me a general idea. Others gave me comments and advices. These are the people I want to thank.

Thanks to Sophia Kayunova for the wish to learn how to program. It led me to the idea of writing a programming guide for my friends.

Thanks to Vitaly Lipatov for introducing me to Linux and Bash. He laid the foundation of my professional skills.

Thanks to Ruslan Piasetskyi for consulting on Bash. He explained to me the idioms and pitfalls of the language.

Thanks also to everyone who supported me and motivated me to finish this work.

# Glossary

## A

**Abstraction**[387] is a software module, application, or library that replicates the basic properties of a real object. Abstractions help manage the complexity of software systems. They hide irrelevant details. Abstractions allow the same algorithm to handle different objects.

**Algorithm**[388] is a finite sequence of instructions understandable to the executor. The goal of an algorithm is to calculate something or solve a task.

**Alias**[389] is a built-in Bash command for shortening long strings. It is used when the interpreter works in shell mode.

**Application programming interface**[390] (API) is a set of agreements on interaction components of the information system. The agreements answer the following questions:

- What function the called component performs?
- What data does the function need on the input?
- What data does the function return on the output?

**Argument**[391] is a word or string that the program receives via the command-line interface. Here is an example of arguments for the `grep` utility:

```
grep "GNU" README.txt
```

**Arithmetic expansion**[392] calculates an arithmetic expression and substitutes its result into the Bash command or statement. Here is an example of the expansion:

```
echo $((4+3))
```

**Array**[393] is a data structure consisting of a set of elements. The sequence number or index determines the position of each element. The array elements are placed one by one in computer memory.

**ASCII**[394] is an eight-bit character encoding standard. It includes the following characters:

[387] https://en.wikipedia.org/wiki/Abstraction_(computer_science)
[388] https://en.wikipedia.org/wiki/Algorithm
[389] https://www.gnu.org/software/bash/manual/html_node/Aliases.html#Aliases
[390] https://en.wikipedia.org/wiki/Application_programming_interface
[391] http://linuxcommand.org/lc3_wss0120.php
[392] https://www.gnu.org/software/bash/manual/html_node/Arithmetic-Expansion.html#Arithmetic-Expansion
[393] https://en.wikipedia.org/wiki/Array_data_structure
[394] https://en.wikipedia.org/wiki/ASCII

- decimal digits
- Latin alphabet
- national alphabet
- punctuation marks
- control characters

**Asynchrony**[395] means events that occur independently of the main program flow. Asynchrony also refers to methods for processing such events.

# B

**Background**[396] is a process execution mode in Bash. When activated, the process identifier does not belong to the identifier group[397] of the terminal. Also, the executed process does not handle keyboard interrupts.

**Bash**[398] (Bourne again shell) is a command-line interpreter developed by Brian Fox. Bash has replaced the Bourne shell in Linux distributions and some proprietary Unix systems. Bash is compatible with the POSIX standard. However, the standard does not include some of the interpreter's features.

**Bash script**[399] is a text file containing interpreter commands. Bash executes scripts in non-interactive mode.

**Best practices**[400] are recommended approaches for using a programming language or technology. An example of best practice for Bash is enclosing strings in double-quotes to avoid word splitting.

**Bottleneck**[401] is a component or resource of a computer system that limits its performance or throughput.

**Boolean expression**[402] is a programming language construct. When calculated, It results in either the "true" or "false" value.

**Bourne shell**[403] is a command-line interpreter developed by Stephen Bourne. It replaced the original Ken Thompson's interpreter[404] in Unix version 7[405]. The POSIX standard[406] contains all features of the Bourne shell. But the shell misses some features from the standard.

[395] https://en.wikipedia.org/wiki/Asynchrony_(computer_programming)
[396] https://www.gnu.org/software/bash/manual/html_node/Job-Control-Basics.html#Job-Control-Basics
[397] https://en.wikipedia.org/wiki/Process_group
[398] https://en.wikipedia.org/wiki/Bash_(Unix_shell)
[399] https://www.gnu.org/software/bash/manual/html_node/Shell-Scripts.html#Shell-Scripts
[400] http://mywiki.wooledge.org/BashGuide/Practices
[401] https://en.wikipedia.org/wiki/Bottleneck_(software)
[402] https://en.wikipedia.org/wiki/Boolean_expression
[403] https://en.wikipedia.org/wiki/Bourne_shell
[404] https://en.wikipedia.org/wiki/Thompson_shell
[405] https://en.wikipedia.org/wiki/Version_7_Unix
[406] https://pubs.opengroup.org/onlinepubs/9699919799/utilities/V3_chap02.html

**Brace expansion**[407] is a Bash mechanism for generating words from given parts. The POSIX standard misses this mechanism.
Here is an example of the brace expansion:

```
cp test.{txt,md,log} Documents
```

It generates the following command:

```
cp test.txt test.md test.log Documents
```

**Builtin commands**[408] are the commands that the interpreter executes by itself. It does not call utilities or programs for that. An example is the `pwd` built-in command.

# C

**Child process**[409] a process spawned by another process called the parent.

**Code style**[410] is a set of rules and conventions for writing the source code for programs. The rules help several programmers write, read, and understand common source code.

**Command** is the text entered after the command prompt. This text matches the action that the interpreter performs on its own or with another application's help.

**Command-line parameter**[411] is a type of the command's argument. It passes information to the program. The parameter can also be a part of some option. For example, it specifies the selected mode of the program's operation.
Here is the `find` utility call:

```
find ~/Documents -name README
```

The first parameter `~/Documents` specifies the path to start the search. The second parameter `README` refers to the option `-name`. It specifies the file or directory name for searching.

**Command prompt**[412] is a sequence of characters. The shell prints a prompt when it is ready to process the user's command.

**Command substitution**[413] is the Bash mechanism that replaces a command's call to its output. The subshell executes the command call. Here is an example:

[407] http://mywiki.wooledge.org/BraceExpansion
[408] https://www.gnu.org/software/bash/manual/html_node/Shell-Builtin-Commands.html
[409] https://en.wikipedia.org/wiki/Child_process
[410] https://en.wikipedia.org/wiki/Programming_style
[411] https://stackoverflow.com/a/36495940/6562278
[412] https://www.gnu.org/software/bash/manual/html_node/Controlling-the-Prompt.html#Controlling-the-Prompt
[413] https://www.gnu.org/software/bash/manual/html_node/Command-Substitution.html#Command-Substitution

```
echo "$(date)"
```

**Compiler**[414] is a program for translating source code from a programming language to the machine code.

**Computer program**[415] is a set of instructions that a computer can execute. Each program solves an applied task.

**Conditional statement**[416] is a construct of a programming language. It executes a block of commands depending on the result of the Boolean expression.

**Control character**[417] is a separate character in the control sequence.

**Control flow**[418] is the order in which the program executes its instructions and functions.

# D

# E

**Endianness**[419] is the byte order the computer uses to store numbers in memory. The CPU defines the supported endianness. There are two commonly used options today: big-endian and little-endian. Some CPUs support both (bi-endian). Here is an example of storing a four-byte number 0x0A0B0C0D for different orders:

```
0A 0B 0C 0D     big-endian
0D 0C 0B 0A     little-endian
```

**Environment variables**[420] is an unordered set of variables that the child process copies from the parent one. The `env` utility changes environment variables at the program startup. If you call the utility without parameters, it prints all variables declared in the current shell.

**Error-prone**[421] (error-prone) is a definition of failed programming techniques and solutions. These solutions work correctly in particular cases but cause errors with certain input data or conditions. An example of an error-prone solution is handling the `ls` utility output in the pipeline:

```
ls | grep "test"
```

[414] https://en.wikipedia.org/wiki/Compiler
[415] https://en.wikipedia.org/wiki/Computer_program
[416] https://en.wikipedia.org/wiki/Conditional_(computer_programming)
[417] https://en.wikipedia.org/wiki/Control_character
[418] https://en.wikipedia.org/wiki/Control_flow
[419] https://en.wikipedia.org/wiki/Endianness
[420] http://mywiki.wooledge.org/Environment
[421] https://en.wiktionary.org/wiki/error-prone

**Escape sequence**[422] is a set of characters that have no meaning of their own. Instead, they control the output device. For example, the line break character `\n` commands the output device to start a new line.

**Exit status**[423] is an integer value from 0 to 255 that the shell command returns when finishing. The zero status means successful execution of the command. All other codes indicate an error.

# F

**File descriptor**[424] is an abstract pointer to a file or communication channel (stream, pipeline or network socket). Descriptors are part of the POSIX interface. They are non-negative integers.

**Filename expansion**[425] is a Bash mechanism that replaces patterns to filenames. The patterns can contain the following wildcards: ?, *, [. Here is an example:

```
rm -rf *
```

**Filename extension**[426] is a suffix of the filename. The extension defines the type of file.

**File system**[427] is a set of rules to store and read data from storage media.

**Foreground**[428] is a default process execution mode in Bash. When used, the process identifier belongs to the identifier group[429] of the terminal. The executed process handles keyboard interrupts.

**Function** is another name for a subroutine.

# G

**General-purpose programming language**[430] is a language that you can use to develop applications for various applied domains. It does not contain constructs that are useful in one domain and useless in others.

**Globbing**[431] or glob is another name for filename expansion in Bash.

**Glob pattern**[432] is a search query. It includes regular and wildcard characters[433] (* and ?). The wildcards correspond to any characters. For example, the “R*M?” pattern matches strings that begin with R and whose penultimate letter is M.

[422] https://en.wikipedia.org/wiki/Escape_sequence
[423] https://www.gnu.org/software/bash/manual/html_node/Exit-Status.html#Exit-Status
[424] https://en.wikipedia.org/wiki/File_descriptor
[425] https://www.gnu.org/software/bash/manual/html_node/Filename-Expansion.html#Filename-Expansion
[426] https://en.wikipedia.org/wiki/Filename_extension
[427] https://en.wikipedia.org/wiki/File_system
[428] https://www.gnu.org/software/bash/manual/html_node/Job-Control-Basics.html#Job-Control-Basics
[429] https://en.wikipedia.org/wiki/Process_group
[430] https://en.wikipedia.org/wiki/General-purpose_programming_language
[431] https://mywiki.wooledge.org/glob?action=show&redirect=globbing
[432] https://en.wikipedia.org/wiki/Glob_(programming)
[433] https://en.wikipedia.org/wiki/Wildcard_character

# H

**Hash function**[434] generates a unique sequence of bytes from the input data.

[ calculates the keys for added elements.

# I

**Idiom**[435] is a way to express a typical construct using a specific programming language. An idiom is a template for implementing an algorithm or data structure.
Here is the Bash idiom for processing a list of
files in the `for` loop:

```
1 for file in ./*.txt
2 do
3   cp "$file" ~/Documents
4 done
```

**Input field separator**[436] (IFS) is a list of characters. Bash uses them as separators when processing input strings. For example, it uses them for word splitting. The default separators are space, tab and a line break.

**Interpreter**[437] is a program that executes instructions. Instructions are written in a programming language. You do not need the compilation step for executing them.

**Iteration**[438] is a single execution of a commands block in the loop's body.

# K

# L

**Library**[439] is a collection of subroutines and objects assembled into a standalone module or file. Applications use libraries as building blocks.

**Linked list**[440] is a data structure consisting of elements or nodes. Their positions in the list do not match their placement in memory. Therefore, each node has a pointer to the next node. Such an organization of the list leads to efficient insertion and deletion operations.

[434] https://en.wikipedia.org/wiki/Hash_function
[435] https://en.wikipedia.org/wiki/Programming_idiom
[436] https://mywiki.wooledge.org/IFS
[437] https://en.wikipedia.org/wiki/Interpreter_(computing)
[438] https://en.wikipedia.org/wiki/Iteration#Computing
[439] https://en.wikipedia.org/wiki/Library_(computing)
[440] https://en.wikipedia.org/wiki/Linked_list

**Linux distribution**[441] is an operating system based on the Linux kernel and the GNU packages[442]. The OS is combined from packages of ready-to-run programs and libraries. There is a package manager application for operating them.

**Linux environment** is another name for a POSIX environment.

**Literal**[443] is a notation in the source code of the program. It represents a fixed value. There are different ways to write literals depending on the data type. Most programming languages support literals for integers, floating point numbers[444] and strings. Here is an example of the string literal `~/Documents` in Bash:

```
1 var="~/Documents"
```

**Logical operator**[445] is an operation on one or more Boolean expressions. The operation can combine them into a single expression. Its calculation result depends on the values of the original expressions.

# M

**Multiprogramming**[446] is distributing the computer's workload among several programs. For example, the computer runs a program until it needs some resource. If the resource is busy, the program stops. The computer switches to another program. It returns to running the first program when the resource it needs becomes free.

**Multitasking**[447] is the parallel execution of several tasks (processes) in a certain period of time. The computer does it by switching[448] between tasks and executing them in parts.

# N

**Network protocol**[449] is an agreement on the format of messages between nodes of a computer network.

# O

**Operand**[450] is an argument of a mathematical operation or command. It represents the data to be processed. Here is an example of operands 1 and 4 in the addition operation:

[441] https://en.wikipedia.org/wiki/Linux_distribution
[442] https://en.wikipedia.org/wiki/List_of_GNU_packages
[443] https://en.wikipedia.org/wiki/Literal_(computer_programming)
[444] https://en.wikipedia.org/wiki/Floating-point_arithmetic
[445] https://en.wikipedia.org/wiki/Logical_connective
[446] https://www.geeksforgeeks.org/difference-between-multitasking-multithreading-and-multiprocessing
[447] https://en.wikipedia.org/wiki/Computer_multitasking
[448] https://en.wikipedia.org/wiki/Context_switch
[449] https://en.wikipedia.org/wiki/Communication_protocol
[450] https://en.wikipedia.org/wiki/Operand

```
1 + 4
```

**Option**[451] is an argument in a standardized form, which the program receives on input. The option begins with a dash `-` or a double dash `--`. It switches the mode of the program's operation. You can combine successive options into one group. Here is an example of grouping the `-l`, `-a` and `-h` options of the `ls` utility:

```
ls -lah
```

# P

**Parameter**[452] is an entity that stores some value. A parameter may have no name, unlike a variable.

**Parameter expansion**[453] is the Bash mechanism that replaces a variable name by its value. Here are two examples:

```
echo "$PATH"
echo "${var:-empty}"
```

**Pipeline**[454] is a mechanism for process communication in Unix-like operating systems. The mechanism is based on passing messages. A pipeline is also two or more processes with connected input and output streams. The output stream of one process is sent directly to the other's input stream.

**Portable operating system interface**[455] (POSIX) is a set of standards. They describe the interfaces between programs and OS, its shell and the utility interfaces. POSIX supports the compatibility of the Unix family OSes. It allows programs to migrate between systems easily.

**Positional parameters**[456] contain all command-line arguments that the Bash script receives on calling. Parameter names match the order of arguments. Here is an example of using the first positional parameter in a script:

```
cp "$1" ~
```

**POSIX environment** is a software environment fully or partially compatible with the POSIX standard. The full compatibility is available only when the OS kernel, shell and file system supports POSIX. Environments like Cygwin[457] provide partial compatibility only.

[451] http://linuxcommand.org/lc3_wss0120.php
[452] http://www.gnu.org/savannah-checkouts/gnu/bash/manual/bash.html#Shell-Parameters
[453] https://www.gnu.org/software/bash/manual/html_node/Shell-Parameter-Expansion.html#Shell-Parameter-Expansion
[454] https://en.wikipedia.org/wiki/Pipeline_(Unix)
[455] https://en.wikipedia.org/wiki/POSIX
[456] https://www.gnu.org/software/bash/manual/html_node/Shell-Parameters.html#Shell-Parameters
[457] https://en.wikipedia.org/wiki/Cygwin

**POSIX shell**[458] is a standard for POSIX systems that describes a minimum set of shell features. If the shell provides these features, it is considered POSIX compatible. The standard does not restrict additional features and extensions in any way. The standard is based on the ksh88 implementation of the Korn shell[459]. This interpreter appeared later than the Bourne shell. Therefore, the Bourne shell misses some features of the POSIX standard.

**Process**[460] is an instance of a computer program that is being executed by the CPU.

**Process substitution**[461] is the Bash mechanism that resembles command substitution. It executes the command and provides its output to the Bash process. The mechanism transfers data via temporary files. Here is an example:

```
diff <(sort file1.txt) <(sort file2.txt)
```

**Programming paradigm**[462] a set of ideas, methods and principles that define how to write programs.

# Q

**Quote removal**[463] is the Bash mechanism that removes the following unescaped characters: \, ‘ and “.

# R

**Recursion**[464] is a case when the function calls itself. This is the direct recursion. The indirect recursion happens when the function calls itself through other functions.

**Redirections**[465] are the special Bash and Bourne shell language constructs that redirect I/O streams of built-in commands, utilities and applications. You should specify file descriptors as the source and target of the redirection. The descriptors point to files or standard streams. Here is an example of redirecting the `find` utility output to the `result.txt` file:

```
find / -path */doc/* -name README 1> result.txt
```

**Reserved variables** are the same as shell variables.

[458] https://www.grymoire.com/Unix/Sh.html
[459] https://en.wikipedia.org/wiki/KornShell
[460] https://en.wikipedia.org/wiki/Process_(computing)
[461] https://www.gnu.org/software/bash/manual/html_node/Process-Substitution.html#Process-Substitution
[462] https://en.wikipedia.org/wiki/Programming_paradigm
[463] https://www.gnu.org/software/bash/manual/html_node/Quote-Removal.html#Quote-Removal
[464] https://en.wikipedia.org/wiki/Recursion_(computer_science)
[465] https://www.gnu.org/software/bash/manual/html_node/Redirections.html#Redirections

# S

**Scope**[466] is a part of a program or system where the variable's name remains associated with its value. In other words, the variable name is correctly converted to the memory address where its value is stored. Outside the scope, the same name can point to another memory location.

**Shebang**[467] is a sequence of a number sign and exclamation mark (#!) at the beginning of the script. The program loader treats the string after shebang as the interpreter's name. The loader launches the interpreter and passes the script to it for execution. Here is an example of shebang for the Bash script:

```
1 #!/bin/bash
```

**Shell options**[468] change the behavior of the interpreter when it works in both shell and script modes. The built-in `set` command sets the options. For example, here is the command to enable the debug output of the interpreter:

```
set -x
```

**Shell parameter**[469] is a named area of the shell's memory for storing data.

**Shell variables**[470] are variables that the shell sets (for example, `PATH`). They store temporary data, settings and states of the OS or Unix environment. The user can read the values of these variables. Only some of them are writable. The `set` command prints values of the shell variables.

**Short-circuit evaluation**[471] (short-circuit) is the approach to limit calculations when deducing Boolean expression. The idea is to calculate only those operands that are sufficient to deduce the whole expression's value.

**Special parameters**[472] are set by the interpreter. They perform the following tasks:

1. Store the interpreter's state.
2. Pass parameters to the called program.
3. Store the exit status of the finished program.

Special parameters are read-only. An example of such a parameter is `$?`.

**Standard streams**[473] are the software channels of communication between the program and the environment where it operates. Streams are abstractions of physical channels of input from the

[466] https://en.wikipedia.org/wiki/Scope_(computer_science)
[467] https://en.wikipedia.org/wiki/Shebang_(Unix)
[468] https://www.tldp.org/LDP/abs/html/options.html
[469] http://mywiki.wooledge.org/BashGuide/Parameters
[470] http://tldp.org/LDP/Bash-Beginners-Guide/html/sect_03_02.html
[471] https://en.wikipedia.org/wiki/Short-circuit_evaluation
[472] http://mywiki.wooledge.org/BashGuide/Parameters
[473] https://en.wikipedia.org/wiki/Standard_streams

keyboard and output to the screen. You can operate channels via their descriptors. OS assigns these descriptors.

**Subroutine**[474] is a fragment of a program that performs a single task. The fragment is an independent code block. It can be called from any place in the program.

**Subshell**[475] is a way of grouping shell commands. A child process executes the grouped commands. Variables defined in the child process are not available in the parent process. Here is an example of executing commands in a subshell:

```
(ps aux | grep "bash")
```

**Symbolic link**[476] is a special type of file. Instead of data, it contains a pointer to another file or directory.

**Synchronous**[477] means events or actions that occur in the main program flow.

# T

**Tilde expansion**[478] (tilde expansion) is the Bash mechanism that replaces the ~ character by the user's home directory. The shell takes the path to the directory from the `HOME` variable.

**Time-sharing**[479] is the approach when several users utilize computer resources simultaneously. It is achieved by multitasking and multiprogramming.

# U

**Unix environment**[480] is another name for a POSIX environment.

**Utility software**[481] is a special program for managing the OS or hardware.

# V

**Variable**[482] is an area of memory that can be accessed by its name.
There is another meaning of this term for Bash. A variable is a parameter that is accessible by its name. The user or the interpreter defines variables.
Here is an example of the variable declaration:

[474] https://en.wikipedia.org/wiki/Subroutine
[475] http://mywiki.wooledge.org/SubShell
[476] https://en.wikipedia.org/wiki/Symbolic_link
[477] https://en.wiktionary.org/wiki/synchronous
[478] https://www.gnu.org/software/bash/manual/html_node/Tilde-Expansion.html#Tilde-Expansion
[479] https://en.wikipedia.org/wiki/Time-sharing
[480] https://ccrma.stanford.edu/guides/planetccrma/Unix.html
[481] https://en.wikipedia.org/wiki/Utility_software
[482] https://www.gnu.org/software/bash/manual/html_node/Shell-Parameters.html#Shell-Parameters

```
filename="README.txt"
```

**Vulnerability**[483] is a bug or flaw in the computing system. An attacker can perform unauthorized actions using the vulnerability.

# W

**Word splitting**[484] is the Bash mechanism that splits command-line arguments into words. Then it passes them to the command as separate parameters. The mechanism uses the characters from the `IFS` variable as delimiters. It skips arguments that are enclosed in quotes. Here is an example:

```
cp file1.txt file2.txt "my file.txt" ~
```

[483] https://en.wikipedia.org/wiki/Vulnerability_(computing)
[484] https://www.gnu.org/software/bash/manual/html_node/Word-Splitting.html#Word-Splitting

# Solutions for Exercises

## General Information

### Exercise 1-1. Numbers conversion from binary to hexadecimal

```
* 10100110100110 = 0010 1001 1010 0110 = 2 9 A 6 = 29A6

* 10110001110101000100 11 = 0010 1100 0111 0101 0001 0011 = 2 C 7 5 1 3 = 2C7513

* 11111011100010010101001100000000110101101 =
1111 1011 1000 1001 0101 0011 0000 0001 1010 1101 = F B 8 9 5 3 0 1 A D =
FB895301AD
```

### Exercise 1-2. Numbers conversion from hexadecimal to binary

```
* FF00AB02 = F F 0 0 A B 0 2 = 1111 1111 0000 0000 1010 1011 0000 0010 =
11111111000000001010101100000010

* 7854AC1 = 7 8 5 4 A C 1 = 0111 1000 0101 0100 1010 1100 0001 =
111100001010100101011000001

* 1E5340ACB38 = 1 E 5 3 4 0 A C B 3 8 =
001 1110 0101 0011 0100 0000 1010 1100 1011 0011 1000 =
111100101001101000000101011001011001111000
```

## Bash Shell

### Exercise 2-1. Glob patterns

The correct answer is “README.md”.

The “00_README.txt” string does not fit. It happens because the “*ME.??”” pattern requires two characters after the dot. But the string has three characters.

The “README” string does not fit because it does not have a dot.

### Exercise 2-2. Glob patterns

The following three lines match the "*/doc?openssl*" pattern:

- `/usr/share/doc/openssl/IPAddressChoice_new.html`
- `/usr/share/doc_openssl/IPAddressChoice_new.html`
- `/doc/openssl`

The "doc/openssl" string does not fit. It does not have the slash symbol before the "doc" word.

### Exercise 2-3. Searching for files with the `find` utility

The following command searches text files in the system paths:

```
find /usr -name "*.txt"
```

The `/usr` path stores text files. So, there is no reason to check other system paths.

Now let's count the number of lines in the found files. We should add the `-exec` action with the `wc` utility call to do this. The resulting command looks like this:

```
find /usr -name "*.txt" -exec wc -l {} +
```

You can find all text files on the disk if you start searching from the root directory. Here is the example command:

```
find / -name "*.txt"
```

If you add the `wc` call to the command, it fails when running in the MSYS2 environment. In other words, the following command does not work:

```
find / -name "*.txt" -exec wc -l {} +
```

The problem happens because of the error message that Figure 2-17 shows. The `find` utility passes the message to the `wc` input. The `wc` utility treats each word it receives as a file path. The error message is not a path. Therefore, `wc` fails.

### Exercise 2-4. Searching for files with the `grep` utility

Look for information about application licenses in the `/usr/share/doc` system path. It contains documentation for all installed software.

If the program has the GNU General Public License[485], its documentation contains the "General Public License" line. The following command searches such documents:

[485] https://en.wikipedia.org/wiki/GNU_General_Public_License

```
grep -Rl "General Public License" /usr/share/doc
```

You also should check the files in the `/usr/share/licenses` path. Here is the command for doing that:

```
grep -Rl "General Public License" /usr/share/licenses
```

The MSYS2 environment has two extra paths for installing programs: `/mingw32` and `/mingw64`. They do not match the POSIX standard. The following commands check documents in these paths:

```
grep -Rl "General Public License" /mingw32/share/doc
grep -Rl "General Public License" /mingw64/share
```

You can find applications with other licenses than GNU General Public License. This is the list of licenses and possible lines for searching:

- MIT - "MIT license"
- Apache - "Apache license"
- BSD - "BSD license"

### Exercise 2-6. Operations with files and directories

First, you should create directories for saving photos by each year and month. The following commands do this task:

```
mkdir -p ~/photo/2019/11
mkdir -p ~/photo/2019/12
mkdir -p ~/photo/2020/01
```

Suppose that the `D:\Photo` directory contains all photos. You can use the `find` utility for searching files with a creation date equals to November 2019. The `-newermt` option of the utility checks the creation date. Here is an example command:

```
find /d/Photo -type f -newermt 2019-11-01 ! -newermt 2019-12-01
```

This command looks for files in the `/d/Photo` directory. It corresponds to the `D:\Photo` path in the Windows environment.

The first expression `-newermt 2019-11-01` means to search for files changed since November 1, 2019. The second expression `! -newermt 2019-12-01` excludes files modified since December 1, 2019. The exclamation point before the expression is a negation. There is no condition between expressions. But the find utility inserts the logical AND by default. The resulting expression looks like: "files created after November 1, 2019, but no later than November 30, 2019". It means "files for November".

The file search command is ready. Now add the copy action to it. The result looks like this:

```
find /d/Photo -type f -newermt 2019-11-01 ! -newermt 2019-12-01 -exec cp {} ~/photo/\
2019/11 \;
```

This command copies the November 2019 files into the `~/photo/2019/11` directory.

Here are similar commands for copying the December and January files:

```
find /d/Photo -type f -newermt 2019-12-01 ! -newermt 2020-01-01 -exec cp {} ~/photo/\
2019/12 \;
find /d/Photo -type f -newermt 2020-01-01 ! -newermt 2020-02-01 -exec cp {} ~/photo/\
2020/01 \;
```

Let's assume that you do not need old files in the `D:\Photo` directory. Then replace copying action with renaming. The result is the following commands:

```
find /d/Photo -type f -newermt 2019-11-01 ! -newermt 2019-12-01 -exec mv {} ~/photo/\
2019/11 \;
find /d/Photo -type f -newermt 2019-12-01 ! -newermt 2020-01-01 -exec mv {} ~/photo/\
2019/12 \;
find /d/Photo -type f -newermt 2020-01-01 ! -newermt 2020-02-01 -exec mv {} ~/photo/\
2020/01 \;
```

Note the scalability of our solution. The number of files in the `D:\Photo` directory is not important. You need only three commands to break them up into three months.

### Exercise 2-7. Pipelines and I/O streams redirection

First, let's figure out how the `bsdtar` utility works. Call it with the `--help` option. It shows you a brief help. The help tells you that the utility archives the directory if you apply the `-c` and `-f` options. The name of the archive follows the options. Here is an example call of the `bsdtar` utility:

```
bsdtar -c -f test.tar test
```

This command creates an archive named `test.tar`. It has the contents of the `test` directory inside. Note that the command does not compress data[486]. It means that the archive occupies the same disk space as the files it contains.

The purposes of archiving and compression operations are different. Archiving is for storing and copying large numbers of files. Compression reduces the amount of the occupied disk memory. Often these operations are combined into one. But they are not the same.

To create an archive and compress it, add the `-j` option to the `bsdtar` call. Here is an example:

[486] https://en.wikipedia.org/wiki/Data_compression

```
bsdtar -c -j -f test.tar.bz2 test
```

You can combine the `-c`, `-j` and `-f` options into one group. Then you get the following command:

```
bsdtar -cjf test.tar.bz2 test
```

Let's write a command to navigate through the catalog of photos. It should create a separate archive for each month.

The following `find` utility call finds directories that match specific months:

```
find ~/photo -type d -path */2019/* -o -path */2020/*
```

Next, we redirect the output of this command to the `xargs` utility. It will generate the `bsdtar` call. Our command looks like this now:

```
find ~/photo -type d -path */2019/* -o -path */2020/* | xargs -I% bsdtar -cf %.tar %
```

We can add the `-j` option to force `bsdtar` to compress archived data. The command became like this:

```
find ~/photo -type d -path */2019/* -o -path */2020/* | xargs -I% bsdtar -cjf %.tar.\
bz2 %
```



The data compression takes longer than archiving.

We pass the `-I` parameter to the `xargs` utility. It specifies where to insert the arguments into the generated command. There are two such places in the `bsdtar` utility call: the archive's name and the directory's path to be processed.

Do not forget about filenames with line breaks. To process them correctly, add the `-print0` option to the `find` utility call. This way, we get the following command:

```
find ~/photo -type d -path */2019/* -o -path */2020/* -print0 | xargs -0 -I% bsdtar \
-cjf %.tar.bz2 %
```

Suppose that we want to keep the files in the archives without relative paths (e.g. `2019/11`). The `--strip-components` option of `bsdtar` removes them. Here is the command with this option:

```
find ~/photo -type d -path */2019/* -o -path */2020/* -print0 | xargs -0 -I% bsdtar \
--strip-components=3 -cjf %.tar.bz2 %
```

### Exercise 2-8. Logical operators

Let's implement the algorithm step by step. The first action is to copy the `README` file to the user's home directory. The following command does it:

```
cp /usr/share/doc/bash/README ~
```

Use the && operator and the `echo` command to print the command's result into the log file. The command became like this:

```
cp /usr/share/doc/bash/README ~ && echo "cp - OK" > result.log
```

Call the `bsdtar` or `tar` utility to archive a file. Here is an example:

```
bsdtar -cjf ~/README.tar.bz2 ~/README
```

We can print the result of archiving operation using the `echo` command again:

```
bsdtar -cjf ~/README.tar.bz2 ~/README && echo "bsdtar - OK" >> result.log
```

In this case, the `echo` command appends a line to the end of the existing log file.

Let's combine the `cp` and `bsdtar` calls into one command. We should call the `bsdtar` utility only if the `README` file has been successfully copied. To achieve this dependency, we put the && operator between the commands. We get the following command:

```
cp /usr/share/doc/bash/README ~ && echo "cp - OK" > result.log && bsdtar -cjf ~/READ\
ME.tar.bz2 ~/README && echo "bsdtar - OK" >> result.log
```

The last missing step is deleting the `README` file. We can add it this way:

```
cp /usr/share/doc/bash/README ~ && echo "cp - OK" > ~/result.log && bsdtar -cjf ~/RE\
ADME.tar.bz2 ~/README && echo "bsdtar - OK" >> ~/result.log && rm ~/README && echo "\
rm - OK" >> ~/result.log
```

Run this command. If it succeeds, the log file looks like this:

```
cp - OK
bsdtar - OK
rm - OK
```

The command with calls to three utilities in a row looks cumbersome. It is inconvenient to read and edit. Let's break the command into lines. There are several ways to do this.

The first way is to break lines after logical operators. We apply it and get the following:

```
cp /usr/share/doc/bash/README ~ && echo "cp - OK" > ~/result.log &&
bsdtar -cjf ~/README.tar.bz2 ~/README && echo "bsdtar - OK" >> ~/result.log &&
rm ~/README && echo "rm - OK" >> ~/result.log
```

Try to copy this command into a terminal window and execute it. It will run without errors.

The second way to add line breaks is to use the backslash character. Put a line break right after it. Use this method when there are no logical operators in the command.

For example, let's put backslashes before the && operators in our command. Then we get this result:

```
cp /usr/share/doc/bash/README ~ && echo "cp - OK" > ~/result.log \
&& bsdtar -cjf ~/README.tar.bz2 ~/README && echo "bsdtar - OK" >> ~/result.log \
&& rm ~/README && echo "rm - OK" >> ~/result.log
```

### Exercise 3-2. The full form of parameter expansion

The `find` utility searches for files recursively. It starts from the specified path and passes through all subdirectories. Use the `-maxdepth` option to exclude subdirectories from the search.

The command for searching TXT files in the current directory looks like this:

```
find . -maxdepth 1 -type f -name "*.txt"
```

Let's add an action to copy the found files to the user's home directory. The command became like this:

```
find . -maxdepth 1 -type f -name "*.txt" -exec cp -t ~ {} \;
```

Now create a script and name it `txt-copy.sh`. Copy our `find` call into the file.

The script should receive an input parameter. The parameter defines an action to apply for each found file. There are two possible actions: copy or move. It is convenient to pass the utility's name as a parameter. Thus, it can be `cp` or `mv`. The script will call the utility by its name.

According to our idea, the script copies the files when it is called like this:

```
./txt-copy.sh cp
```

When you want to move files, you call the script like this:

```
./txt-copy.sh mv
```

You can access the first parameter of the script by the `$1` name. Place this name in the `-exec` action of the`find` call. This way, we get the following command:

```
find . -maxdepth 1 -type f -name "*.txt" -exec "$1" -t ~ {} \;
```

If you do not specify an action, the script should copy the files. It means that the following call is acceptable:

```
./txt-copy.sh
```

To make this work, add a default value to the parameter expansion. Listing 5-1 shows the final script that we get this way.

**Listing 5-1. The script for copying TXT files**

```
#!/bin/bash

find . -maxdepth 1 -type f -name "*.txt" -exec "${1:-cp}" -t ~ {} \;
```

### Exercise 3-4. The `if` statement

The original command looks like this:

```
( grep -RlZ "123" target | xargs -0 cp -t . && echo "cp - OK" || ! echo "cp - FAILS"\
 ) && ( grep -RLZ "123" target | xargs -0 rm && echo "rm - OK" || echo "rm - FAILS" \
)
```

Note the negation of the `echo` call with the “cp - FAILS” output. The negation prevents the second `grep` call if the first one fails.

Replace the && operator between `grep` calls with the `if` statement. We get the following code:

```
if grep -RlZ "123" target | xargs -0 cp -t .
then
  echo "cp - OK"
  grep -RLZ "123" target | xargs -0 rm && echo "rm - OK" || echo "rm - FAILS"
else
  echo "cp - FAILS"
fi
```

Now replace the || operators in the second `grep` call with the `if` statement. The result looks like this:

```
if grep -RlZ "123" target | xargs -0 cp -t .
then
  echo "cp - OK"
  if grep -RLZ "123" target | xargs -0 rm
  then
    echo "rm - OK"
  else
    echo "rm - FAILS"
  fi
else
  echo "cp - FAILS"
fi
```

To avoid nested `if` statements, we will apply the early return pattern. We will also add a shebang at the beginning of the script. Listing 5-2 shows the result.

**Listing 5-2. The script for searching a string in files**

```
#!/bin/bash

if ! grep -RlZ "123" target | xargs -0 cp -t .
then
  echo "cp - FAILS"
  exit 1
fi

echo "cp - OK"

if grep -RLZ "123" target | xargs -0 rm
then
  echo "rm - OK"
else
  echo "rm - FAILS"
fi
```

### Exercise 3-5. The [[ operator

Let's compare the contents of the two directories. The result of the comparison is a list of files that are different.

First, we have to go through all the files in each directory. The `find` utility does this task. Here is a command to search files in the `dir1` directory:

```
find dir1 -type f
```

The command's output can look like this:

```
dir1/test3.txt
dir1/test1.txt
dir1/test2.txt
```

We got a list of files in the `dir1` directory. We should check that each of them presents in the `dir2` directory. Add the following `-exec` action for this check:

```
1 cd dir1
2 find . -type f -exec test -e ../dir2/{} \;
```

Here we use the `test` command instead of the [[ operator. The reason is the built-in interpreter of the `find` utility can not handle this operator correctly. This is one of the exceptions when [[ must be replaced by the `test` command. In general, prefer the [[ operator.

If the `dir2` directory does not contain some file, let's print its name. We need two things for doing that. The first one is inverting the `test` command's result. Then we should add the second `-exec` action with the `echo` command call. Place the logical AND between these two actions. This way, we got the following command:

```
1 cd dir1
2 find . -type f -exec test ! -e ../dir2/{} \; -a -exec echo {} \;
```

We found files of the `dir1` directory that miss in `dir2`. Now we should do vice versa. The similar `find` call can print `dir2` files that miss in `dir1`.
s

Listing 5-3 shows the complete `dir-diff.sh` script for directory comparison.

**Listing 5-3. The script for directory comparison**

```
#!/bin/bash

cd dir1
find . -type f -exec test ! -e ../dir2/{} \; -a -exec echo {} \;

cd ../dir2
find . -type f -exec test ! -e ../dir1/{} \; -a -exec echo {} \;
```



We wrote the directory comparison script for training purposes. Please do not use it for real tasks. Prefer the special GNU utility diff.

### Exercise 3-6. The `case` statement

The script for switching between configuration files will create symbolic links.



You cannot create a symbolic link in a Unix environment running on Windows. Instead, the `ln` utility copies the file or directory corresponding to the link. The utility works fine on Linux and macOS.

Symbolic links are useful when you want to access a file or directory from the file system's specific location. When you open a link, you edit the file to which it points. The link to a directory works the same way. Any changes apply to the target directory.

The algorithm for switching between configuration files looks like this:

1. Remove the existing symbolic link or file in the `~/.bashrc` path.
2. Check the command-line option passed to the script.
3. Depending on the option, create a symlink to the `.bashrc-home` or `.bashrc-work` file.

Let's implement this algorithm by using the `case` operator. Listing 5-4 shows the result.

**Listing 5-4. The script for switching configuration files**

```
#!/bin/bash

file="$1"

rm ~/.bashrc

case "$file" in
  "h")
    ln -s ~/.bashrc-home ~/.bashrc
    ;;

  "w")
    ln -s ~/.bashrc-work ~/.bashrc
    ;;

  *)
    echo "Invalid option"
    ;;
esac
```

The `ln` calls differ only by the target filename. This similarity hints that you can replace the case statement with an associative array. Then we get a script similar to Listing 5-5.

**Listing 5-5. The script for switching configuration files**

```
#!/bin/bash

option="$1"

declare -A files=(
  ["h"]="~/.bashrc-home"
  ["w"]="~/.bashrc-work")

if [[ -z "$option" || ! -v files["$option"] ]]
then
  echo "Invalid option"
  exit 1
fi

rm ~/.bashrc

ln -s "${files["$option"]}" ~/.bashrc
```

Consider the last line of the script. Double-quotes are not necessary when inserting an array element here. But they will prevent an error if the filename gets spaces in the future.

### Exercise 3-7. Arithmetic operations with numbers in the two's complement representation

The results of the adding of single-byte integers:

```
* 79 + (-46) = 0100 1111 + 1101 0010 = 1 0010 0001 -> 0010 0000 = 33

* -97 + 96 = 1001 1111 + 0110 0000 = 1111 1111 -> 1111 1110 -> 1000 0001 = -1
```

The result of adding two-byte integers:

```
* 12868 + (-1219) = 0011 0010 0100 0100 + 1111 1011 0011 1101 =
1 0010 1101 1000 0001 -> 0010 1101 1000 0001 = 11649
```

Use the online calculator[487] to check the conversion integers' correctness to the two's complement.

### Exercise 3-8. Modulo and the remainder of a division

```
* 1697 % 13
q = 1697 / 13 ~ 130.5385 ~ 130
r = 1697 - 13 * 130 = 7

* 1697 modulo 13
q = 1697 / 13 ~ 130.5385 ~ 130
r = 1697 - 13 * 130 = 7

* 772 % -45
q = 772 / -45 ~ -17.15556 ~ -17
r = 772 - (-45) * (-17) = 7

* 772 modulo -45
q = (772 / -45) - 1 ~ -18.15556 ~ -18
r = 772 - (-45) * (-18) = -38

* -568 % 12
q = -568 / 12 ~ -47.33333 ~ -47
r = -568 - 12 * (-47) = -4
```

[487] https://planetcalc.com/747/

```
* -568 modulo 12
q = (-568 / 12) - 1 ~ -48.33333 ~ -48
r = -568 - 12 * (-48) = 8

* -5437 % -17
q = -5437 / -17 ~ 319.8235 ~ 319
r = -5437 - (-17) * 319 = -14

* -5437 modulo -17
q = -5437 / -17 ~ 319.8235 ~ 319
r = -5437 - (-17) * 319 = -14
```

### Exercise 3-9. Bitwise NOT

First, let us calculate bitwise NOT for unsigned two-byte integers.

```
 56 = 0000 0000 0011 1000
~56 = 1111 1111 1100 0111 = 65479

 1018 = 0000 0011 1111 1010
~1018 = 1111 1100 0000 0101 = 64517

 58362 = 1110 0011 1111 1010
~58362 = 0001 1100 0000 0101 = 7173
```

If you apply bitwise NOT on signed two-byte integers, the results differ:

```
 56 = 0000 0000 0011 1000
~56 = 1111 1111 1100 0111 -> 1000 0000 0011 1001 = -57

 1018 = 0000 0011 1111 1010
~1018 = 1111 1100 0000 0101 -> 1000 0011 1111 1011 = -1019
```

You cannot represent the 58362 number as a signed two-byte integer. The reason is an overflow. If we write bits of the number in a variable of this type, we get -7174. Conversion this number to two's complement looks like this:

```
58362 = 1110 0011 1111 1010 -> 1001 1100 0000 0110 = -7174
```

Now let's apply bitwise NOT:

```
  -7174  = 1110 0011 1111 1010
~(-7174) = 0001 1100 0000 0101 = 7173
```

The following Bash commands check the results for the signed integers:

```
$ echo $((~56))
-57
$ echo $((~1018))
-1019
$ echo $((~(-7174)))
7173
```

You cannot check the bitwise NOT of a two-byte unsigned integer 58362 with Bash. The interpreter will store the number in a signed four-byte integer. Then the NOT operation gives the following result:

```
$ echo $((~58362))
-58363
```

### Exercise 3-10. Bitwise AND, OR and XOR

Let's calculate bitwise operations for unsigned two-byte integers. Here are the results:

```
1122 & 908 = 0000 0100 0110 0010 & 0000 0011 1000 1100 = 0000 0000 000 0000 = 0

1122 | 908 = 0000 0100 0110 0010 | 0000 0011 1000 1100 = 0000 0111 1110 1110 = 2030

1122 ^ 908 = 0000 0100 0110 0010 ^ 0000 0011 1000 1100 = 0000 0111 1110 1110 = 2030


49608 & 33036 = 1100 0001 1100 1000 & 1000 0001 0000 1100 = 1000 0001 0000 1000 =
33032

49608 | 33036 = 1100 0001 1100 1000 | 1000 0001 0000 1100 = 1100 0001 1100 1100 =
49612

49608 ^ 33036 = 1100 0001 1100 1000 ^ 1000 0001 0000 1100 = 0100 0000 1100 0100 =
16580
```

If the integers are signed, the bitwise operations for the first pair (1122 and 908) are the same. For the second pair, the calculation is different. Let's consider it.

First, we get the value of the 49608 and 33036 numbers in the two's complement:

```
49608 = 1100 0001 1100 1000 -> 1011 1110 0011 1000 = -15928

33036 = 1000 0001 0000 1100 -> 1111 1110 1111 0100 = -32500
```

Then we do bitwise operations:

```
-15928 & -32500 = 1100 0001 1100 1000 & 1000 0001 0000 1100 =
1000 0001 0000 1000 -> 1111 1110 1111 1000 = -32504

-15928 | -32500 = 1100 0001 1100 1000 | 1000 0001 0000 1100 =
1100 0001 1100 1100 -> 1011 1110 0011 0100 = -15924

-15928 ^ -32500 = 1100 0001 1100 1000 ^ 1000 0001 0000 1100 =
0100 0000 1100 0100 = 16580
```

You can check the results with the following Bash commands:

```
$ echo $((1122 & 908))
0
$ echo $((1122 | 908))
2030
$ echo $((1122 ^ 908))
2030

$ echo $((49608 & 33036))
33032
$ echo $((49608 | 33036))
49612
$ echo $((49608 ^ 33036))
16580

$ echo $((-15928 & -32500))
-32504
$ echo $((-15928 | -32500))
-15924
$ echo $((-15928 ^ -32500))
16580
```

### Exercise 3-11. Bit shifts

Here are the results of the bit shifts:

```
* 25649 >> 3 = 0110 0100 0011 0001 >> 3 =
0110 0100 0011 0 = 0000 1100 1000 0110 = 3206

* 25649 << 2 = 0110 0100 0011 0001 << 2 =
10 0100 0011 0001 -> 1001 0000 1100 0100 -> 1110 1111 0011 1100 = -28476

* -9154 >> 4 = 1101 1100 0011 1110 >> 4 =
1101 1100 0011 -> 1111 1101 1100 0011 -> 1000 0010 0011 1101 = -573

* -9154 << 3 = 1101 1100 0011 1110 << 3 =
1 1100 0011 1110 -> 1110 0001 1111 0000 -> 1001 1110 0001 0000 = -7696
```

These Bash commands should calculate the same:

```
$ echo $((25649 >> 3))
3206
$ echo $((25649 << 2))
102596
$ echo $((-9154 >> 4))
-573
$ echo $((-9154 << 3))
-73232
```

Bash gives different results for the second and fourth cases. It happens because Bash stores all integers in eight bytes.

Check your calculations with the online calculator[488].

### Exercise 3-12. Loop Constructs

The player has seven tries to guess a number. The same algorithm handles each try. Therefore, we can apply a loop with seven iterations.

Here is the algorithm for processing a player's action:

1. Read the input with the `read` command.
2. Compare the entered number with the number chosen by the script.
3. If the player makes a mistake, print a hint and go to step 1.
4. If the player guessed the number, finish the script.

The script can pick a random number using the `RANDOM` Bash variable. The variable returns the new random value from 0 to 32767 each time you read it. We need a number between 1 and 100. The following algorithm gets it:

1. Get a random number in the 0 to 99 range. To do this, divide the value from the `RANDOM` variable by 100 using the following formula:

[488] https://onlinetoolz.net/bitshift

```
number=$((RANDOM % 100))
```

2. Get a number in the 1 to 100 range. To do this, add one to the previous result.

Here is the complete formula for calculating a random number from 1 to 100:

```
number=$((RANDOM % 100 + 1))
```

Listing 5-6 shows the script that implements the game's algorithm.

**Listing 5-6. The script for playing More or Fewer**

```
#!/bin/bash

number=$((RANDOM % 100 + 1))

for i in {1..7}
do
  echo "Enter the number:"

  read input

  if (( input < number))
  then
    echo "The number $input is less"
  elif (( number < input))
  then
    echo "The number $input is greater"
  else
    echo "You guessed the number"
    exit 0
  fi
done

echo "You didn't guess the number"
```

To guess a number in seven tries, use the **binary search**[489]. The idea behind it is to divide an array of numbers into halves. Let's look at an example of applying the binary search for the "More or Fewer" game.

Once the game has started, we guess a number in the range from 1 to 100. The middle of this range is the number 50. Enter this value first. The script will give you the first hint. Suppose the script prints that 50 is less than the number you are looking for. This means that you should search in the

[489] https://en.wikipedia.org/wiki/Binary_search_algorithm

range from 50 to 100. Enter the middle of this range, i.e. the number 75. The answer is that 75 is also less than the number you are looking for. The conclusion is that the number you are looking for is between 75 and 100. You can calculate the middle of this range this way:

```
X = 75 + (100 - 75) / 2 = 87.5
```

Round the result up or down. It doesn't matter. Round it down and get the number 87 for the next input. If the number is still not guessed, keep dividing the range of expected numbers in half. Then you will have enough seven steps to find the right number.

### Exercise 3-13. Functions

We have considered the `code_to_error` function in the section "Using Functions in Scripts".

Let's combine the code of the `print_error` and `code_to_error` functions into one file. We will get the script from Listing 5-7.

**Listing 5-7. The script for printing error messages**

```
#!/bin/bash

code_to_error()
{
  case $1 in
    1)
      echo "File not found:"
      ;;
    2)
      echo "Permission to read the file denied:"
      ;;
  esac
}

print_error()
{
  echo "$(code_to_error $1) $2" >> debug.log
}

print_error 1 "readme.txt"
```

Now the function `code_to_error` prints messages in English. Let's rename it to `code_to_error_en`. Then the language of the messages will be clear from the function name.

Let's add a function `code_to_error_de` to the script. It prints the message in German according to the received error code. The function looks like this:

```
code_to_error_de()
{
  case $1 in
    1)
      echo "Der Datei wurde nicht gefunden:"
      ;;
    2)
      echo "Berechtigung zum Lesen der Datei verweigert:"
      ;;
  esac
}
```

Now we have to choose which `code_to_error` function to call from `print_error`. We can check the regional settings to make the right choice. The environment variable `LANG` stores these settings. If the variable's value matches the "de_DE*" pattern, call the function `code_to_error_de`. Otherwise, call `code_to_error_en`.

Listing 5-8 shows the complete code of the script.

**Listing 5-8. The script for printing error messages**

```
#!/bin/bash

code_to_error_de()
{
  case $1 in
    1)
      echo "Der Datei wurde nicht gefunden:"
      ;;
    2)
      echo "Berechtigung zum Lesen der Datei verweigert:"
      ;;
  esac
}

code_to_error_en()
{
  case $1 in
    1)
      echo "File not found:"
      ;;
    2)
      echo "Permission to read the file denied:"
      ;;
```

```
  esac
}

print_error()
{
  if [[ "$LANG" == de_DE* ]]
  then
    echo "$(code_to_error_de $1) $2" >> debug.log
  else
    echo "$(code_to_error_en $1) $2" >> debug.log
  fi
}

print_error 1 "readme.txt"
```

You can replace the `if` statement in the `print_error` function with `case`. For example, do it like this:

```
print_error()
{
  case $LANG in
    de_DE*)
      echo "$(code_to_error_de $1) $2" >> debug.log
      ;;
    en_US*)
      echo "$(code_to_error_en $1) $2" >> debug.log
      ;;
    *)
      echo "$(code_to_error_en $1) $2" >> debug.log
      ;;
  esac
}
```

The `case` statement is convenient if you need to support more than two languages.

There is code duplication in the `print_error` function. The same `echo` command is called in each block of the `case` statement. The only difference between the blocks is the name of the function for converting the error code into text. We can introduce the `func` variable to get rid of the code duplication. The variable will store the function name. Use the variable this way:

```
print_error()
{
  case $LANG in
    de_DE)
      local func="code_to_error_de"
      ;;
    en_US)
      local func="code_to_error_en"
      ;;
    *)
      local func="code_to_error_en"
      ;;
  esac

  echo "$($func $1) $2" >> debug.log
}
```

There is a second option to solve the code duplication problem. We can replace the `case` statements in the functions `code_to_error_de` and `code_to_error_en` with indexed arrays. Here is an example:

```
code_to_error_de()
{
  declare -a messages

  messages[1]="Der Datei wurde nicht gefunden:"
  messages[2]="Berechtigung zum Lesen der Datei verweigert:"

  echo "${messages[$1]}"
}

code_to_error_en()
{
  declare -a messages

  messages[1]="File not found:"
  messages[2]="Permission to read the file denied:"

  echo "${messages[$1]}"
}
```

We can simplify the code and go without `code_to_error` functions. Let's combine messages in all languages into one associative array. Put it in the function `print_error`. The array's keys will

combine the `LANGUAGE` variable's value and the error code. We get the `print_error` function like Listing 5-9 shows.

**Listing 5-9. The script for printing error messages**

```
#!/bin/bash

print_error()
{
  declare -A messages

  messages["de_DE",1]="Der Datei wurde nicht gefunden:"
  messages["de_DE",2]="Berechtigung zum Lesen der Datei verweigert:"

  messages["en_US",1]="File not found:"
  messages["en_US",2]="Permission to read the file denied:"

  echo "${messages[$LANGUAGE,$1]} $2" >> debug.log
}

print_error 1 "readme.txt"
```

### Exercise 3-14. Variable scope

The script in Listing 3-37 prints the following text:

```
main1: var =
foo1: var = foo_value
bar1: var = foo_value
bar2: var = bar_value
foo2: var = bar_value
main2: var =
```

Let's start with the output of "main1" and "main2" strings. The `var` variable is declared as local in the `foo` function. Thus, it is only available in the `foo` and `bar` functions. Hence, Bash supposes that `var` is undeclared before and after calling the `foo` function.

When we print the `var` value from the beginning of the `foo` function, we get it equal to "foo_value".

Next is the output of "bar1". The variable `var` is declared in the `foo` function. This function calls `bar`. Therefore, the body of the `bar` function is also the scope of `var`.

Then we assign the new value "bar_value" to `var`. Note that we are not declaring a new global variable called `var`. We overwrite an existing local variable. We get its value "bar_value" in both outputs "bar2" and "foo2".

# Helpful Links

## General Information

- The history of the electronic computers[490].
- A History of Apple's Operating Systems[491].
- The article about static and dynamic typing[492].

## Bash

- "Bash Guide for Beginners"[493] by Machtelt Garrels.
- "Advanced Bash-Scripting Guide"[494] by Mendel Cooper.
- Bash Shell Cheatsheet[495].
- Bash Reference Manual[496].
- The Bash Hackers Wiki[497].
- Bash Pitfalls[498].
- Useless Use of Cat Award[499].
- Regular expressions in grep with examples[500].
- Bash Best Practices[501].
- Bash Style Guide[502] based on mywiki.wooledge.org[503].
- Shell Style Guide[504] by Google.
- Online Bash Commands Analyzer[505].
- The Bash scripts localization guide[506].

[490] https://technicshistory.com/2017/08/29/the-electronic-computers-part-1-prologue
[491] http://web.archive.org/web/20180702193510/http://kernelthread.com/publications/appleoshistory/
[492] https://www.destroyallsoftware.com/compendium/types?share_key=baf6b67369843fa2
[493] https://tldp.org/LDP/Bash-Beginners-Guide/html/
[494] http://tldp.org/LDP/abs/html
[495] https://github.com/NisreenFarhoud/Bash-Cheatsheet
[496] https://www.gnu.org/software/bash/manual/html_node/index.html#SEC_Contents
[497] https://wiki.bash-hackers.org
[498] http://mywiki.wooledge.org/BashPitfalls
[499] http://www.smallo.ruhr.de/award.html
[500] https://www.cyberciti.biz/faq/grep-regular-expressions/
[501] http://mywiki.wooledge.org/BashGuide/Practices
[502] https://github.com/bahamas10/bash-style-guide
[503] http://mywiki.wooledge.org
[504] https://google.github.io/styleguide/shellguide.html
[505] https://explainshell.com/#
[506] https://mywiki.wooledge.org/BashFAQ/098

# Unix Environment

- "The Art of Unix Programming "[507] by Eric S. Raymond.

[507] http://www.catb.org/~esr/writings/taoup